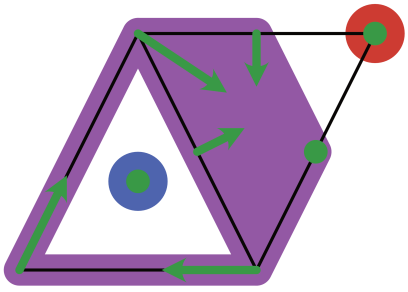
ConleyDynamics.jl

Version 0.8.1

Built by Julia 1.13.3

Thomas Wanner



November 6, 2026

Contents

Contents i

I Overview 1

1 ConleyDynamics.jl 2
- 1.1 Introduction . . . 2
- 1.2 Features . . . 2
- 1.3 Installation . . . 2
- 1.4 Manual Outline . . . 3
- 1.5 Citation . . . 5
- 1.6 License . . . 6

II Manual 7

2 Tutorial 8
- 2.1 Creating Simplicial Complexes . . . 8
- 2.2 Computing Homology and Persistence . . . 10
- 2.3 Forman Vector Fields . . . 12
- 2.4 Isolated Invariant Sets . . . 14
- 2.5 Connection Matrices . . . 15
- 2.6 Multivector Fields . . . 18
- 2.7 Analyzing Planar Vector Fields . . . 21
- 2.8 Analyzing Spatial Vector Fields . . . 25
- 2.9 References . . . 27

3 Lefschetz Complexes 29
- 3.1 Basic Lefschetz Terminology . . . 29
- 3.2 Lefschetz Complex Data Structure . . . 31
- 3.3 Simplicial Complexes . . . 34
- 3.4 Cubical Complexes . . . 37
- 3.5 Golden Ratio Rectangles . . . 41
- 3.6 Euclidean Complexes . . . 44
- 3.7 Lefschetz Complex Operations . . . 45
- 3.8 References . . . 50

4 Homology 51
- 4.1 Lefschetz Complex Homology . . . 51
- 4.2 Relative Homology . . . 56
- 4.3 Persistent Homology . . . 59

4.4 References . . . 63

5 Conley Theory **65**
5.1 Multivector Fields . . . 65
5.2 Invariance and Conley Index . . . 68
5.3 Morse Decompositions . . . 72
5.4 Connection Matrices . . . 75
5.5 Algorithm Comparisons . . . 80
Small Complexes . . . 80
Large Complexes . . . 81
When Parallelization Pays Off . . . 82
5.6 Forman's Morse Complex . . . 83
5.7 References . . . 84

6 Flow Examples **86**
6.1 Extracting Subsystems . . . 86
6.2 Analysis of a Planar System . . . 92
6.3 Analysis of a Spatial System . . . 98
6.4 References . . . 100

7 Further Examples **102**
7.1 A One-Dimensional Forman Field . . . 102
7.2 A Planar Forman Vector Field . . . 104
7.3 The Multivector Field from the Logo . . . 107
7.4 Critical Flow on a Simplex . . . 109
7.5 Flow on a Cylinder and a Moebius Strip . . . 109
7.6 Nonunique Connection Matrices . . . 112
7.7 Forcing Three Connection Matrices . . . 115
7.8 A Lefschetz Multiflow Example . . . 119
7.9 Small Complex with Periodicity . . . 121
7.10 Subdividing a Multivector . . . 124
7.11 A Combinatorial Lorenz System . . . 126
7.12 Chaos in the Dunce Hat . . . 128
7.13 Chaos in a Space with Torsion . . . 130
7.14 References . . . 133

8 Two-Dimensional Meshes **134**
8.1 Loading the Required Packages . . . 134
8.2 DelaunayTriangulation.jl . . . 135
Defining the Bounding Domain . . . 135
Adding Interior Nodes . . . 135
Adding Constraint Curves . . . 136
Mesh Refinement . . . 138
Complete Example . . . 139
Adding Intersecting Constraint Segments . . . 140

9 Three-Dimensional Meshes **144**
9.1 Loading the Required Packages . . . 144
9.2 Basic Notions . . . 146
9.3 Core Geometry Functions . . . 147
Importing Meshes . . . 147
Random Point Samples . . . 149

Triangulated Surfaces . . . 149
Orientation of Surfaces . . . 152
9.4 TetGen.jl: Tetrahedral Meshes . . . 154
Describing a Domain . . . 155
Meshing a Box . . . 156
Mesh Size and Quality . . . 157
Adding Interior Nodes . . . 158
Constraint Surfaces and Curves . . . 158
Holes . . . 161
Regions and Local Refinement . . . 163
Domains Bounded by a Surface . . . 164
Delaunay Triangulations of Random Points . . . 165
Error Messages . . . 166
Complete Example: Conley Indices on a TetGen Mesh . . . 167
9.5 Gmsh.jl: Complex and Intersecting Geometry . . . 169
Basic Usage . . . 169
Physical Groups and Entities . . . 170
Intersecting Constraint Surfaces . . . 171
Restricting the Mesh to Parts of the Model . . . 174
Reading Files . . . 174
Common Refinements of Several Objects . . . 175
Mesh Sizes and Other Options . . . 179
Complete Example: A Periodic Orbit in a Ball . . . 180
9.6 Licences of the Meshing Backends . . . 181

10 Visualization **183**
10.1 Luxor-Based Plotting . . . 183
10.2 Plots-Based Plotting . . . 184
Weak Dependency . . . 184
EuclideanComplex as Input Requirement . . . 185
Plotting a Complex and Forman Vector Fields . . . 185
Plotting Morse Sets for Planar Dynamics . . . 186
Plotting Multivector Field Regions . . . 188
Plotting a Single Multivector . . . 191
10.3 Three-Dimensional Plotting with Makie . . . 192
Makie Backends . . . 192
Plotting a Complex . . . 193
Clipping . . . 195
Combining Plots . . . 195
Highlighting Cell Subsets . . . 196
Plotting Morse Sets . . . 197
Plotting Multivectors and Multivector Fields . . . 198
Plotting Cubical Complexes . . . 199
Practical Tips . . . 200

11 Sparse Matrices **202**
11.1 Sparse Matrix Format . . . 202
11.2 Creating Sparse Matrices . . . 203
11.3 Sparse Matrix Access . . . 205
11.4 Elementary Matrix Operations . . . 206
11.5 Sparse Matrix Information . . . 207

12 References **208**

III Core API **210**

13 Composite Data Structures **211**
13.1 Lefschetz Complex Type . . . 211
13.2 Cell Subset Types . . . 213
13.3 Conley-Morse Graph Type . . . 214

14 Lefschetz Complex Functions **215**
14.1 Lefschetz Complex Creation . . . 215
14.2 Lefschetz Complex Queries . . . 221
14.3 Topological Features . . . 223
14.4 Filters on Lefschetz Complexes . . . 227
14.5 Lefschetz Helper Functions . . . 230

15 Special Complex Functions **232**
15.1 Simplicial Complexes . . . 232
15.2 Point Sets and Triangulated Surfaces . . . 237
15.3 Cubical Complexes . . . 240
15.4 Golden Ratio Rectangles . . . 243
15.5 Cell Subset Helper Functions . . . 244
15.6 Geometry Helper Functions . . . 249

16 Homology Functions **252**
16.1 Regular Homology . . . 252
16.2 Persistent Homology . . . 253
16.3 Reduction Algorithms . . . 254

17 Conley Theory Functions **255**
17.1 Multivector Fields . . . 255
17.2 Multivector Fields via Flows . . . 260
17.3 Conley Index Computations . . . 264
17.4 Connection Matrix Computation . . . 267
17.5 Forman Vector Fields . . . 269

18 Acyclic Partition Functions **278**
18.1 Construction . . . 278
18.2 Conversion . . . 279
18.3 Atomic Refinement . . . 280
18.4 Connectivity . . . 280
18.5 Morse Vector . . . 281
18.6 Block Split Analysis . . . 281
18.7 Stratification . . . 282
18.8 Top Layer . . . 283
18.9 Internal Helpers . . . 285

19 Example Functions **286**
19.1 Examples from Batko et al. . . . 286
19.2 Examples from Mrozek & Wanner . . . 288
19.3 General Examples . . . 291

20 Extension Functions **298**
20.1 DelaunayTriangulation.jl . . . 298
20.2 TetGen.jl . . . 301
20.3 Gmsh.jl . . . 308

21 Plotting Functions **313**
21.1 Visualizing Simplicial Complexes (Luxor) . . . 313
21.2 Visualizing Cubical Complexes (Luxor) . . . 315
21.3 Visualizing Acyclic Partition Strata (Luxor) . . . 317
21.4 Plot Data Types (Plots.jl) . . . 319
21.5 Visualizing Simplicial Complexes (Plots.jl) . . . 320
21.6 Visualizing Cubical Complexes (Plots.jl) . . . 322
21.7 Visualizing Complexes in Three Dimensions (Makie) . . . 323
21.8 Internal Helpers . . . 329

22 Sparse Matrix Functions **330**
22.1 Internal Sparse Matrix Representation . . . 330
22.2 Access Functions . . . 330
22.3 Basic Functions . . . 332
22.4 Conversion Functions . . . 335
22.5 Matrix Creation . . . 337
22.6 Elementary Matrix Operations . . . 340
22.7 Sparse Helper Functions . . . 348

23 Complete API Index **350**
23.1 Composite Data Structures . . . 350
23.2 Lefschetz Complex Functions . . . 350
23.3 Special Complex Functions . . . 352
23.4 Homology Functions . . . 353
23.5 Conley Theory Functions . . . 353
23.6 Acyclic Partition Functions . . . 354
23.7 Example Functions . . . 355
23.8 Extension Functions . . . 356
23.9 Plotting Functions . . . 356
23.10 Sparse Matrix Functions . . . 358

Part I

Overview

Chapter 1

ConleyDynamics.jl

Conley index and multivector fields for Julia.

1.1 Introduction

ConleyDynamics.jl is a Julia package for studying combinatorial multivector fields using Conley theory. The multivector fields can be studied on arbitrary Lefschetz complexes, which include both simplicial and cubical complexes as important special cases. The concept of combinatorial multivector field generalizes Forman vector fields, which were originally introduced to study Morse theory in a discrete combinatorial setting.

Note

This documentation is also available in PDF format: ConleyDynamics.pdf.

1.2 Features

- Data structures for Lefschetz complexes, in particular simplicial and cubical complexes.
- Classical Forman combinatorial vector fields and multivector fields are supported.
- Computation of Conley indices, connection matrices, and Conley-Morse graphs.
- Basic homology algorithms over finite fields and the rationals, including persistent homology and relative homology.
- Algorithms rely on a built-in sparse matrix implementation which is geared towards computations over finite fields and the rationals.

1.3 Installation

To use ConleyDynamics.jl please install Julia 1.10 or higher. See https://julialang.org/downloads/ for instructions on how to obtain Julia for your system.

At the Julia prompt simply type

```
julia> using Pkg; Pkg.add("ConleyDynamics")
```

After Julia has finished downloading and precompiling the package and all of its dependencies, you can start using it by typing

```
julia> using ConleyDynamics
```

Whenever possible, the functions implemented in ConleyDynamics.jl have been parallelized, so as to speed up performance on multi-core computers. This, however, is only taken advantage of if Julia is started with multiple threads. For example, if you would like to start Julia using 8 threads, use the command

```
julia -t 8
```

or

```
julia --threads=8
```

If you want to leave the number of threads selection to the machine, simply use

```
julia --threads=auto
```

Once you are in the Julia REPL, you can see the number of used threads using the command

```
julia> Threads.nthreads()
```

1.4 Manual Outline

The Tutorial briefly explains how to get started with ConleyDynamics.jl. More details, including on the underlying mathematics, are provided in the following three sections, which cover Lefschetz complexes, homology, and Conley theory, including connection matrices and algorithms for their computation. After discussing a number of examples in the sections Flow Examples and Further Examples, the chapters Two-Dimensional Meshes and Three-Dimensional Meshes explain how meshes of planar and spatial domains are created, and the chapter Visualization describes the plotting functions. The manual concludes with a description of the sparse matrix format underlying the package.

- Tutorial
  - Creating Simplicial Complexes
  - Computing Homology and Persistence
  - Forman Vector Fields
  - Isolated Invariant Sets
  - Connection Matrices
  - Multivector Fields
  - Analyzing Planar Vector Fields
  - Analyzing Spatial Vector Fields
  - References

- Lefschetz Complexes
  - Basic Lefschetz Terminology
  - Lefschetz Complex Data Structure
  - Simplicial Complexes
  - Cubical Complexes
  - Golden Ratio Rectangles
  - Euclidean Complexes
  - Lefschetz Complex Operations
  - References
- Homology
  - Lefschetz Complex Homology
  - Relative Homology
  - Persistent Homology
  - References
- Conley Theory
  - Multivector Fields
  - Invariance and Conley Index
  - Morse Decompositions
  - Connection Matrices
  - Algorithm Comparisons
  - Forman's Morse Complex
  - References
- Flow Examples
  - Extracting Subsystems
  - Analysis of a Planar System
  - Analysis of a Spatial System
  - References
- Further Examples
  - A One-Dimensional Forman Field
  - A Planar Forman Vector Field
  - The Multivector Field from the Logo
  - Critical Flow on a Simplex
  - Flow on a Cylinder and a Moebius Strip
  - Nonunique Connection Matrices
  - Forcing Three Connection Matrices
  - A Lefschetz Multiflow Example
  - Small Complex with Periodicity

    - Subdividing a Multivector
    - A Combinatorial Lorenz System
    - Chaos in the Dunce Hat
    - Chaos in a Space with Torsion
    - References
- Two-Dimensional Meshes
    - Loading the Required Packages
    - DelaunayTriangulation.jl
- Three-Dimensional Meshes
    - Loading the Required Packages
    - Basic Notions
    - Core Geometry Functions
    - TetGen.jl: Tetrahedral Meshes
    - Gmsh.jl: Complex and Intersecting Geometry
    - Licences of the Meshing Backends
- Visualization
    - Luxor-Based Plotting
    - Plots-Based Plotting
    - Three-Dimensional Plotting with Makie
- Sparse Matrices
    - Sparse Matrix Format
    - Creating Sparse Matrices
    - Sparse Matrix Access
    - Elementary Matrix Operations
    - Sparse Matrix Information

1.5 Citation

If you use ConleyDynamics.jl, please cite the JOSS paper [Wan25], which is available at https://doi.org/10.21105/joss.08085. The BibTeX entry for this paper is:

```
@article{wanner:25a,
   author = {Thomas Wanner},
   title = {Conley{D}ynamics.jl: {A} {J}ulia package for multivector
            dynamics on {L}efschetz complexes},
   journal = {Journal of Open Source Software},
   doi = {10.21105/joss.08085},
   volume = {10},
   number = {111},
   pages = {8085},
   url = {https://joss.theoj.org/papers/10.21105/joss.08085},
   year = {2025}
   }
```

1.6 License

ConleyDynamics.jl is provided under an MIT license.

Part II

Manual

Chapter 2

Tutorial

This tutorial explains the basic usage of the main components of ConleyDynamics.jl. It is not meant to be exhaustive, and more details will be provided in the more individualized sections. Also, precise mathematical definitions will be delayed until then. The presented examples are taken from the paper [BKMW20] and the book [MW25], with minor modifications. See also [Wan25].

2.1 Creating Simplicial Complexes

The fundamental mathematical object for ConleyDynamics.jl is a Lefschetz complex [Lef42]. For now we note that both simplicial complexes and cubical complexes are special cases, and ConleyDynamics.jl provides convenient interfaces for generating them.

For the sake of simplicity, this tutorial only considers the case of a simplicial complex. Recall that an abstract simplicial complex K is just a collection of finite sets, called simplices, which is closed under taking subsets. In other words, every subset of a simplex is again a simplex. Each simplex σ has an associated dimension $\dim \sigma$, which is one less than the number of its elements. One usually calls simplices of dimension 0 vertices, edges have dimension 1, and simplices of dimension 2 are triangles. It follows easily from these definitions that every simplex is the union of its vertices. The following notions associated with simplicial complexes are important for this introduction:

- A face of a simplex is any of its subsets. Notice that every simplex is a face of itself, and it is the only face that has the same dimension as the simplex. Faces whose dimension is strictly smaller are referred to as proper faces.
- The boundary of a simplex σ is the collection of all proper faces of σ. For a triangle, this amounts to all three edges and all three vertices which are part of it.
- A facet of a simplex σ is any face τ with dimension $\dim \tau = \dim \sigma - 1$. Notice that the facets of a simplex are the faces in its boundary of maximal dimension.
- The closure of a subset K_0 of a simplicial complex K consists of the collection of all faces of simplices in K_0, and we denote the closure by $\operatorname{cl} K_0$.
- A subset K_0 of a simplicial complex K is called closed, if it equals its closure. In other words, K_0 is closed if and only if for every simplex σ in K_0 all of its boundary simplices are part of K_0 as well. Thus, a closed subset of a simplicial complex is a simplicial complex in its own right.

In ConleyDynamics.jl it is easy to generate a simplicial complex. This requires two objects:

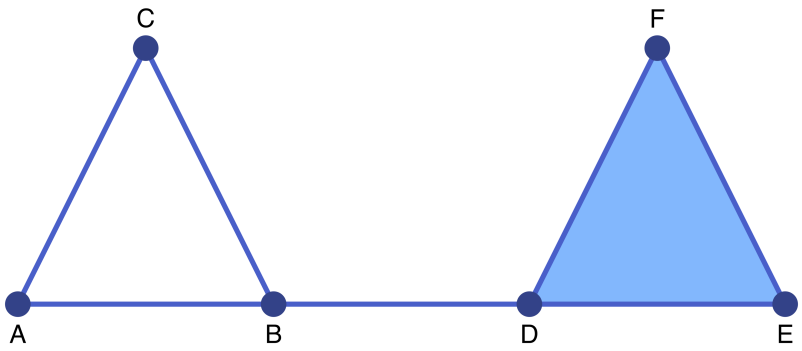


Figure 2.1: A first simplicial complex

- The vertices are described by a vector `labels` of string labels for the vertices of the simplicial complex. Thus, the length of the vector equals the number of vertices, and the k-th entry is the label for the k-th vertex.
- In addition, a second vector `simplices` has to describe enough simplices so that the simplicial complex is determined. This object is a vector of vectors, and the vector `simplices[k]` describes the index values of all the vertices in the k-th simplex. These indices are precisely the corresponding locations of the vertices in `labels`.

Simplices via labels

It is also possible to specify the list of simplices using a `Vector{Vector{String}}`, i.e., as a vector of string vectors. In this case, the entry `simplices[k]` is a list of the labels of the vertices.

Watch the label length

It is expected that the labels in `labels` all have the same number of characters. This is due to the fact that when creating the simplicial complex, ConleyDynamics.jl automatically creates labels for each of the simplices in K, by concatenating the vertex labels. Not using a fixed label size could lead to ambiguities, and will therefore raise an error message.

The following first example creates a simple simplicial complex. The complex is shown in the figure of a hollow triangle ABC connected by the edge BD to a filled triangle DEF, and it has six vertices which we label by the first six letters.

```
labels = ["A","B","C","D","E","F"]
simplices = [["A","B"],["A","C"],["B","C"],["B","D"],["D","E","F"]]
sc = create_simplicial_complex(labels,simplices)
fieldnames(typeof(sc))
```

```
(:labels, :dimensions, :boundary, :ncells, :dim, :indices)
```

Based on the simplex specifications, the generated simplicial complex K consists of three edges connecting each of the vertices A, B, and C, a two-dimensional triangle DEF, as well as the edge BD which connects the

triangle boundary and the filled triangle. The created struct `sc` is of type `LefschetzComplex`, with fieldnames as indicated in the above output. The number of cells in the complex can be seen as follows:

```
println(sc.ncells)
```

```
14
```

Note that the final simplicial complex has a total of seven edges, since also the edges of DEF are part of the simplicial complex. They are automatically generated by `create_simplicial_complex`. The dimension of K is the largest simplex dimensions, and can be recalled via

```
println(sc.dim)
```

```
2
```

The `sc` struct contains a vector of labels, which in this case takes the form

```
println(sc.labels)
```

```
["A", "B", "C", "D", "E", "F", "AB", "AC", "BC", "BD", "DE", "DF", "EF", "DEF"]
```

Finally, the Lefschetz complex data structure for our simplicial complex K includes the dimensions for the corresponding cells in the integer vector `sc.dimensions`, a dictionary `sc.indices` which associates each simplex label with its integer index, and the boundary map `sc.boundary` which will be described in more detail in Lefschetz Complexes. The latter map is internally stored as a sparse matrix over either a finite field or over the rationals. See also the discussion of Sparse Matrices.

2.2 Computing Homology and Persistence

Any simplicial complex, and in fact any Lefschetz complex, has an associated homology. Informally, homology describes the connectivity structure of the simplicial complex. More precisely, the homology consists of a sequence of integers, called the Betti numbers, which are indexed by dimension. There are Betti numbers $\beta_k(K)$ for every $k = 0, \ldots, \dim K$. The zero-dimensional Betti number $\beta_0(K)$ gives the number of connected components of K, while $\beta_1(K)$ counts the number of independent loops that can be found in K. Finally, $\beta_2(K)$ equals the number of cavities. In our case, we have

```
homology(sc)
```

```
3-element Vector{Int64}:
 1
 1
 0
```

This means that the simplicial complex K has one component, as well as one loop, and no cavities. The function `homology` returns a vector of integers, whose k-th entry is $\beta_{k-1}(K)$. We would like to point out that in ConleyDynamics.jl all homology computations are performed over fields, and therefore homology is completely described by the Betti numbers. Two types of fields are supported, and they are selected by the characteristic `p` in the sparse boundary matrix:

- If `p=0`, then the homology computation uses the field of rational numbers.
- For any prime number `p`, homology is determined over the finite field $GF(p)$ with `p` elements.

ConleyDynamics.jl also allows for the computation of relative homology. In the case of relative homology, together with the simplicial complex K one has to specify a closed subcomplex K_0. Intuitively, the relative homology $H_*(K, K_0)$ is the homology of a new space, which is obtained from K by identifying K_0 to a single point, and then decreasing the zero-dimensional Betti number by 1. Consider for example the following command:

```
relative_homology(sc, [1,6])
```

```
3-element Vector{Int64}:
 0
 2
 0
```

In this case, the subcomplex K_0 consists of the two vertices `A` and `F`, which are therefore glued together. This leads to zero Betti numbers in dimension 0 and 2 (remember that the zero-dimensional Betti number is decreased by 1!), and a one-dimensional Betti number of 2. The latter is increased by one since we obtain a second loop by moving from `A` to `F = A` along the edges `AB`, `BD`, and `DF`. Another example is the following:

```
relative_homology(sc, ["DE","DF","EF"])
```

```
3-element Vector{Int64}:
 0
 1
 1
```

Now the subcomplex K_0 consists of the edges `DE`, `DF`, and `EF` – together with the three vertices `D`, `E`, and `F` which are automatically added by `relative_homology`. Identifying them all to one point creates a hollow two-dimensional sphere, and the relative Betti numbers reflect that fact.

As the above two examples demonstrate, the subcomplex can be specified either as a list of simplex indices, or through the simplex labels. Moreover, the specfied subspace simplex list is automatically extended by `relative_homology` to include all simplex faces, i.e., it computes the simplicial closure to arrive at a closed subcomplex. Finally, note that the subcomplex can be empty:

```
relative_homology(sc, [])
```

```
3-element Vector{Int64}:
 1
 1
 0
```

As expected, in this case one obtains the standard homology of `sc`.

In addition to regular and relative homology, ConleyDynamics.jl can also compute persistent homology. For this, one has to specify a filtration of closed Lefschetz complexes

$$K_1 \subset K_2 \subset \ldots \subset K_m.$$

Persistent homology tracks the appearance and disappearance (also often called the birth and death) of topological features as one moves through the complexes in the filtration. In ConleyDynamics.jl, one can specify a Lefschetz complex filtration by assigning the integer k to each simplex that first appears in K_k. Moreover, it is expected that $K_m = K$. Then the persistent homology is computed via the following command:

```
filtration = [1,1,1,2,2,2,1,1,1,3,2,2,2,4]
phsingles, phpairs = persistent_homology(sc, filtration)
```

```
([[1], [1], Int64[]], [[(2, 3)], [(2, 4)], Tuple{Int64, Int64}[]])
```

The function returns the persistence intervals, which give the birth and death indices of each topological feature in each dimension. There are two types of intervals:

- Intervals of the form $[a, \infty)$ correspond to topological features that first appear in K_a and are still present in the final complex. The starting indices of such features in dimension `k` are contained in the list `phsingles[k+1]`.
- Intervals of the form $[a, b)$ correspond to topological features that first appear in K_a and first disappear in K_b. The corresponding pairs `(a,b)` in dimension `k` are contained in the list `phpairs[k+1]`.

In our above example, one observes intervals $[1, \infty)$ in dimensions zero and one – and these correspond to a connected component and the loop generated by the edges `AB`, `AC`, and `BC`. These appear first in K_1 and are still present in K_4. The interval $[2, 3)$ in dimension zero represents the new component created by K_2, and it disappears through merging with the older component from K_1 when the edge `BD` is introduced with K_3. Similarly, the interval $[2, 4)$ in dimension one is the loop created by the triangle `DE`, `DF`, and `EF` in K_2, which disappears with the introduction of the triangle `DEF` in K_4. Note that the interval death times respect the elder rule: When for example a component disappears through merging, the younger interval gets killed, and the older one continues to live. Similarly in higher dimensions.

2.3 Forman Vector Fields

The main focus of ConleyDynamics.jl is on the study of combinatorial topological dynamics on Lefschetz complexes. While the phase space as Lefschetz complex has been discussed above, albeit only for the special

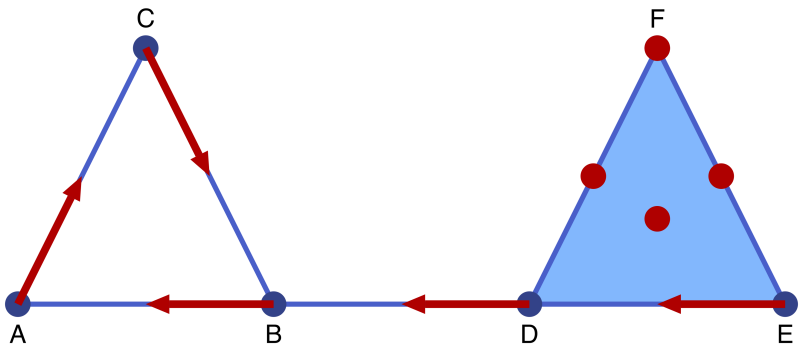


Figure 2.2: A first Forman vector field

case of a simplicial complex, the dynamics part can be given in the simplest form by a combinatorial vector field, also called a Forman vector field [For98a, For98b]. We will soon see that such vector fields are a more restrictive version of multivector fields, but they are easier to start with. The following command defines a simple Forman vector field on our sample simplicial complex K from above:

```
formanvf = [["A","AC"],["B","AB"],["C","BC"],["D","BD"],["E","DE"]]
```

```
5-element Vector{Vector{String}}:
 ["A", "AC"]
 ["B", "AB"]
 ["C", "BC"]
 ["D", "BD"]
 ["E", "DE"]
```

The Forman vector field `formanvf` is visualized in the figure which shows arrows and red critical cells on this simplicial complex.

According to this figure, a Forman vector field is comprised of arrows, as well as critical cells which are indicated by red dots. Every simplex of the underlying simplicial complex is either critical, or it is contained in a unique arrow. In other words, the collection of critical cells and arrows forms a partition of the simplicial complex K. Arrows always have to consist of precisely two simplices: The source of the arrow is a simplex σ^-, while its target is a second simplex σ^+. These two simplices have to be related in the sense that σ^- is a facet of σ^+.

As the above Julia code shows, a forman vector field is described by a vector of string vectors, where each of the latter contains the labels of the two simplices making up an arrow. Note that the critical cells are not explicitly listed, as any simplex of K that is not part of a vector is automatically assumed to be critical. Alternatively, one could define the Forman vector field as a `Vector{Vector{Int}}`, if the labels are replaced by the corresponding indices in `sc.indices`.

Intuitively, the visualization of our sample Forman vector field `formanvf` induces the following dynamical behavior on the simplicial complex `sc`:

- **Critical cells** can be though of as equilibrium states for the dynamics, i.e., they contain a stationary solution. However, depending on their dimension they can also exhibit nonconstant dynamics – which in backward time converges to the equilibrium, and in forward time flows towards the boundary of the simplex.

- **Arrow sources** always lead to flow into the interior of their target simplex σ^+.
- **Arrow targets** create flow towards the boundary of σ^+, except towards the source facet σ^-.

In the figure of the Forman vector field, for example, the simplex EF is a critical cell, so it contains an equilibrium. At the same time, it also allows for flow towards the boundary, which consists of the vertices E and F. A solution flowing to the former then has to enter DE, flow through D to BD, before entering the periodic orbit given by

$$B \to AB \to A \to AC \to C \to BC \to B \to AB \to \dots$$

This heuristic description can be made precise. It was shown in [MW21] that for every Forman vector field on a simplicial complex there exists a classical dynamical system which exhibits dynamics consistent with the above interpretation.

2.4 Isolated Invariant Sets

The global dynamical behavior of a Forman vector field on a simplicial complex can be described by first decomposing it into smaller building blocks. An invariant set is a subset $S \subset K$ of the simplicial complex such that for every simplex $\sigma \in S$ there exists a solution through σ which is contained in S and which exists for all forward and backward time. In our example the following are sample invariant sets:

- Every critical cell σ by itself is an invariant set, since we can choose the constant solution σ in the above definition. Thus, also every union of critical cells is invariant.
- The periodic orbit $S_P = \{A, B, C, AB, AC, BC\}$ is an invariant set, since the periodic orbit mentioned earlier exists for all forward and backward time in S_P and passes through every simplex of the orbit.

While it is tempting to try to decompose the dynamics into invariant sets and "everything else", Conley realized that a better theory can be built around invariant sets which are isolated [Con78]. In our combinatorial setting, an isolated invariant set is an invariant set $S \subset K$ with the following two additional properties:

- The set S is locally closed, i.e., the associated set $\operatorname{mo} S = \operatorname{cl} S \setminus S$ is closed in the simplicial complex. Recall that the closure $\operatorname{cl} A$ of a set $A \subset K$ consists of all simplices which are subsets of simplices in A, and a set is closed if it equals its closure. The set $\operatorname{mo} S$ is called the mouth of S.
- The set S is compatible with the Forman vector field, i.e., the set is the union of critical cells and arrows. In other words, if one of the arrow ends is contained in S, then so is the other.

One can easily see that the periodic orbit S_P is an isolated invariant set, since it is compatible and closed – and therefore $\operatorname{mo} S_P = \emptyset$ is closed. Similarly, the single critical simplex $S_1 = \{DEF\}$ is an isolated invariant set, since in this case the set $\operatorname{mo} S_1 = \{D, E, F, DE, DF, EF\}$ is closed, and S_1 is compatible. On the other hand, the invariant set $S_2 = \{DEF, F\}$ is not an isolated invariant set, since the mouth $\operatorname{mo} S_2 = \{D, E, DE, DF, EF\}$ is not closed – despite the fact that S_2 is compatible. For an example of an invariant set which has a closed mouth but is not compatible, see [KMW16, Figure 5].

It follows from the definition of isolation that for every isolated invariant set $S \subset K$ the two sets $\operatorname{cl} S$ and $\operatorname{mo} S$ are closed, and that the latter is a (possibly empty) subset of the former. Thus, the relative homology of this pair is defined and we let

$$CH_*(S) = H_*(\operatorname{cl} S, \operatorname{mo} S)$$

denote the Conley index of the isolated invariant set. The Conley index can be computed using the command `conley_index`. For the three critical cells `F`, `DF`, and `DEF` one obtains the following Conley indices:

```
println(conley_index(sc, ["F"]))
println(conley_index(sc, ["DF"]))
println(conley_index(sc, ["DEF"]))
```

```
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
```

In other words, the Conley index of a critical cell of dimension k has Betti number $\beta_k = 1$, while the remaining Betti numbers vanish. This is precisely the relative homology of a k-dimensional sphere with respect to a point on the sphere. On the other hand, for the Conley index of the periodic orbit S_P one obtains:

```
conley_index(sc, ["AB", "AC", "BC", "A", "B", "C"])
```

```
3-element Vector{Int64}:
 1
 1
 0
```

This Conley index is nontrivial in dimensions 0 and 1. This is exactly the Conley index of an attracting periodic orbit in classical dynamics.

2.5 Connection Matrices

One of the main features of ConleyDynamics.jl is its capability to take a given combinatorial vector or multivector field on an arbitrary Lefschetz complex and determine its global dynamical behavior. This is done by computing the connection matrix, which in our setting is discussed in detail in [MW25]. For the sample simplicial complex `sc` and the Forman vector field `formanvf` the connection matrix information can be determined as follows:

```
cm = connection_matrix(sc, formanvf)
```

```
ConleyMorseCM{Int64}: struct with the following fields:
  columns, poset, labels, morse, conley, complex
  matrix: 6x6-dimensional matrix with sparsity 0.833
  inspect the connection matrix with sparse_show(cm)
```

This command calculates the connection matrix over the finite field $GF(2) = \mathbb{Z}_2$. The base field for this computation is determined by the data type of the boundary matrix in the underlying simplicial complex `sc`. By default, if one uses the function `create_simplicial_complex` without specifying the field characteristic `p`, the simplicial complex is created over the finite field $\mathbb{Z}_2$, i.e., with p=2.

The variable `cm` is an involved struct which stores various information about the connection matrix. The above output provides a quick overview, which includes the fieldnames. These can also be shown via

```
fieldnames(typeof(cm))
```

```
(:matrix, :columns, :poset, :labels, :morse, :conley, :complex)
```

More precisely, the `connection_matrix` function returns a struct which contains the following information regarding the global dynamics of the combinatorial dynamical system:

- The field `cm.morse` contains the Morse decomposition of the Forman vector field. This is a collection of isolated invariant sets which capture all recurrent behavior. Outside of these sets, the dynamics is gradient-like, i.e., it moves from one Morse set to another.
- Since each of the Morse sets is an isolated invariant set, they all have an associated Conley index. These are contained in the field `cm.conley`.
- In addition, the struct `cm` contains information on the actual connection matrix in the field `cm.matrix`. While the field contains the matrix, the rows and columns of the connection matrix correspond to the simplices in the underlying simplicial complex `sc` listed in `cm.labels`. These simplices represent the basis for the homology groups of all the Morse sets. Moreover, a nonzero entry in the connection matrix indicates that there has to be a connecting orbit between the Morse set containing the column label and the Morse set containing the row label.

The remaining field names of the struct `cm` are described in the section on Conley Theory.

For our example system, the Morse sets are given by

```
cm.morse
```

```
5-element Vector{Vector{String}}:
 ["A", "B", "C", "AB", "AC", "BC"]
 ["F"]
 ["DF"]
 ["EF"]
 ["DEF"]
```

There are five of them: The stable periodic orbit S_P mentioned earlier, the stable critical state `F`, the unstable equilibria `DF` and `EF`, as well as the two-dimensional unstable critical cell `DEF`. The associated Conley indices are

```
cm.conley
```

```
5-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [1, 0, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 0, 1]
```

Clearly these indices are exactly as described in the homology section, since the underlying field is still $\mathbb{Z}_2$, as determined by `sc`. For an example which involves computations over different fields, which also lead to different Conley indices, we refer to the function `example_moebius`.

Finally, the connection matrix itself is contained in `cm.matrix`. Since internally the connection matrix is stored in a sparse format, we display it after conversion to a full matrix:

```
full_from_sparse(cm.matrix)
```

```
6×6 Matrix{Int64}:
 0  0  0  1  1  0
 0  0  0  0  0  0
 0  0  0  1  1  0
 0  0  0  0  0  1
 0  0  0  0  0  1
 0  0  0  0  0  0
```

In order to see which simplices represent the columns of the matrix, we use the command

```
println(cm.labels)
```

```
["C", "AB", "F", "DF", "EF", "DEF"]
```

The above two commands can also be combined in the following way, which provides a more informative display of the connection matrix:

```
sparse_show(cm)
```

```
   ┼   C  AB   F  DF  EF DEF
---┼-----------------------
  C┼   .   .   .   1   1   .
 AB┼   .   .   .   .   .   .
  F┼   .   .   .   1   1   .
 DF┼   .   .   .   .   .   1
 EF┼   .   .   .   .   .   1
DEF┼   .   .   .   .   .   .
```

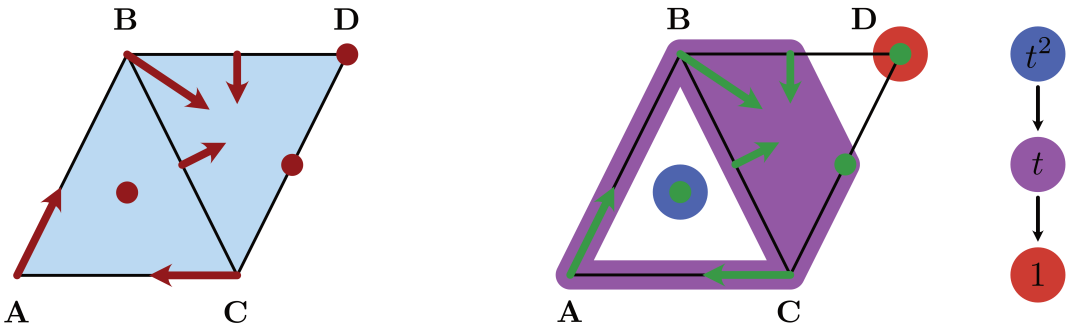


Figure 2.3: The logo multivector field

The right-most column contains two nonzero entries, and they imply that there are connecting orbits between the critical cell DEF and the two critical cells DF and EF, respectively. The second-to-last column establishes connecting orbits originating from EF. One of these ends at the critical vertex F, while the other one leads to A. Notice, however, that since A is part of the Morse set S_P, i.e., the periodic orbit, this second nonzero entry in the column implies the existence of a heteroclinic orbit between the equilibrium and the complete periodic solution. Similarly, there are connections between DF and both F and the periodic orbit, in view of the fourth column of the connection matrix.

A description of the remaining fields of `cm` can also be found in the API entry for `connection_matrix`. We would like to emphasize again that internally, all computations necessary for finding the connection matrix are performed automatically over the rationals or over the finite field $GF(p)$. The choice depends on the data type of the boundary matrix for the underlying Lefschetz complex, in this case the simplicial complex `sc`.

2.6 Multivector Fields

As second example of this tutorial we turn our attention to the logo of ConleyDynamics.jl. It shows a simple multivector field on a simplicial complex, and both the simplicial complex `sclogo` and the multivector field `mvflogo` can be defined using the commands

```
labels = ["A","B","C","D"]
simplices = [["A","B","C"],["B","C","D"]]
sclogo = create_simplicial_complex(labels,simplices)
mvflogo = [["A","AB"],["C","AC"],["B","BC","BD","BCD"]]
```

```
3-element Vector{Vector{String}}:
 ["A", "AB"]
 ["C", "AC"]
 ["B", "BC", "BD", "BCD"]
```

This example is taken from [MW25, Figure 2.1], and is visualized in the figure which shows the two triangles ABC and BCD together with the three multivectors of `mvflogo`.

The multivector field `mvflogo` clearly has a different structure from the earlier Forman vector field. While the latter consists exclusively of arrows and critical cells, the former is made up of multivectors. In this context a multivector is a collection of simplices which form a locally closed set, as defined earlier in the tutorial. One can show that in the case of a simplicial complex, this is equivalent to requiring that if $\sigma_1 \subset \sigma_2$ are two simplices

in the multivector, then so are all simplices τ with $\sigma_1 \subset \tau \subset \sigma_2$. In other words, multivectors are convex with respect to simplex inclusion, i.e., with respect to the face relation. A multivector field is then a partition of the simplicial complex into multivectors. See [LKMW23] for more details.

It is not difficult to see that every Forman vector field is a multivector field. Every critical cell consists of just one simplex, so it trivially satisfies the above convexity condition. In addition, the two simplices contained in an arrow do not allow for any simplex $\sigma^- \subset \tau \subset \sigma^+$ apart from $\tau = \sigma^{\pm}$. As in the case of Forman vector fields, multivector fields in ConleyDynamics.jl only need to list multivectors containing at least two simplices. Any simplex not contained on the list automatically gives rise to a one-element multivector.

One important difference between Forman vector fields and multivector fields is the definition of criticality. In the multivector field case, the types of multivectors are distinguished as follows:

- A multivector V is called critical, if the relative homology $H_*(\operatorname{cl} V, \operatorname{mo} V)$ is not trivial, i.e., at least one Betti number is nonzero.
- A multivector V is called regular, if the relative homology $H_*(\operatorname{cl} V, \operatorname{mo} V)$ is trivial, i.e., it vanishes in all dimensions.

One can show that in the case of a Forman vector field, critical cells are always critical in the above sense, while arrows are always regular. In our above example `mvflogo`, all three multivectors which are not singletons are regular. For example, the following computation shows that the cell `ABC` is a critical cell:

```
cl1, mo1 = lefschetz_clomo_pair(sclogo, ["ABC"])
relative_homology(sclogo, cl1, mo1)
```

```
3-element Vector{Int64}:
 0
 0
 1
```

The first command creates the closure-mouth pair associated with the cell `ABC`, i.e., the variable `cl1` is the closed triangle, while `mo1` is the closed boundary of the triangle. The next command determines the relative homology. Notice that this employs another method under the name `relative_homology`, in contrast to the one used earlier in this tutorial. For more details, see Homology Functions.

Alternatively, since every multivector is locally closed, one can also use the function `conley_index` for the same computation:

```
conley_index(sclogo, ["ABC"])
```

```
3-element Vector{Int64}:
 0
 0
 1
```

Similarly, the next sequence of commands verifies that the third nontrivial multivector `mvflogo[3]` is indeed a regular multivector:

```
cl2, mo2 = lefschetz_clomo_pair(sclogo, mvflogo[3])
relative_homology(sclogo, cl2, mo2)
```

```
3-element Vector{Int64}:
 0
 0
 0
```

The global dynamics can again be determined using the function `connection_matrix`:

```
cmlogo = connection_matrix(sclogo, mvflogo)
cmlogo.morse
```

```
3-element Vector{Vector{String}}:
 ["D"]
 ["A", "B", "C", "AB", "AC", "BC", "BD", "CD", "BCD"]
 ["ABC"]
```

As it turns out, our logo gives rise to three Morse sets, which in fact partition the simplicial complex. Their Conley indices are given by

```
cmlogo.conley
```

```
3-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 1, 0]
 [0, 0, 1]
```

Finally, the connection matrix has the form

```
full_from_sparse(cmlogo.matrix)
```

```
3×3 Matrix{Int64}:
 0  0  0
 0  0  1
 0  0  0
```

While this command only shows the connection matrix itself, the following method provides also information about the column and row labels

```
sparse_show(cmlogo)
```

```
   ┆   D  AC ABC
---┆------------
  D┆   .   .   .
 AC┆   .   .   1
ABC┆   .   .   .
```

Notice that in this example, only the connection between the Morse set ABC and the large index 1 Morse set comprising almost all of the simplicial complex can be detected algebraically. In fact, there are two connections between the large Morse set and the stable equilibrium D, and they cancel algebraically.

2.7 Analyzing Planar Vector Fields

Our third example of the tutorial briefly indicates how ConleyDynamics.jl can be used to analyze the global dynamics of certain planar ordinary differential equations. For this, consider the planar system given by

$$\begin{aligned} \dot{x}_1 &= x_1\left(1 - x_1^2 - 3x_2^2\right) \\ \dot{x}_2 &= x_2\left(1 - 3x_1^2 - x_2^2\right) \end{aligned}$$

The right-hand side of this vector field can be implemented using the Julia function

```
function planarvf(x::Vector{Float64})
    #
    # Sample planar vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end
```

```
planarvf (generic function with 1 method)
```

To analyze the global dynamics of this vector field, we first create a Delaunay triangulation of the square $[-3/2, 3/2]^2$ using the commands

```
lc, coords = create_simplicial_delaunay(300, 300, 10, 30);
coordsN = convert_planar_coordinates(coords,[-1.5,-1.5], [1.5,1.5]);
cx = [c[1] for c in coordsN];
(minimum(cx), maximum(cx))
```

```
(-1.5, 1.5)
```

The first command generates the triangulation in a square box with side length 300, while trying to keep a minimum distance of about 10 between vertices. Once this has been accomplished, the second command transforms the coordinates to the desired square domain. As the last two commands show, the resulting x-coordinates do indeed lie between -3/2 and 3/2.

Next we can create a multivector field which describes the flow behavior through the edges of the triangulation. Basically, for each edge which is traversed in only one direction, the corresponding multivector respects this unidirectionality, while non-transverse edges lead to multivectors which allow for flow in both directions between the adjacent triangles. This is achieved with the commands

```
mvf = create_planar_mvf(lc, coordsN, planarvf);
mvf[1:3]
```

```
3-element Vector{Vector{Int64}}:
 [1, 610, 612, 2387]
 [2, 616, 617, 2392]
 [3, 622, 624, 2397]
```

The first command generates the multivector field, while the second one merely displays the first three resulting multivectors. Note that if the discretization is too coarse, this might lead to large multivectors that cannot resolve the underlying dynamics. In our case, we can analyze the global dynamics of the created multivector field using the commands

```
cm = connection_matrix(lc, mvf);
cm.conley
```

```
9-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [0, 1, 0]
 [1, 0, 0]
 [0, 1, 0]
 [1, 0, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 0, 1]
```

As the output shows, this planar system has nine isolated invariant sets:

- One unstable equilibrium of index 2,
- four unstable equilibria of index 1,
- and four stable equilibria.

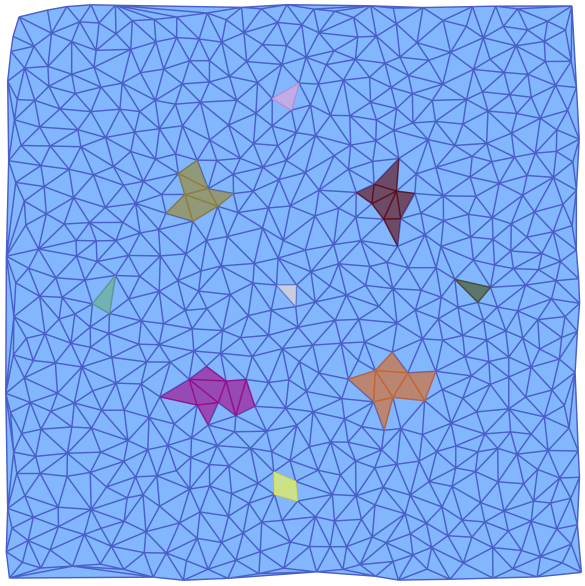


Figure 2.4: Morse sets of a planar vector field

More precisely, this computation does not in fact establish the existence of these equilibria, but of corresponding isolated invariant sets which have the respective Conley indices. The connection matrix is given by

```
full_from_sparse(cm.matrix)
```

```
9×9 Matrix{Int64}:
 0  0  1  0  0  0  1  0  0
 0  0  1  0  1  0  0  0  0
 0  0  0  0  0  0  0  0  1
 0  0  0  0  1  0  0  1  0
 0  0  0  0  0  0  0  0  1
 0  0  0  0  0  0  1  1  0
 0  0  0  0  0  0  0  0  1
 0  0  0  0  0  0  0  0  1
 0  0  0  0  0  0  0  0  0
```

It shows that there are twelve connecting orbits that are forced by the algebraic topology. Finally, we can visualize the Morse sets using the command

```
fname = "tutorialplanar.pdf"
plot_planar_simplicial_morse(lc, coordsN, fname, cm.morse, pv=true)
```

Deprecation warning

The above-described method of manually supplying a coordinate vector for a Lefschetz complex for the purpose of multivector field creation or plotting will eventually be removed from the package. While this approach is direct and simple, it leads to unexpected problems if one wants to work with subcomplexes of a Lefschetz complex that are no longer closed. To remedy this, starting with version `v0.7.5` of ConleyDynamics.jl an alternative workflow is introduced, based on the new complex type `EuclideanComplex`. For the time being, both the old and the new workflow are supported, but the old one will be removed in a future release.

In the above example, the abstract simplicial complex is embedded in the plane for the purpose of creating a multivector field from a classical vector field. This embedding is achieved by not only providing a Lefschetz complex `lc`, but also a coordinate vector `coords` for the vertex coordinates.

Starting with version `v0.7.5` of ConleyDynamics.jl there is another workflow possibility, which will be briefly described in the following. The creation of the vector field proceeds as before:

```
function planarvf(x::Vector{Float64})
    #
    # Sample planar vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end
```

```
planarvf (generic function with 1 method)
```

Instead of creating a separate Lefschetz complex and associated coordinates, the following commands generate a `EuclideanComplex`:

```
ec = create_simplicial_delaunay(300, 300, 10, 30, euclidean=true)
ec = rescale_coords(ec, -1.5, 1.5)
```

```
EuclideanComplex: struct with the following fields:
  labels, indices, dimensions, coords
  dim:      2
  ncells:   3551
  boundary: field GF(2), sparsity 0.999
```

As the output shows, a Euclidean complex has the same fields as a Lefschetz complex, but adds to that a field that contains coordinates for every cell in the complex. Moreover, the function `rescale_coords` can be used to transform the values chosen by the Delaunay function to the square $[-3/2, 3/2]^2$.

Since the variable `ec` contains both the Lefschetz complex and the coordinate information, the multivector field which describes the flow behavior through the edges of the triangulation, as well as its connection matrix, can be computed using the commands

```
mvf = create_planar_mvf(ec, planarvf);
cm  = connection_matrix(ec, mvf)
```

```
ConleyMorseCM{Int64}: struct with the following fields:
  columns, poset, labels, morse, conley, complex
  matrix: 9x9-dimensional matrix with sparsity 0.852
  inspect the connection matrix with sparse_show(cm)
```

The produced connection matrix has the same form as the one above, up to a potential permutation of the rows and columns:

```
cm.matrix
```

```
9x9-dimensional SparseMatrix{Int64}, sparsity=0.852:
 . . 1 . . . 1 . .
 . . 1 . 1 . . . .
 . . . . . . . . 1
 . . . . 1 . . 1 .
 . . . . . . . . 1
 . . . . . . 1 1 .
 . . . . . . . . 1
 . . . . . . . . 1
 . . . . . . . . .
```

Finally, visualizing the Morse sets is achiebd using the command

```
fname = "tutorialplanar.pdf"
plot_planar_simplicial_morse(ec, fname, cm.morse, pv=true)
```

2.8 Analyzing Spatial Vector Fields

The last example of the tutorial shows that the same workflow applies to vector fields in three dimensions. Consider the system

$$\begin{aligned} \dot{x}_1 &= x_1\left(1 - x_1^2\right) \\ \dot{x}_2 &= -x_2 \\ \dot{x}_3 &= -x_3 \end{aligned}$$

which has two stable equilibria at $(\pm 1, 0, 0)$ and an equilibrium at the origin with a one-dimensional unstable manifold. Its right-hand side is implemented by the Julia function

```
function spatialvf(x::Vector{Float64})
    #
    # Sample spatial vector field with nontrivial Morse decomposition
    #
    x1, x2, x3 = x
    y1 = x1 * (1.0 - x1*x1)
    y2 = -x2
    y3 = -x3
    return [y1, y2, y3]
end
```

Instead of a planar Delaunay triangulation, we now need a tetrahedral mesh of the box $[-3/2, 3/2] \times [-1, 1] \times [-1, 1]$, which is a trapping region for the flow. Such meshes can be created with the mesh generator TetGen, which becomes available in ConleyDynamics.jl as soon as the package `TetGen.jl` is loaded. This package is not installed automatically, and it has to be added once with `import Pkg; Pkg.add("TetGen")`, see Loading the Required Packages for details:

```
using TetGen
using ConleyDynamics

dom = tetgen_domain_box([-1.5,-1,-1], [1.5,1,1])
res = tetgen_mesh(dom; maxvolume=0.01)
ec  = tetgen_to_simplicial(res)
lefschetz_cell_count(ec)
# 4-element Vector{Int64}:
#   532
#  3051
#  4727
#  2207
```

The first command describes the box domain, the second one creates a quality tetrahedral mesh whose tetrahedra have volume at most about `0.01`, and the third one converts this mesh into an `EuclideanComplex`. The resulting simplicial complex consists of 532 vertices, 3051 edges, 4727 triangles, and 2207 tetrahedra. The multivector field and its connection matrix are then computed exactly as in the planar case, except that the function `create_spatial_mvf` replaces `create_planar_mvf`:

```
mvf = create_spatial_mvf(ec, spatialvf);
cm  = connection_matrix(ec, mvf);
cm.conley
# 3-element Vector{Vector{Int64}}:
#  [1, 0, 0, 0]
#  [1, 0, 0, 0]
#  [0, 1, 0, 0]
full_from_sparse(cm.matrix)
# 3×3 Matrix{Int64}:
#  0  0  1
#  0  0  1
#  0  0  0
```

The computation finds two Morse sets with the Conley index of a stable equilibrium, and one Morse set with the Conley index of an equilibrium with a one-dimensional unstable manifold. The connection matrix shows

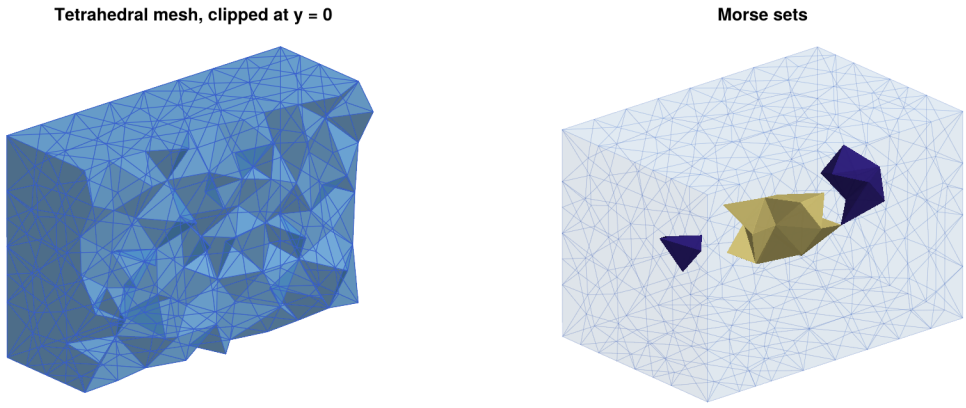


Figure 2.5: A tetrahedral mesh and the Morse sets of a spatial vector field

that the latter is connected to both stable equilibria. Three-dimensional complexes can be visualized with the Makie plotting packages. For example, the commands

```
using GLMakie

fig1 = plot_simplicial3d(ec; clip=([0,-1,0], 0.0), alpha=0.6)
fig2 = plot_simplicial3d_morse(ec, cm.morse; ci=true)
```

show the mesh, clipped at the plane $y = 0$ to reveal its interior, and the Morse sets, colored by their Conley indices. With `GLMakie`, a displayed figure appears in an interactive window, in which it can be rotated with the mouse. By default, this window shows the most recently displayed figure, and `display(fig1)` brings back the first one. Static versions of both plots are shown in the figure of the clipped tetrahedral mesh and the three Morse sets.

Much more on tetrahedral meshes, including embedded constraint surfaces, holes, local refinement, and intersecting geometry, can be found in the chapter Three-Dimensional Meshes. The plotting functions are described in the section Three-Dimensional Plotting with Makie.

2.9 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[BKMW20] B. Batko, T. Kaczynski, M. Mrozek and T. Wanner. Linking combinatorial and classical dynamics: Conley index and Morse decompositions. Foundations of Computational Mathematics **20**, 967–1012 (2020).

[Con78] C. Conley. Isolated Invariant Sets and the Morse Index (American Mathematical Society, Providence, R.I., 1978).

[For98a] R. Forman. Combinatorial vector fields and dynamical systems. Mathematische Zeitschrift **228**, 629–681 (1998).

[For98b] R. Forman. Morse theory for cell complexes. Advances in Mathematics **134**, 90–145 (1998).

[KMW16] T. Kaczynski, M. Mrozek and T. Wanner. Towards a formal tie between combinatorial and classical vector field dynamics. Journal of Computational Dynamics **3**, 17–50 (2016).

[Lef42] S. Lefschetz. Algebraic Topology. Vol. 27 of American Mathematical Society Colloquium Publications (American Mathematical Society, New York, 1942).

[LKMW23] M. Lipinski, J. Kubica, M. Mrozek and T. Wanner. Conley-Morse-Forman theory for generalized combinatorial multivector fields on finite topological spaces. Journal of Applied and Computational Topology **7**, 139–184 (2023).

[MW21] M. Mrozek and T. Wanner. Creating semiflows on simplicial complexes from combinatorial vector fields. Journal of Differential Equations **304**, 375–434 (2021).

[MW25] M. Mrozek and T. Wanner. Connection Matrices in Combinatorial Topological Dynamics. SpringerBriefs in Mathematics (Springer-Verlag, Cham, 2025).

[Wan25] T. Wanner. ConleyDynamics.jl: A Julia package for multivector dynamics on Lefschetz complexes. Journal of Open Source Software **10**, 8085 (2025).

Chapter 3

Lefschetz Complexes

The fundamental structure underlying the functionality of ConleyDynamics.jl is a Lefschetz complex. It provides us with the basic model of phase space for combinatorial topological dynamics. In view of the combinatorial, and therefore discrete, character of the dynamical behavior, a Lefschetz complex is not a typical phase space in the sense of classical dynamics. While the latter one is usually a Euclidean space, a Lefschetz complex is basically a combinatorial model of it. In the following, we provide its precise mathematical definition, and explain how it can be created and modified within the package. We also discuss two important special cases, namely simplicial complexes and cubical complexes.

3.1 Basic Lefschetz Terminology

The original definition of a Lefschetz complex can be found in [Lef42], where it was simply referred to as a complex.

Definition: Lefschetz complex

Let F denote an arbitrary field. Then a pair (X, κ) is called a Lefschetz complex over F if $X = (X_k)_{k\in\mathbb{N}_0}$ is a finite set with $\mathbb{N}_0$-gradation, and $\kappa : X \times X \to F$ is a mapping such that

$$\kappa(x, y) \neq 0 \quad \text{implies} \quad x \in X_k \quad \text{and} \quad y \in X_{k-1},$$

and such that for any $x, z \in X$ one has

$$\sum_{y\in X} \kappa(x, y)\kappa(y, z) = 0 \, .$$

The elements of X are referred to as cells, the value $\kappa(x, y) \in F$ is called the incidence coefficient of the cells x and y, and the map κ is the incidence coefficient map. In addition, one defines the dimension of a cell $x \in X_k$ as the integer k, and denotes it by $k = \dim x$. Whenever the incidence coefficient map is clear from context, we often just refer to X as the Lefschetz complex.

At first glance the above definition can seem daunting. However, it is based on a straightforward geometric idea. A Lefschetz complex is a structure that is built from elementary building blocks called cells. Each cell has a dimension associated with it, and it is topologically an open ball of this dimension. Thus, cells of dimension

zero are points, also called vertices. Cells of dimension one are open curve segments, which we call edges, and two-dimensional cells are called faces and take the form of open two-dimensional membranes.

The incidence coefficient map encodes how these cells are glued together to form the Lefschetz complex X. In order to shed more light on this, consider the boundary map ∂ which is defined on cells via

$$\partial x = \sum_{y \in X} \kappa(x, y) y \ .$$

This map sends a cell x of dimension k to a specific linear combination of cells of dimension $k-1$, called the boundary of x. By using ideas from linear algebra, the boundary map can be extended to map a general linear combination of k-dimensional cells to the corresponding linear combination of the separate boundaries. For example, if one chooses the field $F = \mathbb{Q}$ of rationals, one has $\partial(x_1 - 2x_2) = \partial x_1 - 2\partial x_2$. Notice that using this extended definition of the boundary map, one can rewrite the summation condition in the definition of a Lefschetz complex in the equivalent form

$$\partial(\partial x) = 0 \quad \text{for all cells} \quad x \in X \ .$$

In other words, the boundary of any cell is itself boundaryless.

With the help of the boundary map, one can often infer the overall geometric structure of a Lefschetz complex X. For this, think of a Lefschetz complex as being build from the ground up in the following way. First, start by putting down all vertices of X at different locations in some ambient space. Since the boundary of each one-dimensional cell is made up of a linear combination of vertices, one can then add a curve segment for each one-dimensional cell, which connects the vertices in its boundary. Note that in the general version of a Lefschetz complex it is possible that an edge has only one vertex in its boundary, or maybe even none, and in these cases the edge is either only connected to the one boundary vertex, or it is an open curve segment connected to no vertex at all, respectively. Continue in this fashion to add two-dimensional faces to fill in the space between the edges in its boundary, and so on for higher dimensions. Needless to say, in the case of a general complicated Lefschetz complex this procedure is of limited use, since the boundary of a cell can be an arbitrary linear combination of cells, with coefficients that can be any nonzero numbers in the field F. Yet, in many simple cases the above intuition is sufficient.

In addition to the Lefschetz complex definition, there are a handful of other concepts which will be important for our discussion of Lefschetz complexes. Specifically, the following notions are important:

- A facet of a cell $x \in X$ is any cell y which satisfies $\kappa(x, y) \neq 0$.
- One can define a partial order on the cells of X by letting $x \leq y$ if and only if for some integer $n \in \mathbb{N}$ there exist cells $x = x_1, \ldots, x_n = y$ such that x_k is a facet of x_{k+1} for all $k = 1, \ldots, n-1$. It is not difficult to show that this defines a partial order on X, i.e., this relation is reflexive, antisymmetric, and transitive. We call this partial order the face relation. Moreover, if $x \leq y$ then x is called a face of y.
- A subset $C \subset X$ of a Lefschetz complex is called closed, if for every $x \in C$ all the faces of the cell x are also contained in the subset C.
- The closure of a subset $C \subset X$ is the collection of all faces of all cells in C, and it is denoted by $\operatorname{cl} C$. Thus, a subset of a Lefschetz complex is closed if and only if it equals its closure.
- A subset $S \subset X$ is called locally closed, if its mouth $\operatorname{mo} S = \operatorname{cl} S \setminus S$ is closed. Note that every closed set is automatically locally closed, but the reverse implication is usually false.

While the first two points merely introduce notation for describing the combinatorial boundary of cells, the remaining three points establish important topological concepts. In fact, the above definition of closedness defines a topology on the Lefschetz complex X, which is the so-called Alexandrov topology from [Ale37]. As usual in the field of topology, a subset of a Lefschetz complex will be called open, if and only if its complement is closed.

We would like to point out that while the concept of local closedness is rarely considered in standard topology courses, it is of utmost important for the study of combinatorial topological dynamics. For the moment, we just mention the following result:

Theorem: Lefschetz subcomplexes

Let X be a Lefschetz complex over a field F, and let $\kappa : X \times X \to F$ denote its incidence coefficient map. Then a subset $S \subset X$ is again a Lefschetz complex, with respect to the restriction of κ to $S \times S$, if the subset S is locally closed.

This result goes back to [MB09, Theorem 3.1]. In other words, in the category of Lefschetz complexes local closedness arises naturally. Due to its importance, we also mention the following two equivalent formulations:

- A subset $S \subset X$ is locally closed, if and only if it is the difference of two closed subsets of X.
- A subset $S \subset X$ is locally closed, if and only if it is an interval with respect to the face relation on X, i.e., whenever we have three cells with $S \ni x \leq y \leq z \in S$, then one has to have $y \in S$ as well.

The proof of these characterizations can be found in [MW25, Proposition 3.2.1] and [LKMW23, Proposition 3.10], respectively.

Lefschetz complexes are a very general mathematical concept, and they can be rather confusing at first sight. Nevertheless, they do encompass other complex types, which are more geometric in nature. As we already saw in the tutorial, every simplicial complex is automatically a Lefschetz complex, and we will further elaborate on this connection below. In addition, we will also demonstrate that cubical complexes are Lefschetz complexes. More general, any regular CW complex is a Lefschetz complex as well. For more details on this, we refer to the definition in [Mas91] and the discussion in [DKMW11].

3.2 Lefschetz Complex Data Structure

For the efficient and easy manipulation of Lefschetz complexes in ConleyDynamics.jl we make use of a specific composite data type:

`ConleyDynamics.LefschetzComplex` – Type.

```
LefschetzComplex
LefschetzComplex(labels, dimensions, boundary; validate=true)
```

Collect the Lefschetz complex information in a struct.

The struct is created via the following fields:

- `labels::Vector{String}`: Vector of labels associated with cell indices
- `dimensions::Vector{Int}`: Vector cell dimensions
- `boundary::SparseMatrix`: Boundary matrix, columns give the cell boundaries

It is expected that the dimensions are given in increasing order, and that the square of the boundary matrix is zero. Otherwise, exceptions are raised. In addition, the following fields are created during initialization:

- `ncells::Int`: Number of cells
- `dim::Int`: Dimension of the complex
- `indices::Dict{String,Int}`: Dictionary for finding cell index from label

The coefficient field is specified by the boundary matrix.

The optional keyword argument `validate` controls whether the constructor verifies that the boundary matrix squares to zero. By default this check is enabled. Internal library functions that are guaranteed to produce a valid complex by construction (simplicial, cubical, restriction, permutation, basis-change, etc.) pass `validate=false` explicitly to suppress the check, since for large complexes the matrix multiplication is expensive. Users constructing a `LefschetzComplex` manually can also call `validate_lefschetz_complex` to check an existing complex at any point.

source

The fields of this struct relate to the mathematical definition of a Lefschetz complex X in the following way:

- Internally, every cell of the Lefschetz complex is represented by an integer between 1 and the total number of cells. However, in order to make it easier to interpret the results of computations, each cell in a Lefschetz complex has to also be given a label. These labels are contained in the field `labels::Vector{String}`, where `labels[k]` gives the label of cell k.
- The vector `dimensions` is a `Vector{Int}` and collects the dimensions of the cells. In other words, the cell which is indexed by the integer k has dimension `dimensions[k]`. It is expected that the dimension vector is increasing, and the constructor method will verify this. Otherwise, an error is triggered.
- The incidence coefficient map κ is encoded in the sparse matrix `boundary`. This matrix is a square matrix with `ncells` rows and columns. The k-th column contains the incidence coefficients $\kappa(k, \cdot)$ in the sense that the entry in row m and column k equals the value $\kappa(k, m)$. Since for most Lefschetz complexes the majority of the incidence coefficients is zero, the matrix is represented using the sparse format `SparseMatrix`, which is described in more detail in Sparse Matrices. An exception is raised if the square of the boundary matrix is not zero.

When creating a Lefschetz complex, only the above three items have to be specified, as they define a unique Lefschetz complex X. In other words, a Lefschetz complex is generally created via the command

```
lc = LefschetzComplex(labels, dimensions, boundary)
```

During the construction of the Julia object, additional fields are initialized which simplify working with a Lefschetz complex:

- The integer `ncells` gives the total number of cells in X. Internally, these cells are numbered by integers ranging from 1 to `ncells`.
- The integer `dim` describes the overall dimension of the Lefschetz complex, which is the largest dimension of a cell.
- In order to easily determine the integer index for a cell with a specific label, the field `indices` contains a dictionary of type `Dict{String,Int}` which maps labels to indices. For example, if a cell has the label `"124.010"`, then the associated integer index is given by `indices["124.010"]`.

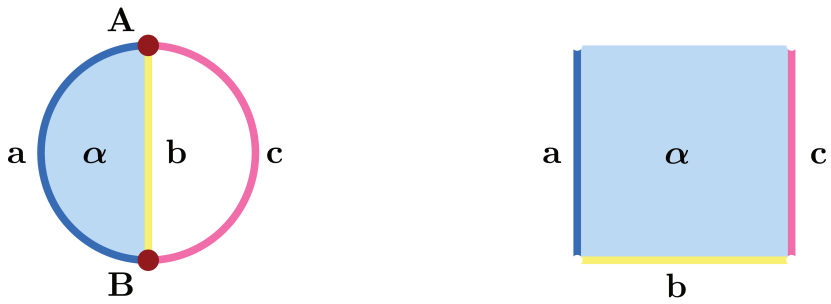


Figure 3.1: Two sample Lefschetz complexes

As mentioned above, note however that an object of type `LefschetzComplex` is created by passing only the first three the field items in the order given in `LefschetzComplex`. Consider for example the Lefschetz complex from Figure 2.4 in [MW25], see also the left complex in the figure with two sample Lefschetz complexes, which shows a circle with a vertical chord, where only the left half of the disk is filled. This complex consists of six cells with labels `A`, `B`, `a`, `b`, `c`, and `alpha`, and we initialize the vector of labels, the cell index dictionary, and the cell dimensions via the commands

```
ncL = 6
labelsL = Vector{String}(["A","B","a","b","c","alpha"])
cdimsL  = [0, 0, 1, 1, 1, 2]
```

The boundary matrix can then be defined using

```
bndmatrixL = zeros(Int, ncL, ncL)
bndmatrixL[[1,2],3] = [1; 1]     # a
bndmatrixL[[1,2],4] = [1; 1]     # b
bndmatrixL[[1,2],5] = [1; 1]     # c
bndmatrixL[[3,4],6] = [1; 1]     # alpha
bndsparseL = sparse_from_full(bndmatrixL, p=2)
```

Notice that we first create the matrix as a regular integer matrix, and then use the function `sparse_from_full` to turn it into sparse format over the field $GF(2)$ with characteristic `p = 2`. This is the most convenient method for small boundary matrices, yet for larger ones it is better to use the function `sparse_from_lists`. Finally, the Lefschetz complex is created using

```
lcL = LefschetzComplex(labelsL, cdimsL, bndsparseL)
```

Lefschetz complexes do not always have to contain cells of all dimensions. For example, the Lefschetz complex shown on the right side of the figure with the two sample Lefschetz complexes, a square with only three of its edges, has no vertices, and it can be created using the commands

```
ncR = 4
labelsR  = Vector{String}(["a","b","c","alpha"])
cdimsR   = [1, 1, 1, 2]
bndmatrixR = zeros(Int, ncR, ncR)
bndmatrixR[[1,2,3],4] = [1; 1; 1]     # alpha
bndsparseR = sparse_from_full(bndmatrixR, p=2)
lcR = LefschetzComplex(labelsR, cdimsR, bndsparseR)
```

While Lefschetz complexes can always be created in ConleyDynamics.jl in this direct way, it is often more convenient to make use of special types, such as simplicial and cubical complexes, and then restrict the complex to a locally closed set using the function `lefschetz_subcomplex`. As an alternative, if one is interested in a fairly small Lefschetz complex over the field $GF(2)$, then the following special function can be used:

- `create_lefschetz_gf2` creates a Lefschetz complex over the two-element field $GF(2)$ by specifying its essential cells and boundaries. The input argument `defcellbnd` of the function has to be a vector of vectors. Each entry `defcellbnd[k]` then has to be of one of the following two forms:
  - `[String, Int, String, String, ...]`: The first `String` contains the label for the cell k, followed by its dimension in the second entry. The remaining entries are for the labels of the cells which make up the boundary.
  - `[String, Int]`: This shorther form is for cells with empty boundary. The first entry denotes the cell label, and the second its dimension.

  The cells of the resulting Lefschetz complex correspond to the union of all occurring labels. Cell labels that only occur in the boundary specification are assumed to have empty boundary, and they do not have to be specified separately in the second form above. However, if their boundary is not empty, they have to be listed via the above first form as well.

Using this function, our earlier Lefschetz complex `lcL` can be created using the commands

```
defcellbndL = [["a",1,"A","B"], ["b",1,"A","B"], ["c",1,"A","B"], ["alpha",2,"a","b"]]
lcL = create_lefschetz_gf2(defcellbndL)
```

while the Lefschetz complex `lcR` is defined via

```
defcellbndR = [["alpha",2,"a","b","c"], ["a",1], ["b",1], ["c",1]]
lcR = create_lefschetz_gf2(defcellbndR)
```

3.3 Simplicial Complexes

One of the earliest types of complexes that have been studied in topology are simplicial complexes. As already mentioned in the tutorial, an abstract simplicial complex X is a finite collection of finite sets, called simplices, which is closed under taking subsets. Each simplex σ has a dimension $\dim \sigma$, which is one less than the number of its elements.

In order to see why every simplicial complex is automatically a Lefschetz complex, we need to be able to define the incidence coefficient map κ. For this, we make use of some notions from [Mun84]. Let X_0 denote the collection of all vertices of the simplicial complex X. Then we use the notation

$$\sigma = [v_0, v_1, \ldots, v_d] \quad \text{with} \quad v_k \in X_0$$

to describe a d-dimensional simplex. Note that even though every simplex in X is just the set of its vertices, in the above representation we pick an order of the vertices, called an orientation of the simplex. This orientation can be chosen arbitrarily, and there are two equivalence classes of orientations. To get from one orientation to the other, one just has to exchange two vertices, and we write

$$[\ldots, v_i, \ldots, v_j, \ldots] \;=\; -[\ldots, v_j, \ldots, v_i, \ldots] \;.$$

For more complicated reorderings, one has to represent the corresponding vertex permutation as a sequence of such exchanges. Using these oriented simplices we can define the boundary operator

$$\partial\sigma \;=\; \partial\,[v_0, \ldots, v_d] \;=\; \sum_{i=0}^{d} (-1)^i\,[v_0, \ldots, \hat{v}_i, \ldots, v_d] \;,$$

where the notation $\hat{v}_i$ means that in the simplex behind the summation sign on the right-hand side the vertex v_i is omitted. For example, for a two-dimensional simplex one obtains

$$\partial\,[v_0, v_1, v_2] \;=\; [v_1, v_2] - [v_0, v_2] + [v_0, v_1] \;.$$

Thus, if one chooses a total order of all the vertices in the simplicial complex, and orients the individual simplices in such a way that their vertices are arranged using this overall order, then the incidence coefficient map is given by

$$\kappa\left([v_0, \ldots, v_i, \ldots, v_d]\,,\; [v_0, \ldots, \hat{v}_i, \ldots, v_d]\right) \;=\; (-1)^i \;.$$

If some or all of the simplices are represented by different orientations, one simply has to multiply the value $(-1)^i$ by the sign of a suitable vertex permutation. In either case, one can show that the so-defined map κ does indeed satisfy the definition of a Lefschetz complex. For more details, see [Mun84, Lemma 5.3].

In ConleyDynamics.jl there are three basic commands for defining a simplicial complex:

- `create_simplicial_complex` is the most general method, and it expects two input arguments. The first is usually called `labels`, and it has to have the data type `Vector{String}`. This vector lists the labels for each vertex. It is important that all of these labels have exactly the same number of characters. The second argument is usually called `simplices`, and it lists as many simplices as necessary for defining the underlying simplicial complex. This means that in practice one only needs to include the simplices which are not faces of higher-dimensional ones, see also the example below. The variable `simplices` can either be of type `Vector{Vector{String}}` or `Vector{Vector{Int}}`, depending on whether the vertices are identified via their labels or integer indices, respectively. Finally, the optional parameter `p` can be used to specify the underlying field for the boundary matrix. If `p` is a prime, then $F = GF(p)$, while for `p = 0` the function uses $F = \mathbb{Q}$. If the argument `p` is ommitted, the function defaults to `p = 2`.
- `create_simplicial_rectangle` expects two integer arguments `nx` and `ny`, and then creates a triangulation of the square $[0, nx] \times [0, ny]$ by subdividing every unit square into four triangles which meet at the center of the square. As before, the optional parameter `p` specifies the underlying field.
- `create_simplicial_delaunay` creates a planar Delaunay triangulation inside a planar rectangle. The function selects a random sample of points inside the box, while either trying to maintain a minimum distance between the points, or just using a prespecified number of points. More details on these two options can be found in the documentation for the function.

To illustrate the first of these functions, consider the commands

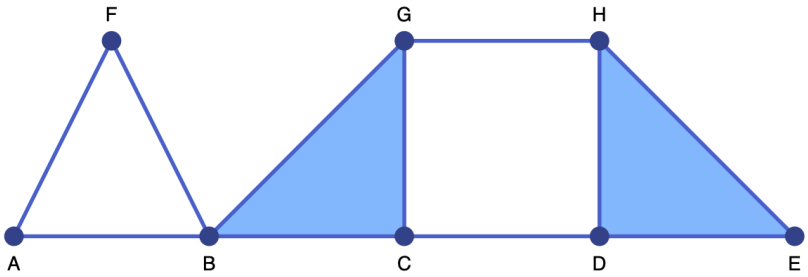


Figure 3.2: First sample simplicial complex

```
labels = ["A","B","C","D","E","F","G","H"]
simplices = [["A","B"],["A","F"],["B","F"],["B","C","G"],["D","E","H"],["C","D"],["G","H"]]
sc = create_simplicial_complex(labels,simplices)
```

These create the simplicial complex `sc`, in the form of a Lefschetz complex. Note that the above commands only specify the labels for the vertices. The labels for simplices of dimension at least one are automatically generated by concatenating the labels for their vertices, sorted in lexicographic order. This can be seen in the following Julia output:

```
julia> sc.labels[end-4:end]
5-element Vector{String}:
 "DH"
 "EH"
 "GH"
 "BCG"
 "DEH"
```

The simplicial complex `sc` can be visualized using the commands

```
coords = [[0,0],[2,0],[4,0],[6,0],[8,0],[1,2],[4,2],[6,2]]
ldir   = [3,3,3,3,3,1,1,1]
fname  = "lefschetzex2.pdf"
plot_planar_simplicial(sc,coords,fname,labeldir=ldir,labeldis=10,hfac=2,vfac=1.5,sfac=50)
```

Similarly, the commands

```
sc2, coords2 = create_simplicial_rectangle(5,2)
fname2 = "lefschetzex3.pdf"
plot_planar_simplicial(sc2,coords2,fname2,hfac=2.0,vfac=1.2,sfac=75)
```

define and illustrate a second simplicial complex, which triangularizes a rectangle in the plane.

For a demonstration of the Delaunay triangulation approach, please see Analyzing Planar Vector Fields.

In addition to the above methods, ConleyDynamics.jl also provides a few special simplicial complexes for illustration purposes:

- `simplicial_torus` constructs a triangulation of the two-dimensional torus.

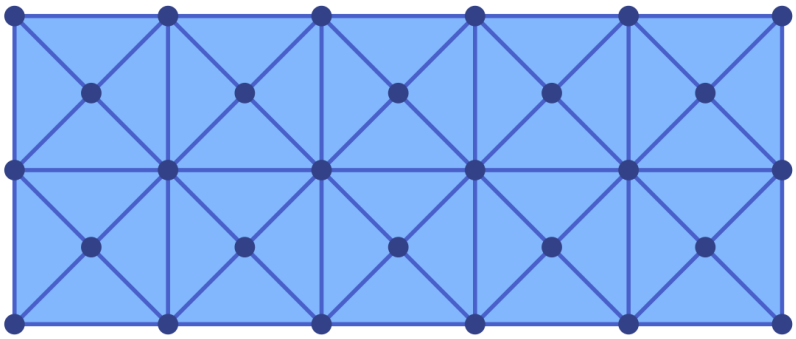


Figure 3.3: Second sample simplicial complex

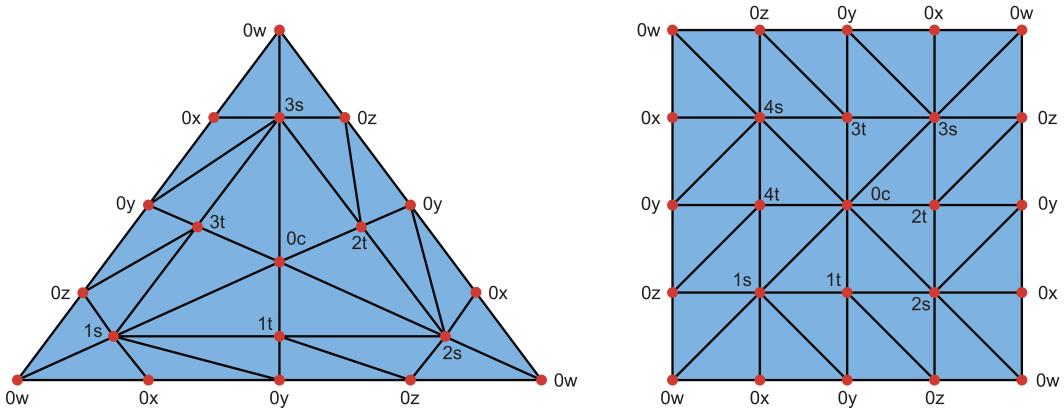


Figure 3.4: Simplicial torsion space

- `simplicial_klein_bottle` returns a triangulation of the two-dimensional Klein bottle.
- `simplicial_projective_plane` constructs a triangulation of the projective plane.
- `simplicial_torsion_space` returns a simplicial complex K with the integer homology groups $H_0(K) \cong \mathbb{Z}$, as well as $H_1(K) \cong \mathbb{Z}_n$ and $H_2(K) \cong 0$. In other words, this simplicial complex has nontrivial torsion in dimension 1. For $n = 3$ and $n = 4$ these triangulations are shown in the figure of the simplicial torsion spaces. Notice that points with the same label have to be identified. This vertex naming scheme is the same as is used in the function.

These examples can be used to illustrate homology over different fields.

3.4 Cubical Complexes

The second important special case of a Lefschetz complex is called cubical complex, and it has been discussed in detail in [KMM04]. In the following, we only present the definitions that are essential for our purposes.

Loosely speaking, a cubical complex is a collection of cubes of varying dimensions in some Euclidean space $\mathbb{R}^d$. More precisely, we say that an interval $I \subset \mathbb{R}$ is an elementary interval if it is of the form

$$I = [\ell, \ell + 1] \quad \text{or} \quad I = [\ell, \ell] \quad \text{for some integer} \quad \ell \in \mathbb{Z}\,.$$

If the elementary interval I consists of only one point, then it is called degenerate, and it is nondegenerate if it is of length one. Elementary intervals are the building blocks for the cubes in a cubical complex. For a complex in $\mathbb{R}^d$, an elementary cube Q is of the form

$$Q \;=\; I_1 \times I_2 \times \ldots \times I_d \;\subset\; \mathbb{R}^d \,,$$

where $I_1, \ldots, I_d$ are elementary intervals. The dimension $\dim Q$ of an elementary cube is given by the number of nondegenerate intervals in its representation. For example, the cube $Q = [0,0]\times[1,1]$ is a zero-dimensional elementary cube in $\mathbb{R}^2$ which contains only the point $(0,1)$, while the elementary cube $R = [2,2] \times [3,4]$ is one-dimensional, and consists of the closed vertical line segment between the points $(2,3)$ and $(2,4)$.

After these preparations, the definition of a cubical complex is now straightforward. A cubical complex X in $\mathbb{R}^d$ is a finite collection of elementary cubes in $\mathbb{R}^d$ which is closed under the inclusion of elementary subcubes. More precisely, if $Q \in X$ is an elementary cube in the cubical complex, and if $R \subset Q$ is any elementary cube contained in Q, then one also has $R \in X$.

The definition of a cubical complex is reminiscent of that of a simplicial complex. It is therefore not surprising that also in the cubical case one can describe the incidence coefficient map κ explicitly, and thus recognize a cubical complex as a Lefschetz complex. For this, we need more notation.

Let $Q = I_1 \times I_2 \times \ldots \times I_d$ denote an elementary cube, and let the nondegenerate elementary intervals in this decomposition be given by $I_{i_1}, \ldots, I_{i_n}$, where $I_{i_j} = [k_j, k_j + 1]$ and $j = 1, \ldots, n = \dim Q$. For every index j, we further define the two $(n-1)$-dimensional elementary cubes

$$\begin{aligned} Q_j^- &= I_1 \times \ldots \times I_{i_j - 1} \times [k_j, k_j] \times I_{i_j+1} \times \ldots \times I_d \,, \\ Q_j^+ &= I_1 \times \ldots \times I_{i_j - 1} \times [k_j + 1, k_j + 1] \times I_{i_j+1} \times \ldots \times I_d \,. \end{aligned}$$

Geometrically, the two elementary cubes Q_j^- and Q_j^+ are directly opposite sides of the elementary cube Q. Using them, one can define the algebraic boundary of the cube as

$$\partial Q \;=\; \sum_{j=1}^{n} (-1)^{j-1} \left(Q_j^+ - Q_j^- \right) .$$

This formula is the cubical analogue of the boundary operator in a simplicial complex, and it allows us to define the incidence coefficient map via

$$\kappa\left(Q,\, Q_j^+\right) \;=\; (-1)^{j-1} \quad \text{and} \quad \kappa\left(Q,\, Q_j^-\right) \;=\; (-1)^{j} \,, \quad \text{for all} \quad j = 1, \ldots, n \,.$$

For all remaining pairs of elementary cubes in X we let $\kappa = 0$. Then it was shown in [KMM04, Proposition 2.37] that the so-defined incidence coefficient map satisfies the summation condition in the definition of a Lefschetz complex, i.e., we have $\partial(\partial Q) = 0$ for every $Q \in X$. This in turn implies that every cubical complex is indeed a Lefschetz complex.

Cubical complexes in ConleyDynamics.jl are a little more restricted. Since a cubical complex in the above sense is always finite, one can assume without loss of generality that the left endpoints of all involved elementary intervals are nonnegative. In other words, we always assume that the cubical complex only contains

elementary cubes from the set $(\mathbb{R}_0^+)^d$. This allows for a simple encoding of elementary cubes via labels of a fixed length, and without having to worry about the sign of an integer.

To describe this, fix a dimension d of the ambient space. Then every elementary cube in $(\mathbb{R}_0^+)^d$ has the following label, which depends on a coordinate width L:

- The first $d \cdot L$ characters of the label encode the starting points of the elementary intervals $I_1, \ldots, I_d$ in the standard representation of the elementary cube. For this, the starting points, which are nonnegative integers, are concatenated without spaces, but with leading zeros. For example, with $L = 2$ the string `"010203"` would correspond to the starting points 1, 2, and 3. Note that for given coordinate width L, one can only encode starting points between 0 and $10^L - 1$.
- The next entry in the label string is a period `.`.
- The remaining d characters of the string are integers 0 or 1, which give the interval lengths of $I_1, \ldots, I_d$.

For example, for $L = 2$ the string `"030600.000"` corresponds to the point $(3, 6, 0)$ in three dimensions. Similarly, the label `"030600.101"` represents the two-dimensional elementary cube $[3, 4] \times [6, 6] \times [0, 1] \subset \mathbb{R}^3$. Note, however, that the label representation is not unique, since it depends on the coordinate width L. Thus, with $L = 1$ the latter cube could also be written as `"360.101"`, or with $L = 3$ as `"003006000.101"`. As we will see in a moment, though, **within a given cubical complex all labels have to use the same coordinate width** L! This implies in particular that for a given coordinate width L one can only represent bounded cubical complexes which are contained in the d-dimensional box $[0, 10^L - 1]^d$.

The following three helper functions simplify the work with these types of cube labels:

- `cube_field_size` determines the field sizes of a given cube label. The first return value gives the dimension d of the ambient space, while the second value returns the coordinate width L.
- `cube_information` returns all information encoded in the cube label. The function returns an integer vector of length $2d + 1$, where d is the dimension of the ambient space. The first d entries give the vector of elementary interval starting points, while the next d values yield the corresponding interval lengths. The last entry specifies the dimension of the cube.
- `cube_label` creates a label from a cube's coordinate information. As function parameters, one has to specify d and L, and then pass an integer vector of length six which specifies the coordinates of the starting points and the interval lengths as in the previous item.

In ConleyDynamics.jl there are four basic commands for defining a cubical complex and working with it:

- `create_cubical_complex` creates a cubical complex in the Lefschetz complex data format. The complex is specified via a list of all the highest-dimensional cubes which are necessary to define the cubical complex. For this, every cube has to be given using the above-described special label format, with the same coordinate width L. In other words, all label strings have to be of the same length! If the optional parameter p is specified, the complex will be defined over a field with characteristic p, analogous to the case of a simplicial complex. If the characteristic is not specified, then the function defaults to the field $GF(2)$.
- `get_cubical_coords` determines the coordinates of all vertices of a given cubical complex from the cube labels. This vector can then be used for plotting purposes, see below.

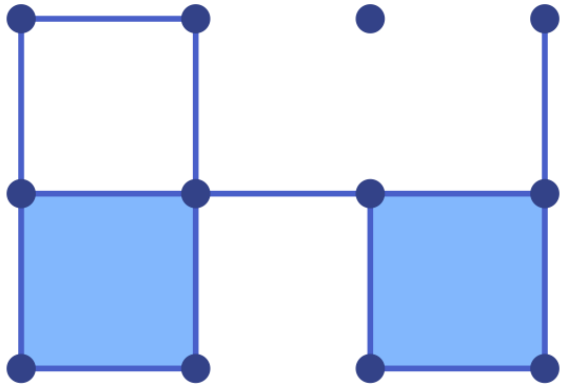


Figure 3.5: First sample cubical complex

- `create_cubical_rectangle` creates a cubical complex covering a rectangle in the plane. The rectangle is given by the subset $[0, nx] \times [0, ny]$ of the plane, where the nonnegative integers nx and ny have to be passed as arguments to the function. The function returns the cubical complex, and a vector of coordinates for the vertices. The latter can also be randomly perturbed as described in more detail in the function documentation.
- `create_cubical_box` creates a cubical complex covering a box in three-dimensional Euclidean space. The box is given by the subset $[0, nx] \times [0, ny] \times [0, nz]$ of space, where the nonnegative integers nx, ny, and nz have to be passed as arguments to the function. The optional parameters are the same as in the planar version.

To illustrate the first of these functions, consider the commands

```
cubes = ["00.11", "01.01", "02.10", "11.10", "11.01", "22.00", "20.11", "31.01"]
cc = create_cubical_complex(cubes)
```

These create the cubical complex cc, in the form of a Lefschetz complex. It can be visualized using the commands

```
coords = get_cubical_coords(cc)
fname  = "lefschetzex4.pdf"
plot_planar_cubical(cc,coords,fname,hfac=2.2,vfac=1.1,cubefac=60)
```

Similarly, the commands

```
cc2, coords2 = create_cubical_rectangle(5,2)
fname2 = "lefschetzex5.pdf"
plot_planar_cubical(cc2,coords2,fname2,hfac=1.7,vfac=1.2,cubefac=75)
```

define and illustrate a second cubical complex.

Finally, it is also possible to perturb the vertices in a cubical rectangle to obtain a Lefschetz complex consisting of quadrilaterals in the plane. This can be accomplied as follows:

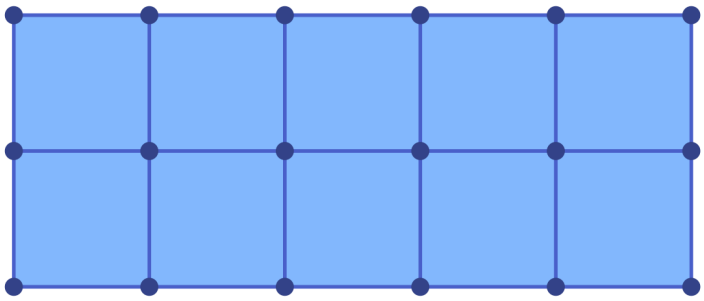


Figure 3.6: Second sample cubical complex

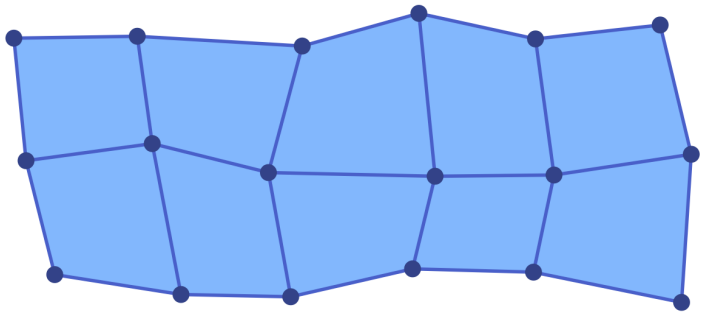


Figure 3.7: A randomly perturbed cubical complex

```
cc3, coords3 = create_cubical_rectangle(5,2,randomize=0.2)
fname3 = "lefschetzex6.pdf"
plot_planar_cubical(cc3,coords3,fname3,hfac=1.7,vfac=1.2,cubefac=75)
```

The resulting Lefschetz complex is visualized in the figure which shows a cubical rectangle with randomly perturbed vertices.

3.5 Golden Ratio Rectangles

The function `create_golden_ratio_rectangle` creates a planar `EuclideanComplex` (described in the following section) by recursively subdividing a given rectangle using a golden ratio subdivision rule, similar to the results of [CWD13]. The subdivision works as follows. Given a rectangle with side lengths $a \geq b$, the longer edge is split into two segments of lengths σa and $(1-\sigma)a$, yielding two smaller rectangles of the same height b. Which piece is placed on which side is chosen uniformly at random at each step.

The default subdivision parameter is $\sigma = (\sqrt{5}-1)/2 \approx 0.618$, which is the golden ratio minus one. Starting from a square, this choice produces subrectangles with aspect ratios $\varphi = (\sqrt{5}+1)/2$ and φ^2, where φ denotes the golden ratio. Crucially, further subdivisions of these rectangles stay within the same set of shapes: Only three rectangle types with aspect ratios 1, φ, and φ^2 ever appear during the process.

The depth of the recursive subdivision is governed by three keyword arguments:

- `sdmin` (default `0`): The minimum number of subdivision levels applied unconditionally to every rectangle. With the default value of zero, no splits are forced.

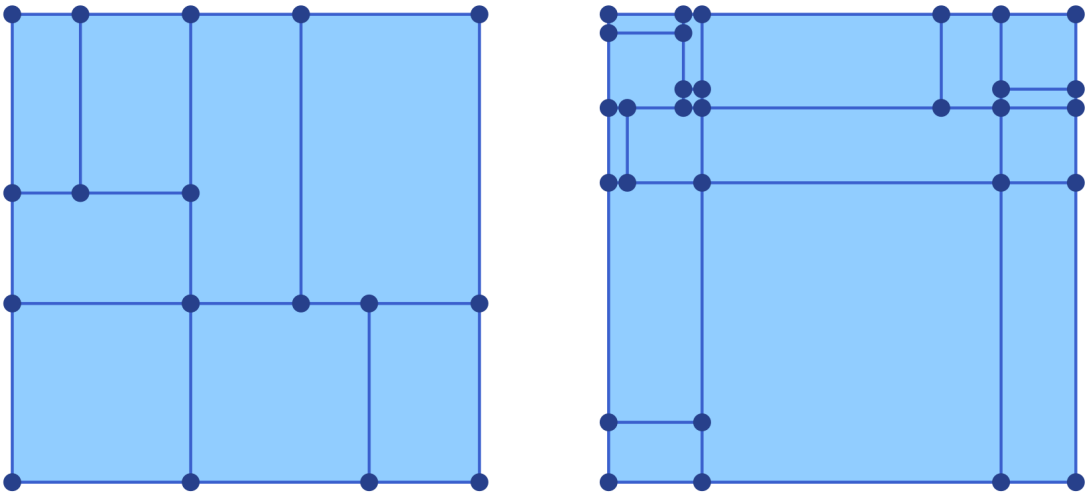


Figure 3.8: Lefschetz complexes via the golden ratio function

- `sdmax` (default 7): A hard upper bound on the number of subdivision levels; no rectangle will be split more than `sdmax` times.
- `sdfunction` (default `(_, _) -> false`): A function `sdfunction(bmin, bmax)` that accepts the lower-left and upper-right corners of a rectangle as `Vector{Float64}` arguments and returns a `Bool`. For depths between `sdmin` and `sdmax` the function decides whether a given rectangle should be subdivided further. The default always returns `false`, so with default arguments exactly `sdmin` uniform subdivision levels are performed. Providing a non-trivial `sdfunction` enables adaptive refinement beyond the forced `sdmin` levels.

The result is a `EuclideanComplex` whose two-dimensional cells are the leaf rectangles produced by the subdivision. Wherever a longer edge is shared between two differently-refined adjacent rectangles, it is automatically represented as a sequence of shorter segments whose endpoints agree with the corners of the finer neighbor. The boundary coefficients follow the same sign convention as in a cubical complex: Bottom and right segments carry coefficient $+1$, top and left segments carry coefficient -1.

To illustrate the basic usage, the following commands create a golden ratio subdivision of the unit square at uniform depth 3:

```
import Random; Random.seed!(42)
ec1 = create_golden_ratio_rectangle([0.0,0.0],[1.0,1.0]; sdmin=3)
lefschetz_information(ec1)
```

With this seed the result is a complex with 8 rectangles, 25 edges, and 18 vertices (51 cells in total). The Euler characteristic is 1 and the homology is $(1, 0, 0)$, confirming that the complex is contractible, as expected. The Lefschetz complex is shown in the left panel of the figure of the Lefschetz complexes created by the golden ratio function.

By default, the algorithm uses the golden ratio for the subdivisions, based on their implications for the rectangle aspect ratios discussed above. The value σ can, however, be changed via the keyword argument `sigma`, as the following example shows:

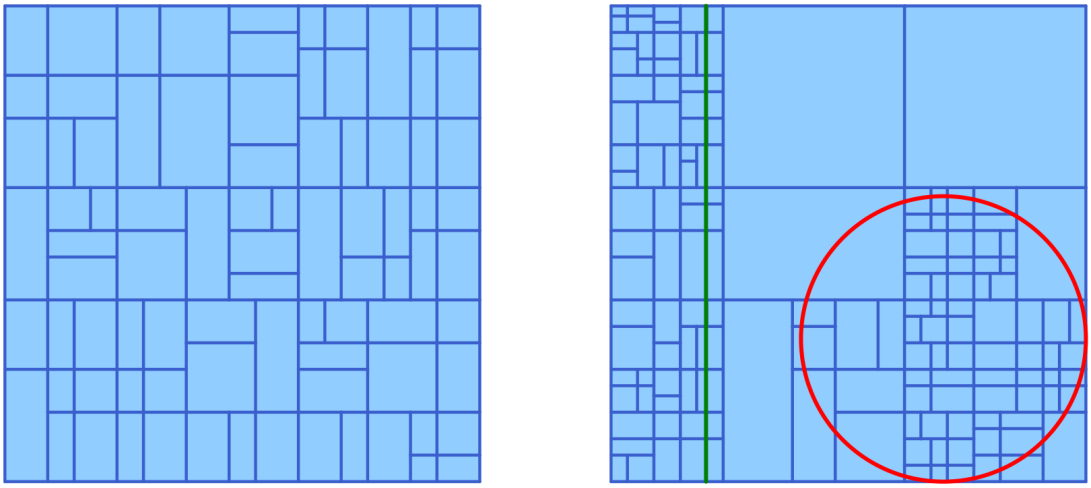


Figure 3.9: Mesh refinement via the golden ratio

```
ec2 = create_golden_ratio_rectangle([0.0,0.0],[1.0,1.0]; sdmin=4, sigma=0.8)
lefschetz_information(ec2)
```

The resulting Lefschetz complex is shown in the right panel of the same figure. Note that this time the aspect ratios of the generated rectangles can become very extreme. Thus, changing the subdivision value `sigma` should only be done in exceptional circumstances.

Adaptive subdivision can be achieved by supplying an `sdfunction`. The following example keeps refining until all rectangles have an x-extent of at most 0.1:

```
import Random; Random.seed!(7)
sdf(bmin, bmax) = bmax[1] - bmin[1] > 0.1
ec3 = create_golden_ratio_rectangle([0.0,0.0],[1.0,1.0]; sdfunction=sdf, sdmax=6)
lefschetz_information(ec3)
```

This produces a complex with 85 rectangles, 230 edges, and 146 vertices (461 cells in total), again with Euler characteristic 1 and trivial higher homology. The resulting complex is shown in the left panel of the figure on mesh refinement via the golden ratio.

Finally, it is also possible to create more complicated refinement functions. For example, the following function `sdfpt` makes sure that rectangles with center within a circle of radius 0.3 and center $(0.7, 0.3)$ have maximum edge length at most 0.07, and rectangles whose x-coordinate are at most 0.2 have maximum edge length at most 0.05. The resulting Lefschetz complex is shown in the right panel of the figure on mesh refinement.

```
function sdfpt(bmin, bmax)
    bcenter = 0.5 .* (bmin .+ bmax)
    bsize   = maximum(bmax .- bmin)
    bdist   = sqrt((bcenter[1]-0.7)^2 + (bcenter[2]-0.3)^2)
    if (bdist <= 0.3) && (bsize > 0.07)
        return true
    elseif (bcenter[1] <= 0.2) && (bsize > 0.05)
        return true
```

```
    else
        return false
    end
end

ec4 = create_golden_ratio_rectangle([0.0,0.0],[1.0,1.0]; sdfunction=sdfpt, sdmin=3)
lefschetz_information(ec4)
```

The figure in the right panel shows the line $x = 0.2$ in green, and the circle of radius 0.3 and center $(0.7, 0.3)$ in red. Moreover, we visualized only the edges and rectangles of this Lefschetz complex. The figure on mesh refinement was created using the following commands:

```
using Plots

theta    = LinRange(0, 2*pi, 500);
circle_x = 0.7 .+ 0.3 .* cos.(theta);
circle_y = 0.3 .+ 0.3 .* sin.(theta);
line_x   = [0.2, 0.2];
line_y   = [0.0, 1.0];

p3 = plot_simplicial(ec3, pdim=[false,true,true])
p4 = plot_simplicial(ec4, pdim=[false,true,true])
plot!(p4, circle_x, circle_y, color=:red,   linewidth=2)
plot!(p4, line_x,   line_y,   color=:green, linewidth=2)
pcombined = plot(p3, p4, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "goldenratio2.png")
```

We would like to point out that the function `plot_simplicial` still displays the golden ratio complex correctly, despite the fact that it is not simplicial.

3.6 Euclidean Complexes

All of the above-described Lefschetz complex types were abstract in the sense that there is no actual geometric realization associated with them. For certain applications, however, such as plotting a complex or using a complex for the analysis of planar and spatial ordinary differential equations, the complex has to be embedded in a suitable Euclidean space. One can usually achieve such an embedding by providing coordinates for each of the vertices of the complex, which then anchor the complex in the Euclidean space. This approach works extremely well for simplicial and cubical complexes, since both of these complex types are closed in the sense that the geometric realization of any higher-dimensional cell can be inferred from the vertex locations.

Unfortunately, however, we will encounter situations in which one would like to use subcomplexes of simplicial or cubical complexes, which are no longer closed in the above sense. For such a complex it is generally not possible to deduce the location of, for example, an edge, by looking at the location of its vertices – since one or both vertices might be missing from the complex. As of version `v0.7.5`, the package ConleyDynamics.jl provides a second complex datatype to address this issue:

`ConleyDynamics.EuclideanComplex` – Type.

```
EuclideanComplex
EuclideanComplex(labels, dimensions, boundary, coords; validate=true)
```

A Lefschetz complex with embedded Euclidean coordinates for every cell.

The struct shares the fields of `LefschetzComplex`:

- `labels::Vector{String}`: Vector of labels associated with cell indices
- `dimensions::Vector{Int}`: Vector of cell dimensions
- `boundary::SparseMatrix`: Boundary matrix, columns give the cell boundaries
- `ncells::Int`: Number of cells
- `dim::Int`: Dimension of the complex (must satisfy `dim ≤ 3`)
- `indices::Dict{String,Int}`: Dictionary for finding cell index from label

In addition it carries:

- `coords::Vector{Vector{Vector{Float64}}}`: Per-cell vertex coordinates. `coords[k]` is the list of vertex coordinate vectors needed to draw cell k:
  - Vertex: `[[x,y]]` (length 1)
  - Edge: `[[x1,y1],[x2,y2]]` (length 2)
  - Triangle: `[[x1,y1],[x2,y2],[x3,y3]]` (length 3)
  - Cubical 2-cell: `[[x1,y1],[x2,y2],[x3,y3],[x4,y4]]` (length 4)

The function `lefschetz_to_euclidean` creates an `EuclideanComplex` from an existing `LefschetzComplex`, and the function `euclidean_to_lefschetz` converts it back.

source

This complex struct has the same fields as a Lefschetz complex, with completely analogous meanings. In addition, however, it includes the field `coords::Vector{Vector{Vector{Float64}}}`. This vector contains embedding information for every cell of the complex, by providing the locations of all determining vertices – regardless of whether they are still contained in the complex or not.

Both `LefschetzComplex` and `EuclideanComplex` are subtypes of the new abstract type `AbstractComplex`. All topology, homology, and dynamics functions accept any `AbstractComplex`. However, if one of these functions manipulates the collection of cells, the coordinate vector in a `EuclideanComplex` is automatically updated as well. Finally, it is possible to transition between the two complex types using the functions `lefschetz_to_euclidean` and `euclidean_to_lefschetz`.

3.7 Lefschetz Complex Operations

Once a Lefschetz complex has been created, there are a number of manipulations and queries that one has to be able to perform on the complex. At the moment, ConleyDynamics.jl supplies a number of functions for this. The following four functions provide basic information:

- `lefschetz_information` provides basic information about the Lefschetz complex, including its Euler characteristic and homology, as well as cell counts by dimension and sparsity percentage of the boundary matrix.
- `lefschetz_cell_count` returns the number of cells in each dimension in a vector of length `lc.dim + 1`. If the optional parameter `bounds=true` is passed, then the function also returns two integer vectors `lo` and `hi`. These contain the beginning and end indices of the cells in each dimension.
- `lefschetz_field` returns the field F over which the Lefschetz complex is defined as a `String`.

- `lefschetz_is_closed` determines whether a given Lefschetz complex cell subset is closed or not.
- `lefschetz_is_locally_closed` checks whether a given Lefschetz complex cell subset is locally closed or not.
- `lefschetz_is_simplicial` checks whether a given Lefschetz complex is a simplicial complex or not.
- `lefschetz_is_cubical` checks whether a given Lefschetz complex is a cubical complex or not, in the sense of this manual. In particular, its labels have to have the correct format.
- `lefschetz_is_subcubical` checks whether a given Lefschetz complex is a locally closed subset of a cubical complex or not.

The next set of functions can be used to extract certain topological features from a Lefschetz complex:

- `lefschetz_boundary` computes the support of the boundary $\partial\sigma$ of a Lefschetz complex cell σ. In other words, it returns the vector of all facets of σ. The cell can either be specified via its index or its label, and the return format corresponds to the input format.
- `lefschetz_coboundary` returns all cells which lie in the coboundary of the specified cell σ, i.e., it returns all cells which have σ as a facet.
- `lefschetz_closure` determines the closure of a given cell subset, i.e., the union of all faces of cells in the cell subset.
- `lefschetz_interior` determines the interior of a given cell subset. For this, the Lefschetz complex is interpreted as a finite topological space, where a set A is open if and only if for every cell $\sigma \in A$ all of its cofaces are also contained in the set A.
- `lefschetz_openhull` computes the open hull of a cell subset, i.e., the smallest open set which contains the given cell subset. For this, we again interpret a Lefschetz complex as a finite topological space as above.
- `lefschetz_topboundary` computes the topological boundary of a cell subset, i.e., the set difference of the closure and the interior of the set. Note that this usually differs from the (algebraic) boundary mentioned above.
- `lefschetz_lchull` finds the locally closed hull of a Lefschetz complex subset. This is the smallest locally closed set which contains the given cell subset. One can show that it is the intersection of the closure and the open hull of the cell subset.
- `lefschetz_clomo_pair` determines the closure-mouth-pair associated with a Lefschetz complex subset.
- `lefschetz_neighbors` finds all cells neighboring a Lefschetz complex subset. More precisely, the function returns all cells which are not contained in the subset specified by `subcomp`, but which are either in the closure or in the open hull of `subcomp`. In other words, if one adds any one of these neighbors to the set `subcomp`, then the number of connected components of the combined set does not increase. It could, however, decrease. In particular, if `subcomp` represents a connected set, then the neighbors are exactly the cells that can be added while preserving connectedness.
- `lefschetz_skeleton` computes the k-dimensional skeleton of a Lefschetz complex or of a given Lefschetz complex subset. There are two possible function calls, namely `lefschetz_skeleton(lc, k)` and `lefschetz_skeleton(lc, subcomp, k)`. While in the first case the k-skeleton of the full Lefschetz complex is returned, in the second case it returns the k-skeleton of the closure of the given subset `subcomp`.

- `manifold_boundary` returns a list of cells which form the "manifold boundary" of the given Lefschetz complex. More precisely, if the complex has dimension d, then it determines all cells of dimension $d-1$ which have at most one cell in their coboundary, as well as all cells of dimensions less than $d-1$ which have no cell in their coboundary, and finally returns the closure of the resulting cell subset.

The following functions create Lefschetz subcomplexes from a Lefschetz complex:

- `lefschetz_subcomplex` determines a Lefschetz subcomplex from a given Lefschetz complex. The subcomplex has to be locally closed, and it is given by a collection of cells. The check for local closedness can be skipped with the optional argument `validate=false`. The computational cost is proportional to the number of cells in the subcomplex.
- `lefschetz_closed_subcomplex` extracts a closed Lefschetz subcomplex from the given Lefschetz complex. The subcomplex is the closure of the specified collection of cells.

In addition, it is possible to create a new Lefschetz complex from an existing one using the following functions:

- `permute_lefschetz_complex` determines a new Lefschetz complex which is obtained from the original one by a permutation of the cells. Note that the permutation has to respect the ordering of the cells by dimension, otherwise an error is raised. In other words, the permutation has to decompose into permutations within each dimension. This is automatically done if no permutation is explicitly specified and the function creates a random one.
- `lefschetz_reduction` uses elementary reductions to transform a Lefschetz complex into a smaller one while preserving homology. In fact, the new complex is chain homotopic to the original one. For this, one has to specify a sequence of reduction pairs, which are disjoint pairs of cells whose dimensions differ by one, and such that one cell is a face of the other, once the earlier reductions have been performed. This function is based on [KMS98]. It returns a new Lefschetz complex which no longer contains the cells contained in the reduction pairs.
- `lefschetz_reduction_maps` is an extension of the previous function `lefschetz_reduction`. While it performs the same basic Lefschetz complex reduction, it does compute useful additional information. Besides the reduced Lefschetz complex, it also returns the chain equivalences between the original and the reduced complex, as well as the chain homotopy relating one of their compositions to the identity. The return values of this function are as follows:
  - `lcred`: The first return variable contains the reduced Lefschetz complex, just as in the previous function.
  - `pp`: This is a sparse matrix representation of the chain equivalence between the original complex and the reduced one.
  - `jj`: This sparse matrix gives the chain equivalence between the reduced complex and the original one.
  - `hh`: The third sparse matrix represents the chain homotopy which shows that the composition `jj * pp` is chain homotopic to the identity.

  Note that these maps are constructed in such a way that the product `pp * jj` always is the identity.
- `lefschetz_newbasis` creates a new Lefschetz complex `lcnew` from the given Lefschetz complex `lc` via a change of basis. The new basis has to be specified in the sparse matrix `basis`, whose columns represent the new basis in terms of the existing one. The basis matrix has to respect the grading by dimension, i.e., the cells which are used to form a new basis chain have to have the same dimensions as the cell which is being replaced.

- `lefschetz_newbasis_maps` extends the previous function in that besides the new Lefschetz complex `lcnew` it also returns the chain maps `pp` and `jj` which are the respective isomorphisms from the original complex `lc` to `lcnew`, and vice versa, as well as the zero chain homotopy `hh`.
- `compose_reductions` can be used to find the composed chain maps and chain homotopy for a pair of subsequent reductions. These can be produced either via the function `lefschetz_reduction_maps` or the function `lefschetz_newbasis_maps`.

It was shown in [EM23] that suitable reduction pairs for `lefschetz_reduction` can be found easily via the concept of filters. A filter on a Lefschetz complex is a function $\varphi : X \to \mathbb{R}$ which has the property that $\varphi(x) \leq \varphi(y)$ if x is a face of y. Every such filter induces shallow pairs, which in the case of an injective filter generate a gradient Forman vector field on the Lefschetz complex. It turns out that these shallow pairs can be used to reduce the complex. In ConleyDynamics.jl there are three functions for working with filters:

- `create_random_filter` creates a random injective filter on a Lefschetz complex. The filter is created by assigning integers to cell groups, increasing with dimension. Within each dimension the assignment is random, but all filter values of cells of dimension k are less than all filter values of cells with dimension $k+1$. The function returns the filter as `Vector{Int}`, with indices corresponding to the cell indices in the Lefschetz complex.
- `filter_shallow_pairs` finds all shallow pairs for the filter φ. These are face-coface pairs (x, y) whose dimensions differ by one, and such that y has the smallest filter value on the coboundary of x, and x has the largest filter value on the boundary of y. For injective filters, these pairs give rise to a Forman vector field on the underlying Lefschetz complex. For noninjective filters this is not true in general.
- `filter_induced_mvf` returns the smallest multivector field which has the property that every shallow pair is contained in a multivector. For injective filters this is a Forman vector field, but in the noninjective case it can be a general multivector field.

Notice that a special class of filters, called filtrations, are used in the context of persistent homology. There are two functions devoted to dealing with them:

- `lefschetz_filtration` computes a filtration on a Lefschetz subset. Based on integer filtration values assigned to some cells of the given Lefschetz complex, it determines the smallest closed subcomplex `lcsub` which contains all cells with nonzero filtration values, as well as filtration values `fvalsub` on this subcomplex, which give rise to a filtration of closed subcomplexes, and which can be used to compute persistent homology.
- `lefschetz_filtration_mvf` determines the multivector field associated with a Lefschetz complex filtration created by the previous function. For the Lefschetz complex `lc`, and the filtration `fvalues` given on the Lefschetz complex, this multivector field is generated by the filtration in the following way. For every filtration value k, the cells which are assigned this value form a locally closed set, which can then be decomposed into connected components that are all locally closed as well. The union of all these connected components, as k ranges from 1 to N, forms the multivector field `mvf` that is returned by the function.

There are also a few helper functions which implement various conversion tasks:

- `lefschetz_gfp_conversion` changes the base field of the given Lefschetz complex from the rationals $\mathbb{Q}$ to a finite field $GF(p)$. Note that it is not possible to perform the reverse conversion.

- `euclidean_to_lefschetz` converts a `EuclideanComplex` to a `LefschetzComplex` by removing the coordinate field.
- `lefschetz_to_euclidean` converts a `LefschetzComplex` to a `EuclideanComplex` by adding a user-provided coordinate field, in one of two forms. A `Vector{Vector{Real}}` is interpreted as vertex coordinates, with the function filling in coordinate vectors for higher-dimensional cells based on the 0-skeleton. A `Vector{Vector{Vector{Real}}}` is instead taken as-is for the new cell coordinate field. Only basic consistency checks are performed.
- `rescale_coords` produces a new `EuclideanComplex` from an existing one by rescaling the coordinates.

In addition, ConleyDynamics.jl provides the following helper functions for the fundamental objects of cells and cell subsets, which can be represented either by integer cell indices or by cell labels:

- `convert_cells` converts a vector of cells from integer to label format, or vice versa.
- `convert_cellsubsets` converts a vector of cell subsets from integer to label format, and vice versa.
- `cellsubsets_to_cells` converts a given vector of cell subsets into a vector of cells, by simply taking the union of all subsets. The cells are returned in ordered form, based on their index form in the underlying Lefschetz complex.
- `cellsubset_bounding_box` computes the bounding box for a cell subset, provided coordinates for the vertices of the Lefschetz complex are provided. The bounding box is purely based on the location of the vertices.
- `cellsubset_distance` computes the distance of a cell subset from a point. More precisely, this function computes the minimal distance of a vertex in the closure of the cell subset to the given point. While this function does not provide the Hausdorff distance, it serves as a simple measure for how far the cell subset is from a give point.
- `cellsubset_planar_area` computes the area of a planar cell subset. This function assumes that the complex is two-dimensional and that the maximal 2-cells in the cell subset are all polygonal with straight boundary edges. If these conditions are not met an error is raised.
- `locate_planar_cellsubsets` expects a list of cell subsets, for example the collection of Morse sets, or a multivector field, and it extracts the subsets whose closure lies in the interior of a geometric object G, and also the ones whose closure intersects the boundary of G. At the moment, G can be either a planar rectangle with sides parallel to the coordinate axes, or a circle in the plane. The same command is used for both, the version of G is selected through the function arguments via multiple dispatch.
- `cellsubset_location_rectangle` determines the location of the closure of a given cell subset relative to a rectangle in the plane. It distinguishes between being in the interior, intersecting the boundary, or lying in the exterior. The rectangle has to be parallel to the coordinate axes.
- `cellsubset_location_circle` determines the location of the closure of a given cell subset relative to a circle in the plane. It distinguishes between being in the interior, intersecting the boundary, or lying in the exterior.

Finally, there are a couple of geometry helper functions which allow for the transformation of vertex coordinates in a Lefschetz complex:

- `convert_planar_coordinates` transforms a given collection of planar coordinates in such a way that the extreme coordinates fit precisely in a given rectangle in the plane.

- `convert_spatial_coordinates` transforms a given collection of spatial coordinates in such a way that the extreme coordinates fit precisely in a given rectangular box in space.
- `signed_distance_rectangle` determines the signed distance from a given point to the boundary of a rectangle in the plane. Negative values correspond to the interior, positive values to the exterior.
- `signed_distance_circle` determines the signed distance from a given point to the boundary of a circle in the plane. Negative values correspond to the interior, positive values to the exterior.
- `segment_intersects_rectangle` determines whether a line segment intersects the boundary of an axes-parallel rectangle in the plane.
- `segment_intersects_circle` determines whether a line segment intersects a circle in the plane.

For more details on the usage of any of these functions, please see their documentation in the API section of the manual.

3.8 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[Ale37] P. Alexandrov. Diskrete Räume. Mathematiceskii Sbornik (N.S.) **2**, 501–518 (1937).

[CWD13] G. S. Cochran, T. Wanner and P. Dłotko. A randomized subdivision algorithm for determining the topology of nodal sets. SIAM Journal on Scientific Computing **35**, B1034–B1054 (2013).

[DKMW11] P. Dłotko, T. Kaczynski, M. Mrozek and T. Wanner. Coreduction homology algorithm for regular CW-complexes. Discrete & Computational Geometry **46**, 361–388 (2011).

[EM23] H. Edelsbrunner and M. Mrozek. The depth poset of a filtered Lefschetz complex (2023), arXiv:2311.14364v2 [math.AT].

[KMM04] T. Kaczynski, K. Mischaikow and M. Mrozek. Computational Homology. Vol. 157 of Applied Mathematical Sciences (Springer-Verlag, New York, 2004).

[KMS98] T. Kaczynski, M. Mrozek and M. Slusarek. Homology computation by reduction of chain complexes. Computers & Mathematics with Applications **35**, 59–70 (1998).

[Lef42] S. Lefschetz. Algebraic Topology. Vol. 27 of American Mathematical Society Colloquium Publications (American Mathematical Society, New York, 1942).

[LKMW23] M. Lipinski, J. Kubica, M. Mrozek and T. Wanner. Conley-Morse-Forman theory for generalized combinatorial multivector fields on finite topological spaces. Journal of Applied and Computational Topology **7**, 139–184 (2023).

[Mas91] W. S. Massey. A Basic Course in Algebraic Topology. Vol. 127 of Graduate Texts in Mathematics (Springer-Verlag, New York, 1991).

[MB09] M. Mrozek and B. Batko. Coreduction homology algorithm. Discrete & Computational Geometry **41**, 96–118 (2009).

[MW25] M. Mrozek and T. Wanner. Connection Matrices in Combinatorial Topological Dynamics. SpringerBriefs in Mathematics (Springer-Verlag, Cham, 2025).

[Mun84] J. R. Munkres. Elements of Algebraic Topology (Addison-Wesley, Menlo Park, 1984).

Chapter 4

Homology

Conley's theory for the qualitative study of dynamical systems is based on fundamental concepts from algebraic topology. One of these is homology, which studies the topological properties of spaces using algebraic means. As part of ConleyDynamics.jl a number of homology methods are included. The first of these is based on the persistence algorithm described in [EH10], which is based on matrix reductions. In addition, we have implemented a version of the method described in [HMMN14], which is based on Morse reductions. Finally, a parallelized version of the Morse reduction algorithm is provided. All of these methods have been extended to work for arbitrary Lefschetz complexes over either the rationals or a finite field of prime order. For more serious applications, one could use professional implementations such as Gudhi, see [GUD24].

4.1 Lefschetz Complex Homology

The most important notion of homology used in ConleyDynamics.jl is Lefschetz homology. It generalizes both simplicial homology as described in [Mun84], and cubical homology in the sense of [KMM04]. In order to fix our notation, we provide a brief introduction in the following. For more details, see [Lef42].

As we saw earlier, a Lefschetz complex X is a collection of cells with associated nonnegative dimensions, together with a boundary map ∂ which is induced by the incidence coefficient map κ. The fundamental idea behind homology is to turn this underlying information into an algebraic form in such a way that the boundary map becomes a linear map. For this, define the k-th chain group as

$$C_k(X) = \left\{ \sum_{i=1}^{m} \alpha_i \sigma_i \ : \ \alpha_1, \ldots, \alpha_m \in F \quad \text{and} \quad \sigma_1, \ldots, \sigma_m \in X_k \right\}.$$

Since X_k denotes the collection of all cells of dimension k, this definition can be rephrased by saying that $C_k(X)$ consists of all formal linear combinations of k-dimensional cells with coefficients in the underlying field F. It is not difficult to see that $C_k(X)$ is in fact a vector space over F. Moreover, its dimension is equal to the number of k-dimensional cells in X. The collection of all chain groups is $C(X) = (C_k(X))_{k \in \mathbb{Z}}$, where we let $C_k(X) = \{0\}$ for all $k < 0$ and $k > \dim X$.

We now turn our attention to the boundary map. It was already explained how the incidence coefficient map κ can be used to define the boundary $\partial\sigma \in C_{k-1}(X)$ for every k-dimensional cell $\sigma \in X_k$. If one further defines

$$\partial \left(\sum_{i=1}^{m} \alpha_i \sigma_i \right) = \sum_{i=1}^{m} \alpha_i \partial \sigma_i \in C_{k-1}(X) \quad \text{for} \quad \sum_{i=1}^{m} \alpha_i \sigma_i \in C_k(X) \ ,$$

then one obtains a map $\partial : C_k(X) \to C_{k-1}(X)$. It is not difficult to verify that this map is both well-defined and linear. Sometimes, we write ∂_k instead of ∂ to emphasize that we consider the boundary map defined on the k-th chain group $C_k(X)$.

Altogether, the above definitions have equipped us with a sequence of vector spaces and maps between them in the form

$$\ldots \xrightarrow{\partial_{k+2}} C_{k+1}(X) \xrightarrow{\partial_{k+1}} C_k(X) \xrightarrow{\partial_k} C_{k-1}(X) \xrightarrow{\partial_{k-1}} \ldots \xrightarrow{\partial_1} C_0(X) \xrightarrow{\partial_0} \{0\} \xrightarrow{\partial_{-1}} \ldots,$$

and the properties of a Lefschetz complex further imply that

$$\partial_k \circ \partial_{k+1} = 0 \quad \text{for all} \quad k \in \mathbb{Z}.$$

In other words, the pair $(C(X), \partial)$ is a chain complex, which consists of a sequence of vector spaces over F and linear maps between them. Recall from linear algebra that any linear map induces two important subspaces, which in the context of algebraic topology are given special names as follows:

- The elements of the subspace $Z_k(X) = \ker \partial_k$ are called the k-cycles of X.
- The elements of the subspace $B_k(X) = \text{im } \partial_{k+1}$ are called the k-boundaries of X.

Both of these vector spaces are subspaces of the k-th chain group $C_k(X)$. Furthermore, in view of the above identity $\partial_k \circ \partial_{k+1} = 0$, one immediately obtains the subspace inclusion $B_k(X) \subset Z_k(X)$. We can therefore define the quotient space

$$H_k(X) \;=\; Z_k(X)/B_k(X) \;=\; \ker \partial_k / \text{im } \partial_{k+1}.$$

This vector space is called the k-th homology group of the Lefschetz complex X. It is again a vector space over F, and therefore its dimension provides important information. In view of this, the dimension of the k-th homology group $H_k(X)$ is called the k-th Betti number of X, and abbreviated as $\beta_k(X) = \dim H_k(X)$.

In order to shed some light on the actual meaning of homology, and in particular the Betti numbers, we turn to an example. Consider the simplicial complex `sc` that was already introduced in the Tutorial, and which can be created using the commands

```
labels = ["A","B","C","D","E","F"]
simplices = [["A","B"],["A","C"],["B","C"],["B","D"],["D","E","F"]]
sc = create_simplicial_complex(labels,simplices)
```

This two-dimensional simplicial complex is shown in the figure from the tutorial, with a hollow triangle `ABC` and a filled triangle `DEF`.

For simplicity, we consider the associated Lefschetz complex X over the field $F = GF(2)$. Then chains in a chain group are just a sum of individual cells of the same dimension.

In this simple example, one can determine the cycles and boundaries for $k = 1$ directly. The vector space $Z_1(X)$ of 1-cycles contains the two nonzero chains $c_1 = \text{AB} + \text{BC} + \text{AC}$ and $c_2 = \text{DE} + \text{EF} + \text{DF}$, since one

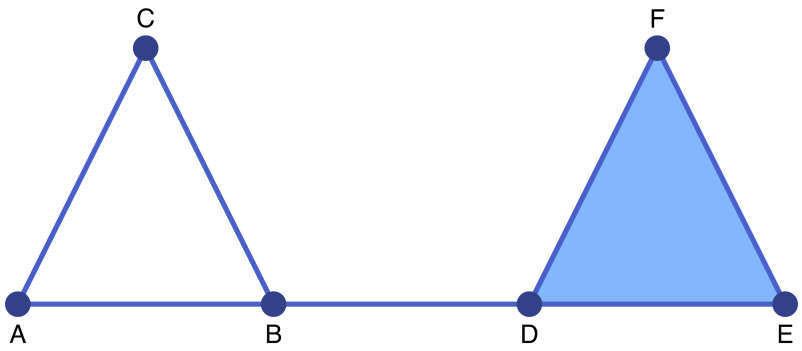


Figure 4.1: The simplicial complex from the tutorial

can verify that $\partial c_1 = \partial c_2 = 0$. These are, however, not all nontrivial 1-cycles, as their sum $c_1 + c_2$ is another one. Thus, the first cycle group is given by $Z_1(X) = \{0, c_1, c_2, c_1 + c_2\}$. It is a vector space over $F = GF(2)$ of dimension two, and any two nonzero elements of $Z_1(X)$ form a basis. What about the 1-boundaries? The simplicial complex X contains only one 2-cell, namely DEF, and its boundary is given by the chain c_2. Thus, the first boundary group is given by $B_1(X) = \{0, c_2\}$, which is a one-dimensional vector space over F.

Combined, one can show that the first homology group $H_1(X)$ consists of the two equivalence classes

$$H_1(X) = \{B_1(X),\ c_1 + B_1(X)\}\ ,$$

where the class $B_1(X)$ is the zero element in $H_1(X)$. This implies that the first homology group is one-dimensional, and we have $\beta_1(X) = 1$. In some sense, the basis element of $H_1(X)$, which is the unique nonzero equivalence class given by $c_1 + B_1(X)$, is represented by the cycle c_1.

The above mathematically precise description can be summarized as follows. All three nontrivial cycles in $Z_1(X)$ have the potential to enclose two-dimensional holes in the simplicial complex X, since they are chains without boundary. However, some of these potential holes have been filled in by two-dimensional cells. Thus, while c_1 does indeed represent a hole, the chain c_2 does not, since its interior is filled in by DEF. Note that the cycle $c_1 + c_2$ does not create a second hole, since we have $(c_1 + c_2) - c_1 = c_2 \in B_1(X)$. In other words, the first Betti number counts the number of independent holes in the complex X.

One can extend this discussion also to other dimensions and to general Lefschetz complexes X. In this way, one obtains the following informal interpretations of the Betti numbers:

- $\beta_0(X)$ counts the number of connected components of X,
- $\beta_1(X)$ counts the number of independent holes in X,
- $\beta_2(X)$ counts the number of independent cavities in X.

In general, one can show that $\beta_k(X)$ represents the number of independent k-dimensional holes in the Lefschetz complex X. For more details, see [Mun84].

The package ConleyDynamics.jl provides two methods to compute standard homology, both of which use the function name `homology`:

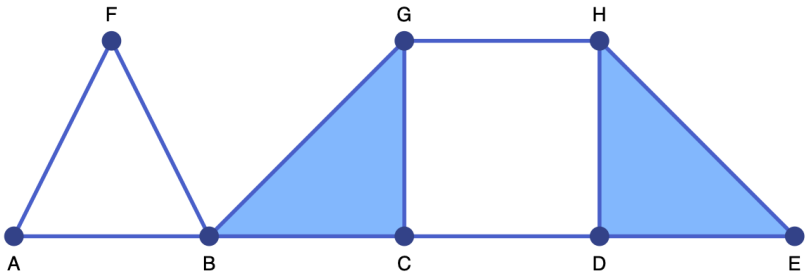


Figure 4.2: Sample simplicial complex

- In the first version, `homology` expects one input argument `lc`, which has to be of the Lefschetz complex type `LefschetzComplex`. It returns a vector `betti` of integers, whose length is one more than the dimension of the complex. The k-th Betti number $\beta_k(X)$ is returned in `betti[k+1]`.
- In the second version, `homology` expects two input arguments: An underlying Lefschetz complex `lc` as above, together with an argument of type `Cells`, which describes a locally closed subset of `lc`. In this case, the function returns the homology of the associated subcomplex.

In both versions of this function, one can also specify the optional argument `algorithm`, which has to be of type `String`. It selects the method used for computing homology:

- `algorithm = "matrix"` selects the standard matrix algorithm, as for example described in [EH10],
- `algorithm = "morse"` selects an algorithm based on Morse reductions, inspired by [HMMN14], and
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions.

By default, the function uses the Morse reduction algorithm `morse`.

We would like to point out that the field F is implicit in the data structure for the Lefschetz complex X, and therefore it does not have to be specified. It can always be queried using the function `lefschetz_field`. For the above example one obtains

```
julia> homology(sc)
3-element Vector{Int64}:
 1
 1
 0
```

This clearly gives the correct Betti numbers, as we have already seen that this simplicial complex has one hole, and it is obviously connected.

The simplicial complex with vertices A to H, which consists of a hollow triangle, two filled triangles, and a square without interior, as shown in the figure of the sample simplicial complex, can be created using the commands

```
labels2 = ["A","B","C","D","E","F","G","H"]
simplices2 = [["A","B"],["A","F"],["B","F"],["B","C","G"],["D","E","H"],["C","D"],["G","H"]]
sc2 = create_simplicial_complex(labels2,simplices2)
```

and its homology can then be determined as follows:

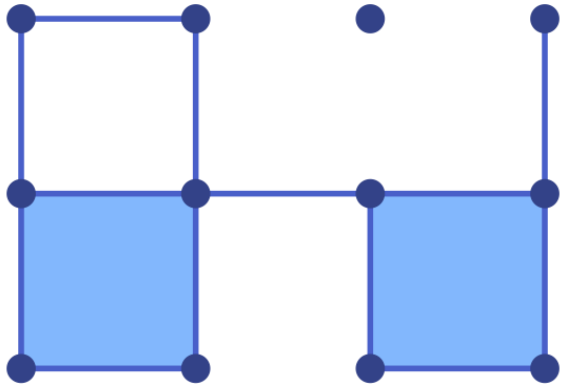


Figure 4.3: Sample cubical complex

```
julia> homology(sc2)
3-element Vector{Int64}:
 1
 2
 0
```

This complex is also connected, and therefore one has $\beta_0(X) = 1$. However, this time one obtains two independent holes, which results in $\beta_1(X) = 2$.

Similarly, the cubical complex depicted in the figure of the sample cubical complex, with two filled squares, can be generated via

```
cubes = ["00.11", "01.01", "02.10", "11.10", "11.01", "22.00", "20.11", "31.01"]
cc = create_cubical_complex(cubes)
```

and its Betti numbers are given by

```
julia> homology(cc)
3-element Vector{Int64}:
 2
 1
 0
```

In this case, the complex has two components and one hole. As a final example, consider a simplicial complex which consists of the manifold boundary of a single cube. Such a complex can be generated using the commands

```
cc2,~ = create_cubical_box(1, 1, 1)
mbcells = manifold_boundary(cc2)
cc2bnd = lefschetz_subcomplex(cc2, mbcells)
```

This time, the homology of the resulting cubical complex is given by the Betti numbers

```
julia> homology(cc2bnd)
3-element Vector{Int64}:
 1
 0
 1
```

This complex is connected and has no holes, but it does have one cavity. As shown, these observations translate into the Betti numbers $\beta_0(X) = 1$ and $\beta_1(X) = 0$, as well as $\beta_2(X) = 1$.

Beyond these simple illustrative examples, homology can be a useful tool in a variety of applied settings. For example, it can be used to quantify the evolution of material microstructures during phase separation processes, see for example [GMW05].

4.2 Relative Homology

For the definition of the Conley index of an isolated invariant set another notion of homology is essential, namely relative homology. For this, we assume that X is a Lefschetz complex, and $Y \subset X$ is a closed subset. In other words, for every cell in Y, all of its faces are contained in Y as well. Then relative homology defines a sequence of groups $H_k(X, Y)$ for $k \in \mathbb{Z}$ which basically measures the topological properties of X if the subset Y is contracted to a point and then forgotten.

This admittedly very vague definition can be made precise in a number of ways. Two of these can easily be described:

- We have already seen that any locally closed subset of a Lefschetz complex is again a Lefschetz complex. Since the subset $Y \subset X$ is closed, its complement $X \setminus Y$ is open, and hence locally closed as well. Thus, the complement $X \setminus Y$ is again a Lefschetz complex. It has been shown in [MB09, Theorem 3.5] that then

$$H_k(X, Y) \cong H_k(X \setminus Y) \quad \text{for all} \quad k \in \mathbb{Z}.$$

  In other words, the relative homology of the pair (X, Y) is just the regular homology of the Lefschetz complex given by the set $X \setminus Y$, with the incidence coefficient map inherited from X.

- On a more topological level, one can also think of the relative homology of (X, Y) in the following way. In the complex X, identify all cells in Y to a single point, in the sense of the quotient space X/Y defined in a standard topology course. Then one can show that

$$H_k(X, Y) \cong \tilde{H}_k(X/Y) \quad \text{for all} \quad k \in \mathbb{Z},$$

  where $\tilde{H}_k(Z)$ denotes the reduced homology of a space Z. While the details of this latter notion of homology can be found in [Mun84, Section 7], for our purposes it suffices to note that the Betti numbers in reduced homology can be obtained from the ones in regular homology by decreasing the 0-th Betti number by 1, and keeping all other Betti numbers unchanged.

The precise mathematical definition of relative homology can be found in [Mun84, Section 9], and it is briefly introduced in the following. Since the k-th chain group of a Lefschetz complex consists of all formal linear combinations of k-dimensional cells, one can consider the vector space $C_k(Y)$ as a subspace of $C_k(X)$. Thus, it makes sense to form the quotient groups

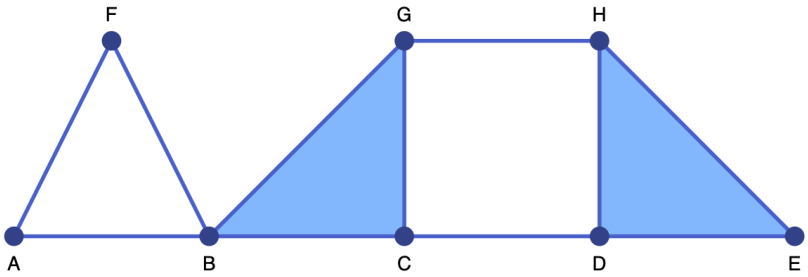


Figure 4.4: Sample simplicial complex

$$C_k(X,Y) = C_k(X)/C_k(Y)$$

as in linear algebra. Moreover, if one considers a class $[x] \in C_k(X,Y)$ represented by some $x \in C_k(X)$, then the definition

$$\partial[x] = [\partial x] \in C_{k-1}(X,Y) \quad \text{for} \quad [x] \in C_k(X,Y)$$

gives a well-defined linear map $\partial : C_k(X,Y) \to C_{k-1}(X,Y)$ which satisfies $\partial \circ \partial = 0$. In other words, the collection $(C_k(X,Y))_{k\in\mathbb{Z}}$ equipped with this boundary operator ∂ is a chain complex, and its associated homology groups $H_k(X,Y)$ are called the relative homology groups of the pair (X,Y). Notice that by forming the quotient spaces $C_k(X)/C_k(Y)$, the chains in the subspace are all identified and set to zero, as mentioned earlier.

In ConleyDynamics.jl, relative homology can be computed using `relative_homology`. There are two possible ways to invoke this function:

- The method `relative_homology(lc::LefschetzComplex, subc::Cells)` expects a Lefschetz complex `lc` which represents X, together with a list of cells `subc`. The closure of this cell list determines the closed subcomplex Y.
- The method `relative_homology(lc::LefschetzComplex, subc::Cells, subc0::Cells)` expects an ambient Lefschetz complex specified by the argument `lc`. The Lefschetz complex X is then the closure of the cell list `subc`, while the subcomplex Y is given by the closure of the cell list `subc0`. These closures are automatically computed by the function.

Both versions of `relative_homology` return the relative homology as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`. Notice also that the necessary cell list arguments have to be variables of the type `Cells = Union{Vector{Int},Vector{String}}`, i.e., they can be given in either label or index form. Finally, in both versions of this function, one can also specify the optional argument `algorithm`, which selects the method used for computing relative homology. It has the same options as `homology`.

In order to briefly illustrate the different usages of the command `relative_homology`, we consider again the simplicial complex with vertices `A` to `H` shown in the figure of the sample simplicial complex, which can be generated using the commands

```
labels2 = ["A","B","C","D","E","F","G","H"]
simplices2 = [["A","B"],["A","F"],["B","F"],["B","C","G"],["D","E","H"],["C","D"],["G","H"]]
sc2 = create_simplicial_complex(labels2,simplices2)
```

If we identify the vertices A and E, then an additional loop is created along the bottom of the original simplicial complex. This leads to the following relative homology:

```
julia> relative_homology(sc2, ["A","E"])
3-element Vector{Int64}:
 0
 3
 0
```

Note that the 0-th Betti number becomes zero, since these identified vertices are considered as zero in the chain group $C_0(X,Y)$. On the other hand, if we consider the boundary of the triangle DEH as the subcomplex Y, then one obtains:

```
julia> relative_homology(sc2, ["DE","DH","EH"])
3-element Vector{Int64}:
 0
 2
 1
```

Again, the 0-th Betti number is reduced by one. But this time, the first Betti number does not change, as no new holes are created. Nevertheless, collapsing the boundary of the triangle to a point does create a cavity, and therefore the 2-nd Betti number is now one. One can also just consider the closure of the triangle BCG as a Lefschetz complex X, and use its boundary as subcomplex Y. In this case we get:

```
julia> relative_homology(sc2, ["BCG"], ["BC","BG","CG"])
3-element Vector{Int64}:
 0
 0
 1
```

This is the reduced homology of a two-dimensional sphere, which is the topological space obtained from the quotient space X/Y. As our final example, consider the closed edge AB as Lefschetz complex X, and the vertex B as subcomplex Y, then the relative homology of the pair (X,Y) is given by

```
julia> relative_homology(sc2, ["AB"], ["B"])
3-element Vector{Int64}:
 0
 0
 0
```

In this case, all Betti numbers are zero. This can also be seen by recalling that this relative homology is isomorphic to the relative homology of the two-element Lefschetz complex which consists only of the edge AB and the vertex A:

```
julia> homology(lefschetz_subcomplex(sc2, ["A","AB"]))
2-element Vector{Int64}:
 0
 0
```

Note that one obtains a Betti number vector of length two, since this subcomplex has dimension one.

4.3 Persistent Homology

Even though the notion of persistence is not strictly necessary for the study of combinatorial topological dynamics, the package ConleyDynamics.jl provides rudimentary support for the computation of persistence intervals for filtrations of Lefschetz complexes. A detailed introduction to persistence can be found in the book [EH10], and we briefly provide an intuitive definition and some examples below.

Persistent homology is concerned with the creation and destruction of topological features in a sequence of nested Lefschetz complexes. More precisely, consider the sequence of Lefschetz complexes

$$X^{(1)} \subset X^{(2)} \subset \ldots \subset X^{(n)},$$

where we assume that $X^{(k)}$ is closed in $X^{(n)}$ for all $1 \leq k < n$. This is called a filtration of Lefschetz complexes. As we have seen above, every one of the complexes $X^{(k)}$ has associated homology groups, and the notion of persistence is concerned with how these groups change as k is increased from 1 to n. More precisely, this is based on the following intuition:

- For $k = 1$, the Betti numbers of the Lefschetz complex $X^{(1)}$ describe how many nontrivial holes the complex has in each dimension. Each of these holes is represented by a cycle which generates the associated homology class. One then says that each of these homology classes is born at $k = 1$.
- As one passes from the complex $X^{(k)}$ to the complex $X^{(k+1)}$, for $k = 1, \ldots, n-1$, these Betti numbers can change in a number of ways:
  - A new homology class is created in $X^{(k+1)}$, which leads to an increase in the corresponding Betti number. As before, this means that a new homology class is born at level $k + 1$.
  - A homology class that was present in $X^{(k)}$ is no longer present in $X^{(k+1)}$. On the one hand, this could be the result of the merging of two separate topological features, such as for example two separate connected components of the complex $X^{(k)}$ which become one connected component in $X^{(k+1)}$. On the other hand, the corresponding hole in the complex could have been filled in through the introduction of cells in the set difference $X^{(k+1)} \setminus X^{(k)}$. In either case, we say that the homology class died at level $k + 1$.
- The persistent homology associated with this filtration then consists of a collection of persistence intervals for each dimension. All of these intervals are of the form $[b, d)$, where b denotes the birth time and d the death time of a topological feature. Note that some homology classes might still be present in the homology of the final Lefschetz complex $X^{(n)}$, and in this case one obtains an interval of the form $[b, \infty)$, i.e., the feature never dies.

With the above intuitive description one usually can work out the collection of persistence intervals for small and simple examples. There is, however, one final ambiguity that has to be resolved. Suppose that two topological features are born at times b_1 and b_2, and they merge to a single feature at time d. Which of these

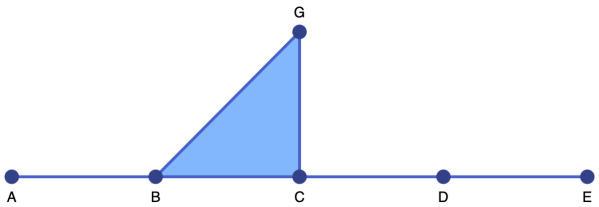


Figure 4.5: The 1-st complex in the filtration

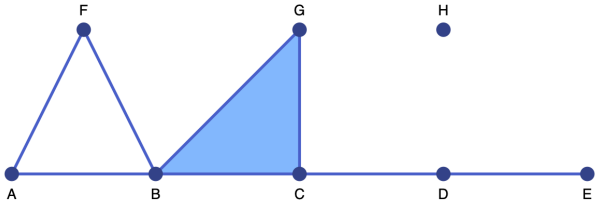


Figure 4.6: The 2-nd complex in the filtration

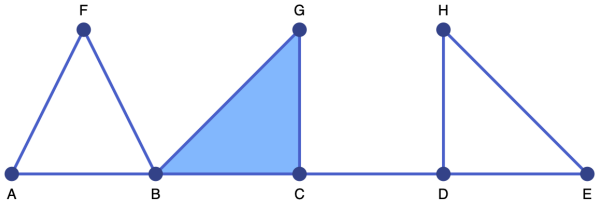


Figure 4.7: The 3-rd complex in the filtration

survives into the next complex, and which dies? In this situation, the elder rule applies, which says that the older feature persists. Thus, if $b_1 \geq b_2$, then one obtains the persistence interval $[b_1, d)$, while the death time e in the interval $[b_2, e)$ will be determined by a later level, i.e., we have $e > d$.

In order to illustrate this informal definition, we consider the filtration given by the four simplicial complexes shown in the four figures of the complexes in the filtration. All of these are subcomplexes of, and the last one is equal to, one of our earlier examples.

In this simple example, the persistence intervals in each dimension can be determined easily:

- **Dimension 0**: The first complex has one connected component. A second component, namely the vertex H, is added in $X^{(2)}$. Both of these components merge in $X^{(3)}$, and no additional components

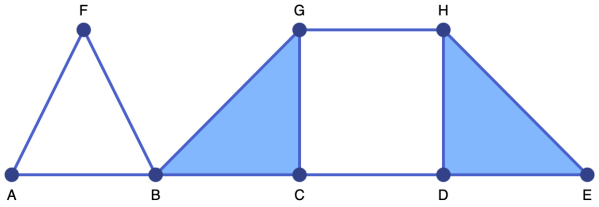


Figure 4.8: The 4-th complex in the filtration

are created. In view of the elder rule, this gives the two persistence intervals $[1, \infty)$ and $[2, 3)$.

- **Dimension 1**: The first hole is created in $X^{(2)}$, given by the cycle $\mathrm{AB} + \mathrm{AF} + \mathrm{BF}$. Moreover, in $X^{(3)}$ the hole determined by the chain $\mathrm{DE} + \mathrm{DH} + \mathrm{EH}$ is added to the mix. Finally, in $X^{(4)}$ the latter hole is removed through filling in the triangle DEH, while the hole bounded by the cycle $\mathrm{CD} + \mathrm{DH} + \mathrm{CG} + \mathrm{GH}$ is created. This gives the unbounded persistence intervals $[2, \infty)$ and $[4, \infty)$, as well as the bounded one $[3, 4)$.
- **Dimension 2**: None of the complexes form any cavities, and therefore there are no persistence intervals in dimension two.

In ConleyDynamics.jl, there are two functions that provide basic persistence functionality:

- The function `persistent_homology` computes the persistence intervals, and it is usually invoked using the command `pinf, ppairs = persistent_homology(lc, filtration)`. It expects two arguments: The first is an underlying Lefschetz complex `lc` of type `LefschetzComplex`, which has to be the complex $X^{(n)}$ in the above notation. The second argument `filtration` is of type `Vector{Int}` and has to have length `lc.ncells`. For each cell index, it contains the integer level k of the the first complex $X^{(k)}$ in which the cell appears. The function returns two vectors, `pinf` and `ppairs`, each of which have length `1 + lc.dim`, and which contain the following information:
  - `pinf[k]` contains a vector of birth times for all unbounded persistence intervals in dimension $k-1$. It is an empty vector if no such intervals exist.
  - `ppairs[k]` contains a vector of pairs `(b,d)` for each of the bounded persistence interval $[b, d)$ in dimension $k - 1$. Again, this vector is empty if no such intervals exist.

  Note that the integer vector `filtration` has to contain every integer between 1 and n at least once, and only these integers. An error is raised if this is not the case, or if the resulting subcomplexes $X^{(k)}$ are not closed. This function also accepts the optional argument `algorithm`, which selects the method used for computing persistent homology. It has the same options as `homology`. For filtrations with a few large levels, the parallelized algorithm `pmorse` can be faster than `morse` if Julia is started with multiple threads, see the section When Parallelization Pays Off for an example.
- The function `lefschetz_filtration` is meant to simplify the construction of the argument `filtration`, especially in the situation that $X^{(n)}$ is a proper subcomplex of some ambient Lefschetz complex X. The function is invoked using the form `lcsub, filtration = lefschetz_filtration(lc, partialfil)`. The argument `lc` contains the Lefschetz complex X. The argument `partialfil` has the type `Vector{Int}` and is of length `lc.ncells`. For each cell index `j` it contains an integer `partialfil[j]` between 0 and n. If the cell appears first in complex $X^{(k)}$, then `partialfil[j] = k`. This time, however, not all cells have to be specified, since the function automatically computes the complex closure at every level. Clearly, this means that the final complex is the closure of all cells with positive `partialfil`-values, and this can be a proper subcomplex of `lc`. The function therefore returns this subcomplex `lcsub`, together with a filtration `filtration` which satisfies the requirements of the function `persistent_homology`.

We close this section with two examples illustrating these functions. As first example, we consider the filtration given above, which consists of four simplicial complexes. In this case the persistence can be computed using the commands:

```
labels     = ["A","B","C","D","E","F","G","H"]
simplices  = [["A","B"],["A","F"],["B","F"],["B","C","G"],["D","E","H"],["C","D"],["G","H"]]
sc         = create_simplicial_complex(labels,simplices)
filtration = [1,1,1,1,1,2,1,2,
              1,2,1,2,1,1,1,1,3,3,4,
              1,4]
pinf, ppairs = persistent_homology(sc, filtration)
```

The first three lines establish the simplicial complex $X^{(4)}$, while the next command defines the filtration. For easier reading, we used different lines for the cells of each dimension. Finally, the last command computes the persistence intervals. The unbounded ones have the birth times

```
julia> pinf
3-element Vector{Vector{Int64}}:
 [1]
 [2, 4]
 []
```

while the bounded ones are given by

```
julia> ppairs
3-element Vector{Vector{Tuple{Int64, Int64}}}:
 [(2, 3)]
 [(3, 4)]
 []
```

This is in accordance with our earlier observations. Notice also that the Betti numbers of the final complex $X^{(4)}$ in the filtration can easily be determined via

```
julia> length.(pinf)
3-element Vector{Int64}:
 1
 2
 0
```

With the second example we illustrate the use of the function `lefschetz_filtration`. For this, suppose that the ambient Lefschetz complex X is the final simplicial complex in the filtration of the previous example. Within this complex, we consider the following new filtration:

- The complex $X^{(1)}$ is the closure of $\{\mathrm{CD}, \mathrm{GH}\}$.
- The complex $X^{(2)}$ is the closure of $X^{(1)} \cup \{\mathrm{BC}, \mathrm{BG}, \mathrm{DEH}\}$.
- The complex $X^{(3)}$ is the closure of $X^{(2)} \cup \{\mathrm{BCG}\}$.

The persistent intervals of this filtration can be determined using the following commands:

```
labels    = ["A","B","C","D","E","F","G","H"]
simplices = [["A","B"],["A","F"],["B","F"],["B","C","G"],["D","E","H"],["C","D"],["G","H"]]
sc        = create_simplicial_complex(labels,simplices)
tmpfil    = fill(Int(0),sc.ncells)
tmpfil[sc.indices["CD"]]  = 1
tmpfil[sc.indices["GH"]]  = 1
tmpfil[sc.indices["BC"]]  = 2
tmpfil[sc.indices["BG"]]  = 2
tmpfil[sc.indices["DEH"]] = 2
tmpfil[sc.indices["BCG"]] = 3
scsub, filtration = lefschetz_filtration(sc, tmpfil)
psinf, pspairs = persistent_homology(scsub, filtration)
```

The unbounded persistence intervals have birth times

```
julia> psinf
3-element Vector{Vector{Int64}}:
 [1]
 [2]
 []
```

while the bounded persistence intervals are

```
julia> pspairs
3-element Vector{Vector{Tuple{Int64, Int64}}}:
 [(1, 2)]
 []
 []
```

Their correctness can immediately be established.

As we mentioned earlier, more information on persistence can be found in [EH10], which also contains a detailed discussion of the implemented persistence algorithm in the context of simplicial complexes. Further examples of persistence computations for general Lefschetz complexes are given in [DW18].

4.4 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[DW18] P. Dłotko and T. Wanner. Rigorous cubical approximation and persistent homology of continuous functions. Computers & Mathematics with Applications **75**, 1648–1666 (2018).

[EH10] H. Edelsbrunner and J. L. Harer. Computational Topology (American Mathematical Society, Providence, 2010).

[GMW05] M. Gameiro, K. Mischaikow and T. Wanner. Evolution of pattern complexity in the Cahn-Hilliard theory of phase separation. Acta Materialia **53**, 693–704 (2005).

[HMMN14] S. Harker, K. Mischaikow, M. Mrozek and V. Nanda. Discrete Morse theoretic algorithms for computing homology of complexes and maps. Foundations of Computational Mathematics **14**, 151–184 (2014).

[KMM04] T. Kaczynski, K. Mischaikow and M. Mrozek. Computational Homology. Vol. 157 of Applied Mathematical Sciences (Springer-Verlag, New York, 2004).

[Lef42] S. Lefschetz. Algebraic Topology. Vol. 27 of American Mathematical Society Colloquium Publications (American Mathematical Society, New York, 1942).

[MB09] M. Mrozek and B. Batko. Coreduction homology algorithm. Discrete & Computational Geometry **41**, 96–118 (2009).

[Mun84] J. R. Munkres. Elements of Algebraic Topology (Addison-Wesley, Menlo Park, 1984).

[GUD24] GUDHI Project. GUDHI User and Reference Manual. 3.10.1 Edition (GUDHI Editorial Board, 2024).

Chapter 5

Conley Theory

The main motivation for ConleyDynamics.jl is the development of an accessible tool for studying the global dynamics of multivector fields on Lefschetz complexes. Having already discussed the latter, we now turn our attention to multivector fields and their global dynamics. This involves a detailed discussion of multivector fields, isolated invariant sets, their Conley index, as well as Morse decompositions and connection matrices. We also briefly compare the different connection matrix algorithms which are provided by the package. Finally, we address Forman's Morse complex and the functions that can be used to work with it.

5.1 Multivector Fields

Suppose that X is a Lefschetz complex as described in Lefschetz Complexes, see in particular the definition in Basic Lefschetz Terminology. Assume further that the Lefschetz complex is defined over a field F, which is either the rational numbers $\mathbb{Q}$ or a finite field of prime order. Then a multivector field on X is defined as follows.

Definition: Multivector field

A multivector field $\mathcal{V}$ on a Lefschetz complex X is a partition of X into locally closed sets.

Recall from our detailed discussion in Basic Lefschetz Terminology that a set $V \subset X$ is called locally closed if its mouth $\operatorname{mo} V = \operatorname{cl} V \setminus V$ is closed, where closedness in turn is defined via the face relation in a Lefschetz complex. This implies that for each multivector $V \in \mathcal{V}$ the relative homology $H_*(\operatorname{cl} V, \operatorname{mo} V)$ is well-defined, and it allows for the following classification of multivectors:

- A critical multivector is a multivector for which $H_*(\operatorname{cl} V, \operatorname{mo} V) \neq 0$.
- A regular multivector is a multivector for which $H_*(\operatorname{cl} V, \operatorname{mo} V) = 0$.

Since a multivector is locally closed, it is a Lefschetz subcomplex of X as well, and we have already seen that its Lefschetz homology satisfies $H_*(V) \cong H_*(\operatorname{cl} V, \operatorname{mo} V)$. For more details, see Relative Homology.

The above classification of multivectors is motivated by the case of classical Forman vector fields. These are a special case of multivector fields, in that they also form a partition of the underlying Lefschetz complex. This time, however, there are only two types of multivectors:

- A critical cell is a multivector consisting of exactly one cell of the Lefschetz complex. One can easily see that in this case the k-th homology group is isomorphic to F, as long as the cell has dimension k. All other homology groups vanish. Thus, every critical cell is a critical multivector.

- A Forman arrow is a multivector consisting of two cells σ^- and σ^+, where σ^- is a facet of σ^+. In other words, one has to have $\kappa(\sigma^+, \sigma^-) \neq 0$, which also implies that $1 + \dim \sigma^- = \dim \sigma^+$. One can show that all homology groups of a Forman arrow are zero, and therefore it is a regular multivector.

In ConleyDynamics.jl, multivector fields can be created in two different ways. The direct method is to specify all multivectors of length larger than one in an array of type `Vector{Vector{Int}}` or `Vector{Vector{String}}`, depending on whether the involved cells are referenced via their indices or labels. Recall that it is easy to convert between these two forms using the command `convert_cellsubsets`. The subsets specified by the vector entries have to be disjoint. They do not, however, have to exhaust the underlying Lefschetz complex X. Any cells that are not part of a specified multivector will be considered as one-element critical cells. This reduces the size of the representation in many situations.

For large Lefschetz complexes, the above method becomes quickly impractical. In such a case it is easier to determine a multivector field indirectly, through a mechanism involving dynamical transitions. This is based on the following result.

Theorem: Multivector fields via dynamical transitions

Let X be a Lefschetz complex and let $\mathcal{D}$ denote an arbitrary collection of subsets of X. Then there exists a uniquely determined minimal multivector field $\mathcal{V}$ which satisfies the following:

- For every $D \in \mathcal{D}$ there exists a $V \in \mathcal{V}$ such that $D \subset V$.

Note that the sets in $\mathcal{D}$ do not have to be disjoint, and their union does not have to exhaust X. One can think of the sets in $\mathcal{D}$ as all allowable dynamical transitions.

The above result shows that as long as one has an idea about the transitions that a system has to be allowed to do, one can always find a smallest multivector field which realizes them. Needless to say, if too many transitions are specified, then it is possible that the result leads to the trivial multivector field $\mathcal{V} = \{X\}$. In most cases, however, the resulting multivector field is more useful. See also the examples later in this section of the manual.

The package ConleyDynamics.jl provides a number of functions for creating and manipulating multivector fields on Lefschetz complexes:

- The function `create_mvf_hull` implements the above theorem on dynamical transitions. It expects two input arguments: A Lefschetz complex `lc`, as well as a vector `mvfbase` that defines the dynamical transitions in $\mathcal{D}$. The latter has to have type `Vector{Vector{Int}}` or `Vector{Vector{String}}`.
- The function `mvf_forward_orbit` determines the forward orbit of a given cell or set of cells, with respect to an underlying multivector field. More precisely, it returns all multivectors that can be reached from the source cells, and critical cells in the forward orbit are included as singletons. This function has four different signatures:

  1. `mvf_forward_orbit(lc::LefschetzComplex, mvf::CellSubsets, cs::Cells)`
  2. `mvf_forward_orbit(lc::LefschetzComplex, mvf::CellSubsets, c::Cell)`
  3. `mvf_forward_orbit(lc::LefschetzComplex, mvf::CellSubsets, cs::Cells, n::Int)`
  4. `mvf_forward_orbit(lc::LefschetzComplex, mvf::CellSubsets, c::Cell, n::Int)`

  In all cases, the underlying Lefschetz complex is `lc`, and `mvf` denotes the multivector field. In the first and third case, `cs` is a vector of starting cells, while in the second and fourth case the orbit starts only

at a single cell `c`. Finally, the first two methods determine the complete forward orbit, while the last two only consider at most `n` transitions. While the forward orbit is given as a collection of multivectors, this can easily be converted to a collection of cells using `cellsubsets_to_cells`.

- The function `mvf_backward_orbit` determines the backward orbit of a given cell or set of cells, i.e., it finds all cells which are transported in forward time to the given source cells. This function has also four signatures, completely analogous to the previous one.

- The function `mvf_neighborhood` computes a multivector neighborhood of a cell or set of cells. It returns all multivectors of the given multivector field that form a neighborhood of the given cells with a collar width of `n`. More precisely, for `n = 0` one obtains the multivectors intersecting the cells in `sources`, while for `n >= 1` the function returns all multivectors from level `n-1`, together with all multivectors whose closure intersects the closure of the level `n-1` neighborhood. Critical cells in this neighborhood will be returned as singletons.

- The function `mvf_length` returns the number of all multivectors of a given multivector field. This not only includes the multivectors explicitly listed in the argument vector `mvf`, but also the number of all singletons, which are always critical multivectors.

- The function `mvf_critical` determines all critical multivectors of the specified multivector field. This includes both the implied singletons, as well as the multivectors explicitly listed in the argument vector `mvf` which have nontrivial homology.

- The function `mvf_regular` returns all regular multivectors of the specified multivector field, i.e., all multivectors whose homology groups are all trivial.

- The function `mvf_information` displays basic information about a given multivector field. It expects both a Lefschetz complex and a multivector field as arguments, and returns a `Dict{String,Any}` with the information. The keys of this dictionary are as follows:

  - `"N mv"`: Number of multivectors
  - `"N critical"`: Number of critcal multivectors
  - `"N regular"`: Number of regular multivectors
  - `"Lengths critical"`: Length distribution of critical multivectors
  - `"Lengths regular"`: Length distribution of regular multivectors

  In the last two cases, the dictionary entries are vectors of pairs `(length,frequency)`, where each pair indicates that there are `frequency` multivectors of length `length`.

- `mvf_is_acyclic` determines whether a multivector field forms an acyclic partition of the underlying Lefschetz complex or not, and returns the corresponding boolean value. If the multivector field is acyclic, then it is gradient-like, otherwise it contains nontrivial recurrent sets.

- The function `extract_multivectors` expects as input arguments a Lefschetz complex and a multivector field, as well as a list of cells specified as a `Vector{Int}` or a `Vector{String}`. It returns a list of all multivectors that contain the specified cells.

- The function `create_planar_mvf` creates a multivector field which approximates the dynamics of a given planar vector field. It expects as arguments a two-dimensional Lefschetz complex, a vector of planar coordinates for the vertices of the complex, as well as a function which implements the vector field. It returns a multivector field based on the dynamical transitions induced by the vector field directions on the vertices and edges of the Lefschetz complex. While the complex does not have to be a triangulation, it is expected that the one-dimensional cells are straight line segments between the two boundary vertices.

- The utility function `planar_nontransverse_edges` expects the same arguments as the previous one, and returns a list of nontransverse edges as `Vector{Int}`, which contains the corresponding edge indices. The optional parameter `npts` determines how many points along an edge are evaluated for the transversality check.
- The function `create_spatial_mvf` creates a multivector field which approximates the dynamics of a given spatial vector field. While it expects the same arguments as its planar counterpart, the Lefschetz complex has to be of one of the following two types:
  - The Lefschetz complex is a tetrahedral mesh of a region in three dimensions, i.e., it is a simplicial complex.
  - The Lefschetz complex is a three-dimensional cubical complex, i.e., it is the closure of a collection of three-dimensional cubes in space.

  In the second case, the vertex coordinates can be slightly perturbed from the original position in the cubical lattice, as long as the overall structure of the complex stays intact. In that case, the faces are interpreted as Bezier surfaces with straight edges.

All of these functions will be illustrated in more detail in the examples which are presented later in this section. See also the Tutorial for another planar vector field analysis.

5.2 Invariance and Conley Index

A multivector field induces dynamics on the underlying Lefschetz complex through the iteration of a multivalued map. This flow map is given by

$$\Pi_{\mathcal{V}}(x) = \operatorname{cl} x \cup [x]_{\mathcal{V}} \qquad \text{for all} \qquad x \in X$$

where $[x]_{\mathcal{V}}$ denotes the unique multivector in $\mathcal{V}$ which contains x. The definition of the flow map shows that the induced dynamics combines two types of behavior:

- From a cell x, it is always possible to flow towards the boundary of the cell, i.e., to any one of its faces.
- In addition, it is always possible to move freely within a multivector.

The multivalued map $\Pi_{\mathcal{V}} : X \multimap X$ naturally leads to a solution concept for multivector fields. A path is a sequence $x_0, x_1, \ldots, x_n \in X$ such that $x_k \in \Pi_{\mathcal{V}}(x_{k-1})$ for all indices $k = 1, \ldots, n$. Paths of bi-infinite length are called solutions. More precisely, a solution of the combinatorial dynamical system induced by the multivector field is then a map $\rho : \mathbb{Z} \to X$ which satisfies $\rho(k+1) \in \Pi_{\mathcal{V}}(\rho(k))$ for all $k \in \mathbb{Z}$. We say that this solution passes through the cell $x \in X$ if in addition one has $\rho(0) = x$. It is clear from the definition of the flow map that every constant map is a solution, since we have the inclusion $x \in \Pi_{\mathcal{V}}(x)$. Thus, rather than considering solutions in the above (classical) sense, we focus on a more restrictive notion.

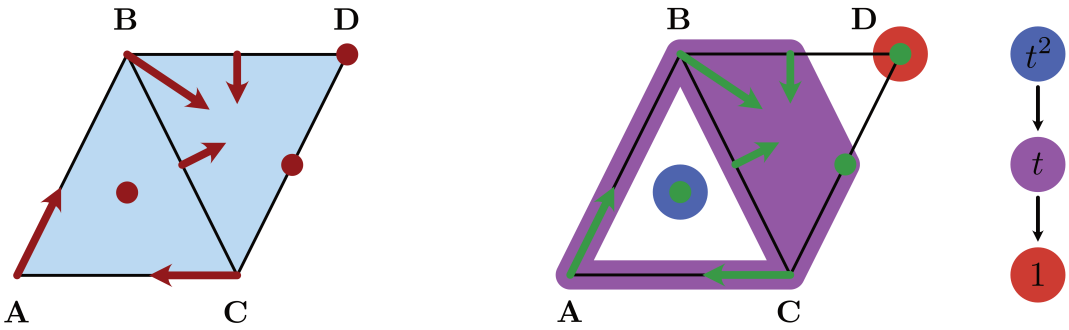


Figure 5.1: The logo multivector field

Definition: Essential solution

Let $\rho : \mathbb{Z} \to X$ be a solution for the multivector field $\mathcal{V}$. Then ρ is an essential solution, if the following holds:

- If for $k \in \mathbb{Z}$ the cell $\rho(k)$ lies in a regular multivector $V \in \mathcal{V}$, then there exist integers $\ell_1 < k < \ell_2$ for which we have $\rho(\ell_i) \notin V$ for $i = 1, 2$.

In other words, an essential solution has to leave a regular multivector both in forward and in backward time. It can, however, stay in a critical multivector for as long as it wants.

The notion of essential solution has its origin in the distinction between critical and regular multivectors. In Forman's theory, which is based on classical Morse theory, critical cells correspond to stationary solutions or equilibria of the underlying flow. Thus, it has to be possible to stay in a critical multivector for all times, whether in forward or backward time, or even for all times. On the other hand, a Forman arrow indicates prescribed non-negotiable motion, and therefore a regular multivector corresponds to motion which goes from the multivector to its mouth.

The multivector field from the package logo, which is shown in the figure of the two triangles ABC and BCD with their multivectors, consists of three critical cells, two Forman arrows, as well as one multivector which consists of four cells. Beyond the constant essential solutions in each of the three critical cells, another essential solution is the periodic orbit

$$\rho_P \text{ given by } \ldots \to \mathbf{A} \to \mathbf{AB} \to \mathbf{B} \to \mathbf{BCD} \to \mathbf{C} \to \mathbf{AC} \to \mathbf{A} \to \ldots$$

Notice that this is just one of many realizations of this particular periodic motion, since an essential solution can take many different paths through a multivector.

Using the concept of essential solutions we can now introduce the notion of invariance. Informally, we say that a subset of a Lefschetz complex is invariant if through every cell in the set there exists an essential solution which stays in the set. In other words, we have the choice of staying in the set, even though there might be other solutions that do leave. More generally, for every subset $A \subset X$ one can ask whether there are elements $x \in A$ for which there exists an essential solution which passes through x and stays in A for all times. This leads to the definition of the invariant part of A as

$$\mathrm{Inv}_{\mathcal{V}}(A) = \{x \in A \ : \text{ there exists an essential solution } \rho : \mathbb{Z} \to A \text{ through } x\}$$

It is certainly possible that the invariant part of a set is empty. If, however, the invariant part of A is all of A, i.e., if we have $\mathrm{Inv}_{\mathcal{V}}(A) = A$, then the set A is called invariant. In the context of our above logo example, the image of the essential solution ρ_P is clearly an invariant set.

Invariant sets are the fundamental building blocks for the global dynamics of a dynamical system. Yet, in general they are difficult to study. Conley realized in [Con78] that if one restricts the attention to a more specialized notion of invariance, then topological methods can be used to formulate a coherent general theory. For this, we need to introduce the notion of isolated invariant set:

Definition: Isolated invariant set

A closed set $N \subset X$ isolates an invariant set $S \subset N$, if the following two conditions are satisfied:

- Every path in N with endpoints in S is a path in S.
- We have $\Pi_{\mathcal{V}}(S) \subset N$.

An invariant set S is an isolated invariant set, if there exists a closed set N which isolates S.

It is clear that the whole Lefschetz complex X isolates its invariant part. Therefore, the set $\mathrm{Inv}_{\mathcal{V}}(X)$ is an isolated invariant set. Moreover, one can readily show that if N is an isolating set for an isolated invariant set S, then any closed set $S \subset M \subset N$ also isolates S. Thus, the closure $\mathrm{cl}\, S$ is the smallest isolating set for S. With these observations in mind, one obtains the following result from [LKMW23]:

Theorem: Characterization of isolated invariant sets

An invariant set $S \subset X$ is an isolated invariant set, if and only if the following two conditions hold:

- S is $\mathcal{V}$-compatible, i.e., it is the union of multivectors.
- S is locally closed.

In this case, the isolated invariant set S is isolated by its closure $\mathrm{cl}\, S$.

Returning to our earlier logo example, notice that the cells visited by the periodic essential solution ρ_P do not form an isolated invariant set, but rather just an invariant set. However, if we consider the larger set S_P which consists of all cells except for the cells $\mathbf{ABC}$ and $\mathbf{D}$, then we do obtain an isolated invariant set which contains the periodic orbit ρ_P.

With this characterization at hand, identifying isolated invariant sets becomes straightforward. In addition, since isolated invariant sets are locally closed, we can now also define their Conley index:

Definition: Conley index

Let $S \subset X$ be an isolated invariant set the multivalued flow map $\Pi_{\mathcal{V}}$. Then the Conley index of S is the relative (or Lefschetz) homology

$$CH_*(S) = H_*(\operatorname{cl} S, \operatorname{mo} S) \cong H_*(S)$$

In addition, the Poincare polynomial of S is defined as

$$p_S(t) = \sum_{k=0}^{\infty} \beta_k(S) t^k \,, \quad \text{where} \quad \beta_k(S) = \dim CH_k(S) \,.$$

The Poincare polynomial is a concise way to encode the homology information.

Since the Conley index is nothing more than the relative homology of the closure-mouth-pair associated with a locally closed set, one could easily use the homology functions described in Homology for its computation. However, we have included a wrapper function to keep the notation uniform. In addition, ConleyDynamics.jl contains a function which provides basic information about an isolated invariant set. These two functions can be described as follows:

- The function `conley_index` determines the Conley index of an isolated invariant set. It expects a Lefschetz complex as its first argument, while the second one has to be a list of cells which specifies the isolated invariant set, and which is either of type `Vector{Int}` or `Vector{String}`. An error is raised if the second argument does not specify a locally closed set. Since the Conley index of a locally closed set S is the Lefschetz homology of S itself, the computation only involves the cells in S, and its cost does not depend on the size of the Lefschetz complex. The function accepts two optional arguments. The argument `algorithm::String` selects the homology algorithm. The choices `"morse"`, `"pmorse"`, and `"matrix"` are as for the function `homology`, while `"rank"` computes the Betti numbers directly from the ranks of the boundary maps restricted to S. The default `"auto"` uses `"rank"` if S contains at most 100 cells, and `"morse"` otherwise. All choices return the same result, and the reason for this default is explained below. The argument `validate::Bool=true` controls whether local closedness is checked. Passing `validate=false` skips this check, which is useful if the set is known to be locally closed, for example if it is a multivector or a Morse set. For sets which are not locally closed, the result is then meaningless.

  The default `algorithm="auto"` is motivated by the different scaling of the two fastest algorithms. The `"rank"` algorithm row reduces the boundary matrix restricted to S, without any further setup. For small sets this is very fast, but due to fill-in during the row reduction its cost grows roughly quadratically with the number of cells in S. In contrast, the `"morse"` algorithm first extracts the subcomplex and constructs a Morse matching. This adds a fixed overhead, but afterwards its cost grows almost linearly. In benchmarks on cubical and simplicial complexes, `"rank"` is three to seven times faster than `"morse"` for sets with up to about ten cells, which is the typical size of a multivector. Both take about the same time for sets with 100 to 150 cells. For larger sets `"morse"` is faster by a quickly growing factor, for example by a factor of 5 to 10 for 1000 cells, and by a factor of more than 100 for 50000 cells. The threshold of 100 cells therefore keeps the advantage of `"rank"` for multivectors, while avoiding its quadratic growth for large sets such as Morse sets or large multivectors. Near the threshold the two algorithms differ by less than a factor of 1.5, so its precise value is not critical, and since the decision only uses the number of cells in S, it has no cost.

- The function `isoinvset_information` expects a Lefschetz complex `lc::LefschetzComplex`, a multivector field `mvf::CellSubsets`, as well as an isolated invariant set `iis::Cells` as its three arguments. It returns a `Dict{String,Any}` with the information. The keys of this dictionary are as follows:
  - `"Conley index"` contains the Conley index of the isolated invariant set.
  - `"N multivectors"` contains the number of multivectors in the isolated invariant set.

5.3 Morse Decompositions

We now turn our attention to the global dynamics of a combinatorial dynamical system. This is accomplished through the notion of Morse decomposition, and it requires some auxilliary definitions:

- Suppose we are given a solution $\varphi : \mathbb{Z} \to X$ for the multivector field $\mathcal{V}$. Then the long-term limiting behavior of φ can be described using the ultimate backward and forward images

$$\operatorname{uim}^{-}\varphi = \bigcap_{t\in\mathbb{Z}^{-}} \varphi\left((-\infty, t]\right) \qquad \text{and} \qquad \operatorname{uim}^{+}\varphi = \bigcap_{t\in\mathbb{Z}^{+}} \varphi\left([t, +\infty)\right).$$

  Notice that since X is finite, there has to exist a $k \in \mathbb{N}$ such that

$$\operatorname{uim}^{-}\varphi = \varphi\left((-\infty, -k]\right) \neq \emptyset \qquad \text{and} \qquad \operatorname{uim}^{+}\varphi = \varphi\left([k, +\infty)\right) \neq \emptyset.$$

- The $\mathcal{V}$-hull of a set $A \subset X$ is the intersection of all $\mathcal{V}$-compatible and locally closed sets containing A. It is denoted by $\langle A \rangle_{\mathcal{V}}$, and is the smallest candidate for an isolated invariant set which contains A.
- The α- and ω-limit sets of φ are then defined as

$$\alpha(\varphi) = \left\langle \operatorname{uim}^{-}\varphi \right\rangle_{\mathcal{V}} \qquad \text{and} \qquad \omega(\varphi) = \left\langle \operatorname{uim}^{+}\varphi \right\rangle_{\mathcal{V}}.$$

While in general the $\mathcal{V}$-hull of a set does not have to be invariant, the following result shows that for every essential solution both of its limit sets are in fact isolated invariant sets.

Theorem: Limit sets are nontrivial

Let φ be an essential solution in X. Then both limit sets $\alpha(\varphi)$ and $\omega(\varphi)$ are nonempty isolated invariant sets.

We briefly pause to illustrate these concepts in the context of the above logo example. For the periodic essential solution ρ_P, both its ultimate backward and forward images are precisely the cells visited by the solution. The $\mathcal{V}$-hull of $\operatorname{im}\rho_P$ is the set S_P which consists of all cells except the index 0 and 2 critical cells. It was already mentioned earlier that this indeed defines an isolated invariant set.

The above notions allow us to decompose the global dynamics of a multivector field. Loosely speaking, this is accomplished by separating the dynamics into a recurrent part given by an indexed collection of isolated invariant sets, and the gradient dynamics between them. This can be abstracted through the concept of a Morse decomposition.

Definition: Morse decomposition

Assume that X is an invariant set for the multivector field $\mathcal{V}$ and that $(\mathbb{P}, \leq)$ is a finite poset. Then an indexed collection $\mathcal{M} = \{M_p : p \in \mathbb{P}\}$ is called a Morse decomposition of X if the following conditions are satisfied:

- The indexed family $\mathcal{M}$ is a family of mutually disjoint, isolated invariant subsets of X.
- For every essential solution φ in X either one has $\operatorname{im}\varphi \subset M_r$ for an $r \in \mathbb{P}$ or there exist two poset elements $p, q \in \mathbb{P}$ such that $q > p$ and

$$\alpha(\varphi) \subset M_q \quad \text{and} \quad \omega(\varphi) \subset M_p.$$

The elements of $\mathcal{M}$ are called Morse sets. We would like to point out that some of the Morse sets could be empty.

Given a combinatorial multivector field $\mathcal{V}$ on an arbitrary Lefschetz complex X, there always exists a finest Morse decomposition $\mathcal{M}$. It can be found by determining those strongly connected components of the digraph associated with the multivalued flow map $\Pi_{\mathcal{V}} : X \multimap X$ which contain essential solutions. The associated Conley-Morse graph is the partial order induced on $\mathcal{M}$ by the existence of connections, and represented as a directed graph labelled with the Conley indices of the isolated invariant sets in $\mathcal{M}$ in terms of their Poincare polynomials.

In order to capture the dynamics between two subsets $A, B \subset X$ one can define the connection set from A to B as the cell collection

$$\mathcal{C}(A, B) = \{x \in X : \exists \text{ essential solution } \varphi \text{ through } x \text{ with } \alpha(\varphi) \subset A \text{ and } \omega(\varphi) \subset B\}.$$

Then $\mathcal{C}(A, B)$ is an isolated invariant set. We would like to point out, however, that the connection set can be, and in fact will be, empty in many cases.

While the Morse sets of a Morse decomposition are the fundamental building blocks for the global dynamics, there usually are many additional isolated invariant sets for the multivector field $\mathcal{V}$. Of particular interest are Morse intervals. To define them, let $I \subset \mathbb{P}$ denote an interval in the index poset. Then

$$M_I = \bigcup_{p \in I} M_p \cup \bigcup_{p,q \in I} \mathcal{C}(M_q, M_p)$$

is always an isolated invariant set. Nevertheless, not every isolated invariant set is of this form. For example, the figure of the Morse decomposition of the planar flow, which shows a simplicial complex next to a Conley-Morse graph, contains the multivector field which was discussed in [BKMW20, Figure 3]. While the underlying simplicial complex and the Forman vector field are depicted in the left panel, the associated Conley-Morse graph is shown on the right. For this combinatorial dynamical system, there exists an isolated invariant set which contains only the four Morse sets within the gray region under the graph. More details can be found in A Planar Forman Vector Field.

Morse decompositions and intervals can be easily computed and manipulated in ConleyDynamics.jl using the following commands:

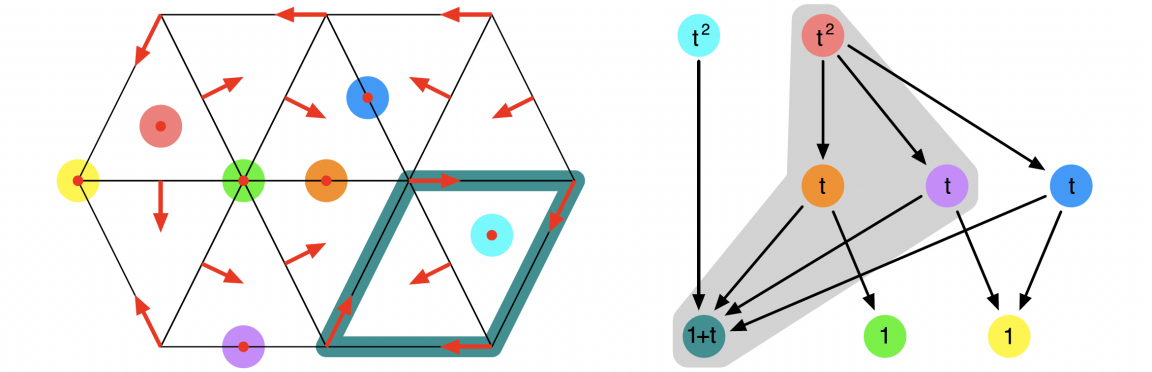


Figure 5.2: Morse decomposition of the planar flow

- The function `morse_sets` expects a Lefschetz complex and a multivector field as arguments, and returns the Morse sets of the finest Morse decomposition as a `Vector{Vector{Int}}` or `Vector{Vector{String}}`, matching the format used for the multivector field. If the optional argument `poset=true` is added, then the function also returns a matrix which encodes the Hasse diagram of the poset $\mathbb{P}$. Note that this is the transitive reduction of the full poset, i.e., it only contains necessary relations. Internally, a strongly connected component of the flow digraph is a Morse set if it contains at least two multivectors, or if it is a single multivector with nontrivial Conley index. In the latter case the index is computed with the `"auto"` algorithm and without validation, so that the cost of the whole computation is close to linear in the size of the Lefschetz complex.

- The function `morse_interval` computes the isolated invariant set for a Morse set interval. The three input arguments are the underlying Lefschetz complex, a multivector field, and a collection of Morse sets. The latter should be determined using the function `morse_sets`. The function returns the smallest isolated invariant set which contains the Morse sets and their connections as a `Vector{Int}`. The result can be converted to label form using `convert_cells`.

- The function `restrict_dynamics` restricts a multivector field to a Lefschetz subcomplex. The function expects three arguments: A Lefschetz complex `lc`, a multivector field `mvf`, and a subcomplex of the Lefschetz complex which is given by the locally closed set represented by `lcsub`. It returns the associated Lefschetz subcomplex `lcreduced` and the induced multivector field `mvfreduced` on the subcomplex. The multivectors of the new multivector field are the intersections of the original multivectors and the subcomplex. An error is raised if `lcsub` is not locally closed. This check can be skipped with the optional argument `validate=false`, for example if `lcsub` was computed with `morse_interval`. The computational cost is linear in the size of the Lefschetz complex and the multivector field.

- Finally, the function `remove_exit_set` removes the exit set for a multivector field on a Lefschetz subcomplex. It is assumed that the Lefschetz complex `lc` is a topological manifold and that `mvf` contains a multivector field that is created via either `create_planar_mvf` or `create_spatial_mvf`. The function identifies cells on the boundary at which the flows exits the region covered by the Lefschetz complex. If this exit set is closed, one has found an isolated invariant set and the function returns a Lefschetz complex `lcr` restricted to it, as well as the restricted multivector field `mvfr`. If the exit set is not closed, a warning is displayed and the function returns the restricted Lefschetz complex and multivector field obtained by removing the closure of the exit set. In the latter case, unexpected results might be obtained.

The first two of these functions rely heavily on the Julia package Graphs.jl.

5.4 Connection Matrices

While a Morse decomposition represents the basic structure of the global dynamics of a combinatorial dynamical system, it does not directly provide more detailed information about the dynamics between them – except for the poset order on the Morse sets. But which of the associated connecting sets actually have to be nonempty? The algebra behind this question is captured by the connection matrix. The precise notion of connection matrix was introduced in [Fra89], see also [HMS21], as well as the book [MW25] which treats connection matrices specifically in the setting of multivector fields and provides a precise definition of connection matrix equivalence, even across varying posets.

Since the precise definition of a connection matrix is beyond the scope of this manual, we only state what it is as an object, what its main properties are, and how it can be computed in ConleyDynamics.jl. Assume therefore that we are given a Morse decomposition $\mathcal{M}$ of an isolated invariant set S. Then the connection matrix is a linear map

$$\Delta \ : \ \bigoplus_{q\in\mathbb{P}} CH_*(M_q) \to \bigoplus_{p\in\mathbb{P}} CH_*(M_p),$$

i.e., it is a linear map which is defined on the direct sum of all Conley indices of the Morse sets in the Morse decomposition. One usually writes the connection matrix Δ as a matrix in the form $\Delta = (\Delta(p,q))_{p,q\in\mathbb{P}}$, which is indexed by the poset $\mathbb{P}$, and where the entries $\Delta(p,q) : CH_*(M_q) \to CH_*(M_p)$ are linear maps between homological Conley indices. If I denotes an interval in the poset $\mathbb{P}$, then one further defines the restricted connection matrix

$$\Delta(I) \ = \ (\Delta(p,q))_{p,q\in I} \ : \ \bigoplus_{p\in I} CH_*(M_p) \to \bigoplus_{p\in I} CH_*(M_p).$$

Any connection matrix Δ has the following fundamental properties:

- The matrix Δ is strictly upper triangular, i.e., if $\Delta(p,q) \neq 0$ then $p < q$.
- The matrix Δ is a boundary operator, i.e., we have $\Delta \circ \Delta = 0$, and Δ maps k-th level homology to $(k-1)$-st level homology for all $k \in \mathbb{Z}$.
- For every interval I in $\mathbb{P}$ we have

  $$H_*\Delta(I) \ = \ \ker \Delta(I)/\mathrm{im}\, \Delta(I) \ \cong \ CH_*(M_I).$$

  In other words, the Conley index of a Morse interval can be determined via the homology of the associated connection matrix minor $\Delta(I)$.
- If $\{p,q\}$ is an interval in $\mathbb{P}$ and $\Delta(p,q) \neq 0$, then the connection set $\mathcal{C}(M_q, M_p)$ is not empty.

We would like to point out that these properties do not characterize connection matrices. In practice, a given multivector field can have several different connection matrices. These in some sense encode different types of dynamical behavior that can occur in the system. Nonuniqueness, however, cannot be observed if the underlying system is a gradient combinatorial Forman vector field on a Lefschetz complex. These are multivector fields in which every multivector is either a singleton, and therefore a critical cell, or a two-element Forman arrow. In addition, a gradient combinatorial Forman vector field cannot have any nontrivial periodic solutions,

i.e., periodic solutions which are not constant and therefore critical cells. For such combinatorial vector fields, the following result was shown in [MW25].

Theorem: Uniqueness of connection matrices

If $\mathcal{V}$ is a gradient combinatorial Forman vector field and $\mathcal{M}$ its finest Morse decomposition, then the connection matrix is uniquely determined.

In ConleyDynamics.jl connection matrices can be computed over arbitrary finite fields or the rationals:

- The function `connection_matrix` computes a connection matrix for the multivector field `mvf` on the Lefschetz complex `lc` over the field associated with the Lefschetz complex boundary matrix. The function returns an object of type `ConleyMorseCM`, which is further described below. If the optional argument `returnbasis=true` is given, then the function also returns a dictionary which gives the basis for the connection matrix columns in terms of the original cell labels.

At the present time, there are four different algorithms implemented for the computation of connection matrices. A specific algorithm can be selected by passing the optional argument `algorithm::String`:

- `algorithm = "DLMS"` selects the original algorithm due to Dey, Lipinski, Mrozek, and Slechta [DLMS24]. This algorithm is based on matrix reductions, and performs a full similarity transformation.
- `algorithm = "DHL"` selects the more efficient algorithm due to Dey, Haas, and Lipinski [DHL26], which does not perform a complete reduction.
- `algorithm = "HMS"` selects the algorithm due to Harker, Mischaikow, and Spendlove [HMS21], which is based on discrete Morse theory, i.e., on the use of gradient Forman vector fields to preprocess the underlying Lefschetz complex.
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions. For this to take effect, Julia has to be started with multi-threading enabled.

By default, the function `connection_matrix` uses the parallel Morse reduction algorithm `pmorse`. However, if the flag `returnbasis::Bool=true` is given the function has to choose the slower matrix-based one, that is, it automatically uses `algorithm = "DLMS"`. A detailed performance comparison of all four algorithms, including reproducible timing examples, is provided in the section Algorithm Comparisons below.

The connection matrix is returned in an object with the composite data type `ConleyMorseCM`. Its docstring is as follows:

`ConleyDynamics.ConleyMorseCM` – Type.

```
ConleyMorseCM{T}
```

Collect the connection matrix information in a struct.

The struct has the following fields:

- `matrix::SparseMatrix{T}`: Connection matrix
- `columns::Vector{Int}`: Corresponding columns in the boundary matrix
- `poset::Vector{Int}`: Poset indices for the connection matrix columns

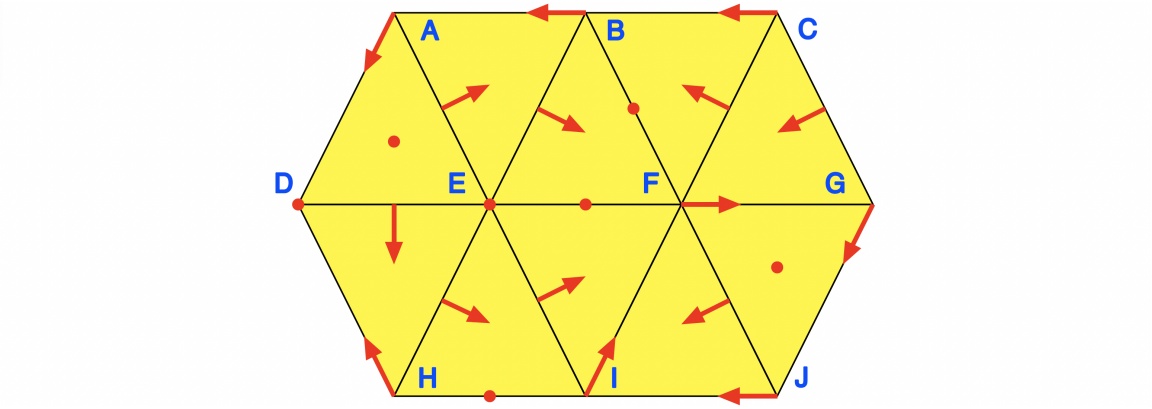


Figure 5.3: A planar simplicial complex flow

- `labels::Vector{String}`: Labels for the connection matrix columns
- `morse::Vector{Vector{String}}`: Vector of Morse sets in original complex
- `conley::Vector{Vector{Int}}`: Vector of Conley indices for the Morse sets
- `complex::LefschetzComplex`: The Conley complex as a Lefschetz complex

source

To illustrate these fields further, we briefly illustrate them for the example associated with the figure of the Morse decomposition of the planar flow, see again A Planar Forman Vector Field. For reference, the underlying simplicial complex and Forman vector field are shown in the figure of the planar simplicial complex flow, which shows the triangulated region with its arrows and critical cells.

The underlying Lefschetz complex, multivector field, and connection matrix can be computed over the field $GF(2)$ as follows:

```
lc, mvf = example_forman2d()
cm = connection_matrix(lc, mvf)
sparse_show(cm.matrix)
```

```
. . . . 1 . 1 . .
. . . . . 1 . . .
. . . . 1 1 1 . .
. . . . . . . . 1
. . . . . . . 1 .
. . . . . . . . .
. . . . . . . 1 .
. . . . . . . . .
. . . . . . . . .
```

The field `cm.poset` indicates which Morse set each column belongs to, while the field `cm.labels` shows which cell label the column corresponds to. For the example one obtains:

```
print(cm.poset)
```

```
[1, 2, 3, 3, 4, 5, 6, 7, 8]
```

```
print(cm.labels)
```

```
["D", "E", "J", "FG", "BF", "EF", "HI", "ADE", "FGJ"]
```

Note that except for the third and fourth column, all columns belong to unique Morse sets whose Conley index is a one-dimensional vector space. The third and fourth column correspond to the periodic orbit, whose Conley index is a two-dimensional vector space. The Conley indices for all eight Morse sets can be seen in the field `cm.conley`:

```
cm.conley
```

```
8-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [1, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 0, 1]
 [0, 0, 1]
```

The full associated Morse sets are list in `cm.morse`:

```
cm.morse
```

```
8-element Vector{Vector{String}}:
 ["D"]
 ["E"]
 ["F", "G", "I", "J", "FG", "FI", "GJ", "IJ"]
 ["BF"]
 ["EF"]
 ["HI"]
 ["ADE"]
 ["FGJ"]
```

As the final struct field, the entry `cm.complex` returns the connection matrix as a Lefschetz complex in its own right. This is useful for determining the Conley indices of Morse intervals. In our example, the cells of the new Lefschetz complex are given by

```
cm.complex.labels
```

```
9-element Vector{String}:
 "D"
 "E"
 "J"
 "FG"
 "BF"
 "EF"
 "HI"
 "ADE"
 "FGJ"
```

The Morse interval consisting of the two index 2 critical cells $\mathbf{ADE}$ and $\mathbf{FGJ}$ should have as Conley index the sum of the two individual indices, and the following computation demonstrates this:

```
conley_index(cm.complex, ["ADE", "FGJ"])
```

```
3-element Vector{Int64}:
 0
 0
 2
```

In contrast, since there is exactly one connecting orbit between $\mathbf{ADE}$ and $\mathbf{BF}$, the Conley index of this interval should be trivial:

```
conley_index(cm.complex, ["ADE", "BF"])
```

```
3-element Vector{Int64}:
 0
 0
 0
```

Finally, there are exactly two connecting orbits between the Morse sets $\mathbf{ADE}$ and $\mathbf{EF}$, and therefore the Conley index of this last interval is again the sum of the separate indices:

```
conley_index(cm.complex, ["ADE", "EF"])
```

```
3-element Vector{Int64}:
 0
 1
 1
```

In order to simplify the inspection of connection matrices, the function `sparse_show` has a special method for an argument of type `ConleyMorseCM`. In our example, it produces the following output:

```
sparse_show(cm)
```

```
   ┆  D   E   J  FG  BF  EF  HI ADE FGJ
───┼──────────────────────────────────
  D┆  .   .   .   .   1   .   1   .   .
  E┆  .   .   .   .   .   1   .   .   .
  J┆  .   .   .   .   1   1   1   .   .
 FG┆  .   .   .   .   .   .   .   .   1
 BF┆  .   .   .   .   .   .   .   1   .
 EF┆  .   .   .   .   .   .   .   .   .
 HI┆  .   .   .   .   .   .   .   1   .
ADE┆  .   .   .   .   .   .   .   .   .
FGJ┆  .   .   .   .   .   .   .   .   .
```

In this way, one can easily see which Morse sets correspond to the columns and rows of the connection matrix.

5.5 Algorithm Comparisons

All four algorithms implement the same mathematical task and are guaranteed to return a valid connection matrix for any given input. However, they differ considerably in their underlying approach and their computational behavior as the size of the Lefschetz complex grows. Note that when the connection matrix is not unique, different algorithms may return different — but equally valid — connection matrices.

The matrix-reduction algorithm `DLMS` works by directly reducing the boundary matrix through column and row operations, while `DHL` uses only column operations. Their structure is relatively straightforward to follow. The Morse-theory-based algorithms `HMS` and `pmorse` first compute a discrete Morse matching to simplify the complex before performing the reduction, which introduces some fixed overhead but can pay off significantly on larger inputs. The parallelized algorithm `pmorse` additionally distributes parts of the Morse matching computation across multiple threads, and can therefore benefit from starting Julia with `julia --threads N` for some suitable number of threads `N`.

The following two examples illustrate how the runtime characteristics differ depending on the problem size. The timings were obtained on a standard laptop; exact results will vary by machine, but the relative scaling behavior is representative.

Small Complexes

For a small example one can use the built-in `example_forman2d`, which creates a Lefschetz complex with 39 cells:

```
using ConleyDynamics
lc, mvf = example_forman2d()
for alg in ["DLMS", "DHL", "HMS", "pmorse"]
    println(alg, ":  ", @elapsed connection_matrix(lc, mvf, algorithm=alg), " s")
end
```

Representative timings on this example are as follows. They are medians of many runs, measured after an initial run which includes compilation:

Algorithm	Approx. time
DLMS	41 µs
DHL	27 µs
HMS	39 µs
pmorse	83 µs

On small complexes all four algorithms are fast. The matrix-reduction algorithm DHL is the fastest, since its column reductions operate on a small matrix, and DLMS and the Morse-based algorithm HMS take about the same time. The parallel algorithm pmorse is about twice as slow as HMS, since the overhead of setting up its threaded tasks does not pay off for such small inputs. This overhead is present even if Julia runs with a single thread.

Large Complexes

For a more demanding example one can use the planar flow from the section Analysis of a Planar System, with a finer triangulation than used there:

```
using ConleyDynamics, Random
Random.seed!(1234)
function planarvf(x::Vector{Float64})
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end
lc  = create_simplicial_delaunay(400, 400, 3, 50, euclidean=true)
lc  = rescale_coords(lc, -1.2, 1.2)
mvf = create_planar_mvf(lc, planarvf)
println("ncells = ", lc.ncells)
for alg in ["DLMS", "DHL", "HMS", "pmorse"]
    println(alg, ":  ", @elapsed connection_matrix(lc, mvf, algorithm=alg), " s")
end
```

This creates a Lefschetz complex with approximately 71,000 cells. Representative timings on this example are as follows. They were measured after an initial run, which includes compilation, and for the fast algorithms they are the median of several runs:

Algorithm	Approx. time
DLMS	39 s
DHL	4.7 s
HMS	0.12 s
pmorse (1 thread)	0.10 s
pmorse (4 threads)	0.10 s
pmorse (8 threads)	0.10 s

At this scale the picture changes substantially. The Morse-theory-based algorithms scale considerably more favorably than the matrix-reduction approaches: both HMS and pmorse are 40 to 50 times faster than DHL, and several hundred times faster than DLMS. Among the matrix-based algorithms, DHL — due to Dey, Haas, and Lipinski — is significantly more efficient than DLMS, which performs a complete similarity transformation of the matrix. The algorithm HMS due to Harker, Mischaikow, and Spendlove, as well as its parallelized variant

`pmorse`, first preprocess the complex using discrete Morse theory, which reduces the remaining work to a small fraction. On this example the two take about the same time, and additional threads do not noticeably reduce the run time of `pmorse`. At this size, most of the remaining time is spent in parts of the computation which are not parallelized, such as the determination of an admissible order of the cells. Timings in the range of a tenth of a second also vary by about 30 percent between runs.

The algorithm `DLMS` is the only one that supports the optional flag `returnbasis=true`, which returns the basis of each connection matrix column in terms of the original cell labels. When this flag is passed to `connection_matrix`, the function automatically selects `DLMS` regardless of the `algorithm` keyword.

When Parallelization Pays Off

The algorithm `pmorse` differs from `HMS` in two places. It constructs the Morse matching for the different levels of the underlying filtration in parallel, and it computes the columns of the reduced boundary matrix in parallel. The matching within a single level is still computed sequentially. Therefore `pmorse` can only be faster than `HMS` if the computation involves several large levels of comparable size, and if Julia is started with multiple threads.

The clearest example of this is persistent homology with a filtration that consists of a small number of large levels. The functions `homology` and `persistent_homology` use the same reduction code as `HMS` for `algorithm = "morse"`, and the same code as `pmorse` for `algorithm = "pmorse"`. The following example creates a simplicial complex with approximately 270,000 cells, together with a filtration with eight levels:

```
using ConleyDynamics, Random
Random.seed!(7)
lc, ~ = create_simplicial_rectangle(150, 150)
lcs, fv = lefschetz_filtration(lc, rand(1:8, lc.ncells))
for alg in ["morse", "pmorse"]
    persistent_homology(lcs, fv, algorithm=alg)
    println(alg, ":  ", @elapsed(persistent_homology(lcs, fv, algorithm=alg)), " s")
end
```

With eight threads, `pmorse` is about one and a half times as fast as `morse`. The following table shows representative timings, which are medians of several runs after an initial run which includes compilation, for this example as well as for the same complex with different numbers of filtration levels:

Filtration levels	`morse` (8 threads)	`pmorse` (8 threads)
4	0.79 s	0.45 s
8	0.51 s	0.35 s
16	0.50 s	0.38 s
64	0.49 s	0.37 s

With a single thread, both algorithms take the same time on these examples. The speedup is largest for four levels, where each of four threads computes the matching for one large level. In all cases the gain is moderate, since the matching within a single level, as well as the remaining parts of the computation, are performed sequentially.

For connection matrices, the levels are given by the Morse sets. The algorithm `pmorse` is therefore faster than `HMS` for flows with several large Morse sets of comparable size, such as a planar flow with a limit cycle in each of a number of identical cells. On a cubical complex with approximately 230,000 cells and 43 or 101 Morse sets, `pmorse` with eight threads was faster than `HMS` by a factor of about 1.3. However, if a single Morse set dominates, or if there is only a single level as in the computation of ordinary homology, then there is no measurable benefit from the parallelization.

5.6 Forman's Morse Complex

The package ConleyDynamics.jl also provides some support for studying Forman gradient vector fields directly, using the notions introduced in [For98b]. In this paper, Forman used the concept of a combinatorial flow Φ to study the forward orbits generated by the Forman vector field. The map Φ is a chain map which is chain homotopic to the identity, and it therefore mimics the concept of a time-1-map associated with a dynamical system. Forman shows that upon iteration this map stabilizes as a map Φ^∞, which encodes the connecting orbits in the associated combinatorial dynamical system. This stabilized combinatorial flow can then be used to find the connection matrix in this case. For more details, we refer to [MW25, Chapter 8]. The combinatorial flow and its stabilized version can be computed using the following two functions.

- `forman_comb_flow` expects a Lefschetz complex and a Forman vector field as input arguments. It returns matrix representations of Forman's combinatorial flow Φ, as well as of the associated chain homotopy Γ between the flow chain map and the identity.
- `forman_stab_flow` also requires both a Lefschetz complex and a Forman vector field as arguments. It then returns matrix representations of Forman's stabilized combinatorial flow Φ^∞, and of the associated chain homotopy Γ^∞. This time, there is a third return argument `stabilized`. This boolean flag indicates whether or not the combinatorial flow stabilized. If it did not, then the returned chain map is the last computed iterate, together with the corresponding chain homotopy. There are two possible reasons for failing stabilization: Either the underlying Forman vector field is not gradient (note that this is not checked!), or the maximal number of iterations has been reached. In the latter case, one just has to pass the optional paramter `maxit` with a larger number of allowed iterations.
- `forman_conley_maps` computes the chain maps and chain homotopies associated with the Conley complex of a Forman gradient vector field. The function expects a Lefschetz complex and a Forman gradient vector field as arguments, and it returns:
  - `pp:SparseMatrix`: The chain equivalence which maps the Lefschetz complex to the Conley complex.
  - `jj:SparseMatrix`: The chain equivalence which maps the Conley complex to the Lefschetz complex.
  - `hh:SparseMatrix`: The chain homotopy between `jj*pp` and the identity map on the Lefschetz complex.
  - `cc:Cells`: The list of critical cells of the Forman vector field which make up the Conley complex.

  These quantities are computed using Forman's stabilized combinatorial flow.
- `forman_gpaths` can be used to find gradient paths in a Forman gradient vector field. There are two methods for this function, which are accessible via multiple dispatch as follows.
  - The call `forman_gpaths(lc, fvf, x)` determines all maximal gradient paths of the Forman gradient field `fvf` on the Lefschetz complex `lc` starting at `x`. These are solution paths which consist exclusively of Forman vectors, i.e., they contain an even number of cells whose dimensions alternate between $\dim x$ and $1 + \dim x$. Every cell of dimension $\dim x$ is an arrow source which is followed by its arrow target, while every cell of dimension $1 + \dim x$ is an arrow target which is succeeded by a cell in its boundary, and which in turn is the source of a different arrow, as long as such a cell exists.
  - The call `forman_gpaths(lc, fvf, x, y)` computes all Forman gradient paths between the cells `x` and `y`. Nontrivial paths are only returned if $|\dim x - \dim y| \leq 1$. More precisely, the following paths are returned:

    * If $\dim x = \dim y - 1$, the function returns all solution paths between x and y which consist entirely of Forman arrows. All the sources have the dimension of x, and the targets the dimension of y.
    * For $\dim x = \dim y$ the function finds all solution paths p between x and y for which `p[1:end-1]` is a Forman gradient path in the above sense, and `p[end]` lies in the boundary of `p[end-1]` and is different from the cell `p[end-2]`. For this, the length of the path has to be at least 3. If on the other hand one has $x = y$, then only a 1-element path is returned.
    * Finally, if $\dim x = \dim y + 1$, then the function returns all solution paths p between x and y for which `p[2:end]` is a Forman gradient path in the sense of the second item, and `p[2]` lies in the boundary of `p[1]`.

  In all other cases an empty collection is returned.

- `forman_path_weight` expects the arguments `lc:LefschetzComplex` and `path::Cell` and computes the weight of the Forman gradient path `path` in the Lefschetz complex `lc`. It is expected that the dimensions of the first and the last cell in the path differ by at most 1. In case they have equal dimension, and the path has length larger than 1, the first cell has to be an arrow source. This function also has a second method associated with its name, in which the second argument is of type `CellSubsets`, i.e., it contains a collection of Forman gradient paths. In the case, the function returns the sum of all weights of these paths.

The following two functions allow one to extract cells of certain types from a Forman vector field:

- `forman_critical_cells` extracts all critical cells from a Forman vector field.
- `forman_all_cell_types` returns lists of cells of all three types in a Forman vector field. More precisely, it returns vectors which contain the critical cells, the arrow sources, and the arrow targets.

For analyzing or applying Forman's combinatorial flow, one needs to work with sparse vector representations of chains. These are elements of the chain groups of the underlying Lefschetz complex, which are then represented as sparse matrices consisting of exactly one column. In this form, they can be multiplied by the matrix of the combinatorial flow, which in turn determines the image of the chain under the flow. In general, these vectors are extremely sparse, and only contain a handfull of nonzero entries. ConleyDynamics.jl therefore provides the following two functions for the creation and analysis of sparse chain vectors:

- `chain_vector` is a function that simplifies the creation of a sparse vector representation of a chain. The chain can be specified by simply listing the cells in the support of the chain. If no coefficients are specified, then the chain is just the sum of these cells, each with coefficient 1. The function has several associated methods: The coefficients of the chain can be omitted or specified, and the cells can be listed in index or label form. More details can be found in the description of each method.
- `chain_support` extracts the support of a chain given as a sparse vector. In its simplest form, the function returns a `Vector{String}` which contains the labels of all the cells which have nonzero coefficients in the chain. If one passes the optional parameter `coeff=true`, then the function returns two arguments: In addition to the vector of labels as above, it also returns the vector of associated coefficients.

5.7 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[BKMW20] B. Batko, T. Kaczynski, M. Mrozek and T. Wanner. Linking combinatorial and classical dynamics: Conley index and Morse decompositions. Foundations of Computational Mathematics **20**, 967–1012 (2020).

[Con78] C. Conley. Isolated Invariant Sets and the Morse Index (American Mathematical Society, Providence, R.I., 1978).

[DHL26] T. K. Dey, A. Haas and M. Lipiński. Computing a connection matrix and persistence efficiently from a Morse decomposition. SIAM Journal on Applied Dynamical Systems **25**, 108–130 (2026).

[DLMS24] T. K. Dey, M. Lipiński, M. Mrozek and R. Slechta. Computing connection matrices via persistence-like reductions. SIAM Journal on Applied Dynamical Systems **23**, 81–97 (2024).

[For98b] R. Forman. Morse theory for cell complexes. Advances in Mathematics **134**, 90–145 (1998).

[Fra89] R. Franzosa. The connection matrix theory for Morse decompositions. Transactions of the American Mathematical Society **311**, 561–592 (1989).

[HMS21] S. Harker, K. Mischaikow and K. Spendlove. A computational framework for connection matrix theory. Journal of Applied and Computational Topology **5**, 459–529 (2021).

[LKMW23] M. Lipinski, J. Kubica, M. Mrozek and T. Wanner. Conley-Morse-Forman theory for generalized combinatorial multivector fields on finite topological spaces. Journal of Applied and Computational Topology **7**, 139–184 (2023).

[MW25] M. Mrozek and T. Wanner. Connection Matrices in Combinatorial Topological Dynamics. SpringerBriefs in Mathematics (Springer-Verlag, Cham, 2025).

Chapter 6

Flow Examples

After the theory described in the last section, we now turn towards a few more detailed examples in the context of ordinary differential equations. More precisely, in the following we describe how a variety of isolated invariant sets can be constructed using Morse decomposition intervals, and apply these tools to the analysis of simple planar and three-dimensional flows. Thus, this section serves as a showcase for the capabilities of ConleyDynamics.jl.

6.1 Extracting Subsystems

We briefly return to one of the examples in the tutorial. More precisely, we consider the planar ordinary differential equation given by

$$\begin{aligned} \dot{x}_1 &= x_1 \left(1 - x_1^2 - 3x_2^2\right) \\ \dot{x}_2 &= x_2 \left(1 - 3x_1^2 - x_2^2\right) \end{aligned}$$

The dynamics of this system is characterized by the existence of a global attractor in the shape of a closed disk. Inside the attractor, there are nine different Morse sets:

- The origin is an equilibrium of index 2, i.e., it is an unstable stationary state with a two-dimensional unstable manifold.
- The four points $(\pm 1/2, \pm 1/2)$ are unstable equilibria of index 1, i.e., with a one-dimensional unstable manifold.
- Finally, the four points $(\pm 1, 0)$ and $(0, \pm 1)$ are asymptotically stable stationary states.

We saw in the tutorial that the Morse decomposition of this system can easily be found using ConleyDynamics.jl, as well as the associated connection matrix. Yet, in certain situations one might only be interested in part of the dynamics on the attractor. Moreover, while the Morse sets describe the recurrent part of the dynamics, they do not provide information on the geometry of the connecting sets between the Morse sets. In the following, we illustrate how this can be analyzed further.

The right-hand side of the above vector field can be implemented using the Julia function

```
function planarvf(x::Vector{Float64})
    #
    # Sample planar vector field with nontrivial Morse decomposition
```

```
    #
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end
```

```
planarvf (generic function with 1 method)
```

To analyze the resulting global dynamical behavior, we first create a simplicial mesh covering the square $[-6/5, 6/5]^2$ using the commands

```
ec = create_simplicial_delaunay(300, 300, 5, 50, euclidean=true);
ec = rescale_coords(ec, [-1.2,-1.2], [1.2,1.2]);
ec.ncells
```

```
14395
```

The integer in the output gives the number of cells in the created Lefschetz complex X. Note that we are using a Delaunay triangulation over an initial box of size 300×300, where the target triangle size is about 5. This box is then rescaled to cover the above square. We would like to point out that in the above form, the commands produce a `EuclideanComplex`, which has the coordinates of all cells embedded. We can then create a multivector field on the simplicial complex `ec` and find its Morse decomposition using the commands

```
mvf = create_planar_mvf(ec, planarvf);
morsedecomp = morse_sets(ec, mvf);
length(morsedecomp)
```

```
9
```

As expected, ConleyDynamics.jl finds exactly nine Morse sets. Their Conley indices can be computed and stored in a `Vector{Vector{Int}}` using the command

```
conleyindices = [conley_index(ec, mset) for mset in morsedecomp]
```

```
9-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [0, 1, 0]
 [1, 0, 0]
 [0, 1, 0]
 [1, 0, 0]
```

```
[0, 1, 0]
[0, 1, 0]
[0, 0, 1]
```

These Conley indices correspond to the dynamical behavior near the equilibrium solutions described above.

Suppose now that rather than finding the connection matrix for the complete Morse decomposition, we would only like to consider a part of it. This can be done as long as we restrict our attention to an interval in the Morse decomposition. Such an interval $\mathcal{I}$ can be created from a selection $\mathcal{S}$ of the Morse sets in the following way:

- In addition to the Morse sets in $\mathcal{S}$, the interval $\mathcal{I}$ contains all Morse sets that lie between two Morse sets in $\mathcal{S}$ with respect to the poset order underlying the Morse decomposition. Recall that this poset order can be computed via `morse_sets` by activating the extra return object `hasse`, which describes the Hasse diagram of the poset.

With every interval $\mathcal{I}$ of the Morse decomposition one can assign a smallest isolated invariant set $X_{\mathcal{I}} \subset X$ which describes the complete dynamics within and between the Morse sets in $\mathcal{I}$. In fact, in can be characterized as follows:

- The set $X_{\mathcal{I}}$ consists of all cells in the underlying Lefschetz complex X through which one can find a solution which originates in one Morse set of $\mathcal{I}$ and ends in another Morse set of $\mathcal{I}$, where the two involved Morse sets can be the same. In other words, one needs to combine the interval Morse sets with all connecting orbits between them.

The two above steps can be performed in ConleyDynamics.jl using the function `morse_interval`.

In our example, we consider two intervals. The first interval consists of the five Morse sets corresponding to all unstable equilibrium solutions, while the second one considers the four index 1 and the four stable stationary states. The associated isolated invariant sets for these two intervals can be computed as follows:

```
subset1 = findall(x -> x[2]+x[3]>0, conleyindices);
subset2 = findall(x -> x[1]+x[2]>0, conleyindices);
ecsub1 = morse_interval(ec, mvf, morsedecomp[subset1]);
ecsub2 = morse_interval(ec, mvf, morsedecomp[subset2]);
[length(subset1), length(subset2), length(ecsub1), length(ecsub2)]
```

```
4-element Vector{Int64}:
    5
    8
 1201
 2256
```

The output shows that we have in fact extracted five and eight Morse sets, respectively. It also shows that the Lefschetz complexes corresponding to these two isolated invariant sets are much smaller than X.

So far, we have just determined the collections of cells that correspond to the two isolated invariant sets for these intervals. We can now restrict the combinatorial dynamics to these subsets. Note that since they are

both isolated invariant sets, they are locally closed in X, and therefore the restrictions provide us with two new Lefschetz complexes `ecr1` and `ecr2`, which have the type `EuclideanComplex`, along with induced multivector fields `mvfr1` and `mvfr2`, respectively. In ConleyDynamics.jl, this is achieved using the commands

```
ecr1, mvfr1 = restrict_dynamics(ec, mvf, ecsub1);
ecr2, mvfr2 = restrict_dynamics(ec, mvf, ecsub2);
[ecr1.ncells, ecr2.ncells]
```

```
2-element Vector{Int64}:
 1201
 2256
```

It is now easy to find the connection matrices for these two intervals. The first connection matrix is given by

```
cmr1 = connection_matrix(ecr1, mvfr1);
cmr1.conley
```

```
5-element Vector{Vector{Int64}}:
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 0, 1]
```

```
full_from_sparse(cmr1.matrix)
```

```
5×5 Matrix{Int64}:
 0  0  0  0  1
 0  0  0  0  1
 0  0  0  0  1
 0  0  0  0  1
 0  0  0  0  0
```

It clearly shows that the unstable index 2 Morse set has connecting orbits to every one of the four index 1 equilibria. Similarly, the second connection matrix can be determined as

```
cmr2 = connection_matrix(ecr2, mvfr2);
cmr2.conley
```

```
8-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [1, 0, 0]
```

```
 [1, 0, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
```

```
full_from_sparse(cmr2.matrix)
```

```
8×8 Matrix{Int64}:
 0  0  0  0  0  1  1  0
 0  0  0  0  0  0  1  1
 0  0  0  0  1  1  0  0
 0  0  0  0  1  0  0  1
 0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0
```

In this case, every index 1 equilibrium is connected two its two neighboring stable stationary states via heteroclinics that are detected by the connection matrix.

The Lefschetz complexes associated with the two Morse decomposition intervals can also be visualized in ConleyDynamics.jl. For this, recall that the function `plot_planar_simplicial_morse` can plot an underlying simplicial complex together with any collection of cell subsets. For our purposes, we use the following commands:

```
show1 = [[ecr1.labels]; cmr1.morse];
show2 = [[ecr2.labels]; cmr2.morse];
fname1 = "/Users/wanner/Desktop/invariantinterval2d1.png"
fname2 = "/Users/wanner/Desktop/invariantinterval2d2.png"
plot_planar_simplicial_morse(ec, fname1, show1, vfac=1.1, hfac=2.0)
plot_planar_simplicial_morse(ec, fname2, show2, vfac=1.1, hfac=2.0)
```

The variable `show1` collects not only the Morse sets that are part of the first connection matrix `cmr1`, but also the support of the Lefschetz complex `ecr1`. This support is accessed via `[ecr1.labels]`, and we add it as a first vector of cells in `show1`. Similarly, we determine the support of the second isolated invariant set, together with the Morse sets of `cmr2`. The remaining four commands create two figures, which show the supports of the two Morse intervals on the whole triangulation.

The figure of the interval support for the first interval shows the five Morse sets surrounding the stationary states at the origin and at $(\pm 1/2, \pm 1/2)$. In addition, it highlights the support of the isolated invariant set associated with this Morse decomposition interval. One can clearly see rough outer approximations for the four heteroclinics which start at the origin and end at the index 1 equilibria. These approximations are necessarily coarse, since we are not working with a very fine triangulation.

Finally, the figure of the interval support for the second interval depicts the eight Morse sets enclosing the index 1 and the stable stationary states. It also shows the support of the Lefschetz complex `ecr2` which is associated with this Morse decomposition interval. In this case, it covers eight different heteroclinic orbits, which are in fact better approximated than the four in the figure for the first interval.

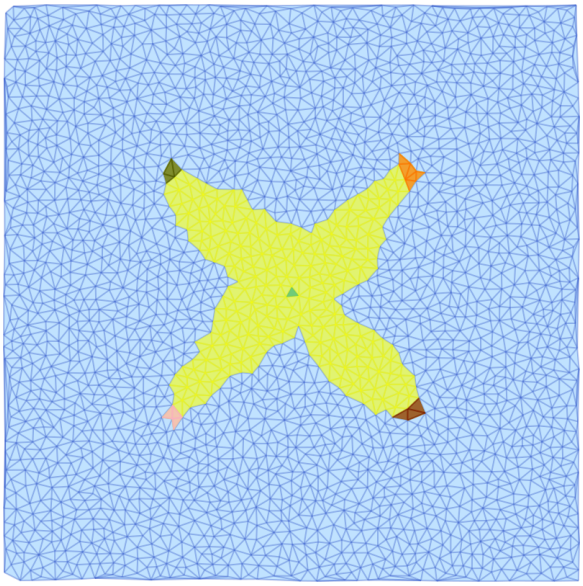


Figure 6.1: Interval support for the first interval

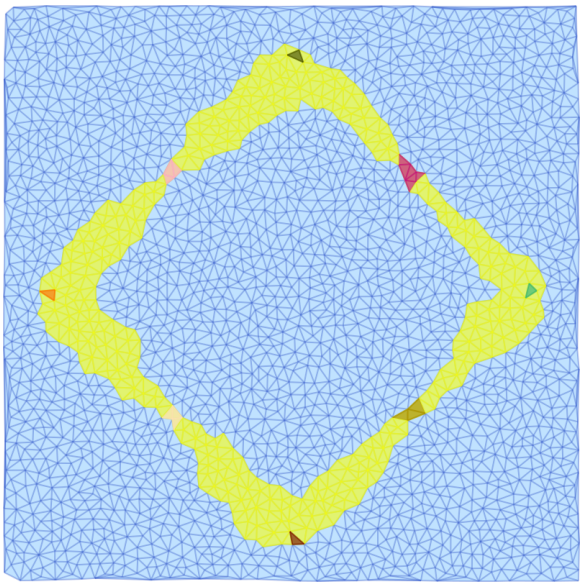


Figure 6.2: Interval support for the second interval

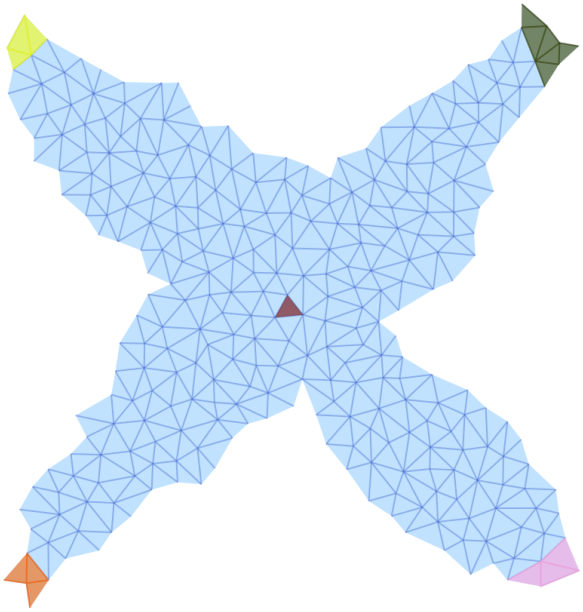


Figure 6.3: Subcomplex representation for the first interval

The two figures of the interval supports were generated by passing the base simplicial complex ec to the plotting command and visualizing sets that were computed using the subcomplexes. This is possible since we use the label form of these sets, and the labels of the restricted complexes are also present in the larger complex. We would like to point out, however, that this example uses the data type `EuclideanComplex`, which has the correct coordinate information for plotting automatically embedded also in the restricted complexes. Therefore, it is possible to only plot these restricted complexes, together with their Morse sets. For this, one just has to change the first argument in each of the above two plot commands as follows:

```
show3 = cmr1.morse;
show4 = cmr2.morse;
fname3 = "/Users/wanner/Desktop/invariantinterval2d3.png"
fname4 = "/Users/wanner/Desktop/invariantinterval2d4.png"
plot_planar_simplicial_morse(ecr1, fname3, show3, vfac=1.1, hfac=2.0)
plot_planar_simplicial_morse(ecr2, fname4, show4, vfac=1.1, hfac=2.0)
```

Notice that in the two figures of the subcomplex representations created by these commands, only the Morse intervals are depicted, together with their Morse sets.

6.2 Analysis of a Planar System

Our next example illustrates how ConleyDynamics.jl can be used to analyze the global dynamics of a planar ordinary differential equations. For this, consider the planar system

$$\begin{aligned} \dot{x}_1 &= x_2 - x_1\left(x_1^2+x_2^2-4\right)\left(x_1^2+x_2^2-1\right) \\ \dot{x}_2 &= -x_1 - x_2\left(x_1^2+x_2^2-4\right)\left(x_1^2+x_2^2-1\right) \end{aligned}$$

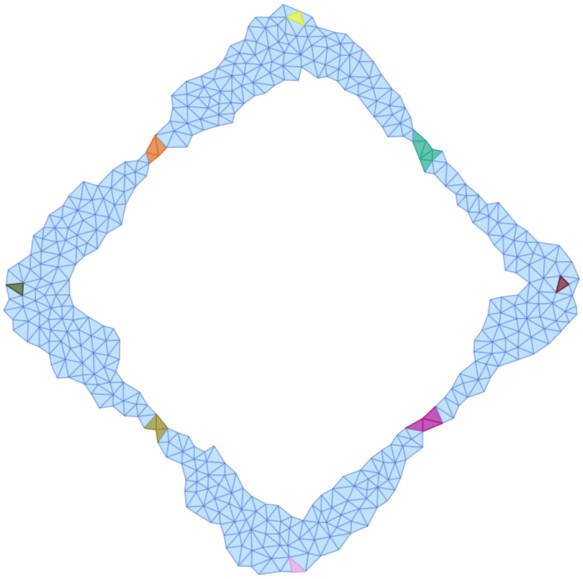


Figure 6.4: Subcomplex representation for the second interval

This system has already been considered in [MSTW22]. The right-hand side of this vector field can be implemented using the Julia function

```
function circlevf(x::Vector{Float64})
    #
    # Sample vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    c0 = x1*x1 + x2*x2
    c1 = (c0 - 4.0) * (c0 - 1.0)
    y1 =  x2 - x1 * c1
    y2 = -x1 - x2 * c1
    return [y1, y2]
end
```

```
circlevf (generic function with 1 method)
```

To analyze the global dynamics of this vector field, we first create a cubical complex covering the square $[-3,3]^2$ using the commands

```
n = 51
lc, coords = create_cubical_rectangle(n,n,p=2);
coordsN = convert_planar_coordinates(coords,[-3.0,-3.0],[3.0,3.0]);
lc.ncells
```

```
10609
```

As the last result shows, this gives a Lefschetz complex with 10609 cells. The multivector field can be generated using

```
mvf = create_planar_mvf(lc, coordsN, circlevf);
length(mvf)
```

```
2449
```

This multivector field consists of 2437 multivectors. Finally, the connection matrix can be determined using the command

```
cm = connection_matrix(lc, mvf);
cm.conley
```

```
3-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [1, 0, 0]
 [0, 1, 1]
```

Therefore, the above planar system has three isolated invariant sets. One has the Conley index of a stable equilibrium, while the other two have that of a stable and an unstable periodic orbit. The columns of the connection matrix correspond to these invariant sets as follows

```
cm.poset
```

```
5-element Vector{Int64}:
 1
 1
 2
 3
 3
```

The connection matrix itself is given by

```
sparse_show(cm)
```

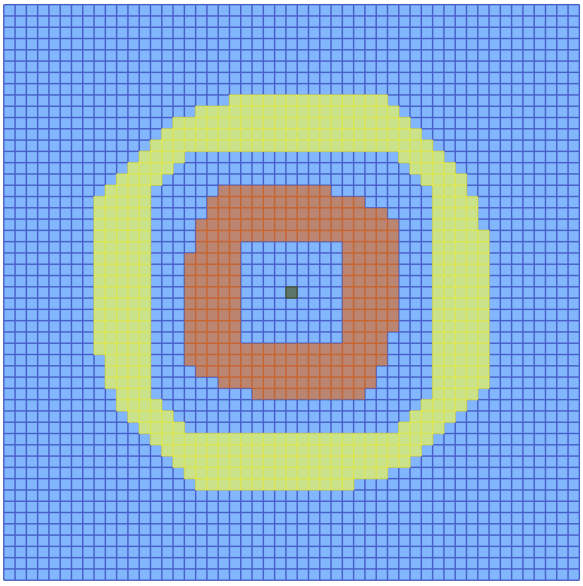


Figure 6.5: Morse sets of the planar circles vector field

```
       ┆ 0825.00 4326.01 2626.00 2418.01 2632.11
-------┆---------------------------------------
0825.00┆       .       .       .       1       .
4326.01┆       .       .       .       .       1
2626.00┆       .       .       .       1       .
2418.01┆       .       .       .       .       .
2632.11┆       .       .       .       .       .
```

This implies that there are connecting orbits from the unstable periodic orbit to both the stable equilibrium, and the stable periodic orbit. To visualize these Morse sets, we employ the commands

```
fname = "cubicalcircles.pdf"
plot_planar_cubical_morse(lc, fname, cm.morse, pv=true)
```

In the above example we used the original fixed cubical grid, which is just a scaled version of the grid on the integer lattice. It is also possible to work with a randomized grid, in which the coordinates of the vertices are randomly perturbed. This can be achieved with the following commands:

```
nR = 75
lcR, coordsR = create_cubical_rectangle(nR,nR,p=2,randomize=0.33);
coordsRN = convert_planar_coordinates(coordsR,[-3.0,-3.0],[3.0,3.0]);
mvfR = create_planar_mvf(lcR, coordsRN, circlevf);
cmR = connection_matrix(lcR, mvfR);
fnameR = "cubicalcirclesR.pdf"
plot_planar_cubical_morse(lcR, coordsRN, fnameR, cmR.morse, pv=true, vfac=1.1, hfac=2.0)
```

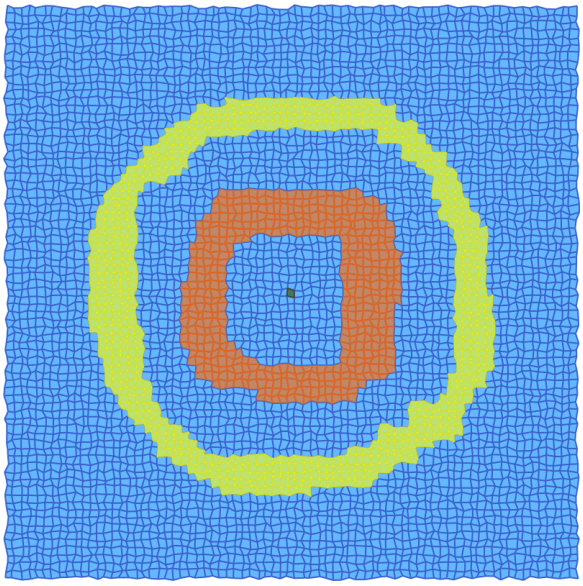


Figure 6.6: Morse sets of the planar circles vector field via randomized cubes

To constrast the above example with the use of a Delaunay triangulation, we reanalyze the vector field in the following way:

```
lc2, coords2 = create_simplicial_delaunay(400, 400, 10, 30, p=2)
coords2N = convert_planar_coordinates(coords2,[-3.0,-3.0], [3.0,3.0])
mvf2 = create_planar_mvf(lc2, coords2N, circlevf)
cm2 = connection_matrix(lc2, mvf2)

fname2 = "cubicalcircles2.pdf"
plot_planar_simplicial_morse(lc2, coords2N, fname2, cm2.morse, pv=true)
```

In this case, the Morse sets can be visualized as in the figure of the Morse sets of the planar circles vector field via a Delaunay triangulation.

Notice that we can also show the individual multivectors in more detail. For the above example, we can plot all multivectors of the multivector field `mvf2` which consist of at least 10 cells using the commands

```
mv_indices = findall(x -> (length(x)>9), mvf2)
large_mv = mvf2[mv_indices]

fname3 = "cubicalcircles3.pdf"
plot_planar_simplicial_morse(lc2, coords2N, fname3, large_mv, pv=true)
```

Note that in this example, there are only 20 large multivectors.

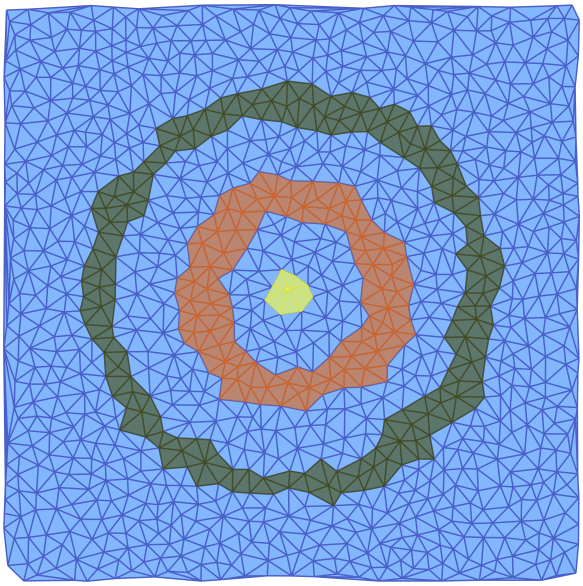


Figure 6.7: Morse sets of the planar circles vector field via Delaunay

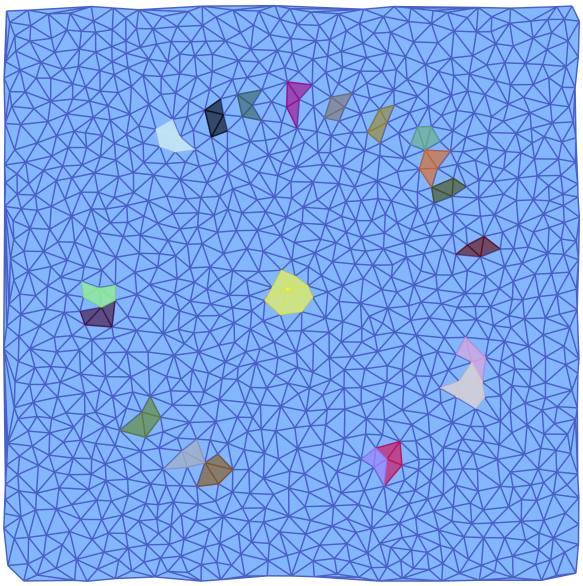


Figure 6.8: Large multivectors in the Delaunay multivector field

6.3 Analysis of a Spatial System

It is also possible to analyze simple three-dimensional ordinary differential equations in ConleyDynamics.jl. To provide one such example, consider the system

$$\begin{aligned}
\dot{x}_1 &= (\lambda-1)x_1 - \tfrac{3\lambda}{2\pi}\left((x_1^3 - x_1^2x_3 + x_2^2x_3 + 2x_1\left(x_2^2+x_3^2\right)\right) \\
\dot{x}_2 &= (\lambda-4)x_2 - \tfrac{3\lambda}{2\pi}\, x_2\left(2x_1^2 + x_2^2 + 2x_1x_3 + 2x_3^2\right) \\
\dot{x}_3 &= (\lambda-9)x_3 + \tfrac{\lambda}{2\pi}\left(x_1\left(x_1^2 - 3x_2^2\right) - 3x_3\left(2x_1^2 + 2x_2^2 + x_3^2\right)\right)
\end{aligned}$$

This system arises in the study of the so-called Allen-Cahn equation, which is the parabolic partial differential equation given by

$$u_t = \Delta u + \lambda\left(u - u^3\right) \quad \text{in} \quad \Omega = (0,\pi) \quad \text{with} \quad u = 0 \quad \text{on} \quad \partial\Omega\,.$$

This partial differential equation can be interpreted as an infinite-dimensional system of ordinary differential equations, see for example [SW26, Section 6.1]. For this, one has to expand the unknown function $u(t,\cdot)$ as a generalized Fourier series with respect to the basis functions

$$\varphi_k(x) = \sqrt{\frac{2}{\pi}}\,\sin(k\pi x) \quad \text{for} \quad k \in \mathbb{N}\,.$$

If one truncates this series representation after three terms, and projects the right-hand side of the partial differential equation onto the linear space spanned by the first three basis functions, then the three coefficients of the approximating sum satisfy the above three-dimensional ordinary differential equation. Thus, this system provides a model for the dynamics of the partial differential equation, at least for sufficiently small values of the parameter λ. It can be implemented in Julia using the following commands:

```
function allencahn3d(x::Vector{Float64})
    #
    # Allen-Cahn projection
    #
    lambda = 3.0 * pi
    c      = lambda / pi
    x1, x2, x3 = x
    y1 = (lambda-1)*x1 - 1.5*c * (x1*x1*x1-x1*x1*x3+x2*x2*x3+2*x1*(x2*x2+x3*x3))
    y2 = (lambda-4)*x2 - 1.5*c * x2 * (2*x1*x1+x2*x2+2*x1*x3+2*x3*x3)
    y3 = (lambda-9)*x3 + 0.5*c * (x1*(x1*x1-3*x2*x2)-3*x3*(2*x1*x1+2*x2*x2+x3*x3))
    return [y1, y2, y3]
end
```

Notice that for our example we use the parameter value $\lambda = 3\pi$. In this particular case, one can show numerically that the system has seven equilibrium solutions. These are approximately given as follows:

- Two equilibria $\pm(1.45165,\ 0,\ 0.24396)$ of index 0.
- Two equilibria $\pm(0,\ 1.09796,\ 0)$ of index 1.
- Two equilibria $\pm(0,\ 0,\ 0.307238)$ of index 2.

- One equilibrium $(0,\ 0,\ 0)$ of index 3.

In order to find the associated Morse decomposition, one can use the commands

```
N = 25
bmax = [1.8, 1.5, 1.0]
lc, coordsI = create_cubical_box(N,N,N);
coordsN = convert_spatial_coordinates(coordsI, -bmax, bmax);
mvf = create_spatial_mvf(lc, coordsN, allencahn3d);
```

These commands create a cubical box of size $25 \times 25 \times 25$ which covers the region $[-1.8, 1.8] \times [-1.5, 1.5] \times [-1.0, 1.0]$. In addition, we construct a multivector field `mvf` which encapsulates the possible dynamics of the system. After these preparations, the Morse decomposition can be computed via

```
morsedecomp = morse_sets(lc, mvf);
morseinterval = morse_interval(lc, mvf, morsedecomp);
lci, mvfi = restrict_dynamics(lc, mvf, morseinterval);
cmi = connection_matrix(lci, mvfi);
```

While the first command finds the actual Morse decomposition, the second one restricts the Lefschetz complex and the multivector field to the smallest isolated invariant set which contains all Morse sets and connecting orbits between them. The last command finds the connection matrix.

To see whether the above commands did indeed find the correct dynamical behavior, we first inspect the computed Conley indices of the Morse sets:

```
julia> cmi.conley
7-element Vector{Vector{Int64}}:
 [1, 0, 0, 0]
 [1, 0, 0, 0]
 [0, 1, 0, 0]
 [0, 1, 0, 0]
 [0, 0, 1, 0]
 [0, 0, 1, 0]
 [0, 0, 0, 1]
```

Clearly, these are the correct indices based on our numerical information concerning the stationary states of the system. The connection matrix is given by:

```
julia> full_from_sparse(cmi.matrix)
7×7 Matrix{Int64}:
 0  0  1  1  0  0  0
 0  0  1  1  0  0  0
 0  0  0  0  1  1  0
 0  0  0  0  1  1  0
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1
 0  0  0  0  0  0  0
```

Thus, there are a total of ten connecting orbits that are induced through algebraic topology. The index 3 equilibrium at the origin has connections to each of the index 2 solutions, which lie above and below the

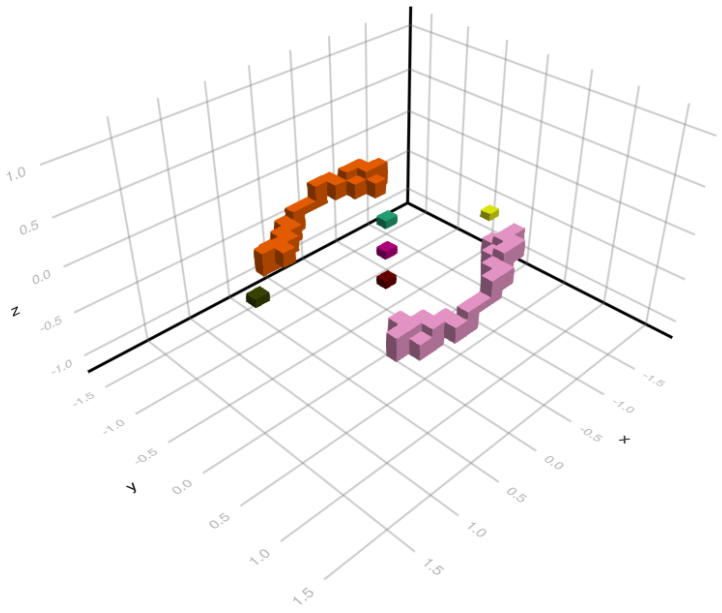


Figure 6.9: The dynamics of an Allen-Cahn model

origin in the direction of the x_3-axis. Each of the latter two stationary states has connections to both index 1 equilibria. Finally, each of these is connected to both stable states.

The location of the computed Morse sets is illustrated in the figure of the dynamics of the Allen-Cahn model, which shows the Morse sets as colored cubes and uses x, y, and z instead of the variable names x_1, x_2, and x_3, respectively. Notice that while the stationary states of index 0, 2, and 3 are all well-localized, this cannot be said about the two equilibria of index 1. The computed enclosures for the latter two are elongated cubical sets which are shown along the upper left and lower right of the figure. This overestimation is a result of the use of a strict cubical grid, combined with the small discretization size `N = 25`. Nevertheless, the above simple code does reproduce the overall global dynamical behavior of the ordinary differential equation correctly.

One can also compute over Morse intervals, rather than the complete Morse decomposition. The two figures of the Allen-Cahn Morse interval show two views of the Morse interval which corresponds to one of the index 1 equilibria, and the two stable stationary states. These computations were performed with the finer resolution `N = 51`.

We would like to emphasize that there are many techniques in the literature that can be used to identify isolated invariant sets and their Conley indices. Rather than giving a detailed list, we refer to [SW14b] and the references therein. For example, in [SW14b] ideas from computational topology were used to rigorously establish candidate sets in three dimensions as an isolating block. The associated Matlab code can be found at [SW14a].

6.4 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[MSTW22] M. Mrozek, R. Srzednicki, J. Thorpe and T. Wanner. Combinatorial vs. classical dynamics: Recurrence. Communications in Nonlinear Science and Numerical Simulation **108**, Paper No. 106226, 30 pages (2022).

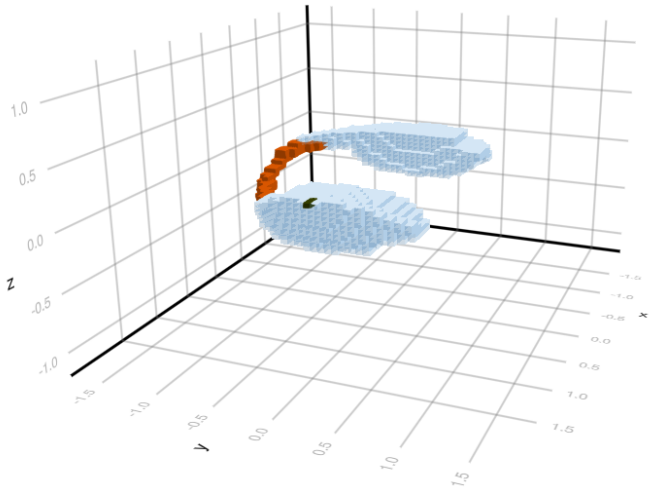


Figure 6.10: Allen-Cahn Morse interval, View 1

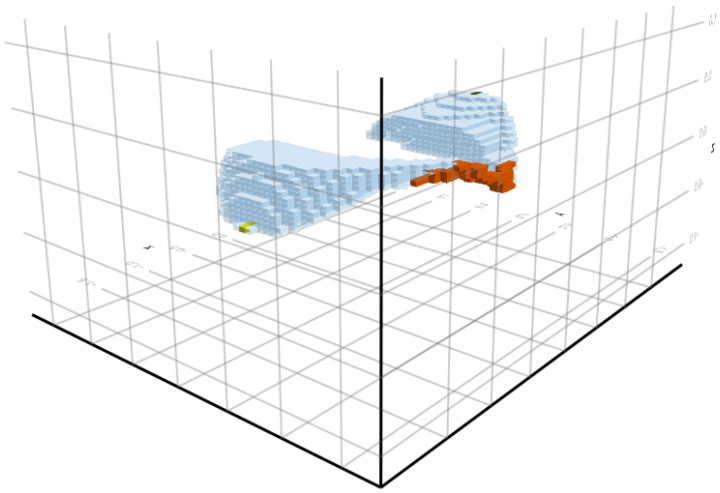


Figure 6.11: Allen-Cahn Morse interval, View 2

[SW26] E. Sander and T. Wanner. Theory and Numerics of Partial Differential Equations (SIAM, Philadelphia, 2026). In preparation, 1023 pages.

[SW14a] T. Stephens and T. Wanner. Isolating block validation in Matlab, https://github.com/almost6heads/isoblockval (2014).

[SW14b] T. Stephens and T. Wanner. Rigorous validation of isolating blocks for flows and their Conley indices. SIAM Journal on Applied Dynamical Systems **13**, 1847–1878 (2014).

Chapter 7

Further Examples

In order to illustrate the basic functionality of ConleyDynamics.jl, this section collects a number of examples. Many of these are taken from the paper [BKMW20] and the book [MW25], and they consider both Forman vector fields and general multivector fields on a variety of Lefschetz complexes. Each example has its own associated function, so that users can quickly create examples on their own by taking the respective source files as templates.

7.1 A One-Dimensional Forman Field

Our first example is taken from [BKMW20, Figure 1], and it is a Forman vector field on a one-dimensional simplicial complex as shown in the figure of the one-dimensional simplicial complex flow.

The simplicial complex and Forman vector field can be created using the function `example_forman1d`:

`ConleyDynamics.example_forman1d` – Method.

```
lc, mvf = example_forman1d()
```

Create the simplicial complex and multivector field for the example from Figure 1 in the FoCM 2020 paper by Batko, Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` and the multivector field `mvf`. The Lefschetz complex is defined over the finite field GF(2). If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case a `EuclidenComplex` with embedded coordinates is returned for `lc`.

Examples

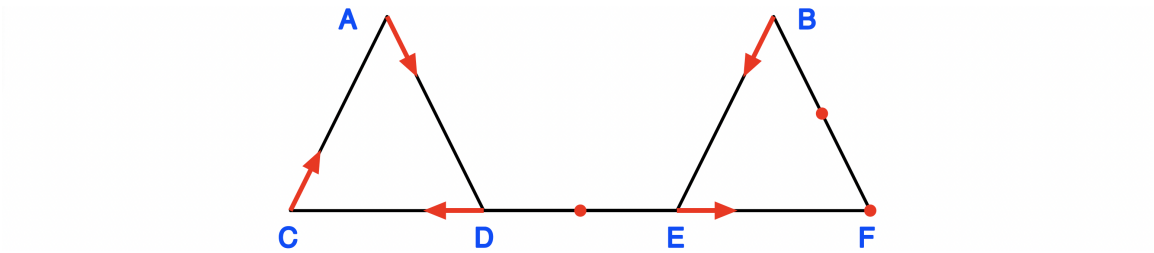


Figure 7.1: A one-dimensional simplicial complex flow

```
julia> lc, mvf = example_forman1d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
  ┆  A CD  F BF DE
--┆---------------
 A┆  .  .  .  .  1
CD┆  .  .  .  .  .
 F┆  .  .  .  .  1
BF┆  .  .  .  .  .
DE┆  .  .  .  .  .

julia> sparse_show(cm.matrix)
 . . . . 1
 . . . . .
 . . . . 1
 . . . . .
 . . . . .

julia> println(cm.labels)
["A", "CD", "F", "BF", "DE"]

julia> lc2, mvf2 = example_forman1d(euclidean=true);

julia> typeof(lc2)
EuclideanComplex

julia> lc2.coords
13-element Vector{Vector{Vector{Float64}}}:
 [[50.0, 100.0]]
 [[250.0, 100.0]]
 [[0.0, 0.0]]
 [[100.0, 0.0]]
 [[200.0, 0.0]]
 [[300.0, 0.0]]
 [[50.0, 100.0], [0.0, 0.0]]
 [[50.0, 100.0], [100.0, 0.0]]
 [[250.0, 100.0], [200.0, 0.0]]
 [[250.0, 100.0], [300.0, 0.0]]
 [[0.0, 0.0], [100.0, 0.0]]
 [[100.0, 0.0], [200.0, 0.0]]
 [[200.0, 0.0], [300.0, 0.0]]
```

source

The commands from the docstring show that the connection matrix has five rows and columns. The last three of these correspond to the critical cells F, BF, and DE, while the first two correspond to the two generators of the homological Conley index of the periodic orbit, given by A and AD.

The full Morse decomposition of this combinatorial dynamical system is depicted in the figure of the Morse decomposition of the one-dimensional example, and all four Morse sets are indicated in the simplicial complex by different colors. They are also listed, together with their Conley indices, in the following Julia output:

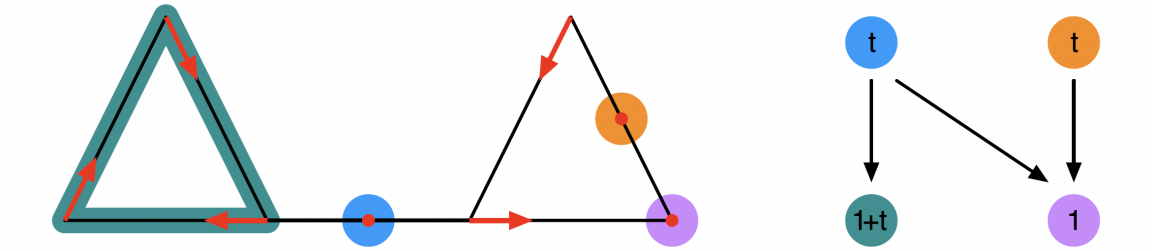


Figure 7.2: Morse decomposition of the one-dimensional example

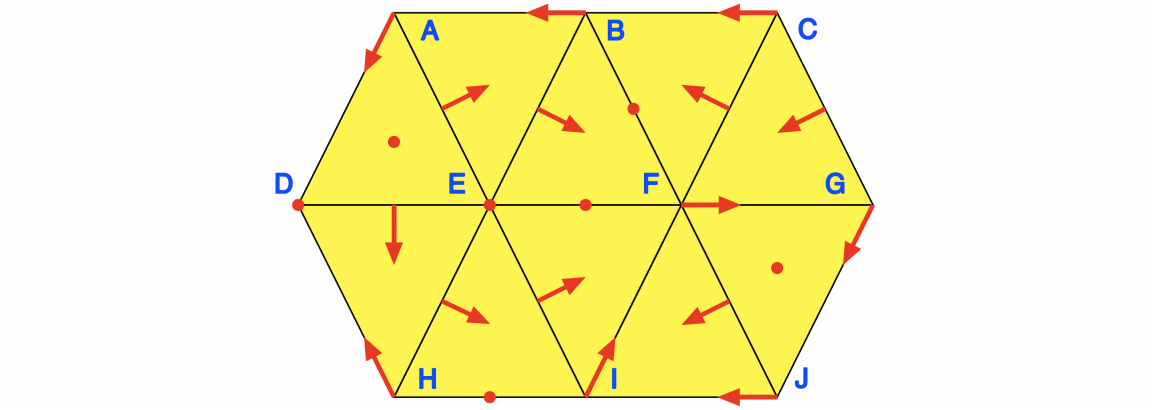


Figure 7.3: A planar simplicial complex flow

```
julia> cm.morse
4-element Vector{Vector{String}}:
 ["A", "C", "D", "AC", "AD", "CD"]
 ["F"]
 ["BF"]
 ["DE"]

julia> cm.conley
4-element Vector{Vector{Int64}}:
 [1, 1]
 [1, 0]
 [0, 1]
 [0, 1]
```

Notice that only two heteroclinic orbits are reflected in the connection matrix. These are the connections between the unstable cell DE and both the equilibrium F and the periodic orbit. In contrast, the two heteroclinics between the index one cell BF and F cancel algebraically.

7.2 A Planar Forman Vector Field

Our second example was originally discussed in the context of [BKMW20, Figure 3], and it consists of a Forman vector field on a topological disk, as shown in the figure of the planar simplicial complex flow.

The disk is represented as a simplicial complex with 10 vertices, 19 edges, and 10 triangles. The Forman vector

field has 7 critical cells, and 16 arrows. Both the simplicial complex and the Forman vector field can be defined using the function `example_forman2d`:

`ConleyDynamics.example_forman2d` – Method.

```
lc, mvf = example_forman2d()
```

Create the simplicial complex and multivector field for the example from Figure 3 in the FoCM 2020 paper by Batko, Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` over the finite field GF(2) and the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case a `EuclidenComplex` with embedded coordinates is returned for `lc`.

Examples

```
julia> lc, mvf = example_forman2d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  D   E   F  IJ  BF  EF  HI ADE FGJ
───┼──────────────────────────────────
  D┆  .   .   .   .   1   .   1   .   .
  E┆  .   .   .   .   .   1   .   .   .
  F┆  .   .   .   .   1   1   1   .   .
 IJ┆  .   .   .   .   .   .   .   .   1
 BF┆  .   .   .   .   .   .   .   1   .
 EF┆  .   .   .   .   .   .   .   .   .
 HI┆  .   .   .   .   .   .   .   1   .
ADE┆  .   .   .   .   .   .   .   .   .
FGJ┆  .   .   .   .   .   .   .   .   .

julia> sparse_show(cm.matrix)
 . . . . 1 . 1 . .
 . . . . . 1 . . .
 . . . . 1 1 1 . .
 . . . . . . . . 1
 . . . . . . . 1 .
 . . . . . . . . .
 . . . . . . . 1 .
 . . . . . . . . .
 . . . . . . . . .

julia> print(cm.labels)
["D", "E", "F", "IJ", "BF", "EF", "HI", "ADE", "FGJ"]

julia> lc2, mvf2 = example_forman2d(euclidean=true);

julia> typeof(lc2)
EuclideanComplex
```

source

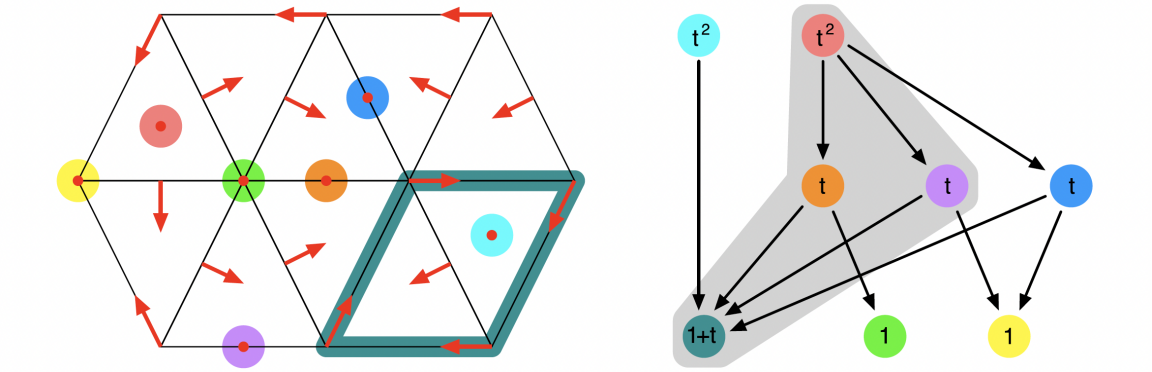


Figure 7.4: Morse decomposition of the planar flow

The Morse decomposition of this example is shown in the figure of the Morse decomposition of the planar flow, together with its Conley-Morse graph. Its eight Morse sets and associated Conley indices are given by

```
julia> cm.morse
8-element Vector{Vector{String}}:
 ["D"]
 ["E"]
 ["F", "G", "I", "J", "FG", "FI", "GJ", "IJ"]
 ["BF"]
 ["EF"]
 ["HI"]
 ["ADE"]
 ["FGJ"]

julia> cm.conley
8-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [1, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 0, 1]
 [0, 0, 1]
```

We would like to point out that the collection of Morse sets does not in general encompass all possible isolated invariant sets for the combinatorial dynamical system. Consider for example the set S shown in light blue in the figure of the nontrivial isolated invariant set.

Its mouth is depicted in dark blue, and it is clearly closed. In addition, the set S decomposes into arrows and critical cells, and one can show that it is invariant. Thus, it is in fact an isolated invariant set for this Forman vector field. Note also that this cell does not correspond to an interval in the Conley-Morse graph either, and this is indicated via gray shading in the Conley-Morse graph in the figure of the Morse decomposition of the planar flow. The set S, together with its closure and its mouth, can be generated using the following commands:

```
S = ["ADE","DEH","EFI","EHI",
     "DE","EF","EH","EI","FG","FI","GJ","HI","IJ",
```

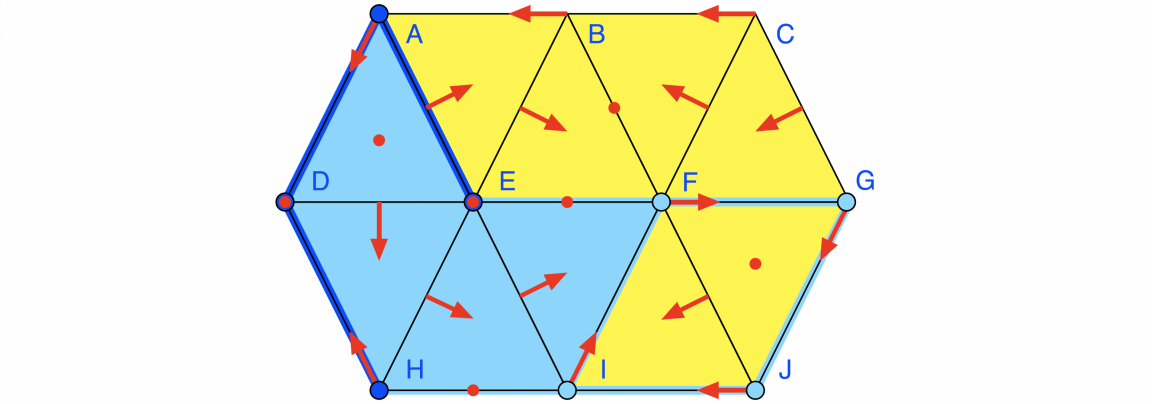


Figure 7.5: A nontrivial isolated invariant set

```
    "F","G","I","J"]
clS, moS = lefschetz_clomo_pair(lc,S)
```

Then the Conley index of S is given by

```
julia> conley_index(lc,S)
3-element Vector{Int64}:
 0
 1
 0

julia> relative_homology(lc,clS,moS)
3-element Vector{Int64}:
 0
 1
 0
```

Notice that this is the same as the relative homology of the pair $(\operatorname{cl} S, \operatorname{mo} S)$, as expected.

7.3 The Multivector Field from the Logo

This example is taken from [MW25, Figure 2.1], and it is visualized in the figure of the logo multivector field, which shows the two triangles ABC and BCD.

Clearly, this is the multivector field from the ConleyDynamics.jl logo. Since it was already discussed in detail in the Tutorial, we only show how the underlying simplicial complex and the multivector field can be created quickly using the function `example_julia_logo`:

`ConleyDynamics.example_julia_logo` – Method.

```
lc, mvf = example_julia_logo()
```

Create the simplicial complex and multivector field for the example from Figure 2.1 in the connection matrix book by Mrozek & Wanner.

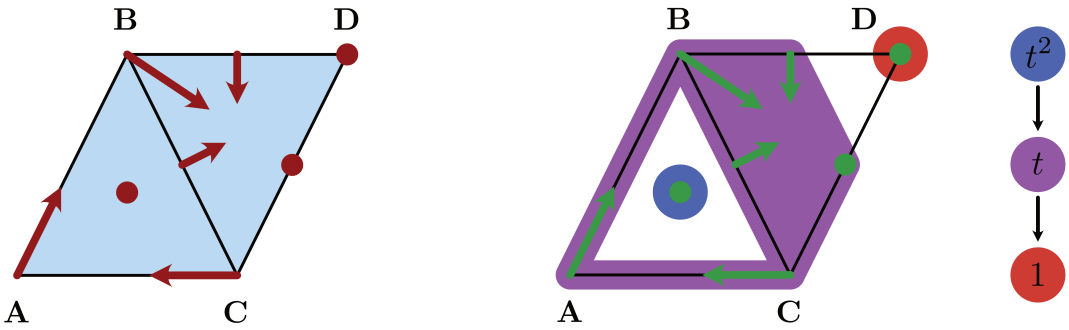


Figure 7.6: The logo multivector field

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_julia_logo();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  D  BC ABC
---┆------------
  D┆  .   .   .
 BC┆  .   .   1
ABC┆  .   .   .

julia> print(cm.labels)
["D", "BC", "ABC"]
```

source

The Morse sets and associated Conley indices can be accessed using the commands:

```
julia> cm.morse
3-element Vector{Vector{String}}:
 ["D"]
 ["A", "B", "C", "AB", "AC", "BC", "BD", "CD", "BCD"]
 ["ABC"]

julia> cm.conley
3-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 1, 0]
 [0, 0, 1]
```

Notice that in this example, every simplex of the underlying simplicial complex is contained in one of the Morse sets.

7.4 Critical Flow on a Simplex

The next example considers the arguably simplest situation of a Forman vector field on a simplicial complex. The simplicial complex X is given by a single simplex of dimension n, together with all its faces, while the Forman vector field on X contains only singletons. In other words, every simplex in the complex is a critical cell. Thus, this combinatorial dynamical system has one equilibrium of index n, and $n+1$ stable equilibria. In addition, there are $2^{n+1} - n - 3$ additional stationary states whose indices lie strictly between 0 and n, as well as a wealth of algebraically induced heteroclinic orbits. All of these can be found by using the connection matrix for the problem, as outlined in the following description for the function `example_critical_simplex`.

`ConleyDynamics.example_critical_simplex` – Method.

```
lc, mvf = example_critical_simplex(dim)
```

Create a simplicial complex of dimension `dim` as well as a multivector field on it in which every cell is critical.

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_critical_simplex(2);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  A   B   C  AB  AC  BC ABC
───┆────────────────────────────
  A┆  .   .   .   1   1   .   .
  B┆  .   .   .   1   .   1   .
  C┆  .   .   .   .   1   1   .
 AB┆  .   .   .   .   .   .   1
 AC┆  .   .   .   .   .   .   1
 BC┆  .   .   .   .   .   .   1
ABC┆  .   .   .   .   .   .   .

julia> print(cm.labels)
["A", "B", "C", "AB", "AC", "BC", "ABC"]
```

source

7.5 Flow on a Cylinder and a Moebius Strip

The next example considers again Forman vector fields, but this time on a cylinder and on a Moebius strip. The underlying simplicial complexes are given by a horizontal strip of eight triangles, whose left and right vertical edges are identified. For the first complex `lc1` these edges are identified without twist, while for the complex `lc2` they are twisted. See also the labels in the figure of the combinatorial flow on the cylinder and the Moebius strip.

Both complexes consist of eight vertices, sixteen edges, and eight triangles. The two complexes and Forman vector fields can be generated using the function `example_critical_simplex`, whose usage can be described as follows.

`ConleyDynamics.example_moebius` – Function.

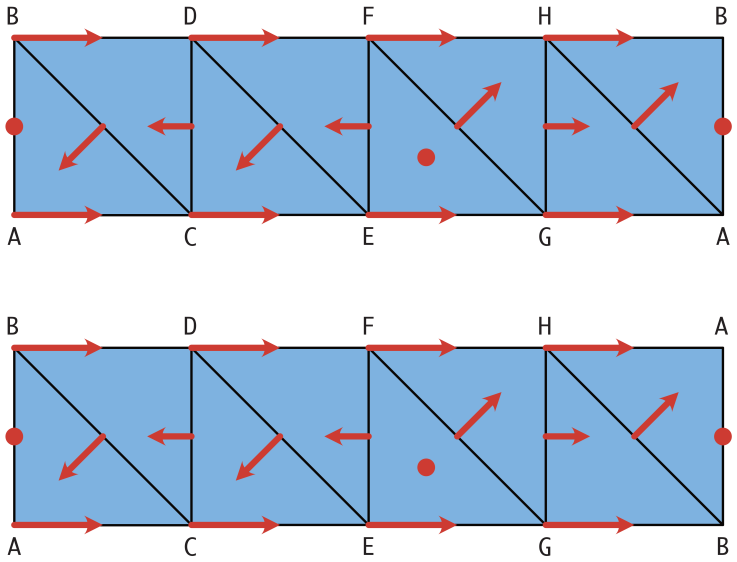


Figure 7.7: Combinatorial flow on cylinder and Moebius strip

```
lc1, mvf1, lc2, mvf2 = example_moebius(p)
```

Create two simplicial complexes for a cylinder and Moebius strip, respectively, together with associated multivector fields on them.

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector fields `mvf1` and `mvf2`. Both complexes are over a field with characteristic `p`. Positive prime characteristic uses the finite field GF(p), while zero characteristic gives the rationals.

The multivector field is the same, and it has one critical cell each in dimension 1 and 2 in the interior of the strip. The boundary consists of two periodic orbits for `lc1` and `mvf1`, and of one periodic orbit in the Moebius case `lc2` and `mvf2`. The latter case leads to different connection matrices for the fields GF(2) and GF(7), for example.

Examples

```
julia> lc1, mvf1, lc2, mvf2 = example_moebius(0);

julia> lc2p2 = lefschetz_gfp_conversion(lc2,2);

julia> lc2p7 = lefschetz_gfp_conversion(lc2,7);

julia> cmp2 = connection_matrix(lc2p2, mvf2, algorithm="DHL");

julia> cmp7 = connection_matrix(lc2p7, mvf2, algorithm="DHL");

julia> sparse_show(cmp2.matrix)
 . . . .
 . . . 1
 . . . .
 . . . .
```

```
julia> sparse_show(cmp7.matrix)
 . . . .
 . . . 1
 . . . 2
 . . . .
```

source

Note that for the combinatorial flow on the Moebius strip `lc2` the choice of field characteristic p leads to potentially different connection matrices. While for characteristic $p = 2$ the connection matrix has only one nontrivial entry, it has two for $p = 7$.

We only briefly include some sample computations for the latter case. One can create the complexes, Forman vector fields, and associated connection matrices for $p = 7$ using the following commands:

```
lc1, mvf1, lc2, mvf2 = example_moebius(7)
cm1 = connection_matrix(lc1,mvf1, algorithm="DHL")
cm2 = connection_matrix(lc2,mvf2, algorithm="DHL")
```

For the first example, the combinatorial flow on the cylinder has four Morse sets. Two critical equilibria of indices 1 and 2, as well as two periodic orbits. This can be shown as follows:

```
julia> cm1.morse
4-element Vector{Vector{String}}:
 ["A", "C", "E", "G", "AC", "AG", "CE", "EG"]
 ["B", "D", "F", "H", "BD", "BH", "DF", "FH"]
 ["AB"]
 ["EFG"]

julia> cm1.conley
4-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [1, 1, 0]
 [0, 1, 0]
 [0, 0, 1]

julia> sparse_show(cm1)
   ┆   A  EG   B  FH  AB EFG
───┼──────────────────────
  A┆   .   .   .   .   6   .
 EG┆   .   .   .   .   .   6
  B┆   .   .   .   .   1   .
 FH┆   .   .   .   .   .   1
 AB┆   .   .   .   .   .   .
EFG┆   .   .   .   .   .   .

julia> print(cm1.labels)
["A", "EG", "B", "FH", "AB", "EFG"]
```

In fact, the connection matrix implies the existence of connecting orbits from both the index 2 and the index 1 equilibrium to the two periodic orbits. The connections between the stationary states cannot be detected algebraically.

For the second example, the combinatorial flow on the Moebius strip, one only obtains three Morse sets. This time, there is only one periodic orbit which loops around both the top and bottom edges of the strip in the figure of the combinatorial flow on the cylinder and the Moebius strip. This is confirmed by the commands

```
julia> cm2.morse
3-element Vector{Vector{String}}:
 ["A", "B", "C", "D", "E", "F", "G", "H", "AC", "AH", "BD", "BG", "CE", "DF", "EG", "FH"]
 ["AB"]
 ["EFG"]

julia> cm2.conley
3-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [0, 1, 0]
 [0, 0, 1]

julia> sparse_show(cm2)
   ┆   A  FH  AB EFG
───┼────────────────
  A┆   .   .   .   .
 FH┆   .   .   .   1
 AB┆   .   .   .   2
EFG┆   .   .   .   .

julia> print(cm2.labels)
["A", "FH", "AB", "EFG"]
```

In this case, the connection matrix is able to identify the connecting orbits between the index 2 stationary state and both the periodic orbit and the index 1 equilibrium. The latter one is not recognized over the field $GF(2)$.

7.6 Nonunique Connection Matrices

Our next example is concerned with another Forman vector field, but this time on a larger simplicial complex, as shown in the figure of the example with nonunique connection matrices.

The simplicial complex is topologically a disk, and it consists of 9 vertices, 18 edges, and 10 triangles. The Forman vector field has 1 critical vertex, 3 critical edges, and 3 critical triangles, as well as 15 Forman arrows. The following example shows that for this combinatorial dynamical system, there are two fundamentally different connection matrices.

`ConleyDynamics.example_nonunique` – Method.

```
lc1, lc2, mvf = example_nonunique()
```

Create two representations of a simplicial complex and one multivector field which illustrates nonunique connection matrices.

The two complexes `lc1` and `lc2` represent the same simplicial complex over GF(2), but differ in the ordering of the labels.

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case, `lc1` and `lc2` are created as `EuclideanComplex` with embedded coordinates.

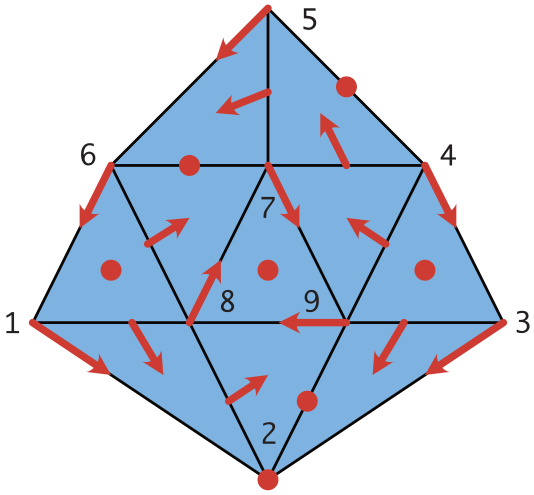


Figure 7.8: An example with nonunique connection matrices

Examples

```
julia> lc1, lc2, mvf = example_nonunique();

julia> cm1 = connection_matrix(lc1, mvf, algorithm="DLMS");

julia> cm2 = connection_matrix(lc2, mvf, algorithm="DLMS");

julia> sparse_show(cm1.matrix)
 . . . 1 . 1 . . .
 . . . 1 . 1 . . .
 . . . . . . . 1 1
 . . . . . . 1 1 .
 . . . . . . . 1 .
 . . . . . . 1 1 .
 . . . . . . . . .
 . . . . . . . . .
 . . . . . . . . .

julia> print(cm1.labels)
["2", "7", "79", "29", "45", "67", "168", "349", "789"]

julia> sparse_show(cm2.matrix)
 . . . 1 . 1 . . .
 . . . 1 . 1 . . .
 . . . . . . 1 . 1
 . . . . . . 1 1 .
 . . . . . . . 1 .
 . . . . . . 1 1 .
 . . . . . . . . .
 . . . . . . . . .
 . . . . . . . . .

julia> print(cm2.labels)
["2", "8", "78", "29", "45", "67", "168", "349", "789"]
```

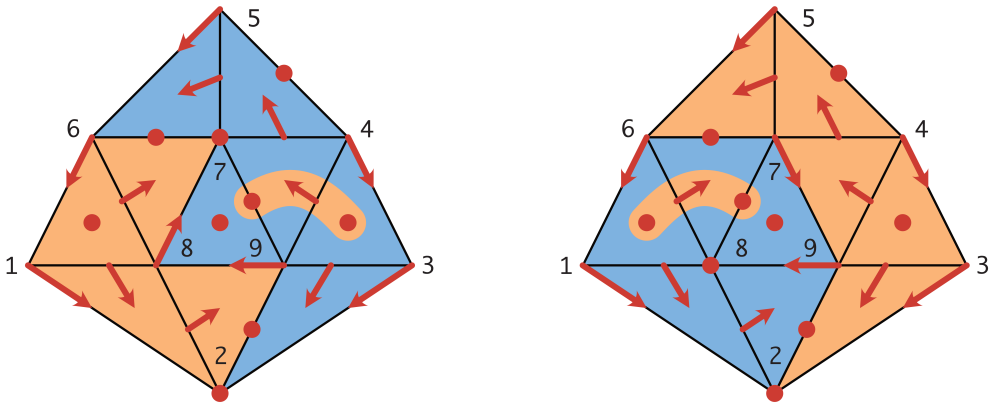


Figure 7.9: Forcing different connection matrices

source

As mentioned in the docstring for the function `example_nonunique`, the two Lefschetz complexes `lc1` and `lc2` both represent the above simplicial complex. However, they differ in the ordering of the vertex labels. This can be seen from the commands

```
julia> print(lc1.labels[1:9])
["1", "2", "3", "4", "5", "6", "7", "8", "9"]
julia> print(lc2.labels[1:9])
["1", "2", "3", "4", "5", "6", "8", "9", "7"]
```

In other words, `lc1` and `lc2` are different representations of the same complex. Nevertheless, computing the connection matrices as in the example gives two distinct connection matrices. This is purely a consequence of the different ordering of the rows and columns in the boundary matrix. Notice also, that for demonstration purposes we compute the connection matrix using the `DLMS24` algorithm.

To shed further light on this issue, notice that the triangle at the center of the complex forms an attracting periodic orbit, whose Conley index has Betti numbers 1 in dimensions 0 and 1. One can break this periodic orbit by removing one of its three arrows, and replacing it with two critical cells of dimensions 0 and 1. The figure on forcing different connection matrices, which consists of two copies of the complex, shows two different ways of doing this.

In the left copy, the vector `["7", "79"]` is removed, while the one on the right breaks up `["8", "78"]`. The corresponding modified Forman vector fields, and their connection matrices, can be created as follows:

```
mvf1 = deepcopy(mvf);
mvf2 = deepcopy(mvf);
deleteat!(mvf1,6);
deleteat!(mvf2,8);
cm1mod = connection_matrix(lc1, mvf1, algorithm="DLMS");
cm2mod = connection_matrix(lc2, mvf2, algorithm="DLMS");
```

Both of the new Forman vector fields are gradient vector fields, and in view of a result in [MW25], their connection matrices are therefore uniquely determined. The connection matrix for the vector field `mvf1` is of the form

```
julia> sparse_show(cm1mod)
   ┆   2   7  29  45  67  79 168 349 789
---┆-----------------------------------
  2┆   .   .   1   .   1   .   .   .   .
  7┆   .   .   1   .   1   .   .   .   .
 29┆   .   .   .   .   .   .   1   1   .
 45┆   .   .   .   .   .   .   .   1   .
 67┆   .   .   .   .   .   .   1   1   .
 79┆   .   .   .   .   .   .   .   1   1
168┆   .   .   .   .   .   .   .   .   .
349┆   .   .   .   .   .   .   .   .   .
789┆   .   .   .   .   .   .   .   .   .

julia> print(cm1mod.labels)
["2", "7", "29", "45", "67", "79", "168", "349", "789"]
```

Notice that this matrix shows that there is a connection from the triangle 349 to the edge 79, but there are no connections from the triangle 168 to the critical edge on the center triangle. In fact, up to reordering the columns and rows, this connection matrix is the same as `cm1` in the example.

Similarly, the connection matrix for the second modified Forman vector field `mvf2` is uniquely determined, and it is given by

```
julia> sparse_show(cm2mod)
   ┆   2   8  29  45  67  78 168 349 789
---┆-----------------------------------
  2┆   .   .   1   .   1   .   .   .   .
  8┆   .   .   1   .   1   .   .   .   .
 29┆   .   .   .   .   .   .   1   1   .
 45┆   .   .   .   .   .   .   .   1   .
 67┆   .   .   .   .   .   .   1   1   .
 78┆   .   .   .   .   .   .   1   .   1
168┆   .   .   .   .   .   .   .   .   .
349┆   .   .   .   .   .   .   .   .   .
789┆   .   .   .   .   .   .   .   .   .

julia> print(cm2mod.labels)
["2", "8", "29", "45", "67", "78", "168", "349", "789"]
```

Now there is a connection from the triangle 168 to the edge 78, but there are no connections from the triangle 349 to the critical edge on the center triangle. This time, up to a permutation of the columns and the rows, this connection matrix is the same as `cm2` in the example.

7.7 Forcing Three Connection Matrices

The next example is taken from [MW25, Figure 2.2], and it revolves around the combinatorial vector field on a simplicial complex shown in the top left part of the figure of the example with three connection matrices.

This combinatorial vector field is again of Forman type. It has a periodic orbit, which is shown in yellow in the top right part of this figure. In addition to three index 1 equilibria, there are two of index 2. The top right part shows that from these two index 2 cells there are a combined total of five connecting orbits to the index 1 cells and to the periodic orbit. Its Morse decomposition is shown in the lower part of this figure. While the Morse sets are indicated by different colors, the Conley-Morse graph is shown on the lower right.

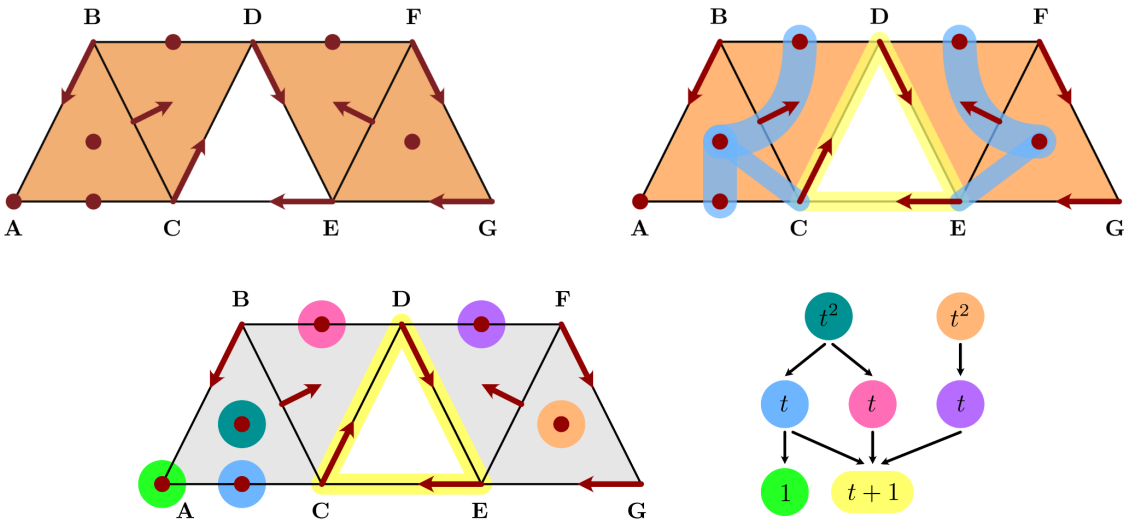


Figure 7.10: An example with three connection matrices

As the following docstring for `example_three_cm` demonstrates, the connection matrix, which this time is computed over the rationals $\mathbb{Q}$, only identifies three of the five connecting orbits between index 2 invariant sets and index 1 sets.

`ConleyDynamics.example_three_cm` – Method.

```
lc, mvf = example_three_cm(mvftype)
```

Create the simplicial complex and multivector field for the example from Figure 2.2 in the connection matrix book by Mrozek & Wanner.

Depending on the value of `mvftype`, return the periodic orbit (0=default) or one of the three gradient (1,2,3) examples.

The function returns the Lefschetz complex `lc` over the rational field and the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`, which creates `lc` as a `EuclideanComplex` with embedded coordinates.

Examples

```
julia> lc, mvf = example_three_cm(0);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> print(cm.labels)
["A", "C", "DE", "AC", "BD", "DF", "ABC", "EFG"]

julia> full_from_sparse(cm.matrix)
8×8 Matrix{Rational{Int64}}:
 0  0  0  -1  -1  0   0   0
 0  0  0   1   1  0   0   0
 0  0  0   0   0  0   0  -1
 0  0  0   0   0  0  -1   0
```

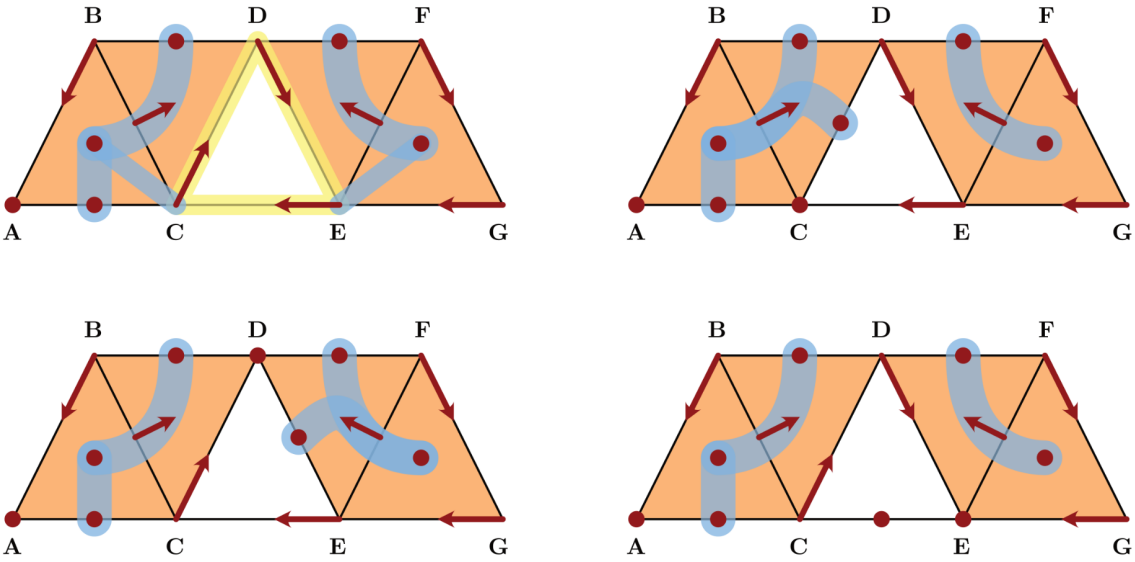


Figure 7.11: Three different ways to break up the periodic orbit

```
0  0  0   0   0  0   1   0
0  0  0   0   0  0   0   1
0  0  0   0   0  0   0   0
0  0  0   0   0  0   0   0
```

source

It turns out that this combinatorial dynamical system has multiple possible connection matrices as well. In fact, it has three. In order to find them we use the same approach as in the last example, and break the periodic orbit by turning one of its arrows into two critical cells. Since there are three arrows in the periodic orbit, this can be accomplished in three different ways. They are indicated in the figure of the three different ways to break up the periodic orbit.

The resulting Forman vector fields are all of gradient type, and therefore have a unique connection matrix. These three vector fields can be obtained via the function `example_three_cm` by specifying the integer argument as 1, 2, or 3. For the first vector field one obtains the following connection matrix:

```
julia> lc1, mvf1 = example_three_cm(1);

julia> cm1 = connection_matrix(lc1, mvf1, algorithm="DLMS");

julia> print(cm1.labels)
["A", "C", "AC", "BD", "CD", "DF", "ABC", "EFG"]

julia> full_from_sparse(cm1.matrix)
8×8 Matrix{Rational{Int64}}:
 0  0  -1  -1  0  0   0  0
 0  0   1   1  0  0   0  0
 0  0   0   0  0  0  -1  0
 0  0   0   0  0  0   1  0
 0  0   0   0  0  0  -1  0
```

```
 0  0   0   0  0  0   0  1
 0  0   0   0  0  0   0  0
 0  0   0   0  0  0   0  0
```

In contrast, the second vector field leads to:

```
julia> lc2, mvf2 = example_three_cm(2);

julia> cm2 = connection_matrix(lc2, mvf2, algorithm="DLMS");

julia> print(cm2.labels)
["A", "D", "AC", "BD", "DE", "DF", "ABC", "EFG"]

julia> full_from_sparse(cm2.matrix)
8×8 Matrix{Rational{Int64}}:
 0  0  -1  -1  0  0   0   0
 0  0   1   1  0  0   0   0
 0  0   0   0  0  0  -1   0
 0  0   0   0  0  0   1   0
 0  0   0   0  0  0   0  -1
 0  0   0   0  0  0   0   1
 0  0   0   0  0  0   0   0
 0  0   0   0  0  0   0   0
```

Finally, the third gradient vector field gives:

```
julia> lc3, mvf3 = example_three_cm(3);

julia> cm3 = connection_matrix(lc3, mvf3, algorithm="DLMS");

julia> print(cm3.labels)
["A", "E", "AC", "BD", "CE", "DF", "ABC", "EFG"]

julia> full_from_sparse(cm3.matrix)
8×8 Matrix{Rational{Int64}}:
 0  0  -1  -1  0  0   0  0
 0  0   1   1  0  0   0  0
 0  0   0   0  0  0  -1  0
 0  0   0   0  0  0   1  0
 0  0   0   0  0  0   0  0
 0  0   0   0  0  0   0  1
 0  0   0   0  0  0   0  0
 0  0   0   0  0  0   0  0
```

This is finally the connection matrix that was originally returned for the Forman vector field with periodic orbit. One could have obtained the remaining two also through cell permutations.

Notice that these three matrices combined do identify all of the above connections. It was shown in [MW25] that these matrices are different connection matrices for the original Forman vector field with periodic orbit, as long as the newly introduced index 1 and 0 equilibria are identified with the Conley index of the periodic solution. For the sake of completeness, the figure of the Morse decompositions for the three gradient vector fields shows the Morse decompositions for all three combinatorial gradient flows. In the Conley-Morse graphs, blue arrows correspond to the heteroclinic orbits that are identified by the associated connection matrix.

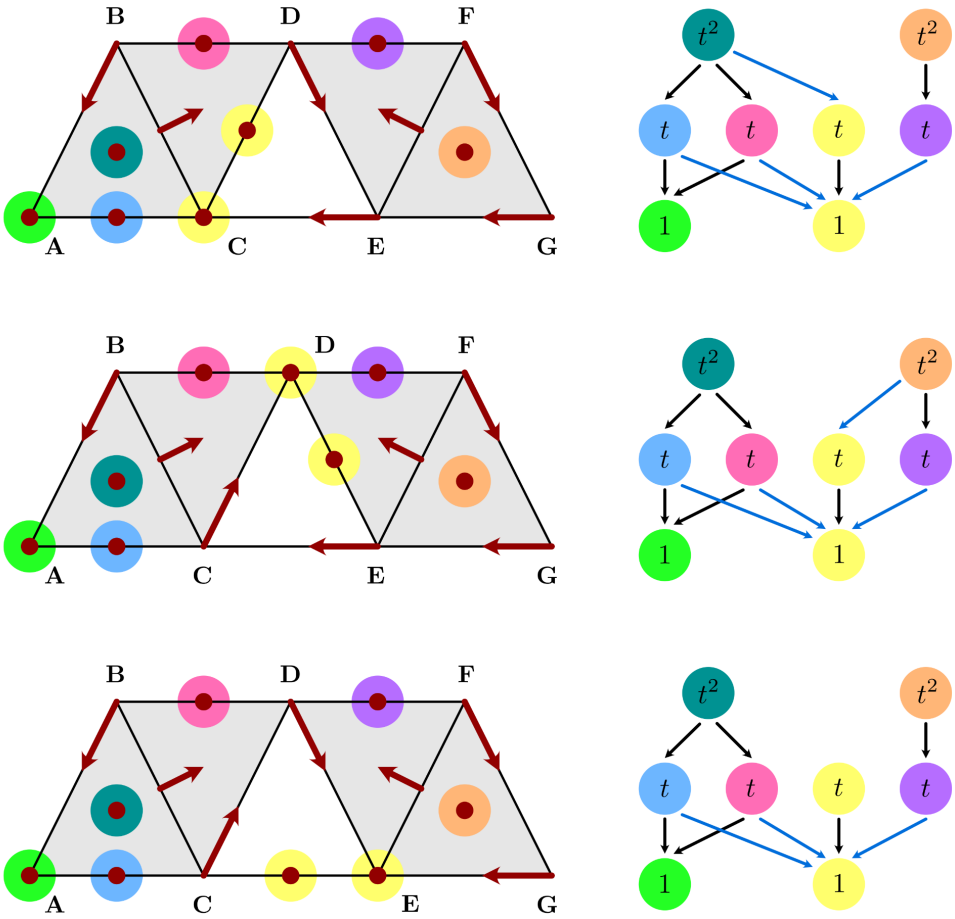


Figure 7.12: Morse decompositions for the three gradient vector fields

In the Conley-Morse graphs, we used the same color yellow for the two Morse sets that are generated by breaking the periodic orbit through the introduction of two critical cells. It can be seen from these Conley-Morse graphs that while the actual Morse set structure stays fixed, the poset order in the Conley-Morse graphs changes from case to case.

7.8 A Lefschetz Multiflow Example

The next example is taken from [MW25, Figure 2.3], and it is a combinatorial multivector field on a true Lefschetz complex, as shown in the left panel of the figure of the multiflow example with trivial connection matrix.

The example is a combinatorial representation of the multiflow shown in the right panel of this figure, which features nonunique forward dynamics at the point labeled 5. Note that the underlying Lefschetz complex is defined as the subset of the depicted simplicial complex, where the vertices A, B, D, E, and F have been removed. This Lefschetz complex `lc` and the depicted multivector field `mvf` can be created using the function `example_multiflow`:

`ConleyDynamics.example_multiflow` – Method.

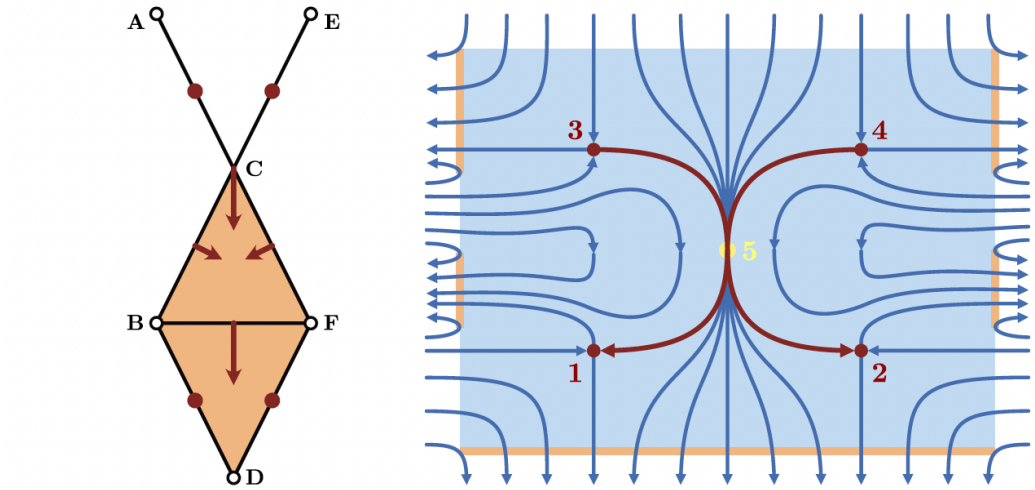


Figure 7.13: A multiflow example with trivial connection matrix

```
lc, mvf = example_multiflow()
```

Create the Lefschetz complex and multivector field for the example from Figure 2.3 in the connection matrix book by Mrozek & Wanner.

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_multiflow();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆ BD DF AC CE
---┆------------
BD ┆  .  .  .  .
DF ┆  .  .  .  .
AC ┆  .  .  .  .
CE ┆  .  .  .  .

julia> print(cm.labels)
["BD", "DF", "AC", "CE"]
```

source

As the docstring shows, this example has a trivial connection matrix. In other words, there are no connecting orbits in this combinatorial dynamical system that are forced by topology. In fact, one can easily see that due to the multivalued nature of the dynamical system, one cannot expect any particular heteroclinic to be present.

The Morse decomposition of the system, and the associated Conley indices encompass precisely the four critical edges:

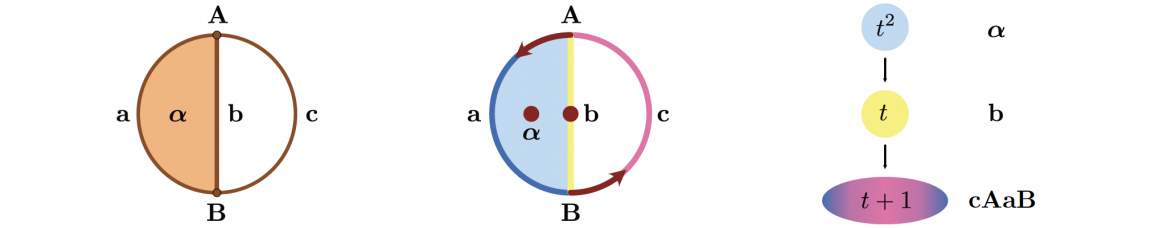


Figure 7.14: A small Lefschetz complex with periodic orbit

```
julia> cm.morse
4-element Vector{Vector{String}}:
 ["BD"]
 ["DF"]
 ["AC"]
 ["CE"]

julia> cm.conley
4-element Vector{Vector{Int64}}:
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
```

Combined with the fact that the connection matrix is trivial, this means that the homology of the underlying Lefschetz complex `lc` is the sum of the Conley indices of the Morse sets. This can be confirmed using the function `homology`:

```
julia> homology(lc)
3-element Vector{Int64}:
 0
 4
 0
```

As we mentioned earlier, this is the same as the relative homology of the full simplicial complex with respect to the union of the five removed vertices.

7.9 Small Complex with Periodicity

In [MW25, Figure 2.4] we introduced a small Lefschetz complex with periodic orbit and nonunique connection matrices. This complex consists of one 2-cell, three 1-cells, and two 0-cells, and it is shown in the leftmost panel of the figure of the small Lefschetz complex with periodic orbit.

On the complex, consider the multivector field depicted in the middle panel of this figure, which consists of the critical cells α and b, as well as the two regular multivectors $\{A, a\}$ and $\{B, c\}$. For this small example, one can easily determine the Morse decomposition, and it is shown in the rightmost panel. The example can be generated in ConleyDynamics.jl using the function `example_small_periodicity`:

`ConleyDynamics.example_small_periodicity` – Method.

```
lc1, lc2, mvf = example_small_periodicity()
```

Create two representations of the Lefschetz complex and the multivector field for the example from Figure 2.4 in the connection matrix book by Mrozek & Wanner.

The complexes `lc1` and `lc2` are just two representations of the same complex, but they lead to different connection matrices. Both Lefschetz complexes are defined over the finite field GF(2).

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector field `mvf`.

Examples

```
julia> lc1, lc2, mvf = example_small_periodicity();

julia> cm1 = connection_matrix(lc1, mvf, algorithm="DLMS");

julia> cm2 = connection_matrix(lc2, mvf, algorithm="DLMS");

julia> full_from_sparse(cm1.matrix)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  1
 0  0  0  1
 0  0  0  0

julia> print(cm1.labels)
["A", "a", "b", "alpha"]

julia> full_from_sparse(cm2.matrix)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  0
 0  0  0  1
 0  0  0  0

julia> print(cm2.labels)
["A", "c", "b", "alpha"]
```

source

The function provides two different representations of the same Lefschetz complex, which only differ in the ordering of the 1-cells. This can be seen from the commands:

```
julia> print(lc1.labels)
["A", "B", "a", "b", "c", "alpha"]

julia> print(lc2.labels)
["A", "B", "c", "a", "b", "alpha"]
```

As the above docstring shows, these different versions lead to two different connection matrices `cm1` and `cm2`.

In this small example, one can easily determine the connection matrices directly, as illustrated in the figure on the direct derivation of connection matrices. While the detailed explanation can be found in [MW25], this

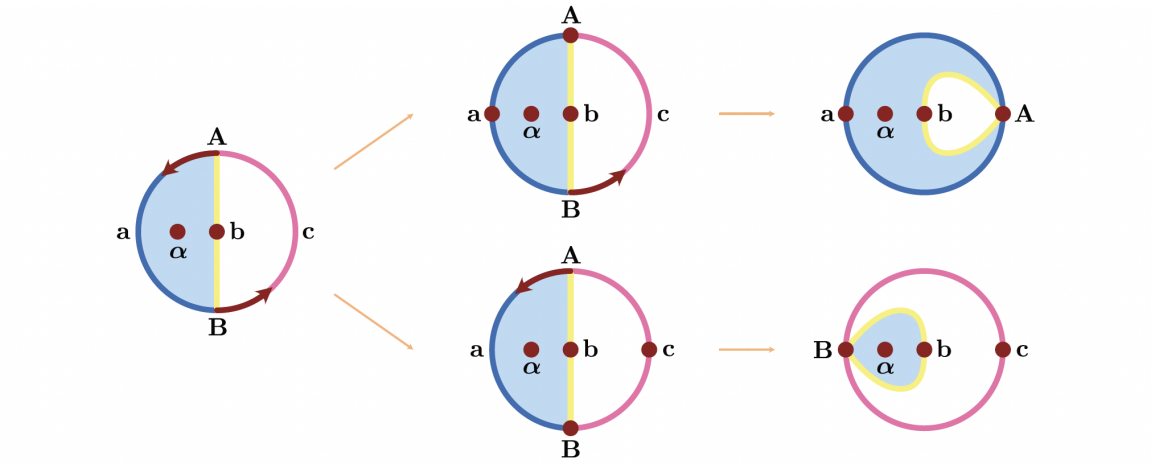


Figure 7.15: Direct derivation of connection matrices

is basically accomplished by contracting one of the two regular multivectors in a process called elementary reduction. While the details of this approach are described in [KMS98], it relies on identifying a reduction pair, which contains a cell and one of its faces of one dimension less. These two cells are then removed from the Lefschetz complex and the boundary is modified in such a way that the new smaller complex still has the same homology as the previous one. If in our above example one uses the reduction pair $\{B, c\}$, then the boundary matrix of the reduced Lefschetz complex is the connection matrix `cm1`, while `cm2` is the result of using the reduction pair $\{A, a\}$. These manipulations can be done in ConleyDynamics.jl using the function `lefschetz_reduction`:

```
julia> rc1 = lefschetz_reduction(lc1, "B", "c");

julia> rc2 = lefschetz_reduction(lc1, "A", "a");
```

These two commands compute the reduced Lefschetz complexes `rc1` and `rc2` for the respective elementary reduction pairs $\{B, c\}$ and $\{A, a\}$. As the following commands show, the first one leads to our earlier connection matrix `cm1`:

```
julia> full_from_sparse(rc1.boundary)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  1
 0  0  0  1
 0  0  0  0

julia> println(rc1.labels)
["A", "a", "b", "alpha"]
```

Similarly, the connection matrix `cm2` is the result of the second reduction:

```
julia> full_from_sparse(rc2.boundary)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  1
```

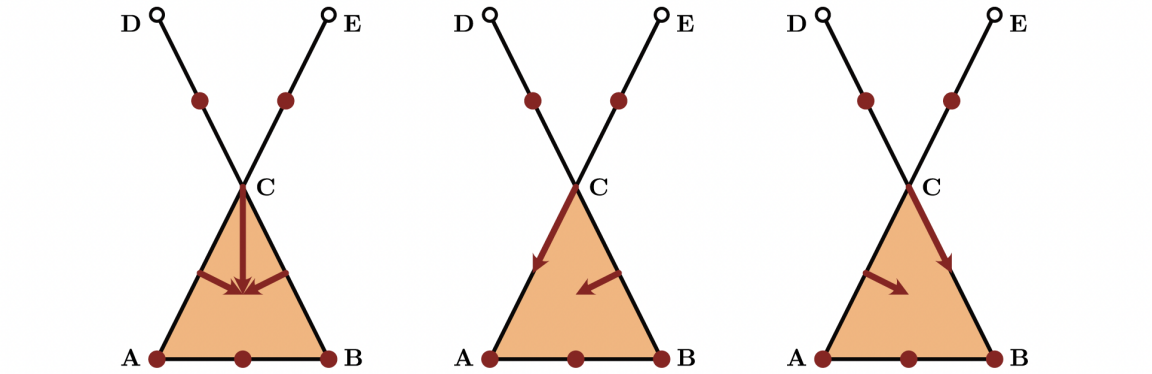


Figure 7.16: Subdividing a multivector

```
 0  0  0  0
 0  0  0  0

julia> println(rc2.labels)
["B", "b", "c", "alpha"]
```

7.10 Subdividing a Multivector

Our next example taken from [MW25] is concerned with turning a given multivector field into a Forman vector field by further subdividing the multivectors. For this, consider the three complexes and combinatorial vector fields shown in the figure on subdividing a multivector, see also [MW25, Figure 7.1].

The depicted combinatorial dynamical systems all use the same Lefschetz complex, which is obtained from a simplicial complex by removing two vertices. In addition to the multivector field shown in the leftmost panel, we also consider two Forman vector fields. Notice that the multivector field has one multivector of size four, which is given by $\{\mathrm{C}, \mathrm{AC}, \mathrm{BC}, \mathrm{ABC}\}$. This vector can be split into two Forman arrows, and in view of the required local closedness this can be achieved in precisely two ways. The splitting into the arrows $\{\mathrm{C}, \mathrm{AC}\}$ and $\{\mathrm{BC}, \mathrm{ABC}\}$ gives the Forman vector field shown in the middle panel, while the one depicted in the rightmost panel uses the arrows $\{\mathrm{C}, \mathrm{BC}\}$ and $\{\mathrm{AC}, \mathrm{ABC}\}$. These fields can be created using the function `example_subdivision`:

`ConleyDynamics.example_subdivision` – Method.

```
lc, mvf = example_subdivision(mvftype)
```

Create the Lefschetz complex and multivector field for the example from Figure 7.1 in the connection matrix book by Mrozek & Wanner.

Depending on the value of `mvftype`, return the multivector (0=default) or one of the two combinatorial vector field (1,2) examples.

The function returns the Lefschetz complex `lc` over the rationals and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_subdivision(1);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> full_from_sparse(cm.matrix)
5×5 Matrix{Rational{Int64}}:
 0  0  -1  -1  -1
 0  0   1   0   0
 0  0   0   0   0
 0  0   0   0   0
 0  0   0   0   0
```

source

The different combinatorial vector fields can be selected via the function argument, which is an integer between 0 and 2, from left to right in the figure on subdividing a multivector. Thus, all fields and connection matrices can be computed using the commands

```
lc0, mvf0 = example_subdivision(0)
lc1, mvf1 = example_subdivision(1)
lc2, mvf2 = example_subdivision(2)
cm0 = connection_matrix(lc0, mvf0, algorithm="DLMS")
cm1 = connection_matrix(lc1, mvf1, algorithm="DLMS")
cm2 = connection_matrix(lc2, mvf2, algorithm="DLMS")
```

All three vector fields give rise to the same Morse decomposition, since in each case the vectors of length at least two are regular. This can be seen for the multivector field below, and is analogous for the two Forman vector fields.

```
julia> cm0.morse
5-element Vector{Vector{String}}:
 ["A"]
 ["B"]
 ["AB"]
 ["CD"]
 ["CE"]

julia> cm0.conley
5-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [1, 0, 0]
 [0, 1, 0]
 [0, 1, 0]
 [0, 1, 0]
```

The three connection matrices are as follows:

```
julia> full_from_sparse(cm0.matrix)
5×5 Matrix{Rational{Int64}}:
 0  0  -1   0   0
 0  0   1  -1  -1
 0  0   0   0   0
```

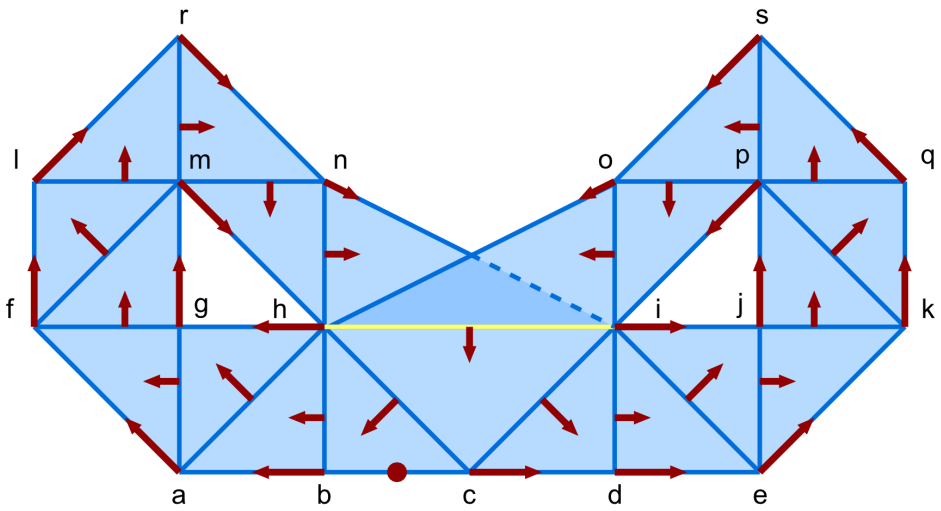


Figure 7.17: A combinatorial Lorenz system

```
 0  0   0   0   0
 0  0   0   0   0

julia> full_from_sparse(cm1.matrix)
5×5 Matrix{Rational{Int64}}:
 0  0  -1  -1  -1
 0  0   1   0   0
 0  0   0   0   0
 0  0   0   0   0
 0  0   0   0   0

julia> full_from_sparse(cm2.matrix)
5×5 Matrix{Rational{Int64}}:
 0  0  -1   0   0
 0  0   1  -1  -1
 0  0   0   0   0
 0  0   0   0   0
 0  0   0   0   0
```

Notice that the connection matrices for the two Forman vector fields are uniquely determined, since both are gradient vector fields. The matrices are, however, different. At first glance, the connection matrix for `mvf0` is equal to the one for `mvf2`. Yet, this is another example of nonuniqueness, and one could produce also the connection matrix for `mvf1` through a reordering of the cells in the Lefschetz complex.

7.11 A Combinatorial Lorenz System

Our next example is taken from [KMW16, Figure 3]. It is a Forman vector field on a two-dimensional simplicial complex, as shown in the figure of the combinatorial Lorenz system.

Notice that the underlying simplicial complex does not represent a manifold in this case, but rather a branched manifold. As indicated in this figure, the two triangles $\mathbf{hin}$ and $\mathbf{hio}$ only intersect in the edge $\mathbf{hi}$, and this edge is also contained in the third triangle $\mathbf{chi}$. This system is a combinatorial version of the famous Lorenz

butterfly, which is well-known for its chaotic behavior. The simplicial complex and the Forman vector field can be created using the function `example_clorenz`:

`ConleyDynamics.example_clorenz` – Method.

```
lc, mvf = example_clorenz()
```

Create the simplicial complex and multivector field for the example from Figure 3 in the JCD 2016 paper by Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` over the finite field GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_clorenz();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
  ┆  i jp  g hm bc
──┆──────────────
 i┆  .  .  .  .  1
jp┆  .  .  .  .  .
 g┆  .  .  .  .  1
hm┆  .  .  .  .  .
bc┆  .  .  .  .  .

julia> print(cm.labels)
["i", "jp", "g", "hm", "bc"]

julia> ms, ps = morse_sets(lc, mvf, poset=true);

julia> [conley_index(lc, mset) for mset in ms]
4-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [1, 1, 0]
 [0, 1, 0]
 [0, 0, 0]

julia> ps
4×4 Matrix{Bool}:
 0  0  1  0
 0  0  1  0
 0  0  0  1
 0  0  0  0
```

source

The first of the above commands creates the simplicial complex `lc` and the Forman vector field `mvf`. These are then analyzed using the following commands:

- Using `cm = connection_matrix(lc, mvf, algorithm="DHL")` one can compute the connection matrix of the example. This connection matrix has two nonzero entries, which indicate connecting orbits from the index 1 critical cell $\mathbf{bc}$ to each of the stable periodic orbits spanned by the vertices $\mathbf{h}$, $\mathbf{g}$, $\mathbf{m}$ and $\mathbf{i}$, $\mathbf{j}$, $\mathbf{p}$, respectively.

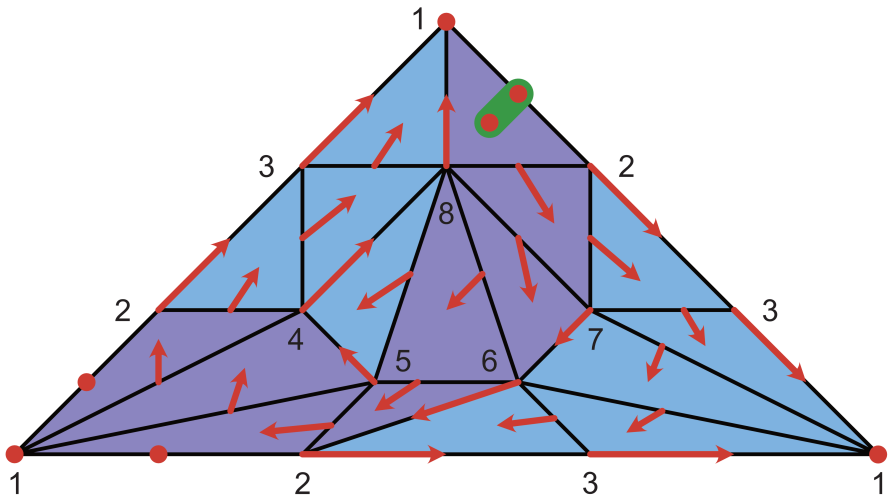


Figure 7.18: Forman chaos in the Dunce hat

- The command `ms, ps = morse_sets(lc, mvf, poset=true)` shows, however, that there is more to this example. While the above connection matrix indicates only three isolated invariant sets, there is actually a fourth one. In view of the Conley index computation for these sets, the additional Morse set has trivial index, and therefore does not show up in the connection matrix. Notice that the partial order given by the flow, which is indicated by the matrix `ps`, implies that the Morse set with trivial index has a connection to the index 1 critical cell.

Upon closer inspection one can see that the last Morse set is comprised of all triangles, as well as all edges which are contained in at least two triangles. This set contains infinitely many periodic orbits. For any bi-infinite sequence of symbols L and R there is a periodic orbit which loops around the left hole for L and around the right hole for R, as one traverses the symbol sequence from left to right. Such behavior is one potential indicator for chaos. In fact, it was shown in [MW21] that it is possible to define a classical semiflow on the branched manifold defined by `lc` which mimics the behavior of the Forman vector field. And according to [MSTW22], any such admissible semiflow does indeed have infinitely many periodic orbits in the set determined by the triangles.

7.12 Chaos in the Dunce Hat

In the above example we observed chaotic behavior in a branched manifold. It is also possible to introduce similar behavior in the Dunce hat.

The required simplicial complex and Forman vector field can be created using the function `example_dunce_chaos`:

`ConleyDynamics.example_dunce_chaos` – Function.

```
sc, vfG, vfC = example_dunce_chaos()
```

Create a minimal simplicial complex representation of the Dunce hat, as well as two Forman vector fields.

The function returns the simplicial representation of the Dunce hat in `sc` over the finite field `GF(2)`. The Forman vector field `vfG` is a gradient vector field with unique connection matrix. The field `vfC` is a modifi-

cation of this field which merges the critical cells of dimensions 1 and 2 into a Forman arrow. The resulting Forman vector field is no longer gradient, and in fact exhibits Lorez-like chaos.

Examples

```
julia> sc, vfG, vfC = example_dunce_chaos();

julia> homology(sc)
3-element Vector{Int64}:
 1
 0
 0

julia> cmG = connection_matrix(sc, vfG, algorithm="DHL");

julia> sparse_show(cmG)
   ┆   1  12 128
---┆------------
  1┆   .   .   .
 12┆   .   .   1
128┆   .   .   .

julia> print(cmG.labels)
["1", "12", "128"]

julia> cmC = connection_matrix(sc, vfC, algorithm="DHL");

julia> sparse_show(cmC)
 ┆ 1
-┆--
1┆ .

julia> print(cmC.labels)
["1"]

julia> msC, psC = morse_sets(sc, vfC, poset=true);

julia> [conley_index(sc, mset) for mset in msC]
2-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 0, 0]

julia> psC
2×2 Matrix{Bool}:
 0  1
 0  0

julia> msC
2-element Vector{Vector{String}}:
 ["1"]
 ["12", "14", "15", "25", "28", "56", "68", "78", "124", "125", "128", "145", "256", "278",
↪  "568", "678"]
```

source

The first of the above commands creates the simplicial complex `sc` and two Forman vector fields. The simplicial

complex contains a minimal triangulation of the Dunce hat, which has 8 vertices, 24 edges, and 17 triangles. The variable `vfG` returns a gradient vector field, as shown in the figure of the Forman chaos in the Dunce hat. This vector field has the three critical simplices 1, 12, and 128. Finally, the chaotic Forman vector field is returned in `vfC`. It is obtained from the gradient field by combining the two critical cells 12 and 128 into a Forman arrow, which is indicated in green in this figure. This reverses the flow between these two simplices. In more detail, the above example provides the following analysis:

- The command `homology(sc)` shows that the Dunce hat has the homology of a point, i.e., it is contractible.
- The next command `cmG = connection_matrix(sc, vfG, algorithm="DHL")` determines the connection matrix for the gradient system. As the entry `cmG.labels` shows, the Morse sets are the three simplices 1, 12, and 128, and the nontrivial entry in the connection matrix indicates at least one connection between the index 2 critical cell and the index 1 critical cell. In fact, there are three such connections.
- The command `cmC = connection_matrix(sc, vfC, algorithm="DHL")` computes the connection matrix. This time, the matrix is the zero matrix with one row and column, which correspond to the stable critical cell 1.
- But there is nontrivial dynamics, as the command `msC, psC = morse_sets(sc, vfC, poset=true)` demonstrates. It produces two Morse sets. In addition to the stable critical simplex, one also obtains an isolated invariant set with trivial index. This set exhibits chaotic behavior, and is shown in purple in the figure of the Forman chaos in the Dunce hat.
- The return variable `psC` gives the flow-induced order between the Morse sets, which shows that there are heteroclinic orbits between the chaotic Morse set and the stable equilibrium.
- The variable `msC` contains the two Morse sets. While the first is the stable equilibrium, the second one consists of 8 edges and 8 triangles.

One can see from the figure of the Forman chaos in the Dunce hat that the chaotic isolated invariant set for the Forman vector field `vfC` has two periodic orbits. Starting with the edge 12 in the upper right, they both traverse the edges 28, 78, 68, 56, and 25. But while the first periodic orbit then returns directly to 12 along the bottom edge, the second one moves first through 15 and 14, before returning to 12 along the left edge.

7.13 Chaos in a Space with Torsion

Our next example describes gradient Forman vector fields on simplicial complexes with torsion whose connection matrices can have large entries, as long as the underlying field has a large enough characteristic. In addition, by merging two critical cells in the Forman gradient vector field into an arrow, once obtains a Forman vector field with chaotic behavior. The simplicial complexes are based on the function `simplicial_torsion_space`, and the Forman vector fields can be seen in the figure of the Forman chaos in a space with torsion for the cases $n = 3$ and $n = 4$.

After specifying the torsion coefficient n and the field characteristic p, the underlying simplicial complex and the two Forman vector fields can be created using the function `example_torsion_chaos`:

`ConleyDynamics.example_torsion_chaos` – Function.

```
sc, vfG, vfC = example_torsion_chaos(n::Int, p::Int)
```

Create a triangulation of a space with 1-dimensional torsion, as well as two Forman vectr fields on this complex.

The function returns a simplicial complex `sc` which has the following integer homology groups:

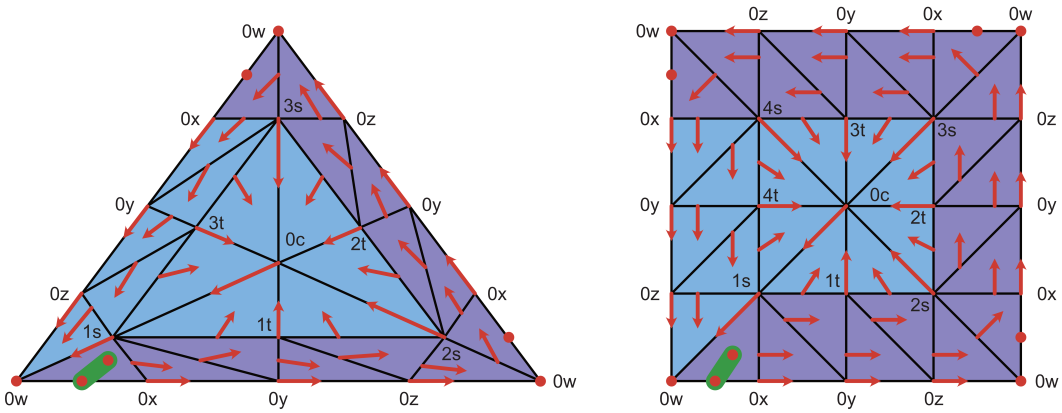


Figure 7.19: Forman chaos in a space with torsion

- In dimension 0 it is the group of integers.
- In dimension 1 it is the integers modulo n.
- In dimension 2 it is the trivial group.

In other words, the simplicial complex has nontrivial torsion in dimension 1. It is a triangulation of an n-gon, in which all boundary edges are oriented counterclockwise, and all of these edges are identified. The parameter p specifies the characteristic of the underlying field.

In addition, two Forman vector fields `vfG` and `vfC` are returned. The first one is a gradient vector field whose connection matrix has a large connection matrix entry. In fact, if p is any prime larger than n then there will be an entry n in the matrix. The scond Forman vector field contains a chaotic Morse set. This Morse set will have trivial Morse index for most p. On the other hand, for prime p = n the set has the Morse index of an unstable periodic orbit.

Examples

```
julia> sc, vfG, vfC = example_torsion_chaos(3,7);

julia> homology(sc)
3-element Vector{Int64}:
 1
 0
 0

julia> cmG = connection_matrix(sc, vfG, algorithm="DHL");

julia> sparse_show(cmG)
       ┆     0w   0w0x 0w0x1s
───────┼─────────────────────
    0w ┆      .      .      .
  0w0x ┆      .      .      3
0w0x1s ┆      .      .      .

julia> print(cmG.labels)
["0w", "0w0x", "0w0x1s"]

julia> cmC = connection_matrix(sc, vfC, algorithm="DHL");
```

```
julia> sparse_show(cmC)
  ┆ 0w
--┆---
0w┆  .

julia> print(cmC.labels)
["0w"]

julia> msC, psC = morse_sets(sc, vfC, poset=true);

julia> [conley_index(sc, mset) for mset in msC]
2-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 0, 0]

julia> psC
2×2 Matrix{Bool}:
 0  1
 0  0

julia> length.(msC)
2-element Vector{Int64}:
  1
 26
```

source

The first of the above commands creates the simplicial complex `sc` and the two Forman vector fields. The gradient vector field is returned in `vfG`, and it corresponds to the Forman vector fields shown in red in the figure of the Forman chaos in a space with torsion. The return variable `vfC` gives the chaotic Forman vector field. It is obtained from the gradient field by combining the two critical cells $0\mathrm{w}0\mathrm{x}$ and $0\mathrm{w}0\mathrm{x}1\mathrm{s}$ into a Forman arrow, which is indicated in green in this figure. Note that this reverses the flow between these two simplices. In the above example, we use $n = 3$ and $p = 7$.

The gradient Forman vector field `vfG` is then analyzed using the following commands:

- The command `homology(sc)` shows that in the field $GF(7)$, the space has the homology of a point.
- The next command `cmG = connection_matrix(sc, vfG, algorithm="DHL")` determines the connection matrix for the gradient system. As the entry `cmG.labels` shows, the Morse sets are the three simplices $0\mathrm{w}$, $0\mathrm{w}0\mathrm{x}$, and $0\mathrm{w}0\mathrm{x}1\mathrm{s}$, and the nontrivial entry in the connection matrix indicates three connections between the index 2 critical cell and the index 1 critical cell.

The next commands analyze the chaotic Forman vector field `vfC`:

- As before, the command `cmC = connection_matrix(sc, vfC, algorithm="DHL")` computes the connection matrix. This time, the matrix is the zero matrix with one row and column, which correspond to the stable critical cell $0\mathrm{w}$.
- But there is nontrivial dynamics, as the command `msC, psC = morse_sets(sc, vfC, poset=true)` demonstrates. It produces two Morse sets. In addition to the stable critical cell, one also obtains an isolated invariant set with trivial index. This set exhibits chaotic behavior, and is shown in purple in the

figure of the Forman chaos in a space with torsion. In the case $n = 3$, it consists of 26 simplices. In the general case, it will have $12n - 10$ simplices.

- The return variable `psC` gives the flow-induced order between the Morse sets, which shows that there are heteroclinic orbits between the chaotic Morse set and the stable equilibrium.

7.14 References

See the full bibliography for a complete list of references cited throughout this documentation. This section cites the following references:

[BKMW20] B. Batko, T. Kaczynski, M. Mrozek and T. Wanner. Linking combinatorial and classical dynamics: Conley index and Morse decompositions. Foundations of Computational Mathematics **20**, 967–1012 (2020).

[KMS98] T. Kaczynski, M. Mrozek and M. Slusarek. Homology computation by reduction of chain complexes. Computers & Mathematics with Applications **35**, 59–70 (1998).

[KMW16] T. Kaczynski, M. Mrozek and T. Wanner. Towards a formal tie between combinatorial and classical vector field dynamics. Journal of Computational Dynamics **3**, 17–50 (2016).

[MSTW22] M. Mrozek, R. Srzednicki, J. Thorpe and T. Wanner. Combinatorial vs. classical dynamics: Recurrence. Communications in Nonlinear Science and Numerical Simulation **108**, Paper No. 106226, 30 pages (2022).

[MW21] M. Mrozek and T. Wanner. Creating semiflows on simplicial complexes from combinatorial vector fields. Journal of Differential Equations **304**, 375–434 (2021).

[MW25] M. Mrozek and T. Wanner. Connection Matrices in Combinatorial Topological Dynamics. SpringerBriefs in Mathematics (Springer-Verlag, Cham, 2025).

Chapter 8

Two-Dimensional Meshes

This chapter describes how planar regions can be triangulated, possibly with prescribed constraint curves, and how the resulting triangulations are turned into simplicial complexes for the analysis with ConleyDynamics.jl. The core function `create_simplicial_delaunay` already creates Delaunay triangulations of random point samples in rectangles, see the Tutorial. For more control over the mesh, and in particular for constraint curves, the package provides an extension for the mesh generator DelaunayTriangulation.jl, which is described in the remainder of this chapter. Meshes of three-dimensional domains are the topic of the chapter Three-Dimensional Meshes, and the visualization of planar complexes is described in the Visualization chapter.

8.1 Loading the Required Packages

`DelaunayTriangulation.jl` is a weak dependency of ConleyDynamics.jl: It is not installed together with ConleyDynamics.jl, and the functions described in this chapter only become available once both packages have been loaded. The package has to be installed once in the active Julia environment, using

```
import Pkg
Pkg.add("DelaunayTriangulation")
```

Versions `1.x` of `DelaunayTriangulation.jl` are supported. In every Julia session which uses the functions of this chapter, both packages then have to be loaded:

```
using DelaunayTriangulation
using ConleyDynamics
```

The order of the two commands does not matter, and `import` can be used instead of `using`. As soon as both packages are loaded, Julia automatically loads the extension module `ConleyDynamicsDelaunayExt`, which provides the functions `delaunay_points_bnd_rectangle`, `delaunay_points_add_nodes`, `delaunay_points_add_segment`, `delaunay_points_add_split_segs`, and `delaunay_to_simplicial`. The functions `triangulate`, `refine!`, and `get_area` used below belong to `DelaunayTriangulation.jl` itself, which is why it has to be loaded with `using` to call them without the module prefix. If one of the above functions is called without loading `DelaunayTriangulation.jl`, Julia reports that no matching method exists:

```
using ConleyDynamics
delaunay_points_bnd_rectangle([-5, -5], [5, 5])
# ERROR: MethodError: no method matching
#        delaunay_points_bnd_rectangle(::Vector{Int64}, ::Vector{Int64})
```

In that case, it suffices to load `DelaunayTriangulation.jl` and to repeat the call. The function `split_segments`, which is described at the end of this chapter, is part of the core package and can be used without the extension. For the figures in this chapter, the package `Plots.jl` was used in addition, see the Visualization chapter.

8.2 DelaunayTriangulation.jl

The DelaunayTriangulation.jl package computes Delaunay triangulations of planar point sets, optionally with constrained edges and boundary curves. Of the five functions provided by the extension, the following three form the basic workflow:

- `delaunay_points_bnd_rectangle` — initialise a point list and outer boundary for a rectangular domain,
- `delaunay_points_add_segment` — append constraint curves (closed or open) to that point list, and
- `delaunay_to_simplicial` — convert the finished triangulation into a `EuclideanComplex`.

The typical workflow is to assemble the geometry with the first two functions, call `triangulate` (and optionally `refine!`) from `DelaunayTriangulation.jl`, and then convert the result with `delaunay_to_simplicial`.

Defining the Bounding Domain

`delaunay_points_bnd_rectangle` creates a four-vertex rectangular outer boundary. Its two arguments are vectors `bmin = [xmin, ymin]` and `bmax = [xmax, ymax]` giving the lower-left and upper-right corners. It returns a point list and a `bndcurve` in the format expected by the `boundary_nodes` keyword of `triangulate`:

```
using DelaunayTriangulation
using ConleyDynamics

points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])
tri1 = triangulate(points; boundary_nodes = bndcurve)
sc1  = delaunay_to_simplicial(tri1)
```

This already produces a valid `EuclideanComplex` — a triangulated filled rectangle without any internal constraints. It is shown in the left panel of the figure with two sample Delaunay triangulations of a square, i.e., the panel with only the four corners as vertices.

Adding Interior Nodes

A finer triangulation of the underlying rectangle can be achieved by adding interior nodes to the list of points describing the boundary. This can be achieved with the function `delaunay_points_add_nodes`. For example, we can add twelve interior nodes to the above square using the following commands:

```
newpoints = [-4 .+ 8 .* rand(2) for k in 1:12]   # 12 random points
points    = delaunay_points_add_nodes(points, newpoints)

tri2 = triangulate(points; boundary_nodes = bndcurve)
sc2  = delaunay_to_simplicial(tri2)
```

This leads to the triangulation shown in the right panel of the figure with two sample Delaunay triangulations of a square. Note that the new triangulation makes use of all new interior nodes, as well as the corners of the squares. This figure was created using the commands

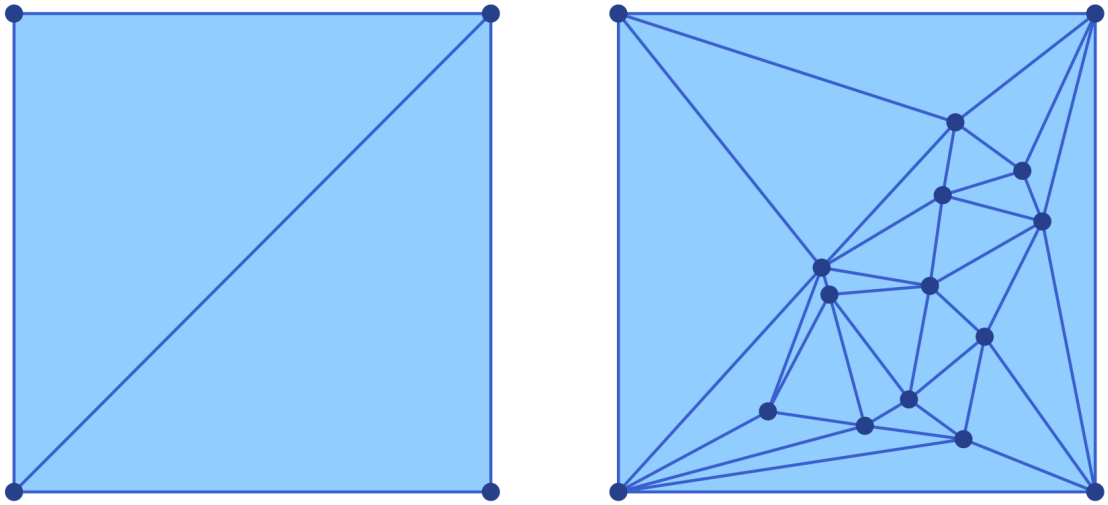


Figure 8.1: Two sample Delaunay triangulations of a square

```
using Plots
p1 = plot_simplicial(sc1)
p2 = plot_simplicial(sc2)
pcombined = plot(p1, p2, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "delaunayext1.png")
```

Adding Constraint Curves

In many applications it is desirable to include certain polygonal curves inside the domain as constraint curves, i.e., their edges have to be part of the created Delaunay triangulation and any of its refinements. Such constraint curves can be added to the initial mesh generation by adding the vertices of the polygons to the point list, and specifying the constraint edges in an argument `segments`, which has to be of type `Set{Tuple{Int,Int}}`.

Such constraint curves can be generated as follows. `delaunay_points_add_segment` appends a polygonal curve to the point list and records successive point pairs as constrained edges. A curve is passed as a `Vector{Vector{<:Real}}` of `[x, y]` coordinates. If the first and last entries coincide (within a tolerance of `1e-9`), the curve is treated as closed and no duplicate endpoint is stored; otherwise it is treated as open.

The function has two calling forms. The two-argument form initialises a fresh segment set and is convenient for the first constraint curve; the three-argument form extends an existing `segments::Set{Tuple{Int,Int}}`:

```
using DelaunayTriangulation
using ConleyDynamics

points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])

# Closed circular constraint (n+1 points with repeated first/last)
n      = 10
theta  = range(0, 2*pi, length = n+1)
circle = [[4.0 * cos(t), 4.0 * sin(t)] for t in theta]

# Open polygonal cut inside the domain
```

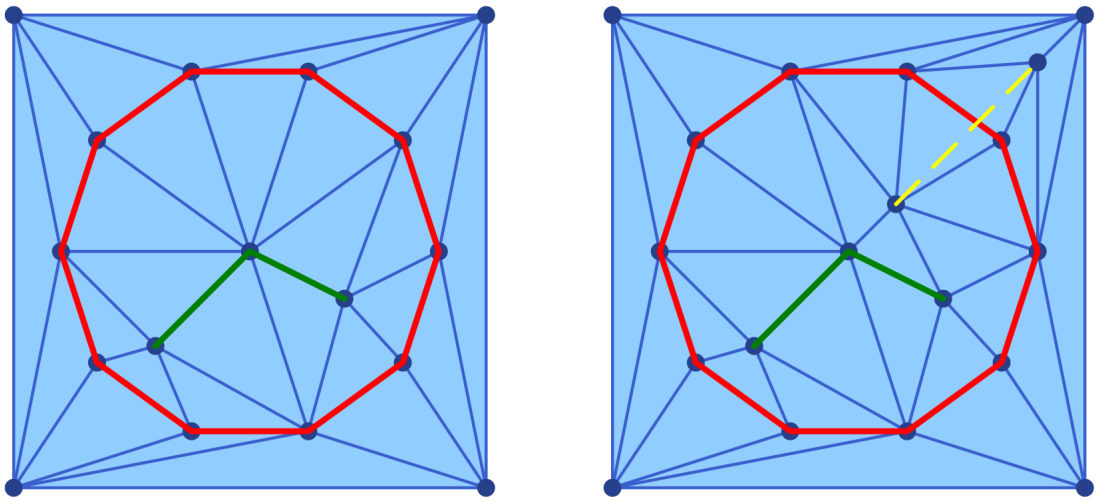


Figure 8.2: Delaunay triangulations with constraint curves

```
cut = [[-2.0, -2.0], [0.0, 0.0], [2.0, -1.0]]

points, segments = delaunay_points_add_segment(points, circle)          # 2-arg form
points, segments = delaunay_points_add_segment(points, segments, cut)   # 3-arg form

tri1 = triangulate(points; boundary_nodes = bndcurve, segments = segments)
sc1  = delaunay_to_simplicial(tri1)
```

The resulting Delaunay triangulation is shown in the left panel of the figure of the Delaunay triangulations with constraint curves. In addition to the generated mesh, it also shows the two constraint curves in red and green. This plot was generated using the following commands:

```
using Plots

cirx = [pt[1] for pt in circle]
ciry = [pt[2] for pt in circle]
cutx = [pt[1] for pt in cut]
cuty = [pt[2] for pt in cut]

p1 = plot_simplicial(sc1)
plot!(p1, cirx, ciry, linewidth=3, color=:red)
plot!(p1, cutx, cuty, linewidth=3, color=:green)
```

We would like to point out that adding constraint curves to a triangulation has to be done with care. More precisely, it is up to the user to ensure that the **curves do not intersect each other** and that each curve is **without interior self-intersections**. If these assumptions are not satisfied, then DelaunayTriangulation.jl will automatically remove some of the constraints. Consider for the example the following additional commands:

```
# Open polygonal cut inside the domain
badcut = [[1.0, 1.0], [4.0, 4.0]]
points, segments = delaunay_points_add_segment(points, segments, badcut)
```

```
tri2 = triangulate(points; boundary_nodes = bndcurve, segments = segments)
sc2  = delaunay_to_simplicial(tri2)

badx = [pt[1] for pt in badcut]
bady = [pt[2] for pt in badcut]

p2 = plot_simplicial(sc2)
plot!(p2, cirx, ciry, linewidth=3, color=:red)
plot!(p2, cutx, cuty, linewidth=3, color=:green)
plot!(p2, badx, bady, linewidth=2, color=:yellow, linestyle=:dash)

pcombined12 = plot(p1, p2, layout=(1,2))
plot!(pcombined12, dpi=300)
savefig(pcombined12, "delaunayext2.png")
```

Clearly the added cut `badcut` intersects the circular constraint `circle` from before. The mesh generated by `DelaunayTriangulation.jl` is shown in the right panel of the figure of the Delaunay triangulations with constraint curves, where `badcut` is drawn as a dashed yellow line. Notice that the third constraint is not respected by the triangulation, even though the two vertices from the constraint have been added to the interior nodes.

Mesh Refinement

After `triangulate`, `DelaunayTriangulation.jl` provides the function `refine!` to improve the quality of the triangulation. Passing `max_area` as a fraction of the total domain area and `min_angle` in degrees is a practical starting point:

```
triarea1 = get_area(tri1)
refine!(tri1; max_area = 0.03 * triarea1, min_angle = 30.0)
sc3 = delaunay_to_simplicial(tri1)
```

The refined triangulation is then converted to a `EuclideanComplex` in the same way as before. The optional argument `min_angle` has to be chosen with care. If one chooses an angle larger than `30` then it is no longer certain that an appropriate triangulation can be generated. The resulting triangulation is shown in the left panel of the figure with two refined Delaunay triangulations of a square.

We can also refine the second mesh from above, the one with the bad constraint curve `badcut`:

```
triarea2 = get_area(tri2)
refine!(tri2; max_area = 0.007 * triarea2, min_angle = 25.0)
sc4 = delaunay_to_simplicial(tri2)
```

The resulting triangulation is shown in the right panel of the figure with two refined Delaunay triangulations of a square. Notice that while the nodes determining `badcut` are still part of the triangulation, the edge is not – just as before. This figure was created using the following commands:

```
p3 = plot_simplicial(sc3)
plot!(p3, cirx, ciry, linewidth=3, color=:red)
plot!(p3, cutx, cuty, linewidth=3, color=:green)
```

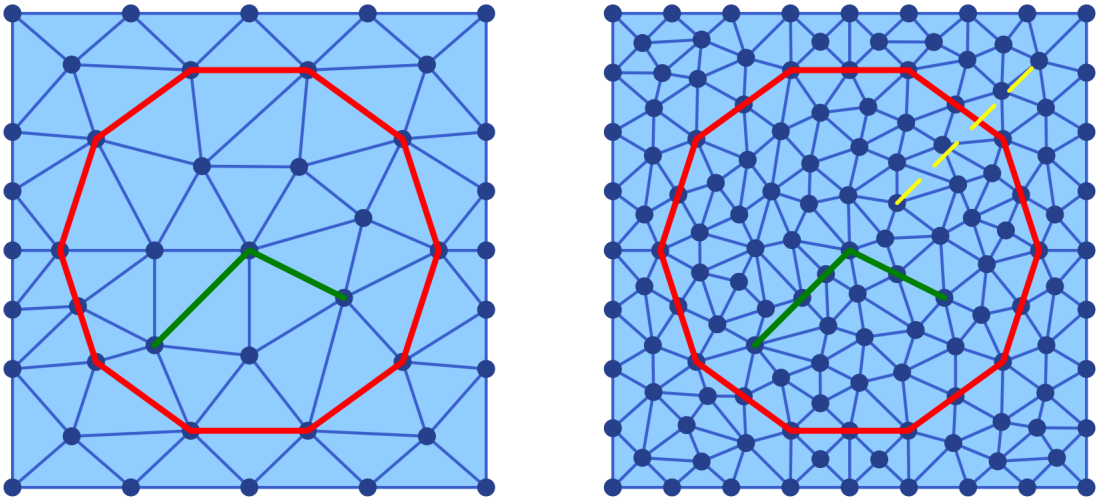


Figure 8.3: Two refined Delaunay triangulations of a square

```
p4 = plot_simplicial(sc4)
plot!(p4, cirx, ciry, linewidth=3, color=:red)
plot!(p4, cutx, cuty, linewidth=3, color=:green)
plot!(p4, badx, bady, linewidth=2, color=:yellow, linestyle=:dash)

pcombined34 = plot(p3, p4, layout=(1,2))
plot!(pcombined34, dpi=300)
savefig(pcombined34, "delaunayext3.png")
```

Complete Example

The following self-contained snippet combines all three steps — bounding rectangle, two closed constraint curves, mesh refinement, and conversion:

```
using Plots
using DelaunayTriangulation
using ConleyDynamics

# Bounding box
points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])

# Outer ring (closed, 30 segments)
n1    = 30
th1   = range(0, 2*pi, length = n1+1)
ring  = [[4.0 * cos(t), 4.0 * sin(t)] for t in th1]
ringx = [pt[1] for pt in ring]
ringy = [pt[2] for pt in ring]
points, segments = delaunay_points_add_segment(points, ring)

# Inner elliptical disc (closed, 15 segments)
n2    = 15
th2   = range(0, 2*pi, length = n2+1)
disc  = [[0.5 + 3.0 * cos(t), 2.0 * sin(t)] for t in th2]
```

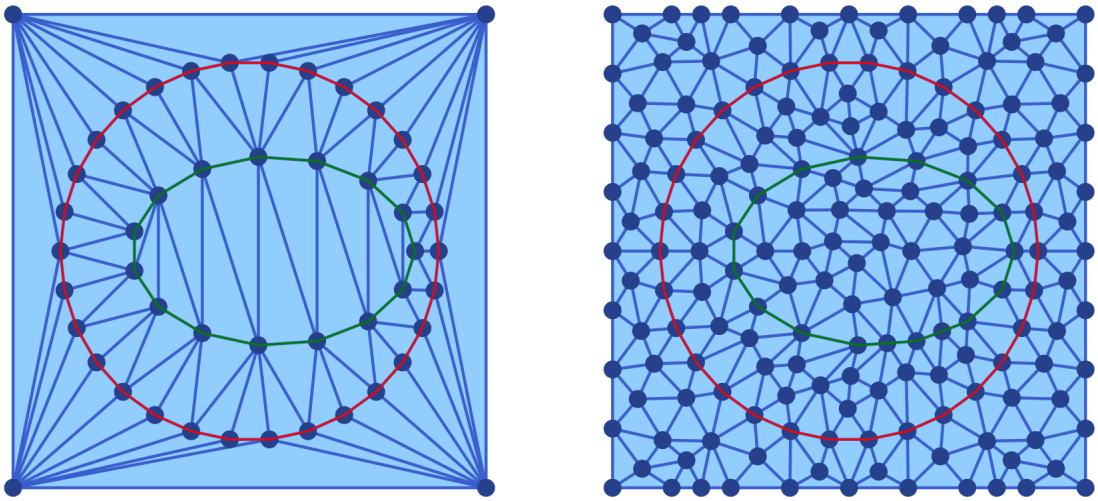


Figure 8.4: A complete Delaunay triangulation example

```
discx = [pt[1] for pt in disc]
discy = [pt[2] for pt in disc]
points, segments = delaunay_points_add_segment(points, segments, disc)

# Triangulate, refine, and convert
tri = triangulate(points; boundary_nodes = bndcurve, segments = segments)
sc1 = delaunay_to_simplicial(tri)

triarea = get_area(tri)
refine!(tri; max_area = 0.005 * triarea, min_angle = 27.0)
sc2 = delaunay_to_simplicial(tri)

# Plot the triangulations
p1 = plot_simplicial(sc1)
plot!(p1, ringx, ringy, color=:red)
plot!(p1, discx, discy, color=:green)

p2 = plot_simplicial(sc2)
plot!(p2, ringx, ringy, color=:red)
plot!(p2, discx, discy, color=:green)

pcombined = plot(p1, p2, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "delaunayext4.png")
```

The resulting Delaunay triangulations, before and after the refinement, are shown in the figure of the complete Delaunay triangulation example, with the ring in red and the elliptical disc in green.

Adding Intersecting Constraint Segments

As noted above, `delaunay_points_add_segment` requires the caller to ensure that constraint curves do not intersect each other. When this condition cannot be guaranteed – for example when the curves are computed

programmatically and may share crossings or collinear overlaps – `delaunay_points_add_split_segs` provides a safe alternative.

Internally it calls `split_segments`, which computes the planar straight-line arrangement of the given segments: All crossings and collinear overlaps are resolved and the input segments are split at every intersection point, yielding a non-crossing set of sub-segments. These vertices and sub-segments are then appended to the existing point list and constraint-edge set exactly as `delaunay_points_add_segment` would.

The function signature mirrors that of `delaunay_points_add_segment`: The two-argument form initialises a fresh segment set, and the three-argument form extends an existing one. An optional keyword argument `verbose=true` prints a brief summary of the splitting process.

Three-argument form: Overlaps with existing segments are not checked

When calling the three-argument form, `split_segments` resolves crossings and overlaps **within newsegments only**. No check is performed for intersections or overlaps between the newly added sub-segments and the segments already present in `segments`. It is the caller's responsibility to ensure that the two sets are compatible. Incorrect use can produce an invalid constraint-edge set that silently causes `DelaunayTriangulation.jl` to drop constraints. Only use the three-argument form if you know that the new segments do not intersect the existing ones.

The basic usage of `delaunay_points_add_segment` is illustrated in the following code segment:

```
using DelaunayTriangulation
using ConleyDynamics

points1, bndcurve1 = delaunay_points_bnd_rectangle([-2, -2], [2, 2])

# Two crossing diagonals - they would violate the no-intersection requirement
# of delaunay_points_add_segment, but split_segments resolves the crossing.
segs1 = [[[-1.0, -1.0], [1.0,  1.0]],
         [[-1.0,  1.0], [1.0, -1.0]]]

points1, segments1 = delaunay_points_add_split_segs(points1, segs1; verbose=true)
# Output:
#   split_segments: 2 input segment(s)
#     intersection points created : 1
#     output vertices             : 5
#     output edges                : 4

tri1 = triangulate(points1; boundary_nodes = bndcurve1, segments = segments1)
sc1  = delaunay_to_simplicial(tri1)
```

The resulting triangulation is shown in the left panel of the figure of the Delaunay triangulations with split constraint curves, with the two diagonals drawn as dashed lines. The five vertices (four original endpoints plus the crossing at the origin) and the four sub-segments are all included as constraints in the triangulation. The result is a valid `EuclideanComplex` whose constraint edges are respected by the mesh.

The following more complex example with three mutually intersecting segments demonstrates that the function `delaunay_points_add_split_segs` also handles multiple simultaneous crossings correctly:

```
using DelaunayTriangulation
using ConleyDynamics
```

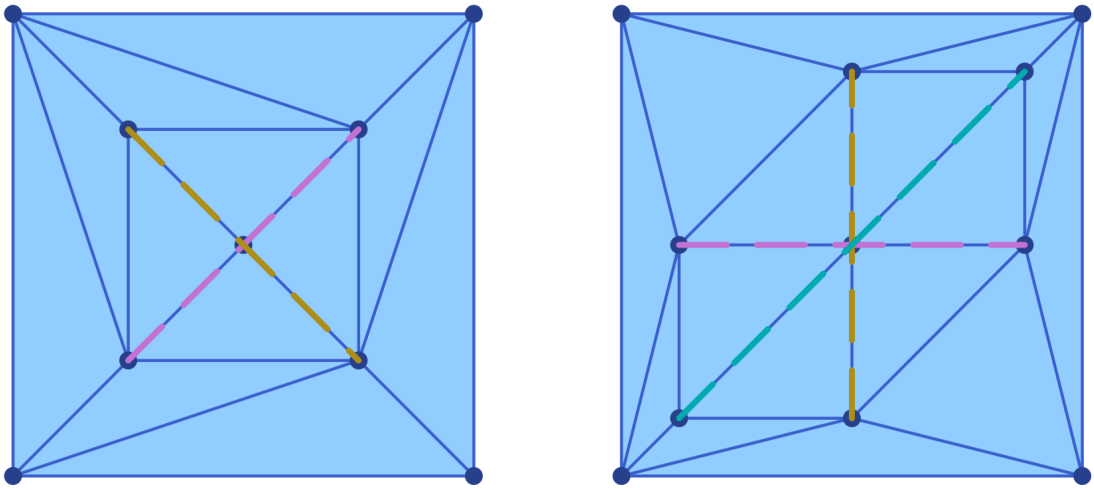


Figure 8.5: Delaunay triangulations with split constraint curves

```
points2, bndcurve2 = delaunay_points_bnd_rectangle([-4, -4], [4, 4])

# Three segments that all cross each other
segs2 = [[[-3.0,  0.0], [3.0,  0.0]],   # horizontal
         [[ 0.0, -3.0], [0.0,  3.0]],   # vertical
         [[-3.0, -3.0], [3.0,  3.0]]]   # diagonal

points2, segments2 = delaunay_points_add_split_segs(points2, segs2; verbose=true)
# Output:
#   split_segments: 3 input segment(s)
#     intersection points created : 3
#     output vertices             : 9
#     output edges                : 6

tri2 = triangulate(points2; boundary_nodes = bndcurve2, segments = segments2)
sc2  = delaunay_to_simplicial(tri2)
```

For the produced triangulation, see the right panel of the figure of the Delaunay triangulations with split constraint curves. The three intersection points (origin, and the two axis crossings with the diagonal) are automatically detected, and each input segment is split into two sub-segments at its crossings. This figure was created using the following commands:

```
using Plots

p1 = plot_simplicial(sc1)
for p1seg in segs1
    pt1, pt2 = p1seg
    plot!(p1, [pt1[1],pt2[1]], [pt1[2],pt2[2]], linewidth=3, linestyle=:dash)
end

p2 = plot_simplicial(sc2)
for p2seg in segs2
    pt1, pt2 = p2seg
```

```
    plot!(p2, [pt1[1],pt2[1]], [pt1[2],pt2[2]], linewidth=3, linestyle=:dash)
end

pcombined = plot(p1, p2, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "delaunayext5.png")
```

Chapter 9

Three-Dimensional Meshes

This chapter is concerned with complexes in three dimensions. While most of the analysis methods of ConleyDynamics.jl, such as for example the homology, Morse decomposition, and connection matrix functions, do not depend on the dimension of the underlying space and can be computed for any `EuclideanComplex`, other functionality is dimension dependent. This is the case for the function `create_spatial_mvf`, which accepts tetrahedral meshes. What is therefore needed in addition is a way to create tetrahedral meshes of interesting domains, and a way to look at them. The package provides three layers of functionality for the first task:

- Core functions, which are available without loading any other package. They import meshes from arbitrary sources, create random point samples, and create triangulated surfaces such as spheres, cylinders, and tori, which can serve as building blocks of more complicated domains.
- The `TetGen.jl` extension, which creates quality tetrahedral meshes of domains bounded by triangulated surfaces, possibly with embedded constraint surfaces and curves, holes, and regions with different mesh sizes. It is the three-dimensional analogue of the `DelaunayTriangulation.jl` extension described in the Two-Dimensional Meshes chapter.
- The `Gmsh.jl` extension, which is the tool of choice for curved geometry described by computer-aided design (CAD) and, above all, for constraint surfaces which intersect each other.

The table below lists all functions involved. The column Pkg shows the package that has to be loaded for them, where `core` means that the function is part of the core package.

The domain builders of `TetGen` and the conversion functions are part of the core package, so that domains can be prepared and stored without loading `TetGen.jl`. Only the actual mesh generation requires it. For the visualization of three-dimensional complexes, see the section Three-Dimensional Plotting with Makie in the Visualization chapter. All figures in this chapter were created with the functions described there, and the script which creates them can be found in the file `docs/imgsrc/meshing3d.jl` of the package repository.

9.1 Loading the Required Packages

The functions in the section Core Geometry Functions only require ConleyDynamics.jl itself, which is loaded as usual with `using ConleyDynamics`.

`TetGen.jl` and `Gmsh.jl`, on the other hand, are weak dependencies: They are not installed together with ConleyDynamics.jl, and the functions which need them only become available once they have been loaded. Each of them has to be installed once in the active Julia environment, using

Function	Pkg	Description
`tetmesh_to_euclidean`	`core`	Convert points and simplices into a complex
`poisson_points_box`	`core`	Random points with a minimum distance
`trisurface_sphere`, `trisurface_ellipsoid`	`core`	Triangulated spheres and ellipsoids
`trisurface_cylinder`, `trisurface_torus`	`core`	Triangulated cylinders and tori
`trisurface_plane_patch`, `trisurface_parametric`	`core`	Planar and general parametric surfaces
`TetDomain`, `tetgen_domain_*`	`core`	Description of a domain for TetGen
`tetgen_to_simplicial`	`core`	Convert a TetGen mesh into a complex
`tetgen_marker_cells`, `tetgen_region_cells`	`core`	Cells on constraints and in regions
`tetgen_mesh`	`TetGen`	Create a constrained quality mesh
`create_simplicial_delaunay3d`	`TetGen`	Delaunay triangulation of random points
`gmsh_to_simplicial`	`Gmsh`	Convert the current `Gmsh` mesh
`gmsh_file_to_simplicial`	`Gmsh`	Read and convert a mesh or geometry file
`gmsh_marker_cells`	`Gmsh`	Cells of physical groups and entities
`gmsh_common_refinement`	`Gmsh`	Split intersecting objects into common pieces
`gmsh_add_trisurface`	`Gmsh`	Add a triangulated surface to the model

```
import Pkg
Pkg.add("TetGen")      # for tetgen_mesh and create_simplicial_delaunay3d
Pkg.add("Gmsh")        # for gmsh_to_simplicial, gmsh_file_to_simplicial,
                       # gmsh_marker_cells, and gmsh_common_refinement
```

Versions `3.x` of `TetGen.jl` and `0.3.x` of `Gmsh.jl` are supported. Note that `Gmsh.jl` downloads large binary dependencies during the installation, about 150 MB including the OpenCASCADE CAD kernel, while `TetGen.jl` is small. Only the package which is actually used needs to be installed. In every Julia session, the meshing package and ConleyDynamics.jl then have to be loaded. For `TetGen.jl`, this is done with

```
using TetGen
using ConleyDynamics
```

and for `Gmsh.jl` with

```
using Gmsh: gmsh
using ConleyDynamics
```

The command `using Gmsh: gmsh` makes the module `gmsh` available, which contains the complete `Gmsh` API, such as `gmsh.initialize()` or `gmsh.model.occ.addBox`. The command `using Gmsh` has the same effect. Both meshing packages can of course be loaded in the same session. The order of the commands does not matter, and `import` can be used instead of `using`. As soon as a meshing package and ConleyDynamics.jl are both loaded, Julia automatically loads the corresponding extension module, `ConleyDynamicsTetGenExt` or `ConleyDynamicsGmshExt`. Whether this has happened can be checked with

```
Base.get_extension(ConleyDynamics, :ConleyDynamicsTetGenExt) !== nothing
```

If one of the extension functions is called without loading the corresponding package, Julia reports that no matching method exists:

```
using ConleyDynamics
dom = tetgen_domain_box([0,0,0], [1,1,1])
tetgen_mesh(dom)
# ERROR: MethodError: no method matching tetgen_mesh(::TetDomain)
```

In that case, it suffices to load the package and to repeat the call. Note that the domain builders `tetgen_domain_*`, as well as `tetgen_to_simplicial`, `tetgen_marker_cells`, and `tetgen_region_cells`, are part of the core package, so that only the mesh generation itself requires `TetGen.jl`. For the figures, a Makie backend such as `CairoMakie` or `GLMakie` has to be installed and loaded in the same way, see Three-Dimensional Plotting with Makie.

TetGen and Gmsh are unreliable on 32-bit systems

On 32-bit Linux, `TetGen` was found to fail with internal errors or to crash Julia with a segmentation fault for domains which are meshed without problems on 64-bit systems, such as a box with a spherical hole, and `Gmsh` was found to hang while adding triangulated surfaces with `gmsh_add_trisurface`. Both extensions should therefore only be used on 64-bit systems. Their tests are skipped on 32-bit systems, and the package is only tested on 64-bit systems.

9.2 Basic Notions

This section collects the few notions from geometry and topology which are used throughout the chapter. Readers who are familiar with meshes and homology can skip it.

Simplices and meshes. The building blocks of all meshes in this chapter are simplices: Vertices (dimension `0`), edges (dimension 1), triangles (dimension 2), and tetrahedra (dimension 3). The faces of a simplex are the simplices spanned by subsets of its vertices, see also the Tutorial. For example, a tetrahedron has four triangles, six edges, and four vertices as proper faces. A tetrahedral mesh of a domain is a collection of tetrahedra which fill the domain and which only meet in common faces, i.e., two tetrahedra share either nothing, a vertex, an edge, or a whole triangle. Since all faces are determined by the tetrahedra, a mesh is described completely by the vertex coordinates and the list of its maximal simplices, i.e., those which are not faces of larger ones. This is exactly the input of `tetmesh_to_euclidean`. The resulting `EuclideanComplex` contains all simplices of the mesh as its cells, together with their coordinates, see the section Euclidean Complexes in the chapter on Lefschetz complexes.

Subcomplexes. A set of cells is closed if it contains all faces of its cells. Closed sets of cells are again simplicial complexes, the subcomplexes, which can be created with `lefschetz_subcomplex`. The closure of a set of cells, which is computed by `lefschetz_closure`, is the smallest closed set containing it. Many functions in this chapter return closed sets of cells, for example all cells on a constraint surface, so that their homology can be computed directly.

Reading homology. The function `homology` returns the Betti numbers `[b0, b1, b2, b3]` of a complex, which were introduced in the Tutorial. In three dimensions, they have an intuitive meaning: `b0` counts the connected components, `b1` the independent closed loops which cannot be contracted, such as the loop around the hole of a donut, and `b2` the enclosed cavities. For a complex in three-dimensional space, `b3` is always zero. The following table lists the homology of the spaces that occur most often in this chapter. Note that the result depends on whether a solid or only its surface is meshed:

Computing the homology of a mesh is therefore a quick way to check that it has the expected shape, and this check is used in almost every example below.

Space	Homology
Solid ball or box	`[1, 0, 0, 0]`
Sphere (the surface of a ball)	`[1, 0, 1]`
Solid torus (a donut)	`[1, 1, 0, 0]`
Torus surface	`[1, 2, 1]`
Box with one spherical cavity	`[1, 0, 1, 0]`
Circle	`[1, 1]`
Open tube, annulus, or Möbius strip	`[1, 1, 0]`

Delaunay tetrahedralizations. For a given set of points, there are many ways to connect them into tetrahedra. A tetrahedralization is called Delaunay if no point lies inside the sphere through the four vertices of any tetrahedron, the so-called circumsphere. Among all tetrahedralizations, Delaunay tetrahedralizations tend to avoid long and thin tetrahedra, and they are the basis of most mesh generators. If certain triangles and edges are prescribed, for example because they describe a surface which has to be contained in the mesh, the Delaunay condition cannot always be satisfied. One then uses a constrained Delaunay tetrahedralization, which contains the prescribed triangles and edges, and which is as close to Delaunay as possible otherwise.

Mesh quality. For the construction of multivector fields, the tetrahedra should generally be well-shaped, i.e., not too flat or too thin. Two standard measures of the shape of a tetrahedron are used below. The radius-edge ratio is the radius of the circumsphere divided by the length of the shortest edge. A regular tetrahedron, whose six edges all have the same length, has the smallest possible ratio `sqrt(6)/4 ≈ 0.61`, and the ratio becomes large for tetrahedra with a very short edge. The dihedral angles of a tetrahedron are the six angles between pairs of its faces, measured along their common edge. For a regular tetrahedron, all of them equal `arccos(1/3) ≈ 70.5°`, while a flat tetrahedron has dihedral angles close to `0°` or `180°`.

9.3 Core Geometry Functions

The functions in this section are part of the core package, so they are available without loading any extension.

Importing Meshes

Every mesh generator describes a simplicial mesh in essentially the same way: By a list of vertex coordinates and a list of maximal simplices, each given by the indices of its vertices. The function `tetmesh_to_euclidean` turns such a description into an `EuclideanComplex`, and it automatically adds all faces of the given simplices. As a first example, consider two tetrahedra which share a common triangle:

```
using ConleyDynamics

points = [[0.0,0.0,0.0], [1.0,0.0,0.0], [0.0,1.0,0.0],
          [0.0,0.0,1.0], [1.0,1.0,1.0]]
tets   = [[1,2,3,4], [2,3,4,5]]
ec     = tetmesh_to_euclidean(points, tets)

lefschetz_cell_count(ec)    # [5, 9, 7, 2]
homology(ec)                # [1, 0, 0, 0]
ec.labels
# 23-element Vector{String}:
#  "1", "2", "3", "4", "5", "12", "13", "14", "23", "24", "25", "34",
#  "35", "45", "123", "124", "134", "234", "235", "245", "345", "1234",
#  "2345"
```

The vertices are labeled by zero-padded decimal numbers, whose width depends on the number of vertices. For example, a mesh with 532 vertices uses the labels `"001"` through `"532"`. The label of a simplex is the concatenation of the labels of its vertices, in increasing order. This is the same convention as in `create_simplicial_delaunay` and `delaunay_to_simplicial`, so that all simplicial complexes created from meshes look alike.

The coordinates can be given as vectors or tuples, and the dimension of the space is arbitrary. Moreover, simplices of different dimensions can be mixed. The following complex consists of a triangle, an edge attached to one of its vertices, and an edge attached to another vertex:

```
mixed = tetmesh_to_euclidean([[0,0,0], [1,0,0], [0,1,0], [0,0,1], [2,0,0]],
                             [[1,2,3], [1,4], [2,5]])
lefschetz_cell_count(mixed)    # [5, 5, 1]
homology(mixed)                # [1, 0, 0]
```

In particular, `tetmesh_to_euclidean` can also be used for triangulated surfaces in three dimensions, which leads to two-dimensional complexes embedded in three-dimensional space. Examples of this can be found in the next section.

Many mesh generators store the mesh in matrix form instead, with one column per vertex and one column per simplex. This form is accepted as well:

```
P = [0.0 1.0 0.0 0.0 1.0;
     0.0 0.0 1.0 0.0 1.0;
     0.0 0.0 0.0 1.0 1.0]
C = [1 2;
     2 3;
     3 4;
     4 5]
ec2 = tetmesh_to_euclidean(P, C)
ec2.labels == ec.labels        # true
```

Mesh generators frequently return vertices which do not belong to any simplex, for example geometry points outside of the meshed region or duplicate points. By default, such orphaned vertices are removed, and the remaining vertices are renumbered in their original order. If the optional argument `droporphans=false` is passed, they are kept as isolated vertices instead:

```
pts6 = vcat(points, [[5.0,5.0,5.0]])                    # unused vertex
tetmesh_to_euclidean(pts6, tets).ncells                 # 23
ec6 = tetmesh_to_euclidean(pts6, tets; droporphans=false)
ec6.ncells                                              # 24
homology(ec6)                                           # [2, 0, 0, 0]
```

Finally, as for all complex creation functions in ConleyDynamics.jl, the optional argument `p` determines the coefficient field. The default `p=2` gives `GF(2)`, any other prime `p` gives `GF(p)`, and `p=0` gives the rationals:

```
ecq = tetmesh_to_euclidean(points, tets; p=0)
lefschetz_field(ecq)           # "QQ"
```

Random Point Samples

The planar function `create_simplicial_delaunay` is based on a random sample of points with a prescribed minimum distance, a so-called Poisson-disk sample. Such samples are useful in any dimension, and they can be created with `poisson_points_box`. Its first two arguments are the lower and upper corners of a box, and the third argument is the minimum distance between the points:

```
using Random

pts2 = poisson_points_box([0,0], [2,1], 0.1; rng=Xoshiro(1234))
length(pts2)                  # 132

pts3 = poisson_points_box([0,0,0], [2,1,1], 0.1; rng=Xoshiro(1234))
length(pts3)                  # 1265
```

The function uses Bridson's algorithm. Starting from a random point, it repeatedly tries to place new points at distances between `pdist` and `2*pdist` from already accepted points, and it gives up on an accepted point after `attempts` unsuccessful tries. The default `attempts=20` gives a good compromise between speed and density. Larger values fill the box more densely:

```
pts3 = poisson_points_box([0,0,0], [2,1,1], 0.1; attempts=50, rng=Xoshiro(1234))
length(pts3)                  # 1353
```

In each case, all pairwise distances are at least `pdist`. The runtime is proportional to the number of generated points, so even samples with hundreds of thousands of points take only a few seconds. If the optional argument `rng` is omitted, Julia's default random number generator is used, and the result changes from call to call. Passing a seeded generator as above makes the sample reproducible, which is recommended for any computation which is to be published.

Poisson-disk samples are used by `create_simplicial_delaunay3d`, and they can be added as interior nodes to a TetGen domain, see Adding Interior Nodes below.

Triangulated Surfaces

Three-dimensional domains are usually described by their boundary surfaces, and the constraints inside a domain are surfaces and curves. The package therefore contains six functions which create triangulated surfaces. Each of them returns a pair `(V, T)`, where `V` is a vector of vertex coordinates and `T` is a vector of triangles, each given by three indices into `V`. Vertices are never duplicated, even along the seams of a parametrization, so that neighboring triangles share their vertices, as required by the mesh generators. Moreover, all surfaces of orientable parametrizations are consistently oriented, and closed surfaces have outward normals. This is needed for surfaces which bound holes or regions of a mesh, as explained in the section on the Orientation of Surfaces below. The figure of the six triangulated surface generators shows one example for each function.

Spheres and ellipsoids. The function `trisurface_sphere` creates an icosphere: An icosahedron whose triangles are subdivided `n` times, with all new vertices projected onto the sphere. This gives `10*4^n+2` vertices and `20*4^n` triangles of nearly equal size and shape, which is much better than a triangulation based on spherical coordinates with its crowded poles. The function `trisurface_ellipsoid` scales such an icosphere along the coordinate axes:

```
V, T = trisurface_sphere([0,0,0], 1.0, 2)          # center, radius, n
length(V), length(T)                               # (162, 320)

V, T = trisurface_ellipsoid([0,0,0], [2.0,1.0,0.5], 2)   # center, semi-axes, n
```

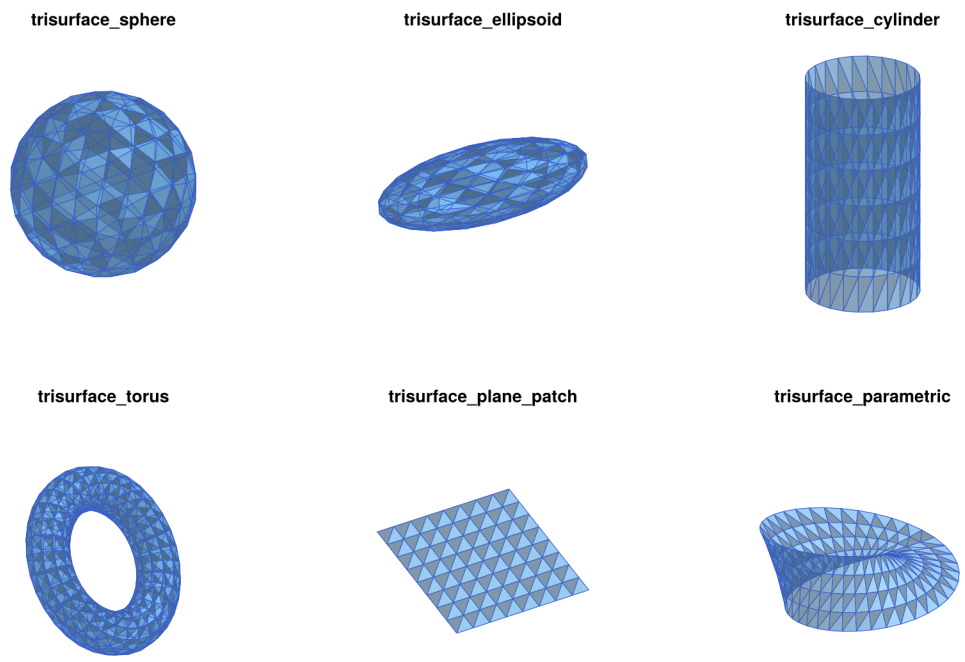


Figure 9.1: The six triangulated surface generators

Cylinders. The function `trisurface_cylinder` creates a cylinder of radius `r` around the segment from `p0` to `p1`, which can have any direction. The lateral surface is divided into `ntheta` parts around the axis and `nz` parts along it. By default, both ends are closed by triangle fans, which gives a closed surface. With `capped=false`, the result is an open tube, as in the figure of the six surface generators:

```
V, T = trisurface_cylinder([0,0,-1], [0,0,1], 0.5, 24, 6)                # closed
V, T = trisurface_cylinder([0,0,-1], [0,0,1], 0.5, 24, 6; capped=false)   # tube
```

Tori. The function `trisurface_torus` expects the center, the major radius R (the distance from the center to the center of the tube), the minor radius `r` (the radius of the tube), and the numbers of subdivisions `nu` around the axis and `nv` around the tube. The axis of rotational symmetry is the z-axis by default, but it can be changed with the optional argument `axis`. The torus in the figure of the six surface generators was created with a tilted axis:

```
V, T = trisurface_torus([0,0,0], 2.0, 0.6, 32, 12; axis=[1,0,1])
```

Plane patches. The function `trisurface_plane_patch` creates the parallelogram of all points `origin + s*u + t*v` with `s` and `t` in the unit interval, subdivided into `nu` times `nv` small parallelograms, each of which is split into two triangles. Its normal vector points in the direction of the cross product of `u` and `v`. The following command creates the patch in the figure of the six surface generators, which is not parallel to any coordinate plane:

```
V, T = trisurface_plane_patch([-1,-1,0], [2,0,0], [0.5,2,0.5], 8, 8)
```

General parametric surfaces. The function `trisurface_parametric` triangulates the image of an arbitrary function `f(u,v)` over a parameter rectangle. The rectangle `urange x vrange` is divided into `nu` times `nv` subrectangles, each of which is split into two triangles. Periodic parametrizations are handled with the optional arguments `uperiodic` and `vperiodic`, which glue the corresponding edges of the parameter rectangle together. In addition, all vertices which are closer than `tol` (default `1e-10`) are merged, and triangles which become degenerate are removed. The value of `tol` has to be positive, and since it is an absolute distance, it has to be scaled with the size of the surface. For a surface of diameter `1e10`, for example, rounding errors in the coordinates are of the order `1e-6`, and points such as the poles of a sphere are only merged with a correspondingly larger `tol`. The seams of `uperiodic` and `vperiodic` are glued by construction, independently of `tol`. As an example, the following commands create a unit sphere in spherical coordinates:

```
fsph(u,v) = [cos(u)*sin(v), sin(u)*sin(v), cos(v)]
V, T = trisurface_parametric(fsph, (0,2*pi), (pi,0), 24, 12; uperiodic=true)
length(V), length(T)                  # (266, 528)
```

Here, `uperiodic=true` glues the seam at `u=0` and `u=2*pi`, and all vertices at the two poles are merged automatically. The triangles are oriented by the cross product of the partial derivatives of `f` with respect to `u` and `v`, so the range `(pi,0)` for `v`, which runs from the south pole to the north pole, gives outward normals. Merging close vertices also handles orientation-reversing gluings. The Möbius strip in the figure of the six surface generators was created with the commands

```
fmoebius(u,v) = [(1+v/2*cos(u/2))*cos(u), (1+v/2*cos(u/2))*sin(u), v/2*sin(u/2)]
V, T = trisurface_parametric(fmoebius, (0,2*pi), (-1,1), 40, 4)
```

Since `fmoebius(2*pi,v) == fmoebius(0,-v)`, the two ends of the strip are glued with a twist, even though `uperiodic` is not set.

Parametrizations have to be injective away from the seams

Since close vertices are merged, any self-intersection of the surface which passes through grid vertices is turned into an identification. For example, the usual figure-eight immersion of the Klein bottle in three dimensions intersects itself along a circle, and the resulting complex is not a Klein bottle. Moreover, a non-orientable surface such as the Möbius strip cannot be consistently oriented. It can be used as a constraint surface for the mesh generators, but it cannot bound a hole or a region.

Surfaces as complexes. Every triangulated surface can be turned into a two-dimensional simplicial complex embedded in three dimensions with `tetmesh_to_euclidean`. This is a convenient way to check the topology of a surface:

```
homology(tetmesh_to_euclidean(trisurface_sphere([0,0,0], 1.0, 2)...))
# [1, 0, 1]
homology(tetmesh_to_euclidean(trisurface_torus([0,0,0], 2.0, 0.6, 32, 12)...))
# [1, 2, 1]
homology(tetmesh_to_euclidean(trisurface_cylinder([0,0,-1], [0,0,1], 0.5, 16, 4)...))
# [1, 0, 1]
homology(tetmesh_to_euclidean(trisurface_cylinder([0,0,-1], [0,0,1], 0.5, 16, 4;
                                                  capped=false)...))
```

```
# [1, 1, 0]
homology(tetmesh_to_euclidean(trisurface_plane_patch([-1,-1,0], [2,0,0],
                                                      [0,2,0], 8, 8)...))
# [1, 0, 0]
homology(tetmesh_to_euclidean(trisurface_parametric(fmoebius, (0,2*pi),
                                                    (-1,1), 40, 4)...))
# [1, 1, 0]
```

Orientation of Surfaces

The triangles of a surface have an orientation, which is easy to get wrong for surfaces which are not created by the `trisurface_*` functions, for example surfaces read from a file or assembled by hand. The orientation is needed in exactly one situation, namely when a closed surface is added to a TetGen domain with `tetgen_domain_add_surface` and the option `hole=true` or `region=(attr, maxvol)`. In all other situations, it is ignored. This section explains what the orientation is, what it means for a surface to be consistently oriented, and why and when this matters.

The orientation of a triangle. A triangle is given by the list of its three vertex indices, such as `[a, b, c]`. The order of this list determines a direction in which the boundary of the triangle is traversed, namely from a to b, from b to c, and from c back to a. This direction is the orientation of the triangle. Only the cyclic order of the vertices matters:

- The lists `[a, b, c]`, `[b, c, a]`, and `[c, a, b]` describe the same closed path with different starting points, and therefore have the same orientation.
- The list `[a, c, b] = reverse([a, b, c])` traverses the boundary in the opposite direction, and therefore has the opposite orientation. The same holds for any list which is obtained by exchanging two vertices.

Every triangle therefore has exactly two orientations, and `reverse` switches between them. This is the same notion as the orientation of a simplex in the chapter on Lefschetz Complexes, where two orientations are equivalent if they differ by an even permutation of the vertices. For a triangle, the even permutations are exactly the cyclic shifts.

The normal vector. The orientation of a triangle `[a, b, c]` determines a normal vector, namely the cross product

```
n = cross(b - a, c - a)
```

which is perpendicular to the triangle. Its direction is given by the right-hand rule: If the fingers of the right hand curl in the direction from a to b to c, then the thumb points in the direction of n. Equivalently, if the triangle is viewed from the side into which n points, then its vertices appear in counterclockwise order. For example, the triangle with the vertices `a = [0,0,0]`, `b = [1,0,0]`, and `c = [0,1,0]` has the normal vector `[0,0,1]`, and seen from above, the path from a to b to c is counterclockwise. Reversing the triangle to `[a, c, b]` gives the normal vector `[0,0,-1]`. Thus, choosing an orientation of a triangle is the same as choosing one of its two sides. The same rule applies to the planar polygons with more than three vertices which are used as facets in a `TetDomain`. For example, the bottom face `[1, 4, 3, 2]` of the unit box created by `tetgen_domain_box` has the vertices `(0,0,0)`, `(0,1,0)`, `(1,1,0)`, and `(1,0,0)` in this order. Seen from above, this path is clockwise, so the normal vector points downward and out of the box.

Consistently oriented surfaces. A triangulated surface is consistently oriented if any two triangles which share an edge traverse this edge in opposite directions. If one triangle contains the step from vertex 3 to

vertex 5, then its neighbor across this edge has to contain the step from 5 to 3. Geometrically, this means that the normal vectors of neighboring triangles point to the same side of the surface, so that the normals of the whole surface point to one side of it. As an example, consider the boundary of the tetrahedron with the vertices `(0,0,0)`, `(1,0,0)`, `(0,1,0)`, and `(0,0,1)`:

```
using LinearAlgebra
Vt = [[0,0,0], [1,0,0], [0,1,0], [0,0,1]]
Tt = [[1,3,2], [1,4,3], [1,2,4], [2,3,4]]
[cross(Vt[t[2]] - Vt[t[1]], Vt[t[3]] - Vt[t[1]]) for t in Tt]
# [[0, 0, -1], [-1, 0, 0], [0, -1, 0], [1, 1, 1]]
```

Every edge occurs once in each direction. For example, the edge between the vertices 1 and 3 is traversed from 1 to 3 in the first triangle, and from 3 to 1 in the second one. All four normal vectors point out of the tetrahedron. If a single triangle is reversed, then two of its edges are traversed in the same direction as in the neighboring triangles, and the surface is no longer consistently oriented. A connected orientable surface has exactly two consistent orientations, which are obtained from each other by reversing all triangles. A non-orientable surface, such as the Möbius strip, has no consistent orientation at all.

Outward and inward normals. For a closed and consistently oriented surface, the normal vectors either all point out of the enclosed volume, or they all point into it. The two cases can be distinguished by the signed volume

```
vol = sum(dot(V[t[1]], cross(V[t[2]], V[t[3]])) for t in T) / 6
```

whose absolute value is the volume enclosed by the surface. It is positive for outward normals and negative for inward normals. For the tetrahedron above, only the last triangle contributes, and its contribution `1/6` is the volume of the tetrahedron. All closed surfaces created by the `trisurface_*` functions have outward normals.

When the orientation is needed. `TetGen` itself does not use the orientation of facets at all. For `TetGen`, a facet is a two-sided wall which tetrahedra cannot cross, and holes and regions are identified by points inside them, see Describing a Domain below. Such a point does not depend on any orientation. The orientation is only needed when the package has to find such a point by itself:

- **Orientation matters:** `tetgen_domain_add_surface` with `hole=true` or with `region=(attr, maxvol)`. In this case, a point inside the surface is computed automatically. The function determines the sign of the signed volume, starts at the center of the largest triangle, and follows the normal vector of this triangle into the enclosed volume until the next surface is hit. The midpoint of this segment is used as the hole or region point. This only works if the surface is closed and consistently oriented, which is therefore checked, and an `ArgumentError` is raised otherwise, see the section on Holes. Since the sign of the signed volume is taken into account, outward normals and inward normals work equally well. A surface with mixed normals, on the other hand, has no well-defined inside direction, and is rejected.
- **Orientation does not matter:** Everything else. This includes `tetgen_domain_add_surface` without `hole` and `region`, where the surface is only a constraint, `tetgen_domain_from_surface`, which only requires a closed surface, `tetgen_domain_add_hole` and `tetgen_domain_add_region`, where the point is given explicitly, as well as `tetmesh_to_euclidean`, which creates the same complex for every orientation of the triangles, and the plotting functions, which orient the drawn triangles themselves.

In particular, a surface whose orientation is inconsistent can still bound a hole or a region, as long as a point inside it is known. It is then added without `hole` or `region`, and the point is given explicitly:

```
V, T = trisurface_sphere([0,0,0], 1.0, 2)
Tflip = deepcopy(T)
Tflip[1] = reverse(Tflip[1])          # one triangle with the wrong orientation

dom = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom = tetgen_domain_add_surface(dom, V, Tflip)
dom = tetgen_domain_add_hole(dom, [0,0,0])
homology(tetgen_to_simplicial(tetgen_mesh(dom; maxvolume=0.05)))
# [1, 0, 1, 0]
```

Orientation of the generated surfaces. The surface generators use the following conventions:

- `trisurface_sphere`, `trisurface_ellipsoid`, `trisurface_torus`, and `trisurface_cylinder` with the default `capped=true` create closed surfaces with outward normals.
- `trisurface_plane_patch` creates triangles whose normal vectors point in the direction of `cross(u, v)`.
- `trisurface_parametric` creates triangles whose normal vectors point in the direction of the cross product of the partial derivatives of `f` with respect to `u` and `v`. Reversing one of the two parameter ranges reverses the orientation. This is why the sphere in spherical coordinates above uses the range `(pi,0)` for `v`: The range `(0,pi)` would give inward normals. Both versions can bound a hole or a region.
- `tetgen_domain_box` creates the six faces of the box with outward normals.

Reversing all triangles of a surface, as in `[reverse(t) for t in T]`, switches between outward and inward normals. For holes and regions, this is never necessary. If a surface is not consistently oriented, however, reversing all triangles does not help. In that case, the triangles have to be reoriented one by one, starting from one triangle and reversing each neighbor which traverses a shared edge in the same direction, or the hole or region point has to be given explicitly, as shown above.

9.4 TetGen.jl: Tetrahedral Meshes

The TetGen.jl package is a Julia interface to Hang Si's mesh generator TetGen. It creates constrained Delaunay tetrahedralizations, see Basic Notions, and quality tetrahedral meshes of three-dimensional domains. A domain is described as a piecewise linear complex: A collection of points, together with planar polygons, segments, and single points (the facets) which have to be contained in the final mesh. Note that the word facet is used here in the sense of TetGen, which differs from the facets of a simplex in the Tutorial. The outer facets bound the domain, and facets inside the domain are constraints. In addition, holes and regions can be specified by points inside them. This data model maps directly onto the `DelaunayTriangulation.jl` workflow in the plane described in the Two-Dimensional Meshes chapter, and the extension follows the same pattern:

1. Describe the domain by a `TetDomain`. Such a domain is usually created with `tetgen_domain_box` or with `tetgen_domain_from_surface`, and then extended with the other `tetgen_domain_*` functions.
2. Create the mesh with `tetgen_mesh`, which returns a `TetGenResult`.
3. Convert the mesh with `tetgen_to_simplicial`, and find the cells on constraint surfaces and curves, or in regions, with `tetgen_marker_cells` and `tetgen_region_cells`.

Only `tetgen_mesh` and `create_simplicial_delaunay3d` require `TetGen.jl`, which has to be installed and loaded as described in Loading the Required Packages. Note that TetGen is distributed under the GNU Affero General Public License, see Licences of the Meshing Backends below.

Describing a Domain

A `TetDomain` is a plain Julia struct with five fields:

- `points`: The vertices of the domain description, as tuples of three numbers,
- `facets`: The facets, each given by a vector of indices into `points`. A facet with at least three vertices is a planar polygon, a facet with two vertices is a segment, and a facet with one vertex is a single point,
- `markers`: An integer marker for every facet,
- `holes`: One point inside every hole, and
- `regions`: For every region, a point inside it, an integer attribute, and a maximal tetrahedron volume.

The markers identify the different parts of the domain description after meshing. By convention, the outer boundary has marker `0`, and the constraint surfaces and curves are numbered `1, 2, 3, ...` in the order in which they are added. The `tetgen_domain_*` functions never change their argument, but return a new `TetDomain`. A domain can therefore be extended step by step, and intermediate versions can be reused.

The simplest domain is a box. The function `tetgen_domain_box` expects the lower and upper corners, and it creates the eight corners and the six faces of the box as quadrilateral facets with outward orientation:

```
dom = tetgen_domain_box([-1.5,-1,-1], [1.5,1,1])
# TetDomain with 8 points, 6 facets, markers [0], 0 holes, 0 regions

dom.facets
# [[1, 4, 3, 2], [5, 6, 7, 8], [1, 2, 6, 5], [2, 3, 7, 6], [3, 4, 8, 7], [4, 1, 5, 8]]
dom.markers
# [0, 0, 0, 0, 0, 0]
```

The one-line summary shown above is printed whenever a `TetDomain` is displayed in the REPL. It is a quick way to check that every added constraint received the intended marker.

The numbers of holes and regions in this summary are easily misunderstood. Closed surfaces divide the domain into several parts, which we call chambers, since tetrahedra cannot cross facets. A box with a sphere inside, for example, has two chambers, namely the inside of the sphere and the box minus the ball. Without further instructions, TetGen meshes all chambers. The fields `holes` and `regions` attach instructions to individual chambers, each of which is identified by a point inside it:

- A hole point means that the chamber containing it is removed.
- A region point means that the chamber containing it is kept, its tetrahedra receive an integer attribute, and they can optionally be given their own maximal volume.

A region in the sense of TetGen is therefore not a chamber, but a label which is attached to a chamber. The summary counts these instructions, i.e., it shows `length(dom.holes)` and `length(dom.regions)`, and not the number of chambers. A box with a spherical hole, as in the section on Holes below, has two chambers, but the summary reads `1 holes, 0 regions`: One chamber is removed, and the other one is meshed without a label. Labels are needed only for local refinement and for finding the tetrahedra of a chamber after meshing, as described in Regions and Local Refinement. As soon as at least one region is specified, TetGen labels all chambers, and the chambers without a region point receive attributes which it chooses itself. Without any region points, no attributes are assigned at all.

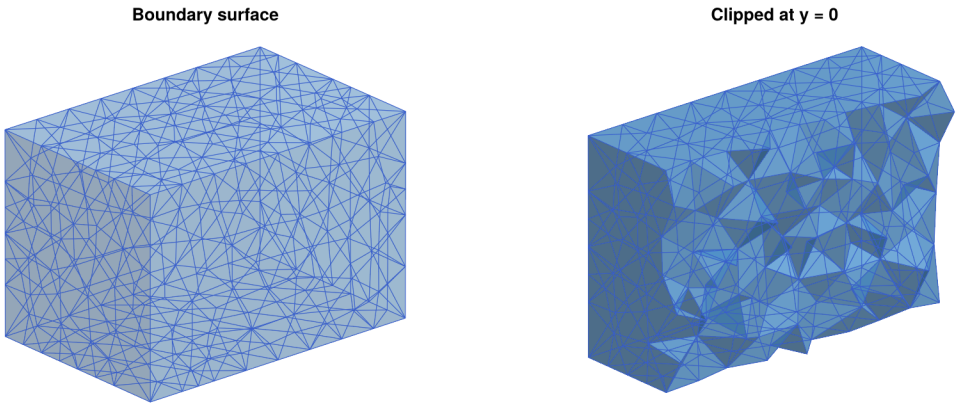


Figure 9.2: A tetrahedral mesh of a box

Meshing a Box

The domain is meshed with `tetgen_mesh`. Without any further arguments, TetGen only performs a mild quality refinement, and since the box contains no constraints, the result is the coarsest possible tetrahedralization:

```
res = tetgen_mesh(dom)
length(res.points), length(res.tetrahedra)     # (8, 6)
```

The optional argument `maxvolume` bounds the volume of the tetrahedra, and it is the main way of controlling the size of the mesh. The volume of the box is 12, so the following mesh has about two thousand tetrahedra:

```
res = tetgen_mesh(dom; maxvolume=0.01)
length(res.points), length(res.tetrahedra)     # (532, 2207)

ec = tetgen_to_simplicial(res)
lefschetz_cell_count(ec)                       # [532, 3051, 4727, 2207]
homology(ec)                                   # [1, 0, 0, 0]
```

The function `tetgen_to_simplicial` converts the mesh into an `EuclideanComplex`, using the labeling scheme of `tetmesh_to_euclidean`. As usual, the optional argument `p` selects the coefficient field. The resulting mesh is shown in the figure of the tetrahedral mesh of a box, both as a whole and clipped at the plane `y = 0`, which reveals the tetrahedra in the interior.

The returned `TetGenResult` contains the raw mesh in its fields `points` (the vertex coordinates, including all points inserted by TetGen), `tetrahedra` (vectors of four indices into `points`), `trifaces` and `trifacemarkers` (the triangles on all facets of the domain, together with their markers), `tetattributes` (the region attributes of the tetrahedra), and `curvesegments` (the constraint curves). These fields are usually only accessed through the functions described below, but they can be useful for exporting a mesh to other software.

Volume constraints are approximate

TetGen treats volume bounds as targets rather than strict limits. In typical meshes, about one percent of the tetrahedra exceed `maxvolume`, by up to about fifty percent. For the box mesh above, the largest tetrahedron has volume `0.0129`. If a strict bound is required, pass a correspondingly smaller value.

Mesh Size and Quality

Besides `maxvolume`, the function `tetgen_mesh` accepts the following optional arguments:

- `minratio`: An upper bound for the radius-edge ratio of the tetrahedra, see Basic Notions. This ratio is the standard quality measure of TetGen, and well-shaped tetrahedra have small ratios. The default value `1.414` (approximately `sqrt(2)`) is the standard value of TetGen. Smaller values lead to more refinement. Even though a regular tetrahedron has the ratio `0.61`, the refinement algorithm of TetGen is only guaranteed to terminate for bounds which are not too small. Values below 1 are therefore rejected with an `ArgumentError` (with `minratio=0.8`, TetGen was still running after two minutes in our tests), and values close to 1 lead to very fine meshes.
- `mindihedral`: A lower bound for the dihedral angles of the tetrahedra, in degrees, strictly between `0` and `90`, see Basic Notions. It is used in addition to `minratio`. If `minratio=nothing`, the ratio bound `2.0` is used together with it. TetGen treats this bound as a goal, but it does not guarantee it. In the table below, the mesh created with `mindihedral=15` still contains a dihedral angle of `2.33°`. Large values can make TetGen fail. On the box, `mindihedral=30` works, but `mindihedral=50` leads to an error.
- `steiner_on_boundary`: Whether TetGen may insert additional points on the boundary and on constraint facets. Points which are added by the mesh generator, rather than given by the user, are called Steiner points. They are needed both to satisfy the quality bounds and to split long boundary edges. This is `true` by default.
- `quiet`: Whether the output of TetGen is suppressed. Passing `quiet=false` prints the statistics of the mesh generation.
- `switches`: A complete TetGen command line switch string, which overrides all other optional arguments.

Passing `minratio=nothing` without `mindihedral` switches the quality refinement off. The following table shows the effect of these options on the box mesh from above, always with `maxvolume=0.01`. The last column contains the smallest dihedral angle in the mesh:

Optional arguments	Points	Tetrahedra	Smallest angle
`minratio=nothing`	511	2159	5.86°
(defaults)	532	2207	1.86°
`minratio=1.2`	547	2243	4.61°
`mindihedral=15`	608	2528	2.33°
`steiner_on_boundary=false`	8	6	25.24°

The table illustrates two points which are important in practice. First, the quality bounds of TetGen remove most badly shaped tetrahedra, but they do not guarantee the absence of slivers. These are flat tetrahedra with four nearly coplanar vertices, whose radius-edge ratio can be small even though their dihedral angles are close to `0°` or `180°`. A few such tetrahedra are harmless for the construction of multivector fields. Second, disallowing Steiner points on the boundary is a strong restriction. Since no boundary triangle may be split, TetGen cannot insert any point whose neighborhood reaches the boundary. In a coarse box, every tetrahedron touches the

boundary, and therefore no refinement takes place at all, and even the volume constraint is ignored. This option is only useful if the boundary is already fine, or together with interior nodes as in the next section.

The switch string gives access to all options of TetGen. With `maxvolume=0.01` and default values for all other optional arguments, `tetgen_mesh` calls TetGen with the switches `pq1.414a0.01Q`, where `p` tells TetGen to respect the facets, `q` sets the quality bound, `a` the volume bound, and `Q` suppresses the output. The command

```
res = tetgen_mesh(dom; switches="pq1.2a0.01Q")
```

gives the same mesh as `minratio=1.2`. The switch string should always contain `p`, and it should never contain `f`, since otherwise interior triangles are reported with marker `0`, which breaks `tetgen_marker_cells`. See the TetGen manual for a description of all switches.

Adding Interior Nodes

The function `tetgen_domain_add_nodes` adds points which become vertices of the final mesh. This gives direct control over the vertex distribution. For example, a Poisson-disk sample gives a mesh whose vertices are evenly spread throughout the domain:

```
using Random

nodes = poisson_points_box([-1.4,-0.9,-0.9], [1.4,0.9,0.9], 0.3;
                           rng=Xoshiro(1234))
length(nodes)                                  # 223
domn = tetgen_domain_add_nodes(dom, nodes)
# TetDomain with 231 points, 6 facets, markers [0], 0 holes, 0 regions

res = tetgen_mesh(domn; minratio=nothing)
length(res.points), length(res.tetrahedra)    # (231, 1374)
res = tetgen_mesh(domn)
length(res.points), length(res.tetrahedra)    # (409, 2129)
```

Without quality refinement, the mesh is the constrained Delaunay tetrahedralization of exactly the given points. With the default quality bound, TetGen adds further points, mostly close to the boundary, where the sample is less regular. Note that the sample was taken in a slightly smaller box, so that no node lies too close to the boundary. Nodes outside of the domain or inside holes are ignored, and nodes which coincide with existing vertices are merged with them. However, a node must not lie exactly on a face or an edge of the boundary or of a constraint, unless it is one of the vertices there, since TetGen then reports a self-intersection and fails. Combined with `steiner_on_boundary=false`, interior nodes do allow refinement, while the boundary stays as coarse as it is:

```
res = tetgen_mesh(domn; steiner_on_boundary=false, maxvolume=0.01)
length(res.points), length(res.tetrahedra)    # (672, 3913)
length(res.trifaces)                           # 12
```

Constraint Surfaces and Curves

The main reason for using a mesh generator rather than a regular grid is the ability to adapt the mesh to geometric features. In the context of dynamics, these are often invariant manifolds, isolating blocks, or surfaces on which the vector field is known to be transverse. The function `tetgen_domain_add_surface` adds a triangulated surface whose triangles become facets of the domain, so that the final mesh contains the surface as a

union of its triangles. Similarly, `tetgen_domain_add_curve` adds a polygonal curve whose segments become edges of the final mesh. The following example embeds a sphere, a closed square curve above it, and a short open segment below it into a box:

```
dom2 = tetgen_domain_box([-2,-2,-2], [2,2,2])

V, T = trisurface_sphere([0,0,0], 1.0, 2)
dom2 = tetgen_domain_add_surface(dom2, V, T)                # marker 1

square = [[-1.8,-1.8,1.5], [1.8,-1.8,1.5], [1.8,1.8,1.5],
          [-1.8,1.8,1.5], [-1.8,-1.8,1.5]]
dom2 = tetgen_domain_add_curve(dom2, square)                # marker 2

spoke = [[0.0,0.0,-1.8], [0.0,0.0,-1.3]]
dom2 = tetgen_domain_add_curve(dom2, spoke; marker=7)       # marker 7
# TetDomain with 176 points, 331 facets, markers [0, 1, 2, 7], 0 holes, 0 regions

res2 = tetgen_mesh(dom2; maxvolume=0.05)
ec2  = tetgen_to_simplicial(res2)
lefschetz_cell_count(ec2)                     # [950, 5616, 8865, 4198]
```

A curve is closed if its first and last points agree, up to `1e-9` times the size of the curve. In that case, the last point is not added a second time. Repeated consecutive points and segments which occur more than once are ignored. The sphere and the square automatically received the markers 1 and 2, while the segment was given the marker 7 explicitly with the optional argument `marker`. Explicit markers are useful if several surfaces or curves form one geometric object: They can all be given the same marker, and they are then treated as one constraint.

After meshing, the function `tetgen_marker_cells` returns the indices of all cells of the complex which lie on the constraints with a given marker. By default, the result is the closure of the triangles on the constraint surfaces and the edges on the constraint curves, i.e., a closed subcomplex. Extracting these subcomplexes shows that the mesh contains a sphere, a circle, and an arc:

```
sphere_cells = tetgen_marker_cells(res2, ec2, 1)
square_cells = tetgen_marker_cells(res2, ec2, 2)
spoke_cells  = tetgen_marker_cells(res2, ec2, 7)

homology(lefschetz_subcomplex(ec2, sphere_cells))      # [1, 0, 1]
homology(lefschetz_subcomplex(ec2, square_cells))      # [1, 1]
homology(lefschetz_subcomplex(ec2, spoke_cells))       # [1, 0]
```

The mesh and the constraint cells are shown in the figure of the sphere and the two curves embedded as constraints. The left panel shows the mesh clipped at the plane `x = 0`, together with the part of the sphere in the remaining half. The right panel shows only the constraint cells. Note that TetGen subdivided the long edges of the square, but the sphere triangles and the short segment remained intact, since their size is comparable to the mesh size inside the box.

With the optional argument `closure=false`, only the cells of maximal dimension on the constraint are returned, i.e., the triangles of the constraint surfaces and the edges of the constraint curves:

```
sphere_tris = tetgen_marker_cells(res2, ec2, 1; closure=false)
length(sphere_tris), unique(ec2.dimensions[sphere_tris])   # (320, [2])
```

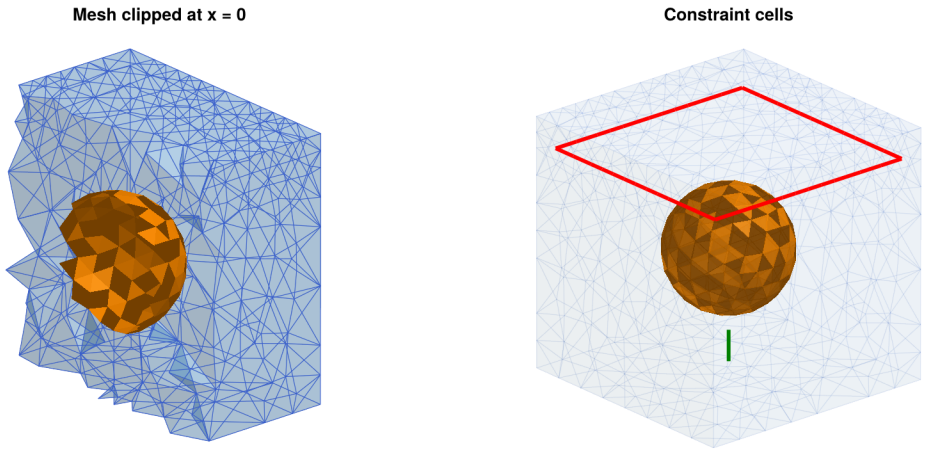


Figure 9.3: A sphere and two curves embedded as constraints

The marker `0` gives the outer boundary of the domain. This makes it easy to work with exit sets or relative homology. Recall from the section Relative Homology that the relative homology of a complex with respect to a closed subcomplex measures the shape of the complex if the subcomplex is collapsed to a single point. For the box mesh from the section Meshing a Box, the boundary is a sphere, and collapsing the boundary of a solid box to a point gives a three-dimensional sphere. Accordingly, the relative homology has a single nonzero entry in dimension three. The second argument of `homology` restricts the computation to the given closed subcomplex:

```
resbox = tetgen_mesh(tetgen_domain_box([-1.5,-1,-1], [1.5,1,1]); maxvolume=0.01)
ecbox  = tetgen_to_simplicial(resbox)
bnd    = tetgen_marker_cells(resbox, ecbox, 0)
homology(ecbox, bnd)           # [1, 0, 1, 0], the homology of the boundary
relative_homology(ecbox, bnd)  # [0, 0, 0, 1]
```

The faces of a box can also be given individual markers with the optional argument `markers` of `tetgen_domain_box`, in the order `x = bmin[1]`, `x = bmax[1]`, `y = bmin[2]`, `y = bmax[2]`, `z = bmin[3]`, and `z = bmax[3]`. This is useful for exit sets. For example, the linear vector field `(x1, -x2, -x3)` has a saddle at the origin, and it leaves the cube `[-1,1]^3` exactly through the two faces `x1 = -1` and `x1 = 1`. These two faces form the exit set of the cube, and by Conley theory, the relative homology of the cube with respect to its exit set is the Conley index of the saddle:

```
domx = tetgen_domain_box([-1,-1,-1], [1,1,1]; markers=[1,1,0,0,0,0])
# TetDomain with 8 points, 6 facets, markers [0, 1], 0 holes, 0 regions
resx = tetgen_mesh(domx; maxvolume=0.02)
ecx  = tetgen_to_simplicial(resx)
exitset = tetgen_marker_cells(resx, ecx, 1)
homology(lefschetz_subcomplex(ecx, exitset))   # [2, 0, 0], two squares
relative_homology(ecx, exitset)                # [0, 1, 0, 0]
```

The same Conley index is obtained from the multivector field of the vector field, using `remove_exit_set`, which determines the exit set automatically:

```
saddlevf(x) = [x[1], -x[2], -x[3]]
lcr, mvfr = remove_exit_set(ecx, create_spatial_mvf(ecx, saddlevf))
connection_matrix(lcr, mvfr).conley           # [[0, 1, 0, 0]]
```

The complex passed to `tetgen_marker_cells` has to be the one created from the given mesh by `tetgen_to_simplicial`. This is checked by comparing the vertex coordinates, and an `ArgumentError` is raised for a mismatch, such as a complex from a different mesh.

Constraints must not intersect

TetGen requires that all facets intersect only in common vertices and edges. Vertices of a new surface or curve which coincide exactly with existing points of the domain are identified with them, so surfaces can share boundary curves if they are created with the same vertex coordinates. Apart from this, constraint surfaces must not cross each other, and they must not touch the outer boundary. In particular, a surface whose boundary curve lies in a face of the box, such as an open cylinder whose ends are on two opposite faces, is rejected, since its boundary curve is not made of edges of the box facets. Similarly, a constraint curve cannot lie inside a face of the box, although it can run along an edge of the box. Curves outside of the domain are ignored by TetGen, and `tetgen_mesh` prints a warning in this case. For intersecting geometry, use `Gmsh.jl` instead, see Gmsh.jl: Complex and Intersecting Geometry.

Holes

A closed constraint surface can be turned into the boundary of a hole by passing `hole=true` to `tetgen_domain_add_surface`. The interior of the surface is then not meshed:

```
dom3 = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom3 = tetgen_domain_add_surface(dom3, V, T; hole=true)
ec3  = tetgen_to_simplicial(tetgen_mesh(dom3; maxvolume=0.05))
homology(ec3)                 # [1, 0, 1, 0]
```

For the above code segment, recall that `V` and `T` are the vertices and triangles of a spherical surface.

TetGen identifies holes by a point inside them, which it then uses to remove all tetrahedra that can be reached from this point without crossing a facet. With `hole=true`, such a point is determined automatically: Starting at the center of the largest triangle of the surface, a ray is followed in the direction of the inward normal until it hits the surface again, and the midpoint of this segment is used. This requires a closed and consistently oriented surface, as created by the `trisurface_*` functions, and both outward and inward normals are fine, see Orientation of Surfaces. The point is stored in the field `holes` of the domain. Holes of any topology are possible, such as the toroidal hole in the right panel of the figure of the domains with holes:

```
Vh, Th = trisurface_torus([0,0,0], 1.2, 0.4, 32, 12)
dom5 = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom5 = tetgen_domain_add_surface(dom5, Vh, Th; hole=true)
ec5  = tetgen_to_simplicial(tetgen_mesh(dom5; maxvolume=0.05))
homology(ec5)                 # [1, 1, 1, 0]
```

The automatically chosen point refers to the chamber enclosed by the surface at the time it is added. The ray stops at all surfaces which are already part of the domain, so a hole surface around a previously added surface removes only the space between the two. If further surfaces are placed inside a hole surface later, however,

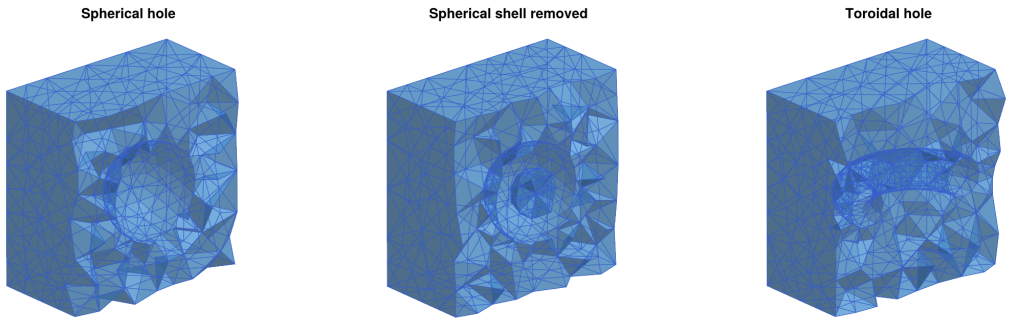


Figure 9.4: Domains with holes, clipped at y = 0

the hole point might end up in the wrong region. Consider two concentric spheres. Adding the outer sphere with `hole=true` creates a hole point at the origin. After adding the inner sphere, this point lies inside the inner ball, and only the inner ball is removed:

```
V1, T1 = trisurface_sphere([0,0,0], 1.0, 2)
V2, T2 = trisurface_sphere([0,0,0], 0.5, 1)

domw = tetgen_domain_box([-2,-2,-2], [2,2,2])
domw = tetgen_domain_add_surface(domw, V1, T1; hole=true)
domw = tetgen_domain_add_surface(domw, V2, T2)
domw.holes                      # [(-1.1e-16, -2.8e-16, 0.0)]
homology(tetgen_to_simplicial(tetgen_mesh(domw; maxvolume=0.05)))
# [1, 0, 1, 0]
```

In such cases, the inner surface should be added first, or the hole point should be given explicitly with `tetgen_domain_add_hole`. The following commands remove the spherical shell between the two spheres, and adding `V2, T2` first and then `V1, T1` with `hole=true` gives the same result. The resulting mesh consists of two components, namely the inner ball and the box with a spherical cavity, as the middle panel of the figure of the domains with holes shows:

```
dom4 = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom4 = tetgen_domain_add_surface(dom4, V1, T1)
dom4 = tetgen_domain_add_surface(dom4, V2, T2)
dom4 = tetgen_domain_add_hole(dom4, [0.75,0,0])
ec4  = tetgen_to_simplicial(tetgen_mesh(dom4; maxvolume=0.05))
homology(ec4)                   # [2, 0, 1, 0]
```

A hole point must lie in a chamber which is completely enclosed by facets. Otherwise, TetGen removes everything that can be reached from it, which is usually the whole domain. Holes and regions also require closed and consistently oriented surfaces, which is checked when the surface is added:

```
Vp, Tp = trisurface_plane_patch([-1,-1,0], [2,0,0], [0,2,0], 4, 4)
tetgen_domain_add_surface(dom3, Vp, Tp; hole=true)
# ERROR: ArgumentError: tetgen_domain_add_surface: holes and regions
#        require a closed surface
```

```
Tflip = deepcopy(T)
Tflip[1] = reverse(Tflip[1])          # one triangle with the wrong orientation
tetgen_domain_add_surface(dom3, V, Tflip; hole=true)
# ERROR: ArgumentError: tetgen_domain_add_surface: holes and regions
#        require a consistently oriented surface
```

A surface may consist of several connected components. In that case, a hole point is determined for every component, so that all of them become holes. For example, two disjoint spheres combined into a single surface give two cavities:

```
Va, Ta = trisurface_sphere([ 1.0,0,0], 0.5, 2)
Vb, Tb = trisurface_sphere([-1.0,0,0], 0.5, 2)
VV = vcat(Va, Vb)
TT = vcat(Ta, [t .+ length(Va) for t in Tb])
dom8 = tetgen_domain_add_surface(tetgen_domain_box([-2,-2,-2], [2,2,2]),
                                 VV, TT; hole=true)
# TetDomain with 332 points, 646 facets, markers [0, 1], 2 holes, 0 regions
homology(tetgen_to_simplicial(tetgen_mesh(dom8; maxvolume=0.05)))
# [1, 0, 2, 0]
```

The same applies to regions. For nested components within one surface, such as two concentric spheres, both the inner ball and the shell between the spheres become holes. To remove only the shell, add the spheres separately, as shown above.

Regions and Local Refinement

Instead of removing the inside of a closed surface, one can declare it a region. Every tetrahedron in a region receives an integer attribute, and every region can have its own maximal tetrahedron volume. This is the standard way of refining a mesh locally, for example close to the equilibria of a vector field. With `tetgen_domain_add_surface`, a region is created by passing `region=(attr, maxvol)`:

```
dom6 = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom6 = tetgen_domain_add_surface(dom6, V, T; region=(1, 0.002))
res6 = tetgen_mesh(dom6; maxvolume=0.1)
ec6  = tetgen_to_simplicial(res6)
lefschetz_cell_count(ec6)      # [1467, 10212, 17276, 8530]
unique(res6.tetattributes)     # [1, 2]
```

Inside the unit sphere, the tetrahedra have volume at most `0.002`, while the global bound `0.1` applies everywhere else, both up to the tolerance described in the note on volume constraints above. Note that the second attribute 2 was not specified by us. TetGen assigns new attributes, larger than all given ones, to all parts of the domain which do not contain a region point. Here, this is the part of the box outside of the sphere. The function `tetgen_region_cells` returns the tetrahedra with a given attribute, by default together with all their faces:

```
inner = tetgen_region_cells(res6, ec6, 1)
outer = tetgen_region_cells(res6, ec6, 2)
count(==(3), ec6.dimensions[inner])                  # 4048
count(==(3), ec6.dimensions[outer])                  # 4482
homology(lefschetz_subcomplex(ec6, inner))           # [1, 0, 0, 0]
homology(lefschetz_subcomplex(ec6, outer))           # [1, 0, 1, 0]
```

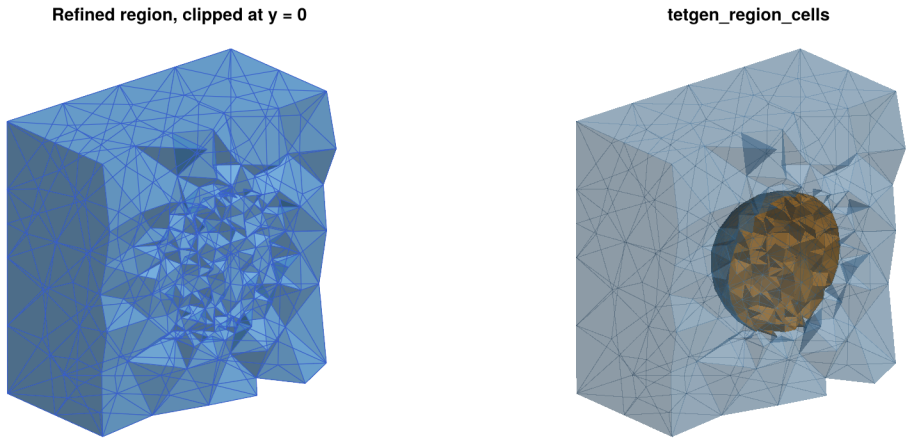


Figure 9.5: A refined region inside a sphere

As before, `closure=false` returns only the tetrahedra. The left panel of the figure of the refined region inside a sphere shows the refined mesh, and the right panel shows the two regions in different colors.

Regions can also be specified by an explicit point with `tetgen_domain_add_region`. Its arguments are the domain, the point, the attribute, and optionally the maximal volume. A nonpositive maximal volume, which is the default, means that only the global bound applies. The following domain gives the same mesh as above, but the outer part now has the attribute 5:

```
dom7 = tetgen_domain_box([-2,-2,-2], [2,2,2])
dom7 = tetgen_domain_add_surface(dom7, V, T)
dom7 = tetgen_domain_add_region(dom7, [0,0,0], 1, 0.002)
dom7 = tetgen_domain_add_region(dom7, [1.9,1.9,1.9], 5)
res7 = tetgen_mesh(dom7; maxvolume=0.1)
unique(res7.tetattributes)     # [1, 5]
```

Domains Bounded by a Surface

Not every domain is a box. The function `tetgen_domain_from_surface` creates a domain whose outer boundary is a closed triangulated surface. All its facets get marker `0`, and the domain can then be extended in the same way as before. The following commands create a solid torus, and an ellipsoid with two spherical holes:

```
Vt, Tt = trisurface_torus([0,0,0], 2.0, 0.8, 48, 16)
domt = tetgen_domain_from_surface(Vt, Tt)
# TetDomain with 768 points, 1536 facets, markers [0], 0 holes, 0 regions
ect  = tetgen_to_simplicial(tetgen_mesh(domt; maxvolume=0.02))
homology(ect)                  # [1, 1, 0, 0]

Ve, Te = trisurface_ellipsoid([0,0,0], [2.0,1.0,0.5], 3)
dome = tetgen_domain_from_surface(Ve, Te)
dome = tetgen_domain_add_surface(dome, trisurface_sphere([ 0.8,0,0], 0.3, 2)...;
                                 hole=true)
dome = tetgen_domain_add_surface(dome, trisurface_sphere([-0.8,0,0], 0.3, 2)...;
```

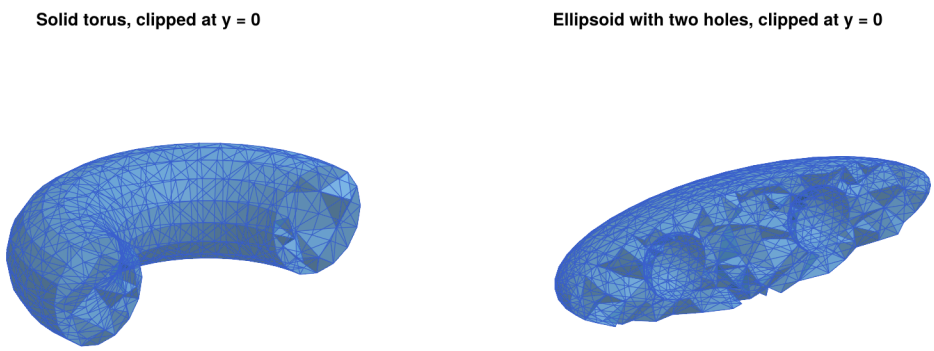


Figure 9.6: Domains bounded by a torus and an ellipsoid

```
                                hole=true)
ece  = tetgen_to_simplicial(tetgen_mesh(dome; maxvolume=0.01))
homology(ece)                  # [1, 0, 2, 0]
```

Note the splatting operator `...`, which passes the pair `(V, T)` returned by `trisurface_sphere` as two separate arguments. The boundary surface has to be closed, i.e., every edge has to belong to exactly two triangles:

```
tetgen_domain_from_surface(trisurface_cylinder([0,0,-1], [0,0,1], 0.5, 16, 4;
                                               capped=false)...)
# ERROR: ArgumentError: tetgen_domain_from_surface: the surface is not closed
```

Surfaces from other sources, such as STL files, often store every triangle with its own copies of its three vertices. Both `tetgen_domain_from_surface` and `tetgen_domain_add_surface` identify vertices with the same coordinates before they check the surface, so that such surfaces can be used directly. Triangles which then have repeated vertices are rejected with an `ArgumentError`.

Delaunay Triangulations of Random Points

The function `create_simplicial_delaunay3d` computes the Delaunay tetrahedralization of a random point sample in a box, and it returns it directly as an `EuclideanComplex`. It is the three-dimensional analogue of the planar function `create_simplicial_delaunay`. There are two methods. The first one uses a Poisson-disk sample with minimum distance `pdist`, and the second one uses `npoints` uniformly distributed points:

```
using Random

ecd1 = create_simplicial_delaunay3d([0,0,0], [2,1,1], 0.15, 20; rng=Xoshiro(1))
lefschetz_cell_count(ecd1)     # [393, 2733, 4622, 2281]

ecd2 = create_simplicial_delaunay3d([0,0,0], [2,1,1], 300; box_corners=true,
                                    rng=Xoshiro(1))
lefschetz_cell_count(ecd2)     # [308, 2239, 3858, 1926]
```

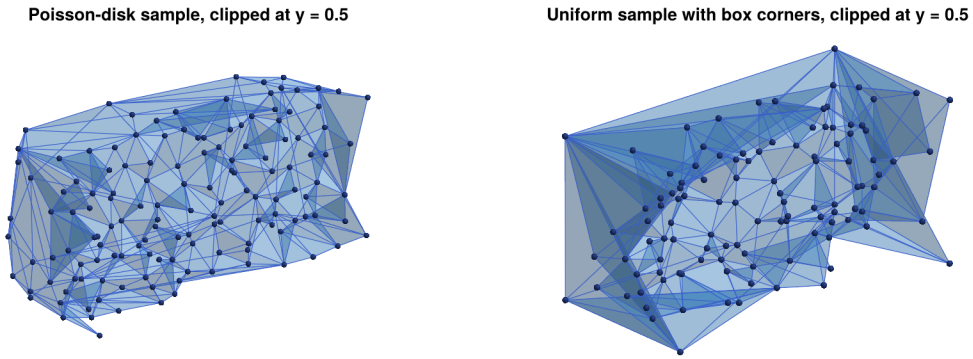


Figure 9.7: Delaunay triangulations of random points

As in the planar case, the complex covers only the convex hull of the points, and no quality refinement takes place. The convex hull of a random sample does not fill the box, and the tetrahedra close to its boundary are often very flat. The optional argument `box_corners=true` adds the eight corners of the box to the sample, so that the convex hull is the whole box, which explains the `308 = 300 + 8` vertices in the second example. Both complexes are shown in the figure of the Delaunay triangulations of random points, clipped at the plane `y = 0.5`. The optional arguments `p` and `rng` have the same meaning as before. If the minimum distance `pdist` is so large that the sample contains fewer than four points, an `ArgumentError` is raised.

These triangulations are useful for experiments with unstructured meshes, since they are easy to create. For computations on a box, a mesh created by `tetgen_mesh` and `tetgen_domain_box` is usually the better choice, since it fills the box and has well-shaped tetrahedra.

Error Messages

When TetGen cannot mesh a domain, `tetgen_mesh` raises an `ArgumentError` with an explanation, instead of the internal error codes of TetGen. The most common reason is a constraint which intersects another constraint or the boundary. The following example adds a plane patch which crosses the embedded sphere:

```
Vp, Tp = trisurface_plane_patch([-1.5,-1.5,0.2], [3,0,0], [0,3,0], 4, 4)
dombad = tetgen_domain_box([-2,-2,-2], [2,2,2])
dombad = tetgen_domain_add_surface(dombad, V, T)
dombad = tetgen_domain_add_surface(dombad, Vp, Tp)
tetgen_mesh(dombad)
# ERROR: ArgumentError: tetgen_mesh: TetGen failed (TetGen error 3: a
#   self-intersection was detected. ...). Possible causes are constraint
#   surfaces or curves which intersect each other or the boundary (for
#   intersecting geometry, use Gmsh.jl instead), nodes which lie on a
#   facet or a curve, or quality bounds which are too strict, such as
#   minratio close to 1 or a large mindihedral.
```

Without the sphere, the same plane patch is a valid constraint, since it lies completely inside the box. Depending on the configuration, TetGen can also report an internal error (error code 2) for intersecting constraints, and the same internal error occurs for quality bounds which TetGen cannot achieve, such as `mindihedral=50`. When TetGen detects intersecting facets, it writes the debugging files `tetgen-tmpfile_skipped.node` and `tetgen-tmpfile_skipped.face` into the current directory. Since they are of no use without the TetGen com-

mand line tools, `tetgen_mesh` removes them again; files with these names which existed before the call are not touched. The second typical failure is a mesh without any tetrahedra, which happens if the outer boundary is not closed, or if a hole point does not lie in an enclosed region:

```
tetgen_mesh(tetgen_domain_add_hole(tetgen_domain_box([-2,-2,-2], [2,2,2]),
                                   [0,0,0]))
# ERROR: ArgumentError: tetgen_mesh: the mesh contains no tetrahedra. Is the
#   outer boundary closed, and does every hole point lie inside a closed
#   constraint surface?
```

Complete Example: Conley Indices on a TetGen Mesh

We close this section with a complete example, which combines a domain with refined regions, the construction of a multivector field, and a connection matrix computation. The vector field

$$\dot{x}_1 = x_1(1 - x_1^2), \qquad \dot{x}_2 = -x_2, \qquad \dot{x}_3 = -x_3$$

has three equilibria at `(-1,0,0)`, `(0,0,0)`, and `(1,0,0)`. The outer two are stable, and the one at the origin has a one-dimensional unstable manifold, which connects it to the two stable equilibria. The box `[-1.5,1.5] x [-1,1] x [-1,1]` is a trapping region, i.e., the vector field points into the box everywhere on its boundary, so that no solution leaves the box in forward time. Since the interesting dynamics is concentrated close to the equilibria, we refine the mesh in small balls around them, and keep the mesh coarse everywhere else:

```
using TetGen
using ConleyDynamics

function samplevf(x::Vector{Float64})
    x1, x2, x3 = x
    return [x1 * (1.0 - x1*x1), -x2, -x3]
end

domc = tetgen_domain_box([-1.5,-1,-1], [1.5,1,1])
for (k, c) in enumerate([[-1.0,0,0], [0.0,0,0], [1.0,0,0]])
    Vk, Tk = trisurface_sphere(c, 0.25, 1)
    global domc = tetgen_domain_add_surface(domc, Vk, Tk; region=(k, 0.0005))
end

resc = tetgen_mesh(domc; maxvolume=0.02)
ecc  = tetgen_to_simplicial(resc)
ecc.ncells                      # 13607

mvfc = create_spatial_mvf(ecc, samplevf)
cmc  = connection_matrix(ecc, mvfc)
cmc.conley
# 3-element Vector{Vector{Int64}}:
#  [1, 0, 0, 0]
#  [1, 0, 0, 0]
#  [0, 1, 0, 0]
full_from_sparse(cmc.matrix)
# 3×3 Matrix{Int64}:
#  0  0  1
#  0  0  1
#  0  0  0
```

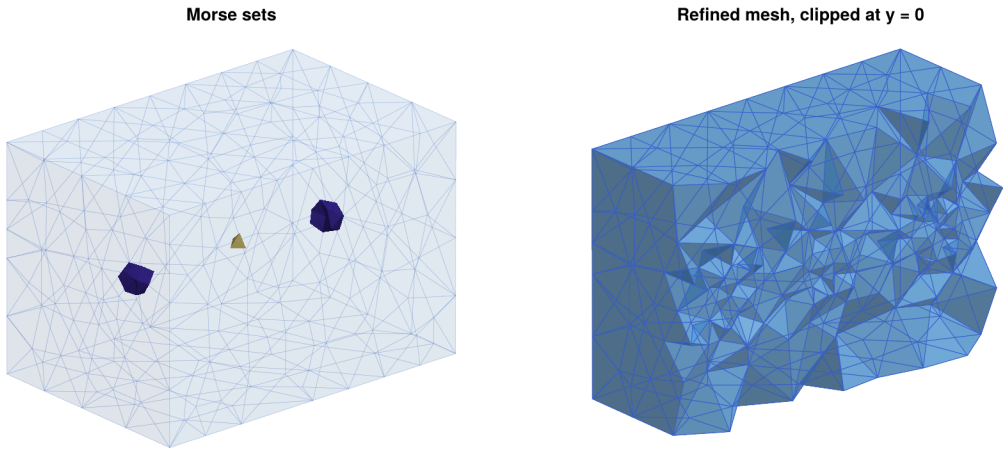


Figure 9.8: Morse sets of a vector field on a locally refined TetGen mesh

The `global` keyword is needed when this loop is part of a script file, since `domc` is a global variable which is reassigned inside the loop. In the REPL and inside of functions, it can be omitted. The connection matrix computation finds three Morse sets. Their Conley indices are given as vectors of Betti numbers, just like homology. For a hyperbolic equilibrium with a k-dimensional unstable manifold, the Conley index has a single nonzero entry 1 in dimension k, i.e., at position k+1 of the vector. Two of the Morse sets therefore have the Conley index `[1, 0, 0, 0]` of an attracting equilibrium (`k = 0`), and the third one has the Conley index `[0, 1, 0, 0]` of a saddle with a one-dimensional unstable manifold (`k = 1`). The rows and columns of the connection matrix correspond to the nonzero entries of the Conley indices, in the order of the Morse sets. Its nonzero entries in the third column show the two connecting orbits from the saddle to the two attractors. The Morse sets are shown in the left panel of the figure of the Morse sets on a locally refined TetGen mesh, colored by their Conley indices. The right panel shows the mesh clipped at the plane `y = 0`, which reveals the three refined balls.

This figure was created with the following commands, using the plotting functions described in Three-Dimensional Plotting with Makie:

```
using CairoMakie

fig = Figure(size=(1200,550))
ax1 = Axis3(fig[1,1]; aspect=:data, title="Morse sets")
plot_simplicial3d_morse!(ax1, ecc, cmc.morse; ci=true)
ax2 = Axis3(fig[1,2]; aspect=:data, title="Refined mesh, clipped at y = 0")
plot_simplicial3d!(ax2, ecc; clip=([0,-1,0], 0.0), alpha=0.6)
for ax in (ax1, ax2)
    hidedecorations!(ax)
    hidespines!(ax)
end
save("meshing3d_conley.png", fig; px_per_unit=1.5)
```

9.5 Gmsh.jl: Complex and Intersecting Geometry

Gmsh is a finite element mesh generator with a built-in CAD engine, and the Gmsh.jl package makes its complete API available in Julia. Compared to TetGen, it offers two important advantages. First, geometry is described by CAD entities such as boxes, spheres, cylinders, and arbitrary curved surfaces, rather than only by triangulations. Second, and more importantly in our context, it can compute boolean operations on such entities. In particular, it can handle constraint surfaces which intersect each other.

To see why this matters, consider a box which contains a plane at height `z = 0.3`, as well as a sphere around the origin intersecting it in a circle. A conforming mesh which contains both surfaces has to contain this circle as a union of edges, and the triangles of the plane and the sphere have to be split along it. TetGen cannot do this, since it requires non-intersecting facets, see the Error Messages above. Computing such an intersection by hand is a difficult problem in computational geometry. Gmsh, on the other hand, computes it with a single `fragment` operation, which splits all entities along their mutual intersections. The function `gmsh_common_refinement` performs this operation and keeps track of the resulting pieces, see Intersecting Constraint Surfaces and Common Refinements of Several Objects below.

The extension provides five functions: `gmsh_to_simplicial` converts the mesh of the current Gmsh model into an `EuclideanComplex`, `gmsh_marker_cells` finds the cells of physical groups and entities, `gmsh_common_refinement` splits intersecting objects along their intersections and keeps track of the pieces, `gmsh_add_trisurface` adds a triangulated surface to the model, so that it can be intersected with other objects, and `gmsh_file_to_simplicial` reads mesh and geometry files. Everything else, i.e., the construction of the geometry and the meshing itself, is done with the Gmsh API. Its documentation can be found on the Gmsh web page, and the examples below cover the most important commands. All of them assume that `Gmsh.jl` has been installed and loaded with `using Gmsh: gmsh` and `using ConleyDynamics`, as described in Loading the Required Packages. Note that Gmsh is distributed under the GNU General Public License, see Licences of the Meshing Backends below.

Basic Usage

A Gmsh session is started with `gmsh.initialize()` and ended with `gmsh.finalize()`. In between, one or more models can be created. A model consists of geometric entities of dimensions 0 to 3 (points, curves, surfaces, and volumes), each identified by its dimension and an integer tag. Gmsh contains two geometry engines, its own built-in one and the CAD library OpenCASCADE, whose functions are collected in the module `gmsh.model.occ`. We always use OpenCASCADE, since only it supports boolean operations. The following commands create a box with `gmsh.model.occ.addBox`, mesh it with a maximal element size of `0.2`, and convert the mesh into a complex:

```
gmsh.initialize()
gmsh.option.setNumber("General.Terminal", 0)     # no output

gmsh.model.add("box")
gmsh.model.occ.addBox(0, 0, 0, 2, 1, 1)          # corner and side lengths
gmsh.model.occ.synchronize()
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.2)
gmsh.model.mesh.generate(3)

ec  = gmsh_to_simplicial()
lefschetz_cell_count(ec)       # [407, 2111, 3089, 1384]
homology(ec)                   # [1, 0, 0, 0]
```

The call to `gmsh.model.occ.synchronize()` transfers the CAD geometry to the model, and it is needed after every change of the geometry. The argument of `gmsh.model.mesh.generate` is the largest dimension to

be meshed, and Gmsh builds the mesh from the bottom up. The command `gmsh.model.mesh.generate(1)` divides all curves of the model into line segments. The command `gmsh.model.mesh.generate(2)` does this as well, and then covers all surfaces with triangles, which fit the segments on their boundary curves. Finally, `gmsh.model.mesh.generate(3)` performs both of these steps, and then fills all volumes with tetrahedra, which fit the triangles on their boundary surfaces. The resulting mesh therefore contains vertices at all points, edges on all curves, triangles on all surfaces, and tetrahedra in all volumes.

The function `gmsh_to_simplicial` only reads the current model. It neither initializes nor finalizes Gmsh, so that the model is still available afterwards. By default, it converts the tetrahedra, and adds all their faces. With the optional argument `dim=2`, it converts the triangles instead, and with `dim=1` the edges. Note, however, that Gmsh does not store all faces of the tetrahedra as mesh elements. Every element belongs to one geometric entity of the model, i.e., tetrahedra belong to volumes, triangles to surfaces, and edges to curves, while the triangles and edges in the interior of a volume are not stored. Therefore, `dim=2` only converts the triangles on the surfaces of the model, and not all triangles of the tetrahedral mesh. For the box, this gives its boundary surface as a two-dimensional complex. If the model contains interior surfaces, such as the interface between two volumes, their triangles are included as well. Similarly, `dim=1` only converts the edges on the curves of the model, which for the box gives the graph formed by its twelve edges:

```
ecs = gmsh_to_simplicial(dim=2)
lefschetz_cell_count(ecs)      # [323, 963, 642]
homology(ecs)                  # [1, 0, 1]

ecg = gmsh_to_simplicial(dim=1)
lefschetz_cell_count(ecg)      # [76, 80]
homology(ecg)                  # [1, 5]
```

Compared to the tetrahedral mesh, the surface complex lacks the 84 vertices and 1148 edges in the interior of the box, and contains only 642 of its 3089 triangles. The graph has the first Betti number five, since the eight corners and twelve edges of the box give the Euler characteristic `8 - 12 = -4 = 1 - 5`.

The optional argument `p` selects the coefficient field as usual. The Gmsh node numbering can have gaps, and nodes which do not belong to the converted elements are dropped. The vertices are then numbered consecutively, and they are labeled as in `tetmesh_to_euclidean`, for example `"001"`, `"002"`, and so on for a mesh with between 100 and 999 vertices.

Only linear simplices are supported, i.e., edges, triangles, and tetrahedra whose nodes are their vertices. If the elements of dimension `dim` include other types, an `ArgumentError` is raised. This happens for quadrangles, hexahedra, prisms, and pyramids, which Gmsh creates with its recombination options. It also happens for second order elements, which `gmsh.model.mesh.setOrder(2)` creates by adding nodes on the edges to follow curved geometry. Only the elements of dimension `dim` are checked. For example, a hexahedral mesh can still be converted with `dim=1`, since its edges are linear.

Physical Groups and Entities

After meshing, one usually needs the cells on particular surfaces or in particular volumes. Gmsh provides two mechanisms for this. Every mesh element belongs to exactly one geometric entity, and entities can be collected into physical groups, which are again identified by a dimension and a tag. The function `gmsh_marker_cells` returns the cells of the complex which belong to a physical group or an entity. For the box above, the six faces are the surface entities with tags 1 through 6:

```
gmsh.model.getEntities(2)
# [(2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6)]
[gmsh.model.occ.getCenterOfMass(2, t) for (d, t) in gmsh.model.getEntities(2)]
```

```
# [(0.0, 0.5, 0.5), (2.0, 0.5, 0.5), (1.0, 0.0, 0.5), (1.0, 1.0, 0.5),
#  (1.0, 0.5, 0.0), (1.0, 0.5, 1.0)]

face1 = gmsh_marker_cells(ec, 2, 1; physical=false)      # the face x = 0
homology(lefschetz_subcomplex(ec, face1))                # [1, 0, 0]

pg  = gmsh.model.addPhysicalGroup(2, [1, 2, 3, 4, 5, 6])
bnd = gmsh_marker_cells(ec, 2, pg)                       # all six faces
homology(lefschetz_subcomplex(ec, bnd))                  # [1, 0, 1]

gmsh.finalize()
```

The second and third arguments of `gmsh_marker_cells` are the dimension and the tag. By default, they refer to a physical group. With `physical=false`, they refer to a geometric entity instead. As for `tetgen_marker_cells`, the result is a closed subcomplex, and `closure=false` returns only the cells of dimension `dim`. Physical groups can be added after meshing, as in this example. The cells are identified by the coordinates of their vertices, so the numbering of the Gmsh nodes does not matter.

Intersecting Constraint Surfaces

We now return to the example from the introduction of this section. Consider a box `[-2,2]^3` which contains the plane `z = 0.3` and the unit sphere, both as constraint surfaces. What we are looking for is a common refinement of the box, the plane, and the sphere, i.e., a tetrahedral mesh of the box in which the plane and the sphere are both unions of triangles, and in which their intersection, the circle, is a union of edges. In the language of Basic Notions, the plane, the sphere, and the circle are then subcomplexes of the mesh. The geometry is constructed as follows:

```
using Gmsh: gmsh
using ConleyDynamics

gmsh.initialize()
gmsh.option.setNumber("General.Terminal", 0)
gmsh.model.add("intersecting")

# 1. Geometry as CAD entities. Each call returns the tag of the entity.

box    = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
sphere = gmsh.model.occ.addSphere(0, 0, 0, 1.0)
plane  = gmsh.model.occ.addRectangle(-2, -2, 0.3, 4, 4)
gmsh.model.occ.synchronize()

# 2. We only need the sphere as a surface: Take its boundary surface
#    and delete the volume itself.

sphsurf = gmsh.model.getBoundary([(3, sphere)], false, false)
gmsh.model.occ.remove([(3, sphere)])

# 3. Split all objects along their intersections, and create one
#    physical group for each of them.

pg_box, pg_plane, pg_sphere = gmsh_common_refinement([(3, box), (2, plane), sphsurf];
                                                     names=["box", "plane", "sphere"])
```

The function addSphere creates a solid ball, whose boundary surface is returned by getBoundary. Its two optional arguments combined and oriented are both set to false, so that the boundary of every given entity is returned separately, rather than the boundary of their union, and the tags come without orientation signs. The function gmsh_common_refinement expects a vector of objects, where every object is either a single entity (dim, tag), such as the box and the plane, or a vector of entities of the same dimension, such as the boundary surface of the sphere. It splits all objects along their mutual intersections, and it returns one physical group for every object, which contains all pieces of this object. The optional argument names gives these groups names, which are also stored in mesh files. In our case, the box is split into four volumes (above and below the plane, each inside and outside the sphere), the plane into a disk inside the sphere and the rest, and the sphere surface into two caps. The pieces of the box can be listed with gmsh.model.getEntitiesForPhysicalGroup, and we collect the ones above the plane, which are identified by their centers of mass, into a further physical group:

```
volumes  = gmsh.model.getEntitiesForPhysicalGroup(3, pg_box)    # [1, 2, 3, 4]
upper    = [v for v in volumes if gmsh.model.occ.getCenterOfMass(3, v)[3] > 0.3]
pg_upper = gmsh.model.addPhysicalGroup(3, upper)
```

Now the model is meshed and converted:

```
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.25)
gmsh.model.mesh.generate(3)

ec = gmsh_to_simplicial()
lefschetz_cell_count(ec)       # [4335, 26847, 43139, 20626]
homology(ec)                   # [1, 0, 0, 0]

plane_cells  = gmsh_marker_cells(ec, 2, pg_plane)
sphere_cells = gmsh_marker_cells(ec, 2, pg_sphere)
circle_cells = intersect(plane_cells, sphere_cells)

homology(lefschetz_subcomplex(ec, plane_cells))      # [1, 0, 0]
homology(lefschetz_subcomplex(ec, sphere_cells))     # [1, 0, 1]
homology(lefschetz_subcomplex(ec, circle_cells))     # [1, 1]
```

Even though the four volumes are separate entities in Gmsh, their meshes match along the common surfaces, and the complex is a single contractible tetrahedral mesh of the box. The plane and the sphere are both subcomplexes, and since the mesh is a common refinement, their intersection is exactly the circle, made of edges of the mesh. This is precisely the kind of mesh that TetGen cannot create. The plane, the sphere, and the circle are shown in the left panel of the figure of the plane and the sphere intersecting in a circle.

Touching or intersecting entities have to be refined together

If entities which touch or intersect are meshed without gmsh_common_refinement or a fragment operation, Gmsh meshes them independently. The resulting mesh contains distinct nodes at the same positions, and the complex falls apart along them. For example, two boxes [0,1]^3 and [1,2] x [0,1] x [0,1] created with addBox and meshed directly give a complex with homology [2, 0, 0, 0], i.e., two separate cubes, and cells on the common face are not found reliably by gmsh_marker_cells. In such cases, gmsh_to_simplicial prints a warning. Calling gmsh_common_refinement([(3, b1), (3, b2)]) before meshing gives one connected mesh with homology [1, 0, 0, 0].

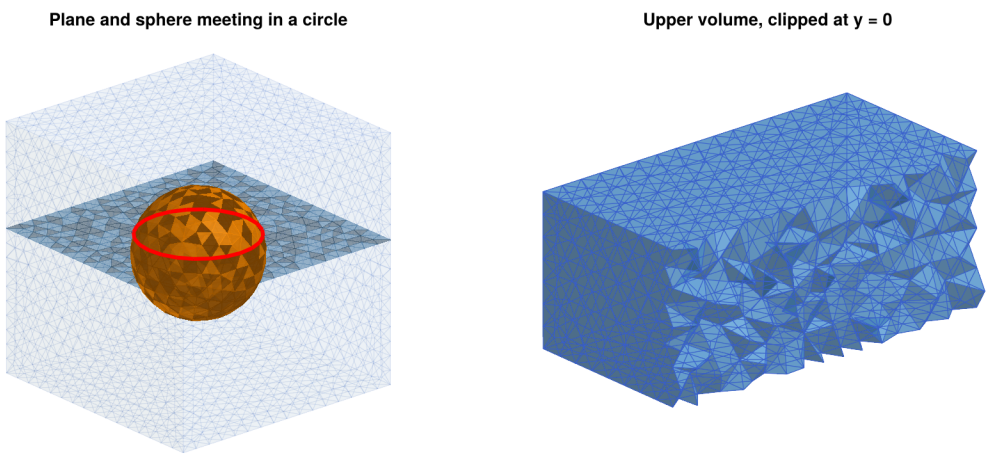


Figure 9.9: A plane and a sphere intersecting in a circle, meshed with Gmsh

Individual pieces can be accessed through their entities. For example, the upper cap of the sphere is the sphere piece whose center of mass lies above the plane:

```
caps   = gmsh.model.getEntitiesForPhysicalGroup(2, pg_sphere)
topcap = [t for t in caps if gmsh.model.occ.getCenterOfMass(2, t)[3] > 0.3][1]
top    = gmsh_marker_cells(ec, 2, topcap; physical=false)
homology(lefschetz_subcomplex(ec, top))              # [1, 0, 0]

toptri = gmsh_marker_cells(ec, 2, topcap; physical=false, closure=false)
unique(ec.dimensions[toptri]), length(toptri)        # ([2], 206)
```

Finding the intersection circle as an entity is slightly more involved, since the OpenCASCADE sphere has a seam, i.e., a meridian curve along which the parametrization of the sphere is glued. Its pieces belong to the boundary of the caps as well. The intersection circle is the curve which the two caps have in common:

```
capcurves(t) = Set(c for (d, c) in gmsh.model.getBoundary([(2, t)], false, false))
circle_tags  = collect(intersect(capcurves(caps[1]), capcurves(caps[2])))
circle2 = gmsh_marker_cells(ec, 1, circle_tags[1]; physical=false)
sort(circle2) == sort(circle_cells)                  # true
```

Since `oriented` is `false`, the curve tags are returned without orientation signs, so that the tags of the two caps can be compared directly. In most cases, however, it is simpler to work with the cell lists of the physical groups and their intersections, as above.

What happens behind the scenes. The function `gmsh_common_refinement` is a thin wrapper around the OpenCASCADE function `gmsh.model.occ.fragment`, which splits a list of object entities and a list of tool entities along all their intersections. It returns two values: A vector `out` with all resulting entities, and a vector `outmap`, whose k-th entry contains the pieces of the k-th input entity, in the order objects first, then tools. The tags of the pieces depend on the internal bookkeeping of OpenCASCADE, and they should never be hard-coded. In a new model with the same steps 1 and 2, one could therefore replace step 3 by

```
out, outmap = gmsh.model.occ.fragment([(3, box)], vcat([(2, plane)], sphsurf))
gmsh.model.occ.synchronize()
outmap
# [[(3, 1), (3, 2), (3, 3), (3, 4)],     # pieces of the box
#  [(2, 3), (2, 14)],                    # pieces of the plane
#  [(2, 13), (2, 7)]]                    # pieces of the sphere surface

pg_box    = gmsh.model.addPhysicalGroup(3, [t for (d, t) in outmap[1] if d == 3])
pg_plane  = gmsh.model.addPhysicalGroup(2, [t for (d, t) in outmap[2] if d == 2])
pg_sphere = gmsh.model.addPhysicalGroup(2, [t for m in outmap[3:end] for (d, t) in m
                                            if d == 2])
```

This gives the same mesh and the same physical groups, except that the groups have no names. Here, `outmap[3:end]` collects the pieces of all boundary surfaces of the sphere, so that the code does not depend on the number of surfaces that form the sphere boundary. The wrapper performs exactly this bookkeeping for any number of objects, which is less error-prone, in particular if objects consist of several entities.

Restricting the Mesh to Parts of the Model

The optional argument `physical=(pdim, ptag)` of `gmsh_to_simplicial` converts only the elements of a physical group. Its dimension has to agree with `dim`. Continuing the example above, the following commands extract the mesh of the upper part of the box, which is shown in the right panel of the figure of the plane and the sphere intersecting in a circle, and the sphere as a surface complex:

```
ecu = gmsh_to_simplicial(physical=(3, pg_upper))
lefschetz_cell_count(ecu)      # [2032, 11917, 18621, 8735]
homology(ecu)                  # [1, 0, 0, 0]

ecsph = gmsh_to_simplicial(dim=2, physical=(2, pg_sphere))
homology(ecsph)                # [1, 0, 1]

gmsh_to_simplicial(physical=(2, pg_sphere))
# ERROR: ArgumentError: gmsh_to_simplicial: the physical group needs to
#        have dimension dim
```

Of course, the cells of the upper part could also be found within the complete complex `ec` with `gmsh_marker_cells(ec, 3, pg_upper)`. The difference is that `gmsh_to_simplicial` creates a new complex with its own labels, while `gmsh_marker_cells` returns cell indices in the given complex.

We finish the example by saving the mesh to a file, which is used again in the next section, and by ending the Gmsh session:

```
gmsh.option.setNumber("Mesh.SaveAll", 1)
gmsh.write("intersecting.msh")
gmsh.finalize()
```

The option `Mesh.SaveAll` is explained at the end of the next section.

Reading Files

Gmsh reads geometry files in its own `.geo` scripting language, CAD files such as STEP or IGES, and mesh files in the `.msh` format and many others. The function `gmsh_file_to_simplicial` opens any such file, generates a

mesh if the file does not contain elements of the requested dimension, and converts the mesh. For example, the following `.geo` file describes a ball of radius 2, together with a mesh size:

```
SetFactory("OpenCASCADE");
Sphere(1) = {0, 0, 0, 2};
Mesh.MeshSizeMax = 0.3;
```

If it is saved as `ball.geo`, then

```
ecb = gmsh_file_to_simplicial("ball.geo")
lefschetz_cell_count(ecb)      # [1344, 8079, 12777, 6041]
homology(ecb)                  # [1, 0, 0, 0]

homology(gmsh_file_to_simplicial("ball.geo"; dim=2))     # [1, 0, 1]
```

creates a tetrahedral mesh of the ball, and a triangulation of its boundary sphere. Since Gmsh is not running at this point, the function initializes it and finalizes it at the end, so no other Gmsh commands are needed.

If Gmsh is already running, the file is opened in the running session. This creates a new model, which becomes the current model, while the previous models remain available, see `gmsh.model.list()` and `gmsh.model.setCurrent`. The new model also remains available after the conversion, including the physical groups stored in a mesh file. This allows the use of `gmsh_marker_cells` after reading a file. For example, the mesh of the intersecting surfaces which was saved at the end of the previous section can be read and analyzed as follows:

```
gmsh.initialize()
ecm = gmsh_file_to_simplicial("intersecting.msh")
lefschetz_cell_count(ecm)            # [4335, 26847, 43139, 20626]
gmsh.model.getPhysicalGroups()       # [(2, 2), (2, 3), (3, 1), (3, 4)]
gmsh.model.getPhysicalName(2, 3)     # "sphere"
homology(lefschetz_subcomplex(ecm, gmsh_marker_cells(ecm, 2, 3)))   # [1, 0, 1]
gmsh.finalize()
```

The physical groups are numbered in the order in which they were created, i.e., the box, the plane, and the sphere by `gmsh_common_refinement`, and the upper part of the box afterwards. Their tags are therefore 1 to 4, and each tag comes with the dimension of its group.

Saving meshes with physical groups

As soon as a model contains physical groups, Gmsh by default only saves the elements which belong to some physical group. In the example above, all tetrahedra belong to the physical group of the box which was created by `gmsh_common_refinement`, so the complete tetrahedral mesh would be saved even without the option `Mesh.SaveAll`. If the only three-dimensional physical group were the upper part of the box, however, the file would only contain the tetrahedra of this part. To be on the safe side, either set `Mesh.SaveAll` to 1 as above, or define physical groups for all volumes.

Common Refinements of Several Objects

The function `gmsh_common_refinement` accepts any number of objects of dimensions 0 to 3, i.e., volumes, surfaces, curves, and single points. All of them are refined together, so that the mesh of every object is a

subcomplex, and the intersection of any two objects is the intersection of their subcomplexes. As a second example, consider two unit spheres around the points `(-1/2, 0, 0)` and `(1/2, 0, 0)`, which intersect in a circle in the plane `x = 0`, together with the segment of the x-axis from `-1.9` to `1.9`, which crosses both spheres twice. The segment is a curve, i.e., an entity of dimension 1, which is created from its two endpoints:

```
gmsh.initialize()
gmsh.option.setNumber("General.Terminal", 0)
gmsh.model.add("twospheres")

function spheresurface(x, y, z, r)
    s = gmsh.model.occ.addSphere(x, y, z, r)
    gmsh.model.occ.synchronize()
    bnd = gmsh.model.getBoundary([(3, s)], false, false)
    gmsh.model.occ.remove([(3, s)])
    return bnd
end

box   = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
left  = spheresurface(-0.5, 0, 0, 1.0)
right = spheresurface( 0.5, 0, 0, 1.0)
axis  = gmsh.model.occ.addLine(gmsh.model.occ.addPoint(-1.9, 0, 0),
                               gmsh.model.occ.addPoint( 1.9, 0, 0))

pgs = gmsh_common_refinement([(3, box), left, right, (1, axis)])
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.25)
gmsh.model.mesh.generate(3)
ec2 = gmsh_to_simplicial()
lefschetz_cell_count(ec2)                              # [4303, 26845, 43256, 20713]

cl = gmsh_marker_cells(ec2, 2, pgs[2])                 # left sphere
cr = gmsh_marker_cells(ec2, 2, pgs[3])                 # right sphere
ca = gmsh_marker_cells(ec2, 1, pgs[4])                 # axis
homology(lefschetz_subcomplex(ec2, cl))                # [1, 0, 1]
homology(lefschetz_subcomplex(ec2, intersect(cl, cr))) # [1, 1]
homology(lefschetz_subcomplex(ec2, union(cl, cr)))     # [1, 0, 3]
homology(lefschetz_subcomplex(ec2, ca))                # [1, 0]

crossings = intersect(ca, union(cl, cr))
sort([ec2.coords[k][1][1] for k in crossings])         # [-1.5, -0.5, 0.5, 1.5]
```

Both spheres are subcomplexes with the homology of a sphere, and their intersection is a circle. Their union consists of four disks, namely the two caps of each sphere, which are glued along the common circle. It therefore encloses three cavities, which explains the Betti number `b2 = 3`. The axis is a chain of edges of the mesh, and it meets the spheres in exactly four vertices, at the four points where the segment crosses them. Since the segment was refined together with the spheres, these crossing points are vertices of the mesh, and not just points somewhere inside of triangles.

Volumes can be used as objects as well. They are the natural way to describe regions, in which, for example, a vector field has a known behavior. Moreover, an object does not have to lie inside of the box. In the following example, the plane is larger than the box, and the solid unit ball is a region:

```
gmsh.model.add("region")
box   = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
plane = gmsh.model.occ.addRectangle(-3, -3, 0.2, 6, 6)
```

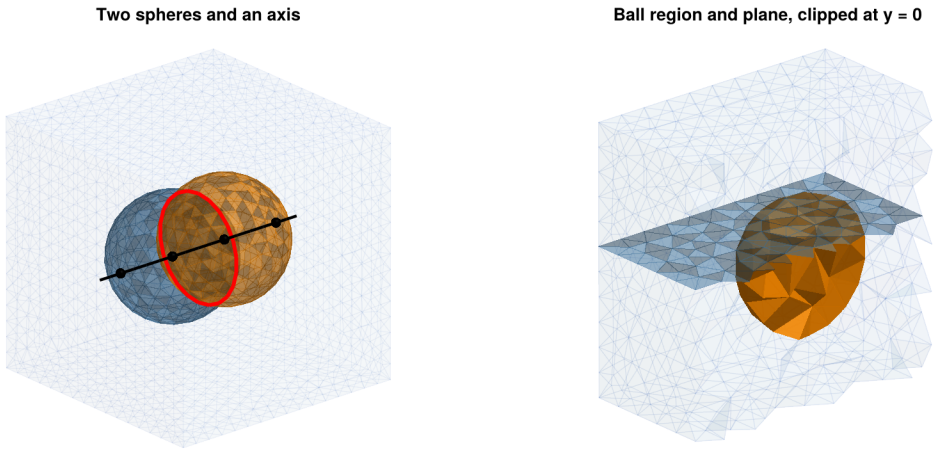


Figure 9.10: Common refinements of several objects, meshed with Gmsh

```
ball  = gmsh.model.occ.addSphere(0, 0, 0, 1.0)

pg_box, pg_plane, pg_ball = gmsh_common_refinement([(3, box), (2, plane), (3, ball)])
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.4)
gmsh.model.mesh.generate(3)
ec3 = gmsh_to_simplicial()

plane_cells = gmsh_marker_cells(ec3, 2, pg_plane)
ball_cells  = gmsh_marker_cells(ec3, 3, pg_ball)
homology(lefschetz_subcomplex(ec3, ball_cells))                          # [1, 0, 0, 0]
homology(lefschetz_subcomplex(ec3, intersect(plane_cells, ball_cells)))  # [1, 0, 0]
maximum(maximum(abs.(ec3.coords[k][1])) for k in plane_cells)            # 2.0
```

The cells of the ball form a closed subcomplex, and the intersection of the ball with the plane is a disk. The part of the plane outside of the box is split off by the refinement and meshed only as a surface. Since it does not belong to any tetrahedron, it is not part of the complex, and `gmsh_marker_cells` ignores it, so that all vertices of `plane_cells` lie in the box.

Both examples are shown in the figure of the common refinements of several objects. In the left panel, the two spheres are drawn in blue and orange, their intersection circle in red, and the axis together with its four crossing points in black. The right panel shows the second mesh clipped at the plane `y = 0`, with the ball region in orange and the plane in blue.

In applications, surfaces are often only known as triangulations, for example as the output of the `trisurface_*` functions, or as read from an STL file. Such a surface can be added to the model with `gmsh_add_trisurface`. It turns every triangle into a flat surface of `gmsh.model.occ`, where adjacent triangles share their points and edges, so that the surface is the piecewise linear interpolation of its vertices. The function returns the vector of these surfaces, which is one object of `gmsh_common_refinement`. As a third example, consider a box which contains the unit sphere, a rectangle in the plane `x = 0.4`, and a triangulation of the paraboloid `z = -0.6 + (x^2 + y^2)/2` over the square `[-1.5, 1.5]^2`. The rectangle is created in a horizontal plane, then rotated by ninety degrees around the y-axis, and finally moved to the plane `x = 0.4`:

```
gmsh.model.add("paraboloid")

fpar(x, y) = -0.6 + 0.5*(x^2 + y^2)
Vp, Tp = trisurface_parametric((x, y) -> [x, y, fpar(x, y)],
                               (-1.5, 1.5), (-1.5, 1.5), 12, 12)

box    = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
sphere = spheresurface(0, 0, 0, 1.0)
rect   = gmsh.model.occ.addRectangle(-1.6, -1.6, 0, 3.2, 3.2)
gmsh.model.occ.rotate([(2, rect)], 0, 0, 0, 0, 1, 0, pi/2)
gmsh.model.occ.translate([(2, rect)], 0.4, 0, 0)
parab  = gmsh_add_trisurface(Vp, Tp)
length(parab)                                          # 288

pgs = gmsh_common_refinement([(3, box), sphere, (2, rect), parab])
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.25)
gmsh.model.mesh.generate(3)
ec4 = gmsh_to_simplicial()
lefschetz_cell_count(ec4)                              # [6114, 39797, 65538, 31854]

cs = gmsh_marker_cells(ec4, 2, pgs[2])                 # sphere
cr = gmsh_marker_cells(ec4, 2, pgs[3])                 # rectangle
cp = gmsh_marker_cells(ec4, 2, pgs[4])                 # paraboloid
gmsh.finalize()

homology(lefschetz_subcomplex(ec4, cp))                # [1, 0, 0]
homology(lefschetz_subcomplex(ec4, intersect(cs, cr))) # [1, 1]
homology(lefschetz_subcomplex(ec4, intersect(cs, cp))) # [1, 1]
homology(lefschetz_subcomplex(ec4, intersect(cr, cp))) # [1, 0]

triple = intersect(cs, cr, cp)
[round.(ec4.coords[k][1]; digits=3) for k in triple]
# 2-element Vector{Vector{Float64}}:
#  [0.4, -0.912, -0.089]
#  [0.4, 0.912, -0.089]
```

The 12 x 12 grid of the parameter square gives 288 triangles, and thus 288 surfaces of the model. After the refinement, the paraboloid is a disk. It meets the sphere in a closed curve, and the rectangle in an arc of a parabola, while the sphere and the rectangle meet in a circle. All three surfaces have exactly two points in common. For the smooth paraboloid, these points would be `(0.4, -0.910, -0.106)` and `(0.4, 0.910, -0.106)`, but the piecewise linear interpolation of a convex function lies above the function, so that the points of the triangulated paraboloid are slightly higher. On the other hand, every edge of the triangulation becomes a curve of the model, and the mesh has to contain it as a chain of edges. The mesh near a triangulated surface is therefore at least as fine as the triangulation, regardless of `Mesh.MeshSizeMax`. In our example, the mesh has about a third more vertices than with a smooth paraboloid, and with a 48 x 48 grid, it has almost three times as many.

The figure of the common refinement of a sphere, a rectangle, and a triangulated paraboloid shows the result. In the left panel, the sphere is drawn in orange, the rectangle in blue, and the paraboloid in green, together with the three intersection curves in red and the two common points in black. The right panel shows the tetrahedral mesh, clipped at the plane `y = 0`, together with the three surfaces.

The following remarks summarize how common refinements are used in practice:

- **Describing the objects.** Constraint surfaces and regions have to be created as CAD entities with

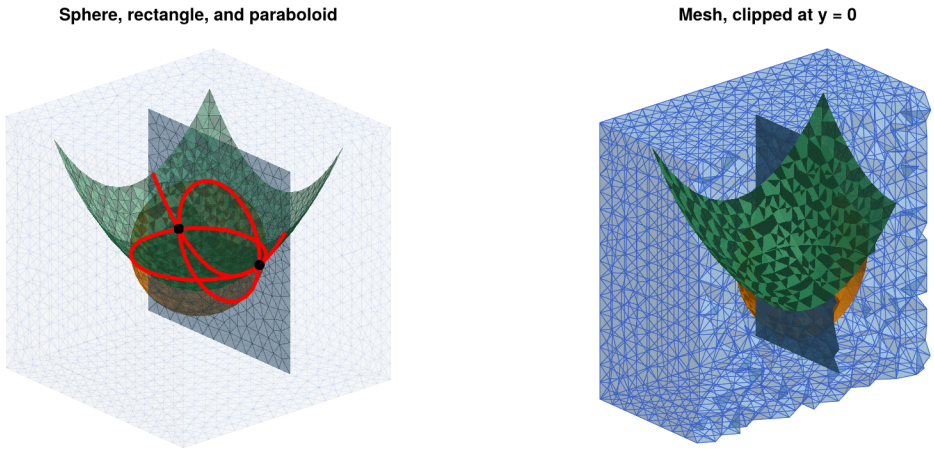


Figure 9.11: Common refinement of a sphere, a rectangle, and a triangulated paraboloid, meshed with Gmsh

the functions in `gmsh.model.occ`, or, for triangulated surfaces, with `gmsh_add_trisurface`, see the next remark. The functions in `gmsh.model.occ` include `addBox`, `addSphere`, `addCylinder`, `addCone`, `addTorus`, and `addWedge` for solids, `addRectangle` and `addDisk` for planar surfaces, and `addPoint`, `addLine`, `addCircle`, and `addSpline` for points and curves. The surface of a solid is obtained with `getBoundary`, as for the spheres above. Curved surfaces which are not of this form can be created, for example, with `addBSplineSurface`, `addSurfaceFilling`, or `addThruSections`, and CAD files in the STEP or IGES format can be loaded with `gmsh.model.occ.importShapes`. See the `Gmsh` documentation for details.

- **Triangulated surfaces.** The surfaces created by the `trisurface_*` functions, or read from STL files, are triangulations, not CAD entities. They can be added to the model with `gmsh_add_trisurface`, as for the paraboloid above. Since every triangle becomes a surface of the model, and every edge a curve, this is slower than a CAD entity for fine triangulations, and it forces a fine mesh. If an analytic description is available, as for spheres, cylinders, and tori, use the corresponding CAD function instead.

- **Positioning.** Entities can be moved and rotated with `gmsh.model.occ.translate` and `gmsh.model.occ.rotate`, for example `gmsh.model.occ.rotate([(2, plane)], 0, 0, 0, 1, 0, 0, pi/2)` rotates a plane by ninety degrees around the x-axis through the origin.

- **Mesh size.** The intersection curves are meshed like all other curves, so their edges have length at most `Mesh.MeshSizeMax`. If two objects intersect at a very small angle, or if they are very close to each other without intersecting, `Gmsh` needs small elements to resolve the geometry, which can lead to large meshes.

- **Using the cells.** The cell lists returned by `gmsh_marker_cells` are closed subcomplexes. Their unions and intersections, computed with `union` and `intersect`, are again closed, and they can be used as exit sets, as arguments of `relative_homology`, or to restrict a multivector field to a part of the mesh.

Mesh Sizes and Other Options

The size of a `Gmsh` mesh is controlled by options, which are set with `gmsh.option.setNumber` before calling `gmsh.model.mesh.generate`. The most important ones are `Mesh.MeshSizeMax` and `Mesh.MeshSizeMin`, which bound the edge lengths of the mesh, and `Mesh.MeshSizeFromCurvature`, which sets the number of elements

per 2*pi of curvature on curved surfaces. Local refinement is possible with mesh size fields, which are described in the Gmsh documentation. Unlike TetGen, Gmsh controls edge lengths rather than volumes. In the examples of this section, a mesh with maximal edge length h has roughly 5/h^3 tetrahedra per unit volume, and about five times as many cells in total. This is useful for estimating the size of a complex before creating it.

When Gmsh fails to mesh a model

For complicated configurations, for example with many intersection curves close to each other, the three-dimensional mesher of Gmsh can fail with messages such as `PLC Error: Two facets intersect at point.` The reason is usually a surface mesh which is too coarse for the geometry, and a smaller value of `Mesh.MeshSizeMax` typically resolves the problem. In our tests with random configurations of spheres and planes in a box, about one in twenty-five configurations failed with the mesh size `0.3`, and a failing configuration could be meshed with `0.25`. After such a failure, the Gmsh session has to be restarted with `gmsh.finalize()` and `gmsh.initialize()`, since later calls of `gmsh.model.mesh.generate` in the same session do not create any tetrahedra. Other choices of the three-dimensional meshing algorithm, which is selected with the option `Mesh.Algorithm3D`, can also fail silently and leave some volumes without tetrahedra. In this case, `gmsh_to_simplicial` prints a warning, since the complex then has holes which are not part of the geometry.

Complete Example: A Periodic Orbit in a Ball

As a final example, we analyze a vector field with an attracting periodic orbit on a curved domain. The vector field

$$\dot{x}_1 = x_1 - \tfrac{1}{2}x_2 - x_1(x_1^2 + x_2^2), \qquad \dot{x}_2 = \tfrac{1}{2}x_1 + x_2 - x_2(x_1^2 + x_2^2), \qquad \dot{x}_3 = -x_3$$

has an attracting periodic orbit, the unit circle in the plane `x_3 = 0`, and an equilibrium at the origin with a two-dimensional unstable manifold, which is the open unit disk in the same plane. The ball of radius 2 is a trapping region. We mesh it with Gmsh and compute the connection matrix:

```
using Gmsh: gmsh
using ConleyDynamics

function cyclevf(x::Vector{Float64})
    x1, x2, x3 = x
    r2 = x1*x1 + x2*x2
    return [x1 - 0.5*x2 - x1*r2, 0.5*x1 + x2 - x2*r2, -x3]
end

gmsh.initialize()
gmsh.option.setNumber("General.Terminal", 0)
gmsh.model.occ.addSphere(0, 0, 0, 2.0)
gmsh.model.occ.synchronize()
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.2)
gmsh.model.mesh.generate(3)
ecb = gmsh_to_simplicial()
gmsh.finalize()

ecb.ncells                    # 92169
mvfb = create_spatial_mvf(ecb, cyclevf)
```

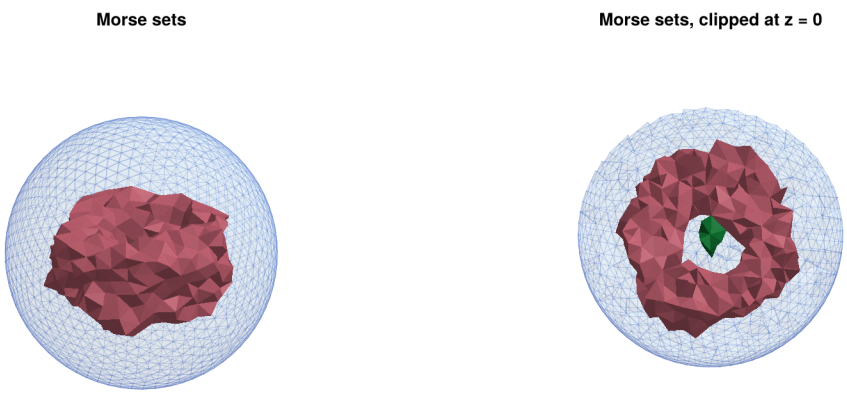


Figure 9.12: Morse sets of a vector field with a periodic orbit in a ball meshed by Gmsh

```
cmb  = connection_matrix(ecb, mvfb)
cmb.conley
# 2-element Vector{Vector{Int64}}:
#  [1, 1, 0, 0]
#  [0, 0, 1, 0]
full_from_sparse(cmb.matrix)
# 3×3 Matrix{Int64}:
#  0  0  0
#  0  0  1
#  0  0  0
```

The Morse decomposition consists of two Morse sets. The first one has the Conley index `[1, 1, 0, 0]` of an attracting periodic orbit, which agrees with the homology of a circle, and the second one has the Conley index `[0, 0, 1, 0]` of an equilibrium with a two-dimensional unstable manifold. The connection matrix has three rows and columns, one for each nonzero entry of the two Conley indices. Its nonzero entry in row 2 and column 3 connects the two-dimensional entry of the equilibrium with the one-dimensional entry of the periodic orbit, and records the connection from the equilibrium to the periodic orbit. The computation takes a few seconds. The Morse sets are shown in the figure of the Morse sets of the vector field with a periodic orbit in a ball. The right panel shows the lower half of the ball from above. The Morse set of the periodic orbit is a solid torus around the unit circle, and the small Morse set of the equilibrium lies in its hole, separated from it by a gap. With a coarser mesh size of `0.3`, the multivector field is not fine enough to separate the two invariant sets, and one obtains a single Morse set with the Conley index of an attractor.

9.6 Licences of the Meshing Backends

ConleyDynamics.jl itself is distributed under the MIT licence. The meshing backends `TetGen.jl` and `Gmsh.jl` are optional weak dependencies, which have to be installed and loaded separately by the user, and they are distributed under different licences:

- TetGen is distributed under the GNU Affero General Public License, version 3 (AGPLv3). Commercial licences are available from its author.

- Gmsh is distributed under the GNU General Public License, version 2 or later, with some exceptions described on its web page.

For computations and publications based on the generated meshes, these licences place no restrictions on the user. However, anyone who redistributes software which includes or links to `TetGen` or `Gmsh` should check the respective licence terms.

Chapter 10

Visualization

ConleyDynamics.jl provides two independent visualization backends for planar complexes: a Luxor.jl-based backend that produces PDF, PNG, or SVG files directly, and a Plots.jl-based backend that integrates with the Julia plotting ecosystem. In addition, complexes embedded in three dimensions, such as tetrahedral meshes, cubical complexes in space, and triangulated surfaces, can be visualized with a Makie.jl-based backend. The three backends are described in separate sections below.

10.1 Luxor-Based Plotting

The functions

- `plot_planar_simplicial`,
- `plot_planar_simplicial_morse`,
- `plot_planar_cubical`, and
- `plot_planar_cubical_morse`

produce publication-quality plots by writing directly to a PDF, PNG, or SVG file via the Luxor.jl package. They require an explicit output filename and a separate coordinate vector for the vertices of the complex. The type of the image file is indicated by the filename ending.

As a simple example, the following commands create a PDF image of a small simplicial complex together with its Morse sets:

```
using ConleyDynamics

lc, mvf = example_forman2d()
cm = connection_matrix(lc, mvf)
coords = [[50,200],[150,200],[250,200],[0,100],[100,100],
          [200,100],[300,100],[50,0],[150,0],[250,0]]
fname  = "forman2d_morse.pdf"
plot_planar_simplicial_morse(lc, coords, fname, cm.morse)
```

For cubical complexes the workflow is analogous. The function `get_cubical_coords` extracts vertex coordinates from the cube labels, so no separate coordinate vector needs to be specified manually:

```
using ConleyDynamics

cc, coords = create_cubical_rectangle(4, 3)
fname = "cubical_rectangle.pdf"
plot_planar_cubical(cc, coords, fname)
```

Both families of Luxor functions also accept a `EuclideanComplex` as their first argument. In that case the coordinate vector can be omitted entirely. For example, the first of the above two plots can alternatively be created using the following commands:

```
using ConleyDynamics

ec, mvf = example_forman2d(euclidean=true)
cm = connection_matrix(ec, mvf)
fname  = "forman2d_morse.pdf"
plot_planar_simplicial_morse(ec, fname, cm.morse)
```

10.2 Plots-Based Plotting

The Plots.jl backend provides eight convenience functions:

Function	Description
`plot_simplicial`	Simplicial complex with optional Forman overlay
`plot_cubical`	Cubical complex with optional Forman overlay
`plot_simplicial_morse`	Simplicial complex with colored Morse sets
`plot_cubical_morse`	Cubical complex with colored Morse sets
`plot_simplicial_mvf`	Simplicial complex with multivector field regions
`plot_cubical_mvf`	Cubical complex with multivector field regions
`plot_simplicial_mv`	Single multivector on a simplicial complex
`plot_cubical_mv`	Single multivector on a cubical complex

All of these functions are described with short examples in the remainder of this section.

Weak Dependency

Plots.jl is a weak dependency of ConleyDynamics.jl. This means it is not loaded automatically when you do `using ConleyDynamics`, and it does not need to be installed unless you want to use the Plots.jl backend. The eight functions above are stub definitions that become active only after Plots.jl has been loaded. The required loading order is:

```
using Plots
using ConleyDynamics
```

If ConleyDynamics is loaded before Plots.jl, the plotting functions will still work because Julia activates package extensions lazily. However, loading Plots first is the recommended order.

All eight functions return a `Plots.Plot` object, so the full Plots.jl interface applies: the result can be displayed with `display(p)`, saved with `savefig(p, "output.png")`, or composed with other plots. In particular, several plots can be combined in one plot, and it is possible to increase the plot resolution to generate high-quality images. Some of this functionality is included in the examples below.

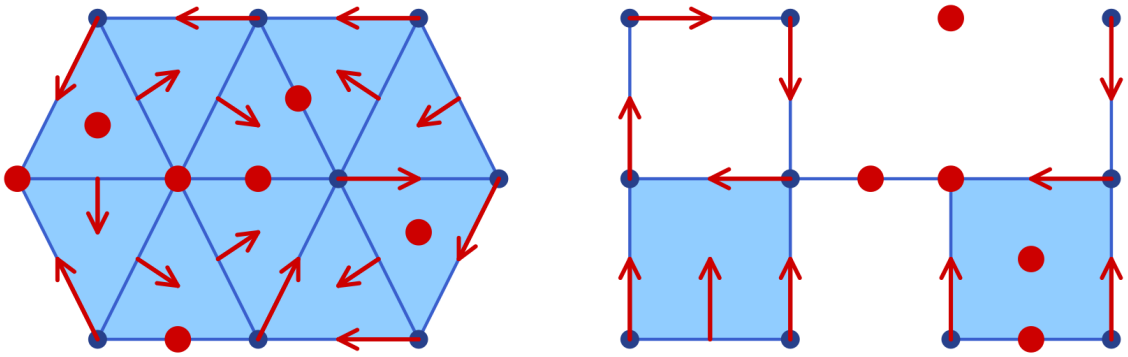


Figure 10.1: Sample Forman vector field plots using 'Plots.jl'

EuclideanComplex as Input Requirement

The Plots.jl backend works exclusively with `EuclideanComplex` objects, which carry embedded vertex coordinates. The simplest way to obtain such a complex is to pass `euclidean=true` when creating a complex:

```
ec, mvf = example_forman2d(euclidean=true)
cc = create_cubical_rectangle(4, 3, euclidean=true)
```

Alternatively, `lefschetz_to_euclidean` converts any `LefschetzComplex` together with a coordinate vector into a `EuclideanComplex`.

Plotting a Complex and Forman Vector Fields

The functions `plot_simplicial` and `plot_cubical` are the main functions for displaying a planar simplicial or cubical complex. In addition, both of them accept an optional `mvf` keyword argument that overlays the Forman part of a multivector field on the complex. More precisely, two-element multivectors are drawn as red arrows from the lower-dimensional cell's barycenter to the higher-dimensional one. Explicitly critical cells (singletons) and cells absent from the multivector field are rendered as red dots at their barycenters. Multivectors of size larger than two are not visualized. Thus, the functions `plot_simplicial` and `plot_cubical` will mainly be used for pure complex visualization, as well as for Forman vector fields on such complexes.

As a first example, the following commands are used to create the simplicial complex and associated Forman vector field shown in the left panel of the figure with the sample Forman vector field plots:

```
using Plots
using ConleyDynamics

sc, smvf = example_forman2d(euclidean=true)
sp = plot_simplicial(sc, mvf=smvf)
display(sp)
```

For the cubical case, the proceeding is similar. In order to generate the right panel of this figure, we used the commands

```
using Plots
using ConleyDynamics
```

```
cubes = ["00.11", "01.01", "02.10", "11.10",
         "11.01", "22.00", "20.11", "31.01"]
cc = create_cubical_complex(cubes, euclidean=true)
cmvf = [["01.00","01.01"], ["02.00","02.10"], ["12.00","11.01"],
        ["11.00","01.10"], ["00.00","00.01"], ["00.10","00.11"],
        ["10.00","10.01"], ["20.00","20.01"], ["32.00","31.01"],
        ["30.00","30.01"], ["31.00","21.10"]]
cp = plot_cubical(cc, mvf=cmvf)
display(cp)
```

Note that the figure with the sample Forman vector field plots contains both plots side-by-side. This was achieved directly using the Plots.jl framework. One can combine both plots in a single one, change the resolution to `dpi = 300`, and then save the figure using the following commands:

```
pcombined = plot(sp, cp, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "plots_forman.png")
```

In the above examples, the vertices, edges, and two-dimensional cells are drawn separately using different shades of blue. In both functions, the keyword argument `pdim::Vector{Bool}` controls which dimensions of the simplicial or cubical complex are drawn. The three entries correspond to vertices (dimension 0), edges (dimension 1), and faces (dimension 2), respectively. For example, `pdim = [false,true,true]` suppresses vertices:

```
p = plot_cubical(create_cubical_rectangle(4, 3, euclidean=true),
                 pdim=[false,true,true])
display(p)
```

Both plots in the figure with the sample Forman vector field plots were created using `pdim = [true,true,true]`, which is also the default if the argument is not specified.

Plotting Morse Sets for Planar Dynamics

The functions `plot_simplicial_morse` and `plot_cubical_morse` are used to visualize Morse sets of multivector fields on planar simplicial or cubical complexes. In addition to an argument of type `EuclideanComplex`, they expect a `CellSubsets` argument which contains a list of locally closed sets, usually the Morse sets associated with a multivector field.

To keep the visualization simple, the vertices, edges, and faces in a Morse set are all colored with the same color. While by default all cells are highlighted, both functions accept the optional argument `pdim::Vector{Bool}`, which allows the user to select the dimensions that should be drawn. In the case of large complexes, the choice `pdim = [false, false, true]` is most appropriate, since then only the two-dimensional cells are highlighted.

The Morse plot functions color each Morse set in a distinct, automatically chosen color. By default only the explicitly listed Morse sets are rendered. Passing the optional argument `addcritical = true` additionally draws cells absent from any Morse set as implicit singletons in separate colors. Since this is mostly used when visualizing complete multivector fields in the above way, and since the focus of the functions `plot_simplicial_morse` and `plot_cubical_morse` is on plotting Morse sets, the default value of this optional argument is `addcritical = false`.

To demonstrate both functions, we return to an earlier example.

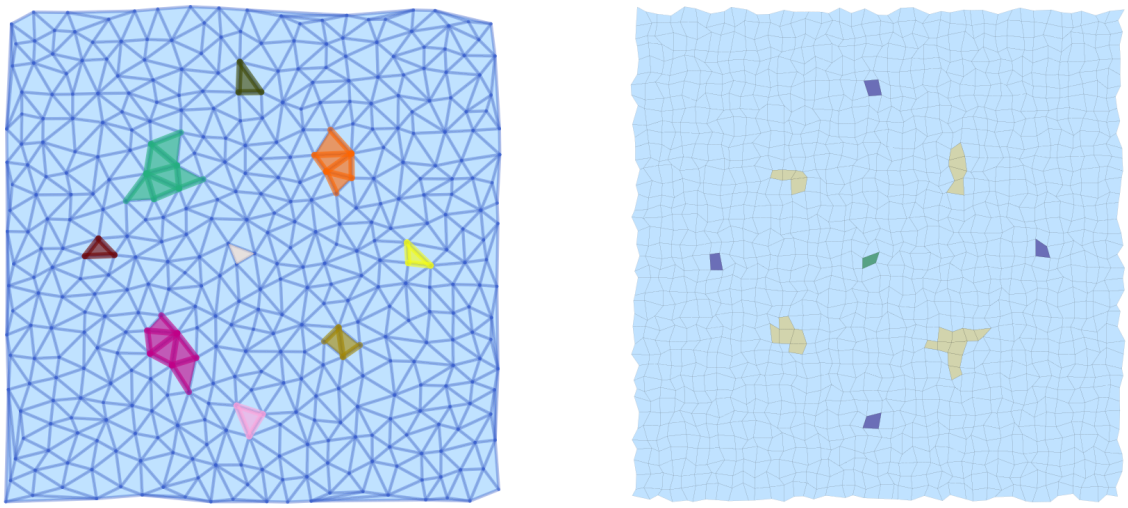


Figure 10.2: Sample Morse plots using ‘Plots.jl‘

```
using Plots
using ConleyDynamics

function planarvf(x::Vector{Float64})
    #
    # Sample planar vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end

sc  = create_simplicial_delaunay(300, 300, 12, 30, euclidean=true);
sc  = rescale_coords(sc, -1.5, 1.5);
svf = create_planar_mvf(sc, planarvf);
scm = connection_matrix(sc, svf);
pls = plot_simplicial_morse(sc, scm.morse);
display(pls)
```

The above commands define a vector field in the plane, create a Delaunay triangulation of the region of interest, and then associate a multivector field to these choices. Finally, the connection matrix is computed, which gives the Morse sets in `scm.morse`. Note that the plot command only colors the Morse sets, but in randomly chosen colors. We would like to point out that all vertices, edges, and faces of the Morse sets are emphasized, see the left panel of the figure with the sample Morse plots. Moreover, the vertices and edges of the underlying simplicial complex are highlighted as well.

The same vector field can also be analyzed using a cubical complex with randomly perturbed vertex locations:

```
cc  = create_cubical_rectangle(40, 40, randomize=0.4, euclidean=true);
cc  = rescale_coords(cc, -1.5, 1.5);
cvf = create_planar_mvf(cc, planarvf);
ccm = connection_matrix(cc, cvf);
```

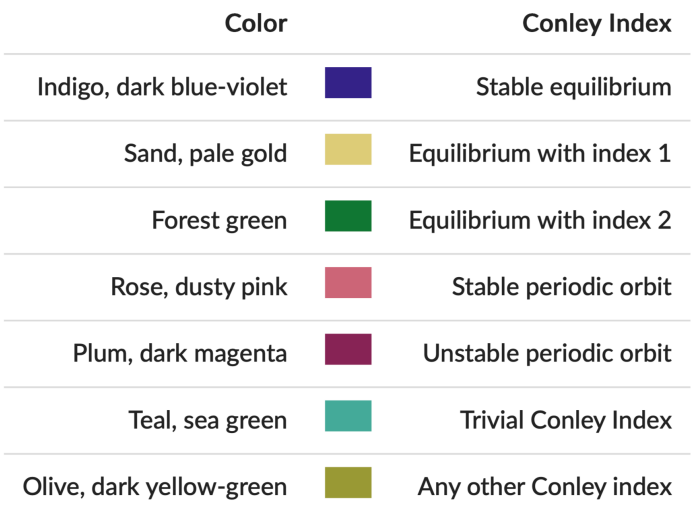


Figure 10.3: Color palette for Conley indices

```
plc = plot_cubical_morse(cc, ccm.morse, ci=true, pdim=[false,false,true]);
display(plc)
```

The resulting plot is shown in the right panel of the figure with the sample Morse plots. Note that due to the optional argument `pdim`, only the faces of the simplicial complex and the Morse sets are highlighted. In addition, this command passes the argument `ci = true`, which forces that the Morse set colors correspond to the Conley index of the respective Morse set as explained in the figure of the color palette for Conley indices.

All of these colors are taken from Paul Tol's qualitative palette, which is designed for colorblind-friendly data visualization.

As before, both plots `pls` and `plc` where combined side-by-side, and then printed:

```
pcombined = plot(pls, plc, layout=(1,2))
plot!(pcombined, dpi=300)
savefig(pcombined, "plots_morse.png")
```

We would like to point out that the argument `ci=true` can be passed to both `plot_simplicial_morse` and `plot_cubical_morse`.

Plotting Multivector Field Regions

While the above-introduced functions are suitable for the visualization of Forman vector fields and Morse sets, they often are not adequate in the setting of multivector fields, especially when the focus is on the depiction of individual multivectors. This is due to the fact that a multivector and its behavior relies crucially on which parts of its boundary are in the mouth. Thus, simply rendering the top-dimensional cell often buries the important information.

In view of this, ConleyDynamics.jl provides both `plot_simplicial_mvf` and `plot_cubical_mvf`. Both of these functions render each multivector as a single colored region. The region geometry is computed as an inflated convex hull of the cell barycenters, so adjacent multivectors are visually separated by a visible gap.

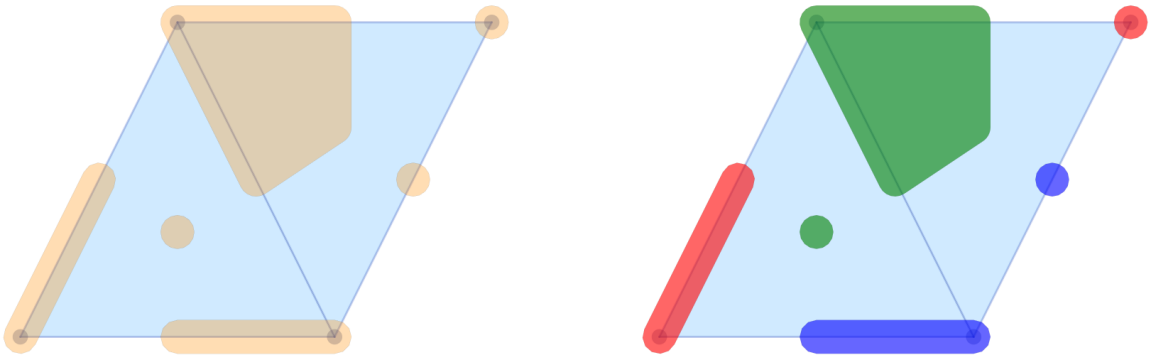


Figure 10.4: Multivector field visualizations

As a first example, consider the following code snippet:

```
using Plots
using ConleyDynamics

lc, mvf = example_julia_logo()
coords = [[0,0], [1,2], [2,0], [3,2]]
ec = lefschetz_to_euclidean(lc, coords)

p1 = plot_simplicial_mvf(ec, mvf)
p2 = plot_simplicial_mvf(ec, mvf, mvfalpha=0.6,
                         mvfcolor=["red", "blue", "green"])

pcombined12 = plot(p1, p2, layout=(1,2))
plot!(pcombined12, dpi=300)
savefig(pcombined12, "plots_mvf_vectors.png")
```

The left panel of the figure of the multivector field visualizations shows the multivectors from the logo example in the above-described form. In the default call, all multivectors are plotted in the same color and with high opacity. It is possible to change these choices, as the second panel demonstrates. The optional argument `mvfalpha` allows one to change the opacity, with values closer to 1 being more solid. In addition, the vector of strings `mvfcolor` can be used to specify different colors for the multivectors. As the multivectors are drawn, the function cycles through the provided colors.

While their main purpose is the visualization of complete multivector fields, the above two functions can, however, also be used for the depiction of Morse decompositions. For the logo example, this can be achieved as follows.

```
cm = connection_matrix(ec, mvf)
p3 = plot_simplicial_mvf(ec, cm.morse, mvfalpha=0.5,
                         tubefac=0.1)
p4 = plot_simplicial_mvf(ec, cm.morse,
                         mvfcolor=["red","purple","darkblue"],
                         mvfalpha=0.8)

pcombined34 = plot(p3, p4, layout=(1,2))
plot!(pcombined34, dpi=300)
savefig(pcombined34, "plots_mvf_morsesets.png")
```

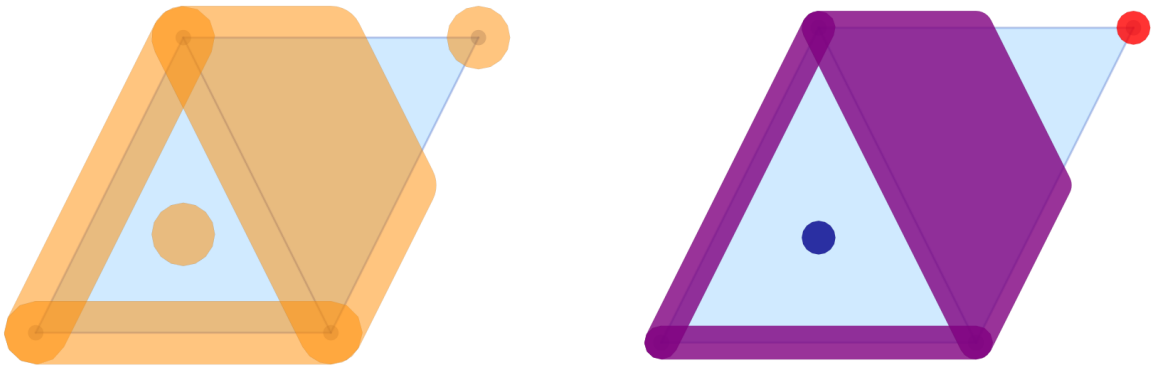


Figure 10.5: Multivector field Morse set visualizations

In this example, the Morse decomposition consists of two critical cells, as well as one large locally closed invariant set, which is shown in purple in the right panel of the figure of the multivector field Morse set visualizations. Note that the `tubefac` parameter controls the inflation radius as a fraction of the average edge length (default `0.05`). As mentioned before, the `mvfalpha` parameter sets the fill opacity (default `0.3`), and distinct colors can be assigned to individual Morse sets by passing a vector `mvfcolor` of color names.

Since the main application of the functions `plot_simplicial_mvf` and `plot_cubical_mvf` is the visualization of multivector fields, by default, cells absent from the multivector field are shown as implicit singleton regions. One can pass `addcritical=false` to suppress this and display only the explicitly listed multivectors. This is particularly useful for displaying Morse sets.

The treatment of multivector fields on cubical complexes is completely analogous, as the following example demonstrates.

```
using Plots
using ConleyDynamics

cubes = ["00.11", "01.01", "02.10", "11.10",
         "11.01", "22.00", "20.11", "31.01"]
cc  = create_cubical_complex(cubes, euclidean=true)
mvf = [["01.00","01.01"], ["02.00","02.10"], ["12.00","11.01"],
       ["11.00","01.10"], ["00.00","00.01"], ["00.10","00.11"],
       ["10.00","10.01"], ["21.00","11.10"], ["32.00","31.01"],
       ["31.00","21.10"], ["20.11","20.01","30.01","20.10"]]
cm  = connection_matrix(cc, mvf)

c1 = plot_cubical_mvf(cc, mvf, ci=true,
                      pdim=[false,true,true])
display(c1)

c2 = plot_cubical_mvf(cc, cm.morse, addcritical=false,
                      ci=true, mvfalpha=0.7,
                      pdim=[false,true,true])
display(c2)

pc12 = plot(c1, c2, layout=(1,2))
plot!(pc12, dpi=300)
savefig(pc12, "plots_mvf_cubical.png")
```

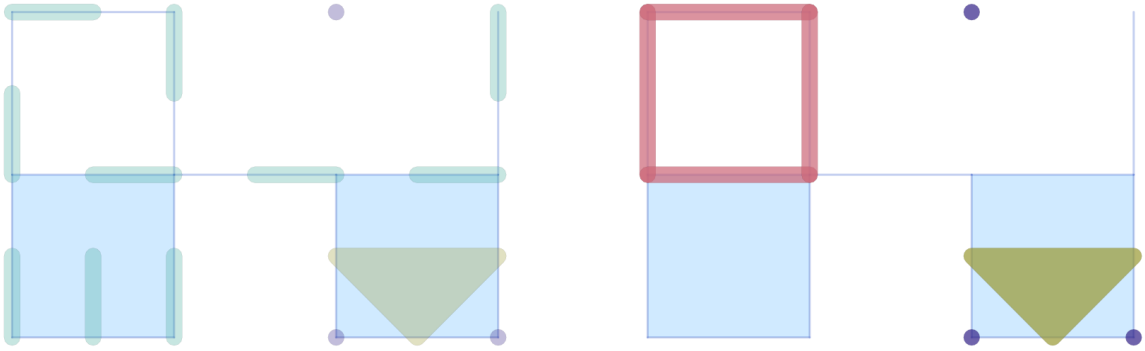


Figure 10.6: Cubical multivector field visualizations

We would like to emphasize that in this case it is essential to pass the optional argument `addcritical = false` in the Morse set visualization, since only the singletons listed in `cm.morse` are actually Morse sets. In fact, one can see that in this example there are five Morse sets, three of which are singletons.

As with the Morse plot functions, passing `ci=true` to `plot_simplicial_mvf` or `plot_cubical_mvf` colors each multivector (or Morse set) by its Conley index instead of using the `mvfcolor` palette.

Plotting a Single Multivector

The `plot_simplicial_mv` and `plot_cubical_mv` functions highlight a single multivector on the complex background:

```
using Plots
using ConleyDynamics

lc, mvf = example_forman2d(euclidean=true)
cm = connection_matrix(lc, mvf)
msmax = argmax(length.(cm.morse))
mv = cm.morse[msmax]      # largest Morse set
p1 = plot_simplicial_mv(lc, mv, mvfalpha=0.5)
title!(p1, "periodic orbit")
display(p1)
```

Cell labels (strings) are also accepted in place of integer indices:

```
lcset = ["ADE", "AE", "DE", "E", "EF", "FJ", "F"]
p2 = plot_simplicial_mv(lc, lcset,
                        mvfcolor="pink", mvfalpha=0.7,
                        pdim=[false,true,true])
title!(p2, "locally closed set")
display(p2)
pcombined12 = plot(p1, p2, layout=(1,2))
plot!(pcombined12, dpi=300)
savefig(pcombined12, "plots_single_mv.png")
```

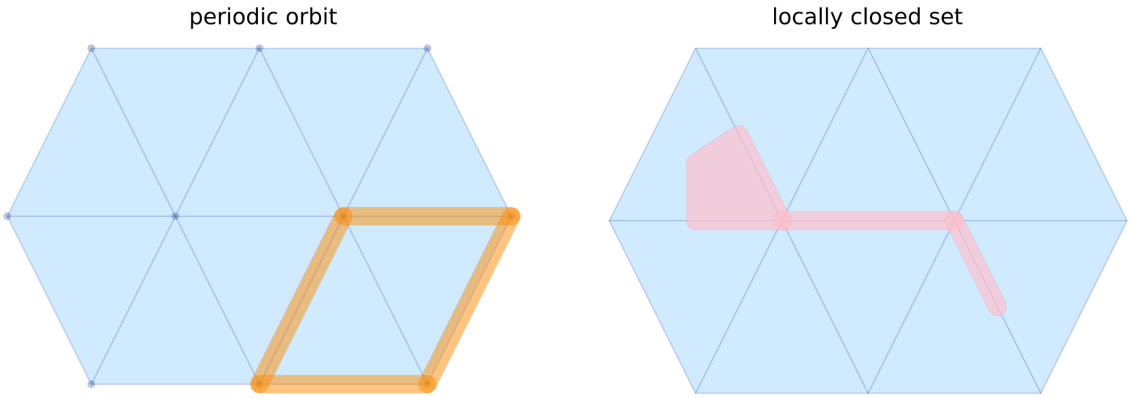


Figure 10.7: Plotting a single multivector or locally closed set

10.3 Three-Dimensional Plotting with Makie

Complexes embedded in three dimensions, such as the tetrahedral meshes described in the chapter Three-Dimensional Meshes, cubical complexes created by `create_cubical_box`, or triangulated surfaces, are visualized with the Makie.jl plotting ecosystem. Makie supports interactive three-dimensional graphics with depth ordering and transparency, which the planar backends above cannot provide. The table below lists the available functions:

Function	Description
`plot_simplicial3d`	Simplicial complex in three dimensions
`plot_cubical3d`	Cubical complex in three dimensions
`plot_simplicial3d_morse`	Simplicial complex with colored Morse sets
`plot_cubical3d_morse`	Cubical complex with colored Morse sets
`plot_simplicial3d_mv`	Single multivector on a simplicial complex
`plot_cubical3d_mv`	Single multivector on a cubical complex
`plot_simplicial3d_mvf`	Multivector field on a simplicial complex
`plot_cubical3d_mvf`	Multivector field on a cubical complex
`plot_cells3d!`	Add an arbitrary cell subset to a plot

Every function except `plot_cells3d!` comes in two variants. The plain variant, such as `plot_simplicial3d`, creates a new figure and returns it. The variant with a trailing exclamation mark, such as `plot_simplicial3d!`, draws into an existing Makie axis, which allows several plots to be combined in one figure or one axis. The simplicial and cubical functions accept the same arguments. They differ only in the way two-dimensional cells are drawn: Triangles are drawn as they are, while squares are split into two triangles.

Makie Backends

Makie itself only describes plots. To display or save them, one of its backends has to be loaded:

- GLMakie opens interactive windows, in which plots can be rotated with the mouse (left button), moved (right button), and zoomed (scroll wheel). It uses OpenGL, and it handles overlapping transparent surfaces much better than CairoMakie, since it blends them independently of the drawing order. This is the recommended backend for exploring three-dimensional complexes.

- CairoMakie creates static, publication-quality images in the PNG, PDF, and SVG formats. It does not need a graphics card, so it also works on remote servers. All figures in this section were created with CairoMakie.
- WGLMakie renders interactive plots in a web browser or a Jupyter notebook.

All three backends load `Makie.jl`, which is a weak dependency of ConleyDynamics.jl, and the plotting functions become available as soon as one of them has been loaded:

```
using GLMakie          # or CairoMakie, or WGLMakie
using ConleyDynamics
```

In the REPL, a figure `fig` returned by one of the plotting functions is displayed automatically. In a script, it can be displayed with `display(fig)`. With GLMakie, `display(GLMakie.Screen(), fig)` opens an additional window, so that several figures can be compared side by side. Figures are saved with `save`, for example

```
save("complex.png", fig; px_per_unit=2)
```

where `px_per_unit` scales the resolution of the image. With GLMakie, the saved image shows the current view, after any interactive rotation. Both GLMakie and CairoMakie can be loaded in the same session, and `CairoMakie.activate!()` and `GLMakie.activate!()` switch between them.

Since only the boundary surface of the tetrahedra is drawn by default, even large tetrahedral meshes can be plotted quickly. For example, a TetGen mesh of a box with about 950,000 cells has 13,500 boundary triangles, and plotting it and saving the result as a PNG file with CairoMakie takes about half a second. Plots which draw many more triangles, such as those with `hull_only=false` or multivector field plots of large complexes, are much slower, in particular with CairoMakie, and the resulting PDF and SVG files can become very large. PNG files are therefore recommended for large complexes.

Plotting a Complex

The function `plot_simplicial3d` draws a complex embedded in three dimensions. For a tetrahedral mesh, it shows the boundary surface of the union of all tetrahedra, semi-transparent, together with its edges. The following commands create a mesh of a box with `tetgen_mesh` and plot it:

```
using TetGen
using GLMakie
using ConleyDynamics

ec  = tetgen_to_simplicial(tetgen_mesh(tetgen_domain_box([-1.5,-1,-1], [1.5,1,1]);
                                       maxvolume=0.05))
fig = plot_simplicial3d(ec)
```

The result is shown in the left panel of the figure with three plots of a tetrahedral mesh. The appearance can be changed with a number of optional arguments:

- `pdim::Vector{Bool}`: which cell dimensions are drawn. The four entries correspond to vertices, edges, two-dimensional cells which are not faces of tetrahedra, and the faces of tetrahedra. The default `[false,true,true,true]` omits the vertices, which are drawn as small spheres if the first entry is `true`. Note that, unlike for the planar plotting functions, `pdim` has four entries, and a vector of a different length raises an `ArgumentError`.

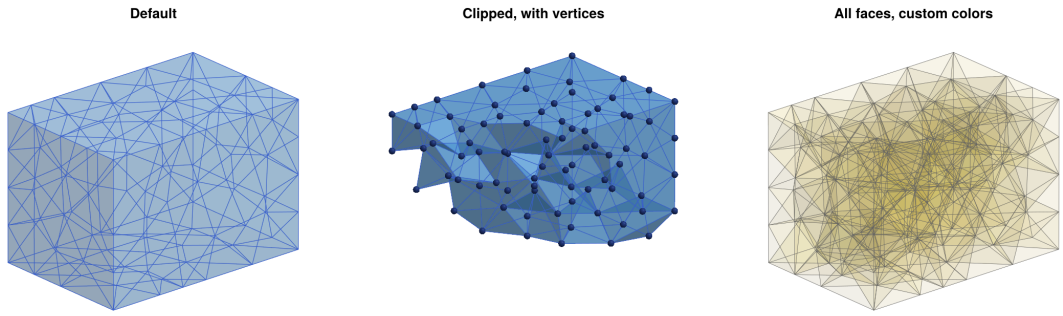


Figure 10.8: Three plots of a tetrahedral mesh

- `hull_only::Bool`: if `true` (default), only the boundary surface of the tetrahedra is drawn. If `false`, all faces and edges are drawn, including the ones in the interior of the mesh.
- `clip`: a pair `(n, offset)` which removes part of the complex, see the next section.
- `alpha`: the opacity of the faces, between `0` (invisible) and `1` (opaque). The default is `0.3`.
- `facecolor`, `edgecolor`, `vertexcolor`: the colors of the cells, which can be given in any form that Makie accepts, such as `:gold`, `"gold"`, or `colorant"gold"`.
- `linewidth` and `linealpha`: the width and the opacity of the edges.
- `markersize`: the radius of the vertex spheres, in the units of the coordinates. By default, it is a fraction of the typical edge length of the complex.

The plain variants additionally accept the following arguments, which determine the figure:

- `axis`: the type of the axis. The default `:scene` creates an LScene, a plain three-dimensional scene without axis decorations, which can be freely rotated. The value `:axis3` creates an `Axis3`, which has ticks, labels, and a fixed vertical direction, and which uses equal scales on all three axes.
- `size`: the size of the figure in pixels, by default `(800,600)`.

The middle and right panels of the same figure illustrate some of these options:

```
fig2 = plot_simplicial3d(ec; clip=([-1,-1,-1], 0.0), pdim=[true,true,true,true],
                         alpha=0.6)
fig3 = plot_simplicial3d(ec; hull_only=false, alpha=0.05,
                         facecolor=:gold, edgecolor=:gray40, linewidth=0.5)
```

With `hull_only=false`, all interior faces and edges are drawn, which gives an impression of the density of the mesh, but which is only readable for small meshes and small values of `alpha`.

If a complex is not embedded in three dimensions, the plotting functions raise an `ArgumentError`. Planar complexes should be plotted with the functions in the previous sections. Note also that all two-dimensional complexes in three dimensions, such as triangulated surfaces, can be plotted in the same way. Their triangles are drawn individually, see the torus in Highlighting Cell Subsets below.

Clipping

Most interesting features of a tetrahedral mesh, such as Morse sets or refined regions, are hidden in its interior. The optional argument `clip=(n, offset)` removes all tetrahedra whose centroid `c`, i.e., the average of the coordinates of their four vertices, lies on the far side of the plane `n·c = offset`, which means that `n·c > offset`. Here `n` is a normal vector of the plane, which points towards the removed part, and `n·c` denotes the dot product. More generally, the same rule applies to all cells which are not faces of other cells, such as the triangles of a triangulated surface. All other cells are kept if and only if they are faces of a kept cell, so that the visible part is again a closed subcomplex, and its boundary surface is drawn as usual. For example,

- `clip=([0,-1,0], 0.0)` keeps the half `y >= 0` of the complex,
- `clip=([0,0,1], 0.5)` keeps the part with `z <= 0.5`, and
- `clip=([-1,-1,-1], 0.0)` cuts the complex diagonally, as in the middle panel of the figure with three plots of a tetrahedral mesh.

Since whole tetrahedra are removed, the cut surface is jagged rather than planar, and it shows the actual tetrahedra at the cut. The cut surface faces the viewer if the normal `n` points towards the viewer. The default view of Makie looks at the complex slightly from above and from the side of negative `x` and `y`. Therefore, the normals `[0,-1,0]`, `[-1,0,0]`, `[0,0,1]`, and `[-1,-1,-1]` show the cut surface, while `[0,1,0]` turns it away from the viewer. The `clip` argument is accepted by all plotting functions, and it applies to the background complex as well as to the highlighted cell subsets. To show the same section in several overlaid plots, pass the same `clip` value to each of them.

Combining Plots

The variants with an exclamation mark draw into an existing axis, which is passed as the first argument. This makes it possible to arrange several plots in one figure, as in all the figures of this section, or to overlay several plots in one axis. The figure with three plots of a tetrahedral mesh was created with the following commands:

```
using CairoMakie

fig = Figure(size=(1500,500))

ax1 = Axis3(fig[1,1]; aspect=:data, title="Default")
plot_simplicial3d!(ax1, ec)

ax2 = Axis3(fig[1,2]; aspect=:data, title="Clipped, with vertices")
plot_simplicial3d!(ax2, ec; clip=([-1,-1,-1], 0.0), pdim=[true,true,true,true],
                   alpha=0.6)

ax3 = Axis3(fig[1,3]; aspect=:data, title="All faces, custom colors")
plot_simplicial3d!(ax3, ec; hull_only=false, alpha=0.05,
                   facecolor=:gold, edgecolor=:gray40, linewidth=0.5)

for ax in (ax1, ax2, ax3)
    hidedecorations!(ax)
    hidespines!(ax)
end
save("makie3d_basic.png", fig; px_per_unit=1.5)
```

In Makie, a `Figure` is the window or image, which contains a grid of axes. The expression `fig[1,2]` refers to the cell in the first row and the second column of this grid, and `Axis3(fig[1,2])` places a three-dimensional

axis there. The argument `title` sets the title of an axis. The argument `aspect=:data` gives all three axes the same scale, which is essential for judging the shape of tetrahedra, and `hidedecorations!` and `hidespines!` remove ticks, labels, and the bounding box. Instead of an `Axis3`, one can also use `LScene(fig[1,1]; show_axis=false)`, which is what the plain variants do by default. An `LScene` can be rotated freely in GLMakie, while an `Axis3` keeps the z-axis vertical. The viewing direction of an `Axis3` is determined by its attributes `azimuth` and `elevation`, in radians, which can be passed to the constructor or changed later, for example with `ax1.azimuth = 0.3*pi`.

The plain variants return the figure, and the axis they created can be retrieved with `content(fig[1,1])`. This allows the plain variants to be combined with the other functions as well:

```
fig = plot_simplicial3d(ec; axis=:axis3)
ax  = content(fig[1,1])
plot_cells3d!(ax, ec, [1, 2, 3]; color=:red)
```

Highlighting Cell Subsets

The function `plot_cells3d!` draws an arbitrary collection of cells, given by their indices or labels, into an existing axis. It is the basic building block of all the plotting functions of this section, and it is the tool of choice for constraint surfaces, exit sets, or the result of any computation that returns cells. Cells are drawn according to their dimension:

- Three-dimensional cells are drawn by the boundary surface of their union (or by all their faces if `hull_only=false`).
- Two-dimensional cells are drawn as triangles or squares, unless they are faces of three-dimensional cells in the collection.
- Edges are drawn as thick lines, unless they are faces of higher-dimensional cells in the collection.
- Vertices are drawn as spheres, unless they are faces of higher-dimensional cells in the collection.

The optional arguments are `color` (default `:darkorange`), `alpha` (default `1.0`), `pdim` (by default all dimensions are drawn), `hull_only`, `clip`, `linewidth` (default 4), and `markersize`. As an example, the following commands plot a triangulated torus surface and three cell subsets on it: the edges of a meridian circle, a band of triangles, and the first three cells of the complex, which are vertices selected by their labels:

```
Vt, Tt = trisurface_torus([0,0,0], 2.0, 0.7, 36, 12)
ecs = tetmesh_to_euclidean(Vt, Tt)

meridian = [k for k in 1:ecs.ncells if ecs.dimensions[k] == 1 &&
            all(abs(c[2]) < 1e-12 && c[1] > 0 for c in ecs.coords[k])]
band = [k for k in 1:ecs.ncells if ecs.dimensions[k] == 2 &&
        all(c[1] < -0.5 && abs(c[2]) < 1.2 for c in ecs.coords[k])]

fig = Figure(size=(1200,500))
ax1 = Axis3(fig[1,1]; aspect=:data, title="Torus surface")
plot_simplicial3d!(ax1, ecs; alpha=0.5)
ax2 = Axis3(fig[1,2]; aspect=:data, title="Added cell subsets")
plot_simplicial3d!(ax2, ecs; alpha=0.15)
plot_cells3d!(ax2, ecs, meridian; color=:red)
plot_cells3d!(ax2, ecs, band; color=:darkorange, alpha=0.9)
plot_cells3d!(ax2, ecs, ecs.labels[1:3]; color=:black)
```

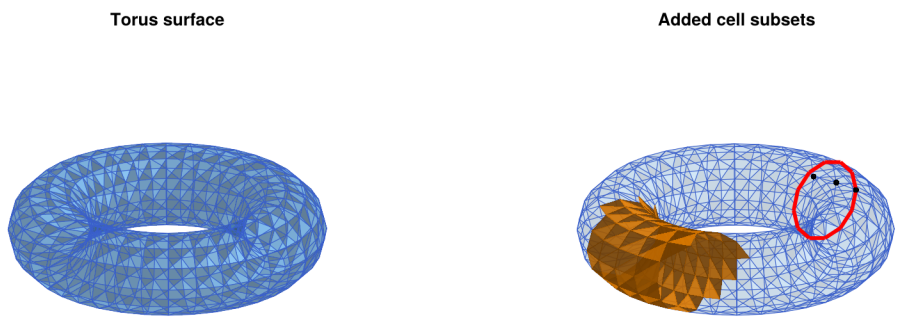


Figure 10.9: A torus surface with highlighted cell subsets

The cell subsets are selected geometrically, using the vertex coordinates which every `EuclideanComplex` stores in its field `coords`. For each cell, `ecs.coords[k]` is the vector of the coordinates of its vertices. The two panels are shown in the figure of the torus surface with highlighted cell subsets.

Further examples of `plot_cells3d!` can be found in the chapter Three-Dimensional Meshes, where it is used to show the constraint surfaces and curves found by `tetgen_marker_cells` and `gmsh_marker_cells`.

Plotting Morse Sets

The functions `plot_simplicial3d_morse` and `plot_cubical3d_morse` draw the complex faintly in the background, and every Morse set in its own color on top of it. Each Morse set is drawn as described for `plot_cells3d!` above. The following example uses the vector field from the documentation of `create_spatial_mvf`:

```
function samplevf(x::Vector{Float64})
    x1, x2, x3 = x
    return [x1 * (1.0 - x1*x1), -x2, -x3]
end

mvf = create_spatial_mvf(ec, samplevf)
ms  = morse_sets(ec, mvf)
[conley_index(ec, m) for m in ms]     # [[1,0,0,0], [1,0,0,0], [0,1,0,0]]

fig1 = plot_simplicial3d_morse(ec, ms)
fig2 = plot_simplicial3d_morse(ec, ms; ci=true)
fig3 = plot_simplicial3d_morse(ec, ms; colors=[:purple, :gold, :seagreen],
                               alpha=0.5, bgalpha=0.02,
                               pdim=[false,false,false,true])
```

The three plots are shown in the figure of the Morse sets of a vector field on a tetrahedral mesh. On this coarse mesh, each Morse set contains a single tetrahedron. For the two attractors, the Morse set consists of this tetrahedron and all its faces, while the Morse set of the saddle contains only six of its fourteen faces.

The optional arguments are:

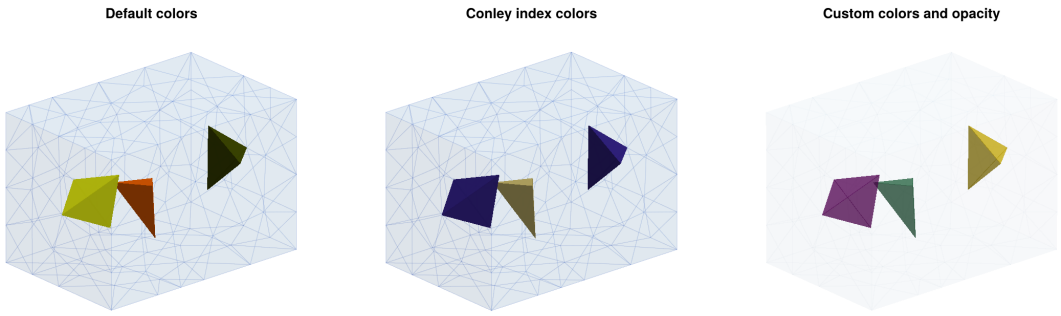


Figure 10.10: Morse sets of a vector field on a tetrahedral mesh

- `ci::Bool`: if `true`, the Morse sets are colored by their Conley index, with the same colors as in the planar plots, see the color palette for Conley indices above. In addition, the indices `[0,0,0,1]` and `[0,0,1,1]` of dimension three have their own colors (purple and light blue), and all remaining indices are shown in olive.
- `colors`: a vector of colors for the Morse sets, which is cycled through if there are more Morse sets than colors. By default, the same distinguishable colors as in the planar plots are used.
- `addcritical::Bool`: if `true`, all cells which do not belong to any of the given sets are added as singleton sets. The default is `false`.
- `alpha`: the opacity of the Morse sets, by default `0.9`.
- `bgalpha`: the opacity of the background faces, by default `0.08`.
- `pdim`: the cell dimensions which are drawn in the background. In the third panel, `pdim=[false,false,false,true]` suppresses the edges of the background.
- `hull_only`, `clip`, `linewidth`, and `markersize`, as before. Here, `linewidth` (default 4) refers to the edges of the Morse sets.

The Morse sets can be given as cell indices or as labels, i.e., as a `Vector{Vector{Int}}` or a `Vector{Vector{String}}`, and in particular the field `morse` of the result of `connection_matrix` can be passed directly. The functions are not restricted to Morse sets: Any collection of cell subsets can be plotted, such as isolated invariant sets or isolating blocks.

Plotting Multivectors and Multivector Fields

The functions `plot_simplicial3d_mv` and `plot_cubical3d_mv` highlight a single multivector, or any other single cell subset, on the faint background complex. The color is set with the optional argument `color`, and all other optional arguments are as for the Morse set plots. The functions `plot_simplicial3d_mvf` and `plot_cubical3d_mvf` draw a complete multivector field. Every multivector with at least two cells is drawn in its own color, and singletons are omitted. The colors are chosen by a greedy coloring, so that adjacent multivectors receive different colors whenever possible. Since the multivectors cover the whole complex, these plots are mostly useful together with clipping:

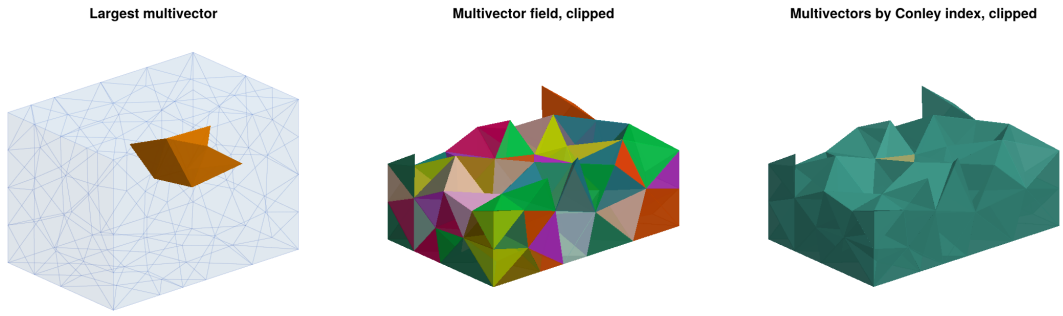


Figure 10.11: A multivector and a multivector field on a tetrahedral mesh

```
mvlargest = mvf[argmax(length.(mvf))]
length(mvf), length(mvlargest)        # (325, 38)

fig1 = plot_simplicial3d_mv(ec, mvlargest)
fig2 = plot_simplicial3d_mvf(ec, mvf; clip=([0,0,1], 0.0))
fig3 = plot_simplicial3d_mvf(ec, mvf; ci=true, clip=([0,0,1], 0.0))
```

The three plots are shown in the figure of the multivector and the multivector field on a tetrahedral mesh.

The optional arguments of the multivector field plots are the same as for the Morse set plots, with the following differences. The argument `colors` specifies the palette for the greedy coloring, which by default consists of twelve light colors. The default opacity is `alpha=0.8`. With `ci=true`, every multivector is colored by its Conley index. Since most multivectors of a typical multivector field are regular, i.e., have trivial Conley index, such a plot is dominated by one color, and the few critical multivectors stand out, as the small sand-colored multivector in the right panel of the figure of the multivector and the multivector field on a tetrahedral mesh shows. With `addcritical=true`, the singleton multivectors and all cells that are not covered by the multivector field are drawn as well.

Plotting Cubical Complexes

The cubical functions work in exactly the same way, and they accept the same optional arguments. Cubical complexes are usually created with `create_cubical_box`, whose integer vertex coordinates are then mapped to the region of interest with `convert_spatial_coordinates`. The following commands plot a small cubical complex with its vertices, and the multivector field of the sample vector field from above, clipped at the plane `z = 0`:

```
lc, coords = create_cubical_box(6, 4, 4)
ecq = lefschetz_to_euclidean(lc, convert_spatial_coordinates(coords,
                                    [-1.5,-1,-1], [1.5,1,1]))
mvfq = create_spatial_mvf(ecq, samplevf)

fig1 = plot_cubical3d(ecq; pdim=[true,true,true,true])
fig2 = plot_cubical3d_mvf(ecq, mvfq; clip=([0,0,1], 0.0))
```

The squares of a cubical complex are drawn as two triangles each. The diagonal along which a square is split is not drawn, since it is not an edge of the complex, but it can be visible in the shading of semi-transparent

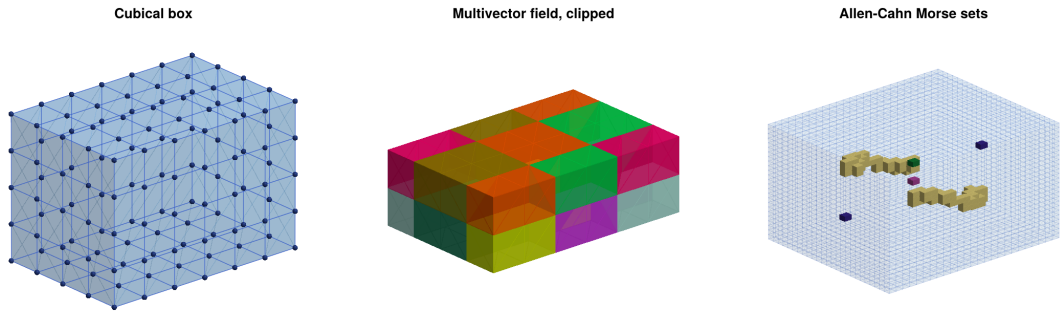


Figure 10.12: Cubical complexes in three dimensions

squares. For a larger example, we return to the three-dimensional projection of the Allen-Cahn equation discussed in the section Analysis of a Spatial System:

```
function allencahn3d(x::Vector{Float64})
    lambda = 3.0 * pi
    c      = lambda / pi
    x1, x2, x3 = x
    y1 = (lambda-1)*x1 - 1.5*c * (x1*x1*x1-x1*x1*x3+x2*x2*x3+2*x1*(x2*x2+x3*x3))
    y2 = (lambda-4)*x2 - 1.5*c * x2 * (2*x1*x1+x2*x2+2*x1*x3+2*x3*x3)
    y3 = (lambda-9)*x3 + 0.5*c * (x1*(x1*x1-3*x2*x2)-3*x3*(2*x1*x1+2*x2*x2+x3*x3))
    return [y1, y2, y3]
end

bmax = [1.8, 1.5, 1.0]
lca, ca = create_cubical_box(25, 25, 25)
eca  = lefschetz_to_euclidean(lca, convert_spatial_coordinates(ca, -bmax, bmax))
mvfa = create_spatial_mvf(eca, allencahn3d)
msa  = morse_sets(eca, mvfa)
[conley_index(eca, m) for m in msa]
# [[1,0,0,0], [1,0,0,0], [0,1,0,0], [0,1,0,0], [0,0,1,0], [0,0,1,0], [0,0,0,1]]

fig3 = plot_cubical3d_morse(eca, msa; ci=true)
```

The Morse decomposition consists of seven Morse sets: two attracting equilibria, two equilibria with one-dimensional unstable manifolds, two equilibria with two-dimensional unstable manifolds, and the trivial equilibrium, whose unstable manifold is three-dimensional. The three plots are shown in the figure of the cubical complexes in three dimensions. In the right panel, the colors of the Conley indices immediately identify the index of each equilibrium: the attractors are dark blue, the Morse sets with index `[0,1,0,0]` are sand-colored, the ones with index `[0,0,1,0]` are green, and the trivial equilibrium is purple. Note that one of the two green Morse sets is hidden behind a sand-colored one in this view. In such cases, an interactive GLMakie window, or the `clip` argument, helps to see all Morse sets.

Practical Tips

The following remarks summarize the experience gained with the figures in this manual:

- **Transparency.** CairoMakie sorts the triangles of a single plot object by depth, but it draws different plot objects in the order in which they were created. All colored triangles of a Morse set or multivector field plot are therefore combined into one plot object, which is rendered correctly. When several plots are overlaid with `plot_cells3d!`, draw the inner objects first, and use opaque colors for objects which should hide others. GLMakie uses order-independent transparency, which handles such overlays much better, although it is also only an approximation.
- **Lines and surfaces.** Edges and curves remain clearly visible through semi-transparent surfaces, and with CairoMakie, lines which are drawn after a surface even appear in front of opaque parts of it. This is usually desirable for constraint curves, but it can be confusing for the edges of large complexes. In such cases, reduce `linealpha` or omit the edges with `pdim`.
- **Large complexes.** For tetrahedral meshes with hundreds of thousands of tetrahedra, the default plot is still fast, since only the boundary surface is drawn. Avoid `hull_only=false` and vertex spheres for such meshes, and use clipping to look inside. Multivector field plots are only readable for small complexes.
- **Reproducible views.** The default view of an `LScene` depends on the bounding box of the plot. For figures which are to be published, an `Axis3` with explicitly set `azimuth` and `elevation` gives reproducible views.

Chapter 11

Sparse Matrices

While Julia provides a data structure for sparse matrix computations, the employed design decisions make it difficult to use this implementation for computations over finite fields. This is mainly due to the fact that in the Julia implementation, it is assumed that one can determine the zero and one elements from the data type alone. However, a finite field data type generally also depends on additional parameters, such as the characteristic of the field.

Since the algorithms underlying ConleyDynamics.jl only require basic row and column operations, a specialized sparse matrix implementation is provided in the package. It is briefly described in the following.

11.1 Sparse Matrix Format

Sparse matrices in this package have to be of the composite data type `SparseMatrix`, which is structured as follows:

`ConleyDynamics.SparseMatrix` – Type.

```
SparseMatrix{T}
```

Composite data type for a sparse matrix with entries of type T.

The struct has the following fields:

- `const nrow::Int`: Number of rows
- `const ncol::Int`: Number of columns
- `const char::Int`: Characteristic of type T
- `const zero::T`: Number 0 of type T
- `const one::T`: Number 1 of type T
- `entries::Vector{Vector{T}}`: Matrix entries corresponding to `columns`
- `columns::Vector{Vector{Int}}`: `columns[k]` points to nonzero entries in column k
- `rows::Vector{Vector{Int}}`: `rows[k]` points to nonzero entries in the k-th row

source

In this struct, the type T has to be either `Int` or `Rational{Int}`, depending on whether the sparse matrix is interpreted as a matrix with entries in the finite field $GF(p)$ for some prime p, or over the field of rationals, respectively. The data type has the following fields:

- `nrow::Int` designates the number of rows.
- `ncol::Int` gives the number of columns.
- `char::Int` specifies the characteristic of the underlying field F. If `char=0`, then the field is the rationals $\mathbb{Q}$, and one has to have `T = Rational{Int}`. If, on the other hand, the finite field $F = GF(p)$ is used, then `char=p` has to be a prime number. In this case, the data type of the matrix entries has to be `T = Int`.
- `zero::T` provides 0 in the data type T.
- `one::T` provides 1 in the data type T.
- `columns::Vector{Vector{Int}}` is a vector of integer vectors, which contains the row indices of nonzero matrix entries in each column. More precisely, `columns[k]` contains an increasing list of row indices, which give the locations of all nonzero entries in column k. Note that the list for each colum has to be strictly increasing.
- `rows::Vector{Vector{Int}}` is a vector of integer vectors, which contains the column indices of nonzero matrix entries in each row. It is the precise dual to the previous field. This time, `rows[k]` contains an increasing list of column indices, which correspond to the nonzero entries of the matrix in the k-th row.
- `entries::Vector{Vector{T}}` is a vector of vectors which contains the actual matrix entries. It is organized in exactly the same way as the field `columns`. In other words, for every `k = 1,..,ncol` the matrix entry in column `k` and row `columns[k][j]` is given by `entries[k][j]`, where `j` indexes the nonzero column entries from top to bottom.

This data structure is clearly redundant, in the sense that the field `rows` is not needed to uniquely determine the matrix. However, the type `SparseMatrix` is fundamental for almost every aspect of ConleyDynamics.jl, as it is used to encode the incidence coefficient map κ, and therefore also the matrix representation of the boundary operator ∂. And for many operations on or queries of Lefschetz complexes, one needs fast access to both the cells in the boundary and the coboundary of a given cells. While the boundary can easily be accessed via the field `columns`, the fast coboundary access is aided by the field `rows`.

We would like to point out that in view of the different underlying fields, sparse matrices should only be manipulated using the specific commands provided by the package. These are described in detail below. If there is a need for additional functionality beyond these first methods, it can be added at a later point in time.

11.2 Creating Sparse Matrices

The package provides a number of methods for creating sparse matrices with the data type `SparseMatrix`. These are geared towards their usage within ConleyDynamics.jl and are therefore by no means exhaustive:

- `sparse_from_full` is usually invoked in the form `A = sparse_from_full(AF, p=PP)`. The first input argument `AF` has to be a regular Julia integer matrix. This matrix is then converted to sparse format and returned as `A`. If the optional parameter `p` is omitted, the resulting sparse matrix is over the rational numbers $\mathbb{Q}$, otherwise it is over the finite field with characteristic `PP`.
- `full_from_sparse` converts a given sparse matrix into a standard full matrix in Julia. The data type of the entries is either `Rational{Int}` or `Int`, depending on whether the sparse input matrix is considered over the rationals $\mathbb{Q}$ or over a finite field, respectively. When invoking this command, be mindful of the size of the sparse matrix!

- sparse_from_lists creates a sparse matrix solely based on its nonzero entries and their locations. It expects the following required input arguments, in the order they are listed:

  - nr::Int: Number of rows
  - nc::Int: Number of columns
  - tchar: Field characteristic, which has to be 0 if $F = \mathbb{Q}$ and a positive prime otherwise
  - tzero::T: Number 0 of type T
  - tone::T: Number 1 of type T
  - r::Vector{Int}: Vector of row indices
  - c::Vector{Int}: Vector of column indices
  - v::Vector{T}: Vector of matrix entries

  The function assumes that the vectors r, c, and v have the same length and that the matrix has entry v[k] at the location (r[k],c[k]). Zero entries will be ignored, and multiple entries for the same matrix position raise an error. Furthermore, if tchar>0, then the entries in v are all replaced by their values modulo tchar. As mentioned before, if tchar=0 then the entry type has to be T = Rational{Int}, otherwise we have T = Int.

- lists_from_sparse takes a sparse matrix and disassembles it into the separate ingredients specified in the discussion of the previous function. In this sense, it is precisely the inverse method of sparse_from_lists.

- sparse_identity creates a sparse identity matrix. It is invoked as A = sparse_identity(n, p=PP), and returns a sparse identity matrix A with n rows and n columns. If the optional characteristic parameter specified and positive, then the matrix is considered over the finite field with characteristic PP, otherwise it is over the rationals $\mathbb{Q}$.

- sparse_zero creates a sparse zero matrix. It is invoked as A = sparse_zero(nr, nc, p=PP), and returns a sparse zero matrix A with nr rows and nc columns. If the optional characteristic parameter specified and positive, then the matrix is considered over the finite field with characteristic PP, otherwise it is over the rationals $\mathbb{Q}$.

- sparse_diagonal creates a sparse diagonal matrix. The function is called as sparse_diagonal(nr, nc, d, p=pp), and it creates a sparse matrix with nr rows and nc columns, over a field with characteristic pp. In this form of the function call, the matrix has the entries in the vector d along the main diagonal. If the optional integer argument offset is given and positive, then the diagonal is placed offset positions above the main diagonal, if it is negative, then it indicates how many positions below the diagonal it is placed. The function places as many entries from d as possible, i.e., if the length of d is too short, the remaining diagonal entries are zero, if it is too long, then the later entries are ignored. The function tries to convert the entries in d to the correct format based on p.

Of these methods, the function sparse_from_lists provides the easiest and quickest way to create a sparse matrix. In addition, it is also possible to combine given sparse matrices using the following functions:

- sparse_hcat concatenates sparse matrices horizontally, as long as they have the same number of rows, and are all defined over the same field. This function can be used with any number of arguments, as long as they are all sparse matrices. If the sparse matrices are collected in a vector vec, use the call sparse_hcat(vec...) to splat the vector entries. Finally, one can also use the usual [A B C...] to concatenate sparse matrices horizontally, just like in the case of Julia arrays.

- sparse_vcat concatenates sparse matrices vertically, as long as they have the same number of columns, and are all defined over the same field. Also this function can be used with any number of arguments, see the previous entry. One can also use the abbreviation `[A; B; C; ...]` to concatenate sparse matrices vertically.

- sparse_hvcat can be used to arrange sparse matrices of varying dimensions in block form, as long as the dimensions are compatible and all matrices are defined over the same field. The specific function calls for `sparse_hvcat` can be found in the API. One can also use Julia's block form abbreviation `[A B; C D E; F]`.

- sparse_hvncat can also be used for the purpose of arranging sparse matrices in blocks, but this time every row and every column have to be made up of the same number of blocks. It is, however, possible to specify whether the rows or the columns are first filled. In addition, one can use the Julia abbreviation `[A; B;; C; D;; E; F]` etc.

- sparse_cat is useful for creating sparse matrices which only have nontrivial blocks along the diagonal of the block matrix. The call `sparse_cat(A1, A2, .., An, dims=(1,2))` arranges the sparse matrices `A1, A2, .., An` along the main diagonal, and fills the remainder entries of the combined matrix with zero. There are no restrictions on the dimensions of the individual matrices.

11.3 Sparse Matrix Access

Access to the entries of sparse matrices is provided via the following commands:

- sparse_get_entry extracts the matrix entry `val` of the matrix `A` located in row `ri` and column `ci`, if it is invoked using the command `val = sparse_get_entry(A,ri,ci)`.

- sparse_set_entry! sets the matrix entry of the matrix `A` located in row `ri` and column `ci` to the value 'val', if it is invoked using the command `sparse_set_entry!(A,ri,ci,val)`. Internally, this commands makes sure that the above-defined format of the fields of a sparse matrix is preserved. Note that the data type of `val` has to match the type of `A.zero`. If the matrix is a sparse matrix over the integers, it is considered as a sparse matrix over a finite field. Thus, the value `val` is automatically reduced modulo `A.char` before being assigned to the entry.

- sparse_get_column is invoked as `Acol = sparse_get_column(A,ci)`, and it returns the full `ci`-th column of the matrix `A` as a `Vector{T}` of length `A.nrow`.

- sparse_get_nz_column returns the row indices for the nonzero entries in the `ci`-th column of the sparse matrix `A`, if invoked as `rivec = sparse_get_nz_column(A,ci)`.

- sparse_get_nz_row returns the column indices for the nonzero entries in the `ri`-th row of the sparse matrix `A`, if invoked as `civec = sparse_get_nz_row(A,ri)`.

- sparse_minor creates a minor from a given sparse matrix `A`. For this, one needs to specify the row and column indices of the minor in the integer vectors `rvec` and `cvec`, respectively, and then invoke the function using the command `AM = sparse_minor(A,rvec,cvec)`. Note that the entries in `rvec` and `cvec` do not have to be in increasing order, but they are not allowed to contain repeated indices.

One can also read and set sparse matrix values using the overloaded methods `y = A[i,j]` and `A[i,j] = val`. In the latter case, if the matrix is defined as `SparseMatrix{Int}`, then it is interpreted as being over a finite field. In this case, the value `val` is automatically reduced via modular arithmetic modulo `A.char` before the assignment.

11.4 Elementary Matrix Operations

The following commands perform the basic sparse matrix operations that are needed for the functionality of the package:

- sparse_add_column! is invoked using the form sparse_add_column!(A,ci1,ci2,cn,cd), and it replaces the ci1-th column column[ci1] of A by column[ci1] + (cn/cd) * column[ci2]. This operation automatically performs the computations over the field F underlying the sparse matrix A. In other words, if this field is finite, then it determines the inverse of the argument cd as part of the computation.
- sparse_add_row! is invoked using the form sparse_add_row!(A,ri1,ri2,cn,cd), and it replaces the ri1-th row row[ri1] of A by row[ri1] + (cn/cd) * row[ri2]. As before, this operation automatically performs the computations over the field F underlying the sparse matrix A.
- sparse_rref computes the reduced row Echelon form of a sparse matrix. There also exists an in-place version of this function which alters the input argument, see sparse_rref!. Both of these functions are most useful for reasonably small matrices.
- sparse_solve computes a solution of the sparse linear system Ax = b. It is expected that A and b are sparse matrices over the same field, and that both have the same number of rows. Usually, this function is probably used with a column vector b. However, if b is a matrix, then the respective matrix equation is solved. The function returns a particular solution of the system, if one exists. Otherwise it returns nothing. In order to obtain all solutions of the system, one has to add an arbitrary linear combination of the basis vectors determined via sparse_basis_kernel(A).
- sparse_basis_kernel determines a basis for the kernel of the given sparse matrix. The basis vectors are returned in a Vector{SparseMatrix}, where all vectors are sparse column vectors.
- sparse_basis_range determines a basis for the column space or range of the given sparse matrix. As above, the basis vectors are returned in a Vector{SparseMatrix}, where all vectors are sparse column vectors.
- sparse_permute creates a new sparse matrix by permuting the row and column indices. It is invoked using the command AP = sparse_permute(A,pr,pc), and the integer vectors pr and pc have to describe the row and column permutations, respectively.
- sparse_transpose creates the transpose of a sparse matrix. In addition to the call sparse_transpose(A) one can also simply use the more natural A'.
- sparse_remove! is invoked as sparse_remove!(A,ri,ci) and removes the sparse matrix entry in the ri-th row and ci-th colum, i.e., it effectively sets the entry equal to zero.
- sparse_add computes the matrix sum of two sparse matrices. Exceptions are raised if the matrix sum is not defined, or if the involved sparse matrices are defined over different fields. One can also use the operator form A+B to compute the sum of sparse matrices.
- sparse_subtract computes the matrix difference of two sparse matrices. Exceptions are raised if the matrix difference is not defined, or if the involved sparse matrices are defined over different fields. One can also use the operator form A-B to compute the sum of sparse matrices.
- sparse_multiply computes the matrix product of two sparse matrices. Exceptions are raised if the matrix product is not defined, or if the involved sparse matrices are defined over different fields. One can also use the operator form A*B to compute the product of sparse matrices.
- sparse_scale computes the scalar product of a number and a sparse matrix. An exception is raised if the scalar and the matrix entries are not of the same type. One can also use the operator form sfac*A to compute the scalar product.

- `sparse_inverse` computes the inverse matrix of a sparse matrix. Exceptions are raised if the matrix is not square or not invertible.

There are also the useful helper functions `scalar_inverse`, `scalar_multiply`, and `scalar_add` which compute the inverse of a scalar, the product of two scalars, or the sum of two scalars, respectively. Depending on the input type, these computations are performed either over the rationals, or in modular arithmetic. See the function documentations for more details.

As mentioned earlier, additional operations can easily be implemented if they become necessary.

11.5 Sparse Matrix Information

Finally, ConleyDynamics.jl provides the following functions for quickly extracting certain information from sparse matrices:

- `sparse_size` is invoked as `size = sparse_size(A,dim)`, and it returns the number of rows if `dim=1`, or the number of columns for `dim=2`.
- `sparse_low` returns the largest row index `ri` of a nonzero entry in the `ci`-th column of the matrix `A`, if used in the form `ri = sparse_low(A,ci)`. In other words, it returns the row index of the lowest nonzero matrix entry in the column.
- `sparse_is_zero` checks whether a sparse matrix is the zero matrix.
- `sparse_is_identity` checks whether a sparse matrix is the identity matrix.
- `sparse_is_equal` checks whether two sparse matrices are the same. For this, they not only have to have the same size and the same entries, they also need to be defined over the same field. This function can also be invoked using `A == B`.
- `sparse_is_rref` checks whether the argument matrix is in reduced row Echelon form, and returns the respective boolean value.
- `sparse_is_sut` checks whether a given sparse matrix is strictly upper triangular, and returns either `true` or `false`.
- `sparse_fullness` returns the fullness of a sparse matrix as a floating point number. Here fullness refers to the ratio of the number of nonzero matrix elements and the total number of matrix entries.
- `sparse_sparsity` computes the sparseness of a sparse matrix, which is defined as 1 minus its fullness, i.e., it is the ratio of the number of zero matrix elements and the total number of matrix entries.
- `sparse_nz_count` determines the number of nonzero matrix entries.
- `sparse_show` can be used to display a sparse matrix in traditional matrix form at the Julia REPL prompt. This function has additional methods which allow the user to display row and column labels, which have to be specified as second and third arguments. In addition, if `cm` is a connection matrix, then the command `sparse_show(cm)` shows the connection matrix including the Conley index labels. In order to increase the readability of the shown matrix, any zero entries will be represented as `.`. In other words, only nonzero entries will be printed as their numeric value.

Chapter 12

References

[Ale37] P. Alexandrov. Diskrete Räume. Mathematiceskii Sbornik (N.S.) **2**, 501–518 (1937).

[BKMW20] B. Batko, T. Kaczynski, M. Mrozek and T. Wanner. Linking combinatorial and classical dynamics: Conley index and Morse decompositions. Foundations of Computational Mathematics **20**, 967–1012 (2020).

[CWD13] G. S. Cochran, T. Wanner and P. Dłotko. A randomized subdivision algorithm for determining the topology of nodal sets. SIAM Journal on Scientific Computing **35**, B1034–B1054 (2013).

[Con78] C. Conley. Isolated Invariant Sets and the Morse Index (American Mathematical Society, Providence, R.I., 1978).

[DKMW11] P. Dłotko, T. Kaczynski, M. Mrozek and T. Wanner. Coreduction homology algorithm for regular CW-complexes. Discrete & Computational Geometry **46**, 361–388 (2011).

[DHL26] T. K. Dey, A. Haas and M. Lipiński. Computing a connection matrix and persistence efficiently from a Morse decomposition. SIAM Journal on Applied Dynamical Systems **25**, 108–130 (2026).

[DLMS24] T. K. Dey, M. Lipiński, M. Mrozek and R. Slechta. Computing connection matrices via persistence-like reductions. SIAM Journal on Applied Dynamical Systems **23**, 81–97 (2024).

[DW18] P. Dłotko and T. Wanner. Rigorous cubical approximation and persistent homology of continuous functions. Computers & Mathematics with Applications **75**, 1648–1666 (2018).

[EH10] H. Edelsbrunner and J. L. Harer. Computational Topology (American Mathematical Society, Providence, 2010).

[EM23] H. Edelsbrunner and M. Mrozek. The depth poset of a filtered Lefschetz complex (2023), arXiv:2311.14364v2 [math.AT].

[For98a] R. Forman. Combinatorial vector fields and dynamical systems. Mathematische Zeitschrift **228**, 629–681 (1998).

[For98b] R. Forman. Morse theory for cell complexes. Advances in Mathematics **134**, 90–145 (1998).

[Fra89] R. Franzosa. The connection matrix theory for Morse decompositions. Transactions of the American Mathematical Society **311**, 561–592 (1989).

[GMW05] M. Gameiro, K. Mischaikow and T. Wanner. Evolution of pattern complexity in the Cahn-Hilliard theory of phase separation. Acta Materialia **53**, 693–704 (2005).

[HMMN14] S. Harker, K. Mischaikow, M. Mrozek and V. Nanda. Discrete Morse theoretic algorithms for computing homology of complexes and maps. Foundations of Computational Mathematics **14**, 151–184 (2014).

[HMS21] S. Harker, K. Mischaikow and K. Spendlove. A computational framework for connection matrix theory. Journal of Applied and Computational Topology **5**, 459–529 (2021).

[KMM04] T. Kaczynski, K. Mischaikow and M. Mrozek. Computational Homology. Vol. 157 of Applied Mathematical Sciences (Springer-Verlag, New York, 2004).

[KMS98] T. Kaczynski, M. Mrozek and M. Slusarek. Homology computation by reduction of chain complexes. Computers & Mathematics with Applications **35**, 59–70 (1998).

[KMW16] T. Kaczynski, M. Mrozek and T. Wanner. Towards a formal tie between combinatorial and classical vector field dynamics. Journal of Computational Dynamics **3**, 17–50 (2016).

[Lef42] S. Lefschetz. Algebraic Topology. Vol. 27 of American Mathematical Society Colloquium Publications (American Mathematical Society, New York, 1942).

[LKMW23] M. Lipinski, J. Kubica, M. Mrozek and T. Wanner. Conley-Morse-Forman theory for generalized combinatorial multivector fields on finite topological spaces. Journal of Applied and Computational Topology **7**, 139–184 (2023).

[Mas91] W. S. Massey. A Basic Course in Algebraic Topology. Vol. 127 of Graduate Texts in Mathematics (Springer-Verlag, New York, 1991).

[MB09] M. Mrozek and B. Batko. Coreduction homology algorithm. Discrete & Computational Geometry **41**, 96–118 (2009).

[MSTW22] M. Mrozek, R. Srzednicki, J. Thorpe and T. Wanner. Combinatorial vs. classical dynamics: Recurrence. Communications in Nonlinear Science and Numerical Simulation **108**, Paper No. 106226, 30 pages (2022).

[MW21] M. Mrozek and T. Wanner. Creating semiflows on simplicial complexes from combinatorial vector fields. Journal of Differential Equations **304**, 375–434 (2021).

[MW25] M. Mrozek and T. Wanner. Connection Matrices in Combinatorial Topological Dynamics. SpringerBriefs in Mathematics (Springer-Verlag, Cham, 2025).

[Mun84] J. R. Munkres. Elements of Algebraic Topology (Addison-Wesley, Menlo Park, 1984).

[SW26] E. Sander and T. Wanner. Theory and Numerics of Partial Differential Equations (SIAM, Philadelphia, 2026). In preparation, 1023 pages.

[SW14a] T. Stephens and T. Wanner. Isolating block validation in Matlab, https://github.com/almost6heads/isoblockval (2014).

[SW14b] T. Stephens and T. Wanner. Rigorous validation of isolating blocks for flows and their Conley indices. SIAM Journal on Applied Dynamical Systems **13**, 1847–1878 (2014).

[Wan25] T. Wanner. ConleyDynamics.jl: A Julia package for multivector dynamics on Lefschetz complexes. Journal of Open Source Software **10**, 8085 (2025).

[GUD24] GUDHI Project. GUDHI User and Reference Manual. 3.10.1 Edition (GUDHI Editorial Board, 2024).

Part III

Core API

Chapter 13

Composite Data Structures

The package relies on a number of basic composite data structures that encompass more complicated objects. For the internal representation of sparse matrices we refer to Internal Sparse Matrix Representation.

`ConleyDynamics` – Module.

```
module ConleyDynamics
```

Collection of tools for computational Conley theory.

source

13.1 Lefschetz Complex Type

`ConleyDynamics.AbstractComplex` – Type.

```
AbstractComplex
```

Abstract base type for Lefschetz complexes.

Both `LefschetzComplex` and `EuclideanComplex` are subtypes of `AbstractComplex`. All topology, homology, and dynamics functions accept any `AbstractComplex`.

source

`ConleyDynamics.LefschetzComplex` – Type.

```
LefschetzComplex
LefschetzComplex(labels, dimensions, boundary; validate=true)
```

Collect the Lefschetz complex information in a struct.

The struct is created via the following fields:

- `labels::Vector{String}`: Vector of labels associated with cell indices
- `dimensions::Vector{Int}`: Vector cell dimensions

- `boundary::SparseMatrix`: Boundary matrix, columns give the cell boundaries

It is expected that the dimensions are given in increasing order, and that the square of the boundary matrix is zero. Otherwise, exceptions are raised. In addition, the following fields are created during initialization:

- `ncells::Int`: Number of cells
- `dim::Int`: Dimension of the complex
- `indices::Dict{String,Int}`: Dictionary for finding cell index from label

The coefficient field is specified by the boundary matrix.

The optional keyword argument `validate` controls whether the constructor verifies that the boundary matrix squares to zero. By default this check is enabled. Internal library functions that are guaranteed to produce a valid complex by construction (simplicial, cubical, restriction, permutation, basis-change, etc.) pass `validate=false` explicitly to suppress the check, since for large complexes the matrix multiplication is expensive. Users constructing a `LefschetzComplex` manually can also call `validate_lefschetz_complex` to check an existing complex at any point.

source

`ConleyDynamics.EuclideanComplex` – Type.

```
EuclideanComplex
EuclideanComplex(labels, dimensions, boundary, coords; validate=true)
```

A Lefschetz complex with embedded Euclidean coordinates for every cell.

The struct shares the fields of `LefschetzComplex`:

- `labels::Vector{String}`: Vector of labels associated with cell indices
- `dimensions::Vector{Int}`: Vector of cell dimensions
- `boundary::SparseMatrix`: Boundary matrix, columns give the cell boundaries
- `ncells::Int`: Number of cells
- `dim::Int`: Dimension of the complex (must satisfy `dim ≤ 3`)
- `indices::Dict{String,Int}`: Dictionary for finding cell index from label

In addition it carries:

- `coords::Vector{Vector{Vector{Float64}}}`: Per-cell vertex coordinates. `coords[k]` is the list of vertex coordinate vectors needed to draw cell k:
  - Vertex: `[[x,y]]` (length 1)
  - Edge: `[[x1,y1],[x2,y2]]` (length 2)
  - Triangle: `[[x1,y1],[x2,y2],[x3,y3]]` (length 3)
  - Cubical 2-cell: `[[x1,y1],[x2,y2],[x3,y3],[x4,y4]]` (length 4)

The function `lefschetz_to_euclidean` creates an `EuclideanComplex` from an existing `LefschetzComplex`, and the function `euclidean_to_lefschetz` converts it back.

source

`Base.show` – Method.

```
Base.show(io::IO, ::MIME"text/plain", lc::LefschetzComplex)
```

Display Lefschetz complex information when hitting return in REPL.

source

`Base.show` – Method.

```
Base.show(io::IO, ::MIME"text/plain", ec::EuclideanComplex)
```

Display EuclideanComplex information when hitting return in REPL.

source

13.2 Cell Subset Types

`ConleyDynamics.Cell` – Type.

```
Cell = Union{Int,String}
```

A cell of a Lefschetz complex.

This data type is used to represent a cell of a Lefschetz complex. The cell can be specified either via its index, or its label.

source

`ConleyDynamics.Cells` – Type.

```
Cells = Union{Vector{Int},Vector{String}}
```

A list of cells of a Lefschetz complex.

This data type is used to represent subsets of a Lefschetz complex. It is used for individual isolated invariant sets, locally closed subsets, and multivectors.

source

`ConleyDynamics.CellSubsets` – Type.

```
CellSubsets = Union{Vector{Vector{Int}},Vector{Vector{String}}}
```

A collection of cell lists.

This data type is used to represent a collection of subsets of a Lefschetz complex. It is used for Morse decompositions and for multivector fields.

source

13.3 Conley-Morse Graph Type

`ConleyDynamics.ConleyMorseCM` – Type.

```
ConleyMorseCM{T}
```

Collect the connection matrix information in a struct.

The struct has the following fields:

- `matrix::SparseMatrix{T}`: Connection matrix
- `columns::Vector{Int}`: Corresponding columns in the boundary matrix
- `poset::Vector{Int}`: Poset indices for the connection matrix columns
- `labels::Vector{String}`: Labels for the connection matrix columns
- `morse::Vector{Vector{String}}`: Vector of Morse sets in original complex
- `conley::Vector{Vector{Int}}`: Vector of Conley indices for the Morse sets
- `complex::LefschetzComplex`: The Conley complex as a Lefschetz complex

source

`Base.show` – Method.

```
Base.show(io::IO, ::MIME"text/plain", cm::ConleyMorseCM)
```

Display connection matrix information when hitting return in REPL.

source

Chapter 14

Lefschetz Complex Functions

14.1 Lefschetz Complex Creation

`ConleyDynamics.create_lefschetz_gf2` – Function.

```
create_lefschetz_gf2(defcellbnd)
```

Create a Lefschetz complex over `GF(2)` by specifying its essential cells and boundaries.

The input argument `defcellbnd` has to be a vector of vectors. Each entry `defcellbnd[k]` has to be of one of the following two forms:

- `[String, Int, String, String, ...]`: The first `String` contains the label for the cell `k`, followed by its dimension in the second entry. The remaining entries are for the labels of the cells which make up the boundary.
- `[String, Int]`: This shorther form is for cells with empty boundary. The first entry denotes the cell label, and the second its dimension.

The cells of the resulting Lefschetz complex correspond to the union of all occurring labels. Cell labels that only occur in the boundary specification are assumed to have empty boundary, and they do not have to be specified separately in the second form above. However, if their boundary is not empty, they have to be listed via the above first form as well.

Examples

```
julia> defcellbnd = [["A",0], ["a",1,"B","C"], ["b",1,"B","C"]];

julia> push!(defcellbnd, ["c",1,"B","C"]);

julia> push!(defcellbnd, ["alpha",2,"b","c"]);

julia> lc = create_lefschetz_gf2(defcellbnd);

julia> lc.labels
7-element Vector{String}:
 "A"
 "B"
 "C"
```

```
 "a"
 "b"
 "c"
 "alpha"

julia> homology(lc)
3-element Vector{Int64}:
 2
 1
 0
```

source

ConleyDynamics.lefschetz_subcomplex – Function.

```
lefschetz_subcomplex(lc::AbstractComplex, subcomp::Vector{Int};
                     [validate::Bool=true])
```

Extract a subcomplex from a Lefschetz complex. The subcomplex has to be locally closed, and is given by the collection of cells in `subcomp`.

If `lc` is a `EuclideanComplex`, the returned subcomplex is also an `EuclideanComplex` with the coordinates restricted to the subcomplex cells.

By default, the function raises an error if `subcomp` is not locally closed. This check can be skipped by passing the optional argument `validate=false`, which should only be done if it is already known that the set is locally closed. The computational cost of the function is proportional to the size of `subcomp`, not to the size of `lc`.

source

```
lefschetz_subcomplex(lc::AbstractComplex, subcomp::Vector{String};
                     [validate::Bool=true])
```

Extract a subcomplex from a Lefschetz complex. The subcomplex has to be locally closed, and is given by the collection of cells in `subcomp`.

If `lc` is a `EuclideanComplex`, the returned subcomplex is also an `EuclideanComplex` with the coordinates restricted to the subcomplex cells.

By default, the function raises an error if `subcomp` is not locally closed. This check can be skipped by passing the optional argument `validate=false`, which should only be done if it is already known that the set is locally closed.

source

ConleyDynamics.lefschetz_closed_subcomplex – Function.

```
lefschetz_closed_subcomplex(lc::AbstractComplex, subcomp::Vector{Int})
```

Extract a closed subcomplex from a Lefschetz complex. The subcomplex is the closure of the collection of cells given in `subcomp`.

source

```
lefschetz_closed_subcomplex(lc::AbstractComplex, subcomp::Vector{String})
```

Extract a closed subcomplex from a Lefschetz complex. The subcomplex is the closure of the collection of cells given in `subcomp`.

If `lc` is a `EuclideanComplex`, the returned subcomplex is also an `EuclideanComplex` with the coordinates restricted to the subcomplex cells.

source

`ConleyDynamics.permute_lefschetz_complex` – Function.

```
permute_lefschetz_complex(lc::AbstractComplex,
                          permutation::Vector{Int})
```

Permute the indices of a Lefschetz complex.

The vector `permutation` contains a permutation of the indices for the given Lefschetz complex `lc`. If no permutation is specified, or if the length of the vector is not correct, then a randomly generated one will be used. Note that the permutation has to respect the ordering of the cells by dimension, otherwise an error is raised. In other words, the permutation has to decompose into permutations within each dimension. This is automatically done if no permutation is explicitly specified.

source

`ConleyDynamics.lefschetz_reduction` – Function.

```
lefschetz_reduction(lc::AbstractComplex, redpairs::Vector{Vector{Int}})
```

Apply a sequence of elementary reductions to a Lefschetz complex.

The reduction pairs have to be specified in the argument `redpairs`. Each entry has to be a vector of length two which contains an elementary reduction pair in index form. In particular, the dimensions of the two cells in the pair have to differ by one, and once the pair is reached in the reduction sequence, one cell has to be a face of the other. The function returns a new Lefschetz complex, where all cells in `redpairs` have been removed.

source

```
lefschetz_reduction(lc::AbstractComplex, redpairs::Vector{Vector{String}})
```

Apply a sequence of elementary reductions to a Lefschetz complex.

The reduction pairs have to be specified in the argument `redpairs`. Each entry has to be a vector of length two which contains an elementary reduction pair in label form. In particular, the dimensions of the two cells in the pair have to differ by one, and once the pair is reached in the reduction sequence, one cell has to be a face of the other. The function returns a new Lefschetz complex, where all cells in `redpairs` have been removed.

source

```
lefschetz_reduction(lc::AbstractComplex, r1::Int, r2::Int)
```

Apply a single elementary reduction to a Lefschetz complex.

This method expects that the two cells `r1` and `r2` which form the reduction pair are given in index form. The function returns the reduced Lefschetz complex.

source

```
lefschetz_reduction(lc::AbstractComplex, r1::String, r2::String)
```

Apply a single elementary reduction to a Lefschetz complex.

This method expects that the two cells `r1` and `r2` which form the reduction pair are given in label form. The function returns the reduced Lefschetz complex.

source

`ConleyDynamics.lefschetz_reduction_maps` – Function.

```
lefschetz_reduction_maps(lc::AbstractComplex, redpairs::Vector{Vector{Int}})
```

Apply a sequence of elementary reductions to a Lefschetz complex and return the associated chain maps.

The reduction pairs have to be specified in the argument `redpairs`. Each entry has to be a vector of length two which contains an elementary reduction pair in index form. In particular, the dimensions of the two cells in the pair have to differ by one, and once the pair is reached in the reduction sequence, one cell has to be a face of the other. The function returns a new Lefschetz complex, where all cells in `redpairs` have been removed, as well as the associated chain maps.

The return values are as follows:

- `lcred`: The first variable contains the reduced Lefschetz complex.
- `pp`: This is a sparse matrix representation of the chain equivalence between the original complex and the reduced one.
- `jj`: This is a sparse matrix representation of the chain equivalence between the reduced complex and the original one.
- `hh`: This is a sparse matrix representation of the chain homotopy which shows that the composition `jj * pp` is chain homotopic to the identity.

Examples

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> sc = create_simplicial_complex(labels, simplices, p=0);

julia> redpairs = [["b", "bc"], ["d", "cd"]];

julia> scr, pp, jj, hh = lefschetz_reduction_maps(sc,redpairs);
```

```
julia> bnd = deepcopy(sc.boundary);

julia> bndr = deepcopy(scr.boundary);

julia> ii = sparse_identity(sc.ncells, p=0);

julia> sparse_nz_count(pp*bnd - bndr*pp)
0

julia> sparse_nz_count(jj*bndr - bnd*jj)
0

julia> full_from_sparse(pp*jj)
3×3 Matrix{Rational{Int64}}:
 1  0  0
 0  1  0
 0  0  1

julia> full_from_sparse(jj*pp)
7×7 Matrix{Rational{Int64}}:
 1  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  1  1  1  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  1  0  0
 0  0  0  0  1  0  0
 0  0  0  0  0  0  0

julia> sparse_nz_count(ii + bnd*hh + hh*bnd - jj*pp)
0
```

source

```
lefschetz_reduction_maps(lc::AbstractComplex, redpairs::Vector{Vector{String}})
```

Apply a sequence of elementary reductions to a Lefschetz complex and return the chain maps.

The reduction pairs have to be specified in the argument `redpairs`. Each entry has to be a vector of length two which contains an elementary reduction pair in label form. In particular, the dimensions of the two cells in the pair have to differ by one, and once the pair is reached in the reduction sequence, one cell has to be a face of the other. The function returns a new Lefschetz complex, where all cells in `redpairs` have been removed, as well as the involved chain maps.

source

```
lefschetz_reduction_maps(lc::AbstractComplex, r1::Int, r2::Int)
```

Apply a single elementary reduction to a Lefschetz complex and return the chain maps.

This method expects that the two cells `r1` and `r2` which form the reduction pair are given in index form. The function returns the reduced Lefschetz complex, as well as the involved chain maps.

source

```
lefschetz_reduction_maps(lc::AbstractComplex, r1::String, r2::String)
```

Apply a single elementary reduction to a Lefschetz complex and return the chain maps.

This method expects that the two cells `r1` and `r2` which form the reduction pair are given in label form. The function returns the reduced Lefschetz complex, as well as the involved chain maps.

source

`ConleyDynamics.lefschetz_newbasis` – Function.

```
lefschetz_newbasis(lc::AbstractComplex, basis::SparseMatrix; maps::Bool=false)
```

Create a new Lefschetz complex via change of basis.

The new basis has to be specified in the sparse matrix `basis`, whose columns represent the new basis in terms of the existing one. This matrix has to respect the grading by dimension, i.e., the cells which are used to form a new basis chain have to have the same dimensions as the cell which is being replaced. The function returns the new Lefschetz complex `lcnew`. If the optional parameter `maps = true` is passed, the the function also returns the chain maps `pp` and `jj` which are the isomorphisms from `lc` to `lcnew`, and vice versa, as well as the zero chain homotopy `hh`.

source

`ConleyDynamics.lefschetz_newbasis_maps` – Function.

```
lefschetz_newbasis_maps(lc::AbstractComplex, basis::SparseMatrix)
```

Create a new Lefschetz complex via change of basis and return the associated chain maps.

The new basis has to be specified in the sparse matrix `basis`, whose columns represent the new basis in terms of the existing one. This matrix has to respect the grading by dimension, i.e., the cells which are used to form a new basis chain have to have the same dimensions as the cell which is being replaced. The function returns the new Lefschetz complex `lcnew`, as well as the chain maps `pp` and `jj` which are the isomorphisms from `lc` to `lcnew`, and vice versa.

source

`ConleyDynamics.compose_reductions` – Function.

```
compose_reductions(pp1::SparseMatrix, jj1::SparseMatrix, hh1::SparseMatrix,
                   pp2::SparseMatrix, jj2::SparseMatrix, hh2::SparseMatrix)
```

Combine the chain maps and chain homotopies of two reductions.

The function expects the chain maps and chain homotopies of two Lefschetz complex reductions, and computes the chain maps and chain homotopy for the composition of both. The maps `ppX`, `jjX`, and `hhX` can be obtained via either `lefschetz_reduction_maps` or `lefschetz_newbasis_maps`.

source

14.2 Lefschetz Complex Queries

ConleyDynamics.lefschetz_information – Function.

```
lefschetz_information(lc::AbstractComplex)
```

Extract basic information about a Lefschetz complex.

The input argument `lc` contains the Lefschetz complex. The function returns the information in the form of a `Dict{String,Any}`. You can use the command `keys` to see the keyset of the return dictionary:

- `"Dimension"`: Dimension of the Lefschetz complex
- `"Coefficient field"`: Underlying coefficient field
- `"Euler characteristic"`: Euler characteristic of the complex
- `"Homology"`: Betti numbers of the Lefschetz complex
- `"Boundary sparsity"`: Sparsity percentage of the boundary matrix
- `"Number of cells"`: Total number of cells in the complex
- `"Cell counts by dim"`: Cell counts by dimension

In the last case, the dictionary entry is a vector of pairs `(dimension, cell count)`.

source

ConleyDynamics.lefschetz_cell_count – Function.

```
lefschetz_cell_count(lc::AbstractComplex; bounds::Bool=false)
```

Returns the number of cells in each dimension.

The function returns the number of cells in each dimension. The return variable is of type `Vector{Int}`, has length `lc.dim + 1`, and its k-th entry contains the number of cells in dimension k-1. If the optional parameter `bounds=true` is passed, then the function also returns two integer vectors `lo` and `hi`. These contain the beginning and end indices of the cells in each dimension.

source

ConleyDynamics.lefschetz_field – Function.

```
fieldstr = lefschetz_field(lc::AbstractComplex)
```

Returns the Lefschetz complex coefficient field.

source

ConleyDynamics.lefschetz_is_closed – Function.

```
lefschetz_is_closed(lc::AbstractComplex, subcomp::Vector{Int})
```

Determine whether a Lefschetz complex subset is closed.

source

```
lefschetz_is_closed(lc::AbstractComplex, subcomp::Vector{String})
```

Determine whether a Lefschetz complex subset is closed.

source

ConleyDynamics.lefschetz_is_locally_closed – Function.

```
lefschetz_is_locally_closed(lc::AbstractComplex, subcomp::Vector{Int})
```

Determine whether a Lefschetz complex subset is locally closed.

source

```
lefschetz_is_locally_closed(lc::AbstractComplex, subcomp::Vector{String})
```

Determine whether a Lefschetz complex subset is locally closed.

source

ConleyDynamics.lefschetz_is_simplicial – Function.

```
lefschetz_is_simplicial(lc::AbstractComplex)
```

Determine whether a Lefschetz complex is a simplicial complex.

source

ConleyDynamics.lefschetz_is_cubical – Function.

```
lefschetz_is_cubical(lc::AbstractComplex)
```

Determine whether a Lefschetz complex is a cubical complex.

This function determines whether a Lefschetz complex is cubical in the sense of this package. This means that the labels of the complex have to satisfy the constraints explained in `create_cubical_complex`.

source

ConleyDynamics.lefschetz_is_subcubical – Function.

```
lefschetz_is_subcubical(lc::AbstractComplex)
```

Determine whether a Lefschetz complex is a locally closed subset of a cubical complex.

This function determines whether a Lefschetz complex is a locally cosed subset of a cubical complex in the sense of this package. This means that the labels of the complex have to satisfy the constraints explained in `create_cubical_complex`, and `lc` is locally closed in the complex obtained from the labels in `lc.labels` via `create_cubical_complex`.

source

`ConleyDynamics.validate_lefschetz_complex` – Function.

```
validate_lefschetz_complex(lc::AbstractComplex)
```

Check whether the boundary matrix of `lc` satisfies $\partial^2 = 0$.

Returns `true` if the complex is valid. Throws an error if the boundary matrix does not square to zero, indicating an incorrectly assembled complex.

This function is the explicit counterpart to the `validate=true` keyword argument of the `LefschetzComplex` constructor. It is useful for checking a complex that was not validated at construction time.

Examples

```
julia> lc = create_simplicial_complex(["A","B","C"], [["A","B"],["B","C"]]);

julia> validate_lefschetz_complex(lc)
true
```

source

14.3 Topological Features

`ConleyDynamics.lefschetz_boundary` – Function.

```
lefschetz_boundary(lc::AbstractComplex, cellI::Int)
```

Compute the support of the boundary of a Lefschetz complex cell.

This method returns the boundary support as a `Vector{Int}`.

source

```
lefschetz_boundary(lc::AbstractComplex, cellS::String)
```

Compute the support of the boundary of a Lefschetz complex cell.

This method returns the boundary support as a `Vector{String}`.

source

`ConleyDynamics.lefschetz_coboundary` – Function.

```
lefschetz_coboundary(lc::AbstractComplex, cellI::Int)
```

Compute the support of the coboundary of a Lefschetz complex cell.

This method returns the boundary support as a `Vector{Int}`.

source

```
lefschetz_coboundary(lc::AbstractComplex, cellS::String)
```

Compute the support of the coboundary of a Lefschetz complex cell.

This method returns the boundary support as a `Vector{String}`.

source

`ConleyDynamics.lefschetz_closure` – Function.

```
lefschetz_closure(lc::AbstractComplex, subcomp::Vector{Int})
```

Compute the closure of a Lefschetz complex subset.

source

```
lefschetz_closure(lc::AbstractComplex, subcomp::Vector{String})
```

Compute the closure of a Lefschetz complex subset.

source

`ConleyDynamics.lefschetz_interior` – Function.

```
lefschetz_interior(lc::AbstractComplex, subcomp::Vector{Int})
```

Compute the interior of a Lefschetz complex subset.

source

```
lefschetz_interior(lc::AbstractComplex, subcomp::Vector{String})
```

Compute the interior of a Lefschetz complex subset.

source

`ConleyDynamics.lefschetz_topboundary` – Function.

```
lefschetz_topboundary(lc::AbstractComplex, subcomp::Vector{Int})
```

Compute the topological boundary of a Lefschetz complex subset.

In contrast to the algebraic boundary defined via the boundary operator, this function computes the boundary of the Lefschetz complex subset specified in `subcomp` if the Lefschetz complex is interpreted as a finite topological space. In other words, the topological boundary is the set difference of the closure and the interior of the subset.

source

```
lefschetz_topboundary(lc::AbstractComplex, subcomp::Vector{String})
```

Compute the topological boundary of a Lefschetz complex subset.

In contrast to the algebraic boundary defined via the boundary operator, this function computes the boundary of the Lefschetz complex subset specified in `subcomp` if the Lefschetz complex is interpreted as a finite topological space. In other words, the topological boundary is the set difference of the closure and the interior of the subset.

source

`ConleyDynamics.lefschetz_openhull` – Function.

```
lefschetz_openhull(lc::AbstractComplex, subcomp::Vector{Int})
```

Compute the open hull of a Lefschetz complex subset.

source

```
lefschetz_openhull(lc::AbstractComplex, subcomp::Vector{String})
```

Compute the open hull of a Lefschetz complex subset.

source

`ConleyDynamics.lefschetz_lchull` – Function.

```
lefschetz_lchull(lc::AbstractComplex, subcomp::Vector{Int})
```

Compute the locally closed hull of a Lefschetz complex subset.

The locally closed hull is the smallest locally closed set which contains the given cells. It is the intersection of the closure and the open hull.

source

```
lefschetz_lchull(lc::AbstractComplex, subcomp::Vector{String})
```

Compute the locally closed hull of a Lefschetz complex subset.

The locally closed hull is the smallest locally closed set which contains the given cells. It is the intersection of the closure and the open hull.

source

`ConleyDynamics.lefschetz_clomo_pair` – Function.

```
lefschetz_clomopair(lc::AbstractComplex, subcomp::Vector{Int})
```

Determine the closure-mouth-pair associated with a Lefschetz complex subset.

The function returns the pair `(closure,mouth)`.

source

```
lefschetz_clomopair(lc::AbstractComplex, subcomp::Vector{String})
```

Determine the closure-mouth-pair associated with a Lefschetz complex subset.

The function returns the pair `(closure,mouth)`.

source

`ConleyDynamics.lefschetz_neighbors` – Function.

```
lefschetz_neighbors(lc::AbstractComplex, subcomp::Vector{Int})
```

Find all cells neighboring a Lefschetz complex subset.

The function returns all cells which are not contained in `subcomp`, but which are either in the closure or the open hull of `subcomp`. In other words, if one adds a neighbor to the set `subcomp`, then the number of connected components of the combined set does not increase. It could, however, decrease.

source

```
lefschetz_neighbors(lc::AbstractComplex, subcomp::Vector{String})
```

Find all cells neighboring a Lefschetz complex subset.

The function returns all cells which are not contained in `subcomp`, but which are either in the closure or the open hull of `subcomp`. In other words, if one adds a neighbor to the set `subcomp`, then the number of connected components of the combined set does not increase. It could, however, decrease.

source

`ConleyDynamics.lefschetz_skeleton` – Function.

```
lefschetz_skeleton(lc::AbstractComplex, subcomp::Vector{Int}, skdim::Int)
```

Compute the `skdim`-dimensional skeleton of a Lefschetz complex subset.

The computed skeleton is for the closure of the subcomplex given by `subcomp`.

source

```
lefschetz_skeleton(lc::AbstractComplex, subcomp::Vector{String}, skdim::Int)
```

Compute the `skdim`-dimensional skeleton of a Lefschetz complex subset.

The computed skeleton is for the closure of the subcomplex given by `subcomp`.

source

```
lefschetz_skeleton(lc::AbstractComplex, skdim::Int)
```

Compute the `skdim`-dimensional skeleton of a Lefschetz complex.

The computed skeleton is for the full Lefschetz complex.

source

`ConleyDynamics.manifold_boundary` – Function.

```
manifold_boundary(lc::AbstractComplex)
```

Extract the manifold boundary from a Lefschetz complex.

The function expects a Lefschetz complex which represents a compact d-dimensional manifold with boundary. It returns a list of all cells which lie on the topological boundary of the manifold, in the form of a `Vector{Int}`.

source

14.4 Filters on Lefschetz Complexes

`ConleyDynamics.create_random_filter` – Function.

```
create_random_filter(lc::AbstractComplex)
```

Create a random injective filter on a Lefschetz complex.

The function creates a random injective filter on a Lefschetz complex. The filter is created by assigning integers to cell groups, increasing with dimension. Within each dimension the assignment is random, but all filter values of cells of dimension k are less than all filter values of cells with dimension k+1. The function returns the filter as `Vector{Int}`, with indices corresponding to the cell indices in the Lefschetz complex.

source

`ConleyDynamics.filter_shallow_pairs` – Function.

```
filter_shallow_pairs(lc::AbstractComplex, phi)
```

Find all shallow pairs for a filter.

This function finds all shallow pairs for the filter `phi`. These are face-coface pairs `(x,y)` whose dimensions differ by one, and such that `y` has the smallest filter value on the coboundary of `x`, and `x` has the largest filter value on the boundary of `y`.

If the filter is injective, these pairs give rise to a Forman vector field on the underlying Lefschetz complex. For noninjective filters this is not true in general.

source

`ConleyDynamics.filter_induced_mvf` – Function.

```
filter_induced_mvf(lc::AbstractComplex, phi)
```

Compute the multivector field induced by a filter.

This function returns the smallest multivector field which has the property that every shallow pair is contained in a multivector. For injective filters this is a Forman vector field, but in the noninjective case it can be a general multivector field.

source

`ConleyDynamics.lefschetz_filtration` – Function.

```
lefschetz_filtration(lc::AbstractComplex, fvalues::Vector{Int})
```

Compute a filtration on a Lefschetz subset.

The considered Lefschetz complex is given in `lc`. The vector `fvalues` assigns an integer between 0 and N to every cell in `lc`. For every k the complex L_k is given by the closure of all cells with values between 1 and k. The function returns the following variables:

- `lcsub`: The subcomplex L_N
- `fvalsub`: The filtration on the subcomplex with values 1,...,N

Example

```
julia> labels = ["A","B","C","D","E","F","G"];

julia> simplices = [["A","B","D"],["B","D","E"],["B","C","E"],["C","E","F"],["F","G"]];

julia> sc = create_simplicial_complex(labels,simplices);

julia> filtration = [0,0,0,0,0,0,0,1,1,0,1,2,0,4,2,4,0,5,3,7,6];

julia> lcsub, fvalsub = lefschetz_filtration(sc,filtration);

julia> phinf, phint = persistent_homology(lcsub,fvalsub);

julia> phinf
3-element Vector{Vector{Int64}}:
 [1]
 []
 []
```

```
julia> phint
3-element Vector{Vector{Tuple{Int64, Int64}}}:
 []
 [(1, 5), (2, 7), (4, 6)]
 []
```

source

```
lefschetz_filtration(lc::AbstractComplex, strfilt::Vector{Vector{String}})
```

Compute a filtration on a Lefschetz subset.

The considered Lefschetz complex is given in `lc`. The vector of string vectors `strfilt` contains the necessary simplices to build the filtration. The list `strfilt[k]` contains the simplices that are added at the k-th step, together with their closures. Thus, for every k the complex L_k is given by the closure of all cells listed in `strfilt[i]` for `i` between 1 and k. The function returns the following variables:

- `lcsub`: The subcomplex L_N, where `N = length(strfilt)`
- `fvalsub`: The filtration on the subcomplex with values 1,...,N

Example

```
julia> labels = ["A","B","C","D","E","F","G"];

julia> simplices = [["A","B","D"],["B","D","E"],["B","C","E"],["C","E","F"],["F","G"]];

julia> sc = create_simplicial_complex(labels,simplices);

julia> strfiltration =
↪  [["AB","AD","BD"],["BE","DE"],["BCE"],["CF","EF"],["ABD"],["CEF"],["BDE"]];

julia> lcsub, fvalsub = lefschetz_filtration(sc, strfiltration);

julia> phinf, phint = persistent_homology(lcsub,fvalsub);

julia> phinf
3-element Vector{Vector{Int64}}:
 [1]
 []
 []

julia> phint
3-element Vector{Vector{Tuple{Int64, Int64}}}:
 []
 [(1, 5), (2, 7), (4, 6)]
 []
```

source

ConleyDynamics.lefschetz_filtration_mvf – Function.

```
lefschetz_filtration_mvf(lc::AbstractComplex, fvalues::Vector{Int})
```

Compute the multivector field associated with a Lefschetz complex filtration.

For the Lefschetz complex `lc`, and the filtration `fvalues` given on the Lefschetz complex, this function determines the multivector field generated by the filtration in the following way. For every filtration value `k`, the cells which are assigned this value form a locally closed set, which can be decomposed into connected components that are all locally closed. The union of all these connected components, as `k` ranges from 1 to `N`, forms the multivector field `mvf` that is returned by the function. The filtration should be generated by the function `lefschetz_filtration`.

source

14.5 Lefschetz Helper Functions

`ConleyDynamics.lefschetz_gfp_conversion` – Function.

```
lcgfp = lefschetz_gfp_conversion(lc::AbstractComplex, p::Int)
```

Convert a Lefschetz complex to the same complex over a finite field.

It is expected that the boundary matrix of the given Lefschetz complex `lc` is defined over the rationals, and that the target characteristic `p` is a prime.

source

`ConleyDynamics.lefschetz_to_euclidean` – Function.

```
lefschetz_to_euclidean(lc::LefschetzComplex,
                       vertex_coords::Vector{<:Vector{<:Real}}) -> EuclideanComplex
```

Embed a `LefschetzComplex` into Euclidean space by attaching coordinates to every cell.

The vector `vertex_coords` contains one coordinate vector per vertex of `lc`, indexed by the cell index of the vertex (i.e., `vertex_coords[k]` gives the coordinates of the vertex whose cell index is `k`). Since vertices are always stored first in the cell list (dimensions are non-decreasing), the indices 1 through `nvertices` correspond to the vertices.

The function raises an `ArgumentError` if the complex is not simplicial or cubical, i.e., if any 1-cell has a vertex count other than 2, or any 2-cell has a vertex count other than 3 or 4.

source

```
lefschetz_to_euclidean(lc::LefschetzComplex,
                       coords::Vector{<:Vector{<:Vector{<:Real}}})
                       -> EuclideanComplex
```

Embed a `LefschetzComplex` into Euclidean space by attaching coordinates to every cell.

The vector `coords` contains coordinates for each of the cells in `lc`, indexed by the cell index. This function only performs basic check on the suitability of this coordinate vector. More precisely, the function raises

an `ArgumentError` if any vertex has coordinate count other than 1, any 1-cell has a coordinate count other than 2, or any 2-cell has a coordinate count other than 3 or 4.

source

`ConleyDynamics.euclidean_to_lefschetz` – Function.

```
euclidean_to_lefschetz(ec::EuclideanComplex) -> LefschetzComplex
```

Convert a `EuclideanComplex` to a `LefschetzComplex` by stripping the embedded coordinates. All other fields are shared unchanged.

source

`ConleyDynamics.rescale_coords` – Function.

```
rescale_coords(ec::EuclideanComplex,
               a::Vector{<:Real}, b::Vector{<:Real}) -> EuclideanComplex
```

Rescale the coordinates of a `EuclideanComplex` so that component `j` maps from its current range `[min_j, max_j]` to `[a[j], b[j]]`.

a and b must be vectors of length equal to the spatial dimension (number of coordinate components). The rescaling is linear and applied uniformly to all vertex coordinates across all cells.

Example

```
# Normalize x to [0,1] and y to [-1,1]:
ec2 = rescale_coords(ec, [0.0, -1.0], [1.0, 1.0])
```

source

```
rescale_coords(ec::EuclideanComplex, a::Real, b::Real) -> EuclideanComplex
```

Rescale all coordinate components uniformly to the interval `[a, b]`.

This is a convenience overload of `rescale_coords` that applies the same target interval to every spatial dimension.

source

Chapter 15

Special Complex Functions

15.1 Simplicial Complexes

`ConleyDynamics.create_simplicial_complex` – Function.

```
create_simplicial_complex(labels::Vector{String},
                          simplices::Vector{Vector{Int}};
                          p::Int=2)
```

Initialize a Lefschetz complex from a simplicial complex. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The vector `labels` contains a label for every vertex, while `simplices` contains all the highest-dimensional simplices necessary to define the simplicial complex. Every simplex is represented as a vector of `Int`, with entries corresponding to the vertex indices.

Warning

Note that the labels all have to have the same character length!

source

```
create_simplicial_complex(labels::Vector{String},
                          simplices::Vector{Vector{String}};
                          p::Int=2)
```

Initialize a Lefschetz complex from a simplicial complex. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The vector `labels` contains a label for every vertex, while `simplices` contains all the highest-dimensional simplices necessary to define the simplicial complex.

source

```
create_simplicial_complex(labels::Vector{String},
                          simplices::Vector{Vector{Int}},
                          vertex_coords::Vector{<:Vector{<:Real}};
                          p::Int=2)
```

Initialize a `EuclideanComplex` from a simplicial complex with embedded vertex coordinates. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The vector `labels` contains a label for every vertex, `simplices` contains all the highest-dimensional simplices, and `vertex_coords` contains the Euclidean coordinates of each vertex (indexed by vertex index, i.e., `vertex_coords[k]` gives the coordinate of vertex k).

source

```
create_simplicial_complex(labels::Vector{String},
                          simplices::Vector{Vector{String}},
                          vertex_coords::Vector{<:Vector{<:Real}};
                          p::Int=2)
```

Initialize a `EuclideanComplex` from a simplicial complex with embedded vertex coordinates (String-simplex variant).

source

`ConleyDynamics.create_simplicial_rectangle` – Function.

```
create_simplicial_rectangle(nx::Int, ny::Int; p::Int=2, euclidean::Bool=false)
```

Create a simplicial complex covering a rectangle in the plane. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The rectangle is given by the subset `[0,nx] x [0,ny]` of the plane. Each unit square is represented by four triangles, which meet in the center point of the square. Labels have the following meaning:

- The label XXXYYYb corresponds to the point `(XXX, YYY)`.
- The label XXXYYYc corresponds to `(XXX + 1/2, YYY + 1/2)`.

The number of characters in XXX and YYY matches the number of digits of the larger number of `nx` and `ny`.

When `euclidean=false` (default), the function returns:

- A simplicial complex `sc::LefschetzComplex`.
- A vector `coords::Vector{Vector{Float64}}` of vertex coordinates.

When `euclidean=true`, it returns a `EuclideanComplex` with embedded coordinates.

source

`ConleyDynamics.create_simplicial_delaunay` – Function.

```
create_simplicial_delaunay(boxw::Real, boxh::Real, pdist::Real, attmpt::Int;
                           p::Int=2, euclidean::Bool=false)
```

Create a planar Delaunay triangulation inside a box. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The function selects a random sample of points inside the rectangular box `[0,boxw] x [0,boxh]`, while trying to maintain a minimum distance of `pdist` between the points. The argument `attmpt` specifies the number of attempts when trying to add points. A standard value is 20, and larger values tend to fill holes better, but at the expense of runtime. From the random sample, the function then creates a Delaunay triangulation.

When `euclidean=false` (default), the function returns:

- A simplicial complex `sc::LefschetzComplex`.
- A vector `coords::Vector{Vector{Float64}}` of vertex coordinates.

When `euclidean=true`, it returns a `EuclideanComplex` with embedded coordinates.

Note that the function does not provide a full triangulation of the given rectangle. Close to the boundary there will be gaps.

source

```
create_simplicial_delaunay(boxw::Real, boxh::Real, npoints::Int;
                           p::Int=2, euclidean::Bool=false)
```

Create a planar Delaunay triangulation inside a box. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The function selects a random sample of `npoints` points inside the rectangular box `[0,boxw] x [0,boxh]`. From the random sample, the function then creates a Delaunay triangulation.

When `euclidean=false` (default), the function returns:

- A simplicial complex `sc::LefschetzComplex`.
- A vector `coords::Vector{Vector{Float64}}` of vertex coordinates.

When `euclidean=true`, it returns a `EuclideanComplex` with embedded coordinates.

Note that the function does not provide a full triangulation of the given rectangle. Close to the boundary there will be gaps.

source

`ConleyDynamics.tetmesh_to_euclidean` – Function.

```
tetmesh_to_euclidean(points::AbstractVector, cells::AbstractVector;
                     p::Int=2, droporphans::Bool=true) -> EuclideanComplex
```

Convert a simplicial mesh, given by vertex coordinates and maximal simplices, into an `EuclideanComplex`. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The vector `points` contains the vertex coordinates, where `points[k]` can be any vector or tuple of real numbers. Usually these are points in three dimensions, but any dimension is allowed. The vector `cells` contains the maximal simplices of the mesh, as vectors or tuples of 1-based indices into `points`. Typically these are the tetrahedra of a tetrahedral mesh, or the triangles of a triangulated surface, but simplices of different dimensions can be mixed. All faces of the given simplices are generated automatically.

If `droporphans=true` (default), vertices which do not belong to any cell are removed, and the remaining vertices are renumbered in their original order. Mesh generators frequently leave such unused vertices, for example geometry points outside of the meshed region. If `droporphans=false`, unused vertices become isolated vertices of the complex.

Vertex labels are zero-padded decimal strings (`"001"`, `"002"`, ...) whose width is determined by the number of vertices, and the label of a higher-dimensional simplex is the concatenation of its sorted vertex labels. This is the same convention as in `create_simplicial_delaunay` and `delaunay_to_simplicial`.

This function can be used to import tetrahedral or surface meshes from any source, such as mesh files or other Julia packages.

Example

```
points = [[0.0,0.0,0.0], [1.0,0.0,0.0], [0.0,1.0,0.0],
          [0.0,0.0,1.0], [1.0,1.0,1.0]]
tets   = [[1,2,3,4], [2,3,4,5]]
ec     = tetmesh_to_euclidean(points, tets)
```

source

```
tetmesh_to_euclidean(points::AbstractMatrix{<:Real},
                     cells::AbstractMatrix{<:Integer};
                     p::Int=2, droporphans::Bool=true) -> EuclideanComplex
```

Convert a simplicial mesh given in matrix form into an `EuclideanComplex`.

Every column of `points` contains the coordinates of one vertex, and every column of `cells` contains the 1-based vertex indices of one maximal simplex. For a tetrahedral mesh in three dimensions, `points` is a `3 x n` and `cells` a `4 x m` matrix. This is the format used by many mesh generators. See the vector version of `tetmesh_to_euclidean` for a description of the keyword arguments.

source

`ConleyDynamics.simplicial_torus` – Function.

```
sc = simplicial_torus(p::Int)
```

Create a triangulation of the two-dimensional torus.

The function returns a simplicial complex which represents a two-dimensional torus. The argument `p` specifies the characteristic of the underlying field. This triangulation is taken from Figure 6.4 in Munkres' book on Algebraic Topology. The boundary vertices are labeled as letters as in the book, the five center vertices are labeled by 1 through 5.

Examples

```
julia> println(homology(simplicial_torus(0)))
[1, 2, 1]

julia> println(homology(simplicial_torus(2)))
[1, 2, 1]

julia> println(homology(simplicial_torus(3)))
[1, 2, 1]
```

source

ConleyDynamics.simplicial_klein_bottle – Function.

```
sc = simplicial_klein_bottle(p::Int)
```

Create a triangulation of the two-dimensional Klein bottle.

The function returns a simplicial complex which represents the two-dimensional Klein bottle. The argument p specifies the characteristic of the underlying field. This triangulation is taken from Figure 6.6 in Munkres' book on Algebraic Topology. The boundary vertices are labeled as letters as in the book, the five center vertices are labeled by 1 through 5.

Examples

```
julia> println(homology(simplicial_klein_bottle(0)))
[1, 1, 0]

julia> println(homology(simplicial_klein_bottle(2)))
[1, 2, 1]

julia> println(homology(simplicial_klein_bottle(3)))
[1, 1, 0]
```

source

ConleyDynamics.simplicial_projective_plane – Function.

```
sc = simplicial_projective_plane(p::Int)
```

Create a triangulation of the projective plane.

The function returns a simplicial complex which represents the projective plane. The argument p specifies the characteristic of the underlying field. This triangulation is taken from Figure 6.6 in Munkres' book on Algebraic Topology. The boundary vertices are labeled as letters as in the book, the five center vertices are labeled by 1 through 5.

Examples

```
julia> println(homology(simplicial_projective_plane(0)))
[1, 0, 0]

julia> println(homology(simplicial_projective_plane(2)))
[1, 1, 1]

julia> println(homology(simplicial_projective_plane(3)))
[1, 0, 0]
```

source

ConleyDynamics.simplicial_torsion_space – Function.

```
sc = simplicial_torsion_space(n::Int, p::Int)
```

Create a triangulation of a space with 1-dimensional torsion.

The function returns a simplicial complex which has the following integer homology groups:

- In dimension 0 it is the group of integers.
- In dimension 1 it is the integers modulo n.
- In dimension 2 it is the trivial group.

In other words, the simplicial complex has nontrivial torsion in dimension 1. It is a triangulation of an n-gon, in which all boundary edges are oriented counterclockwise, and all of these edges are identified. The parameter p specifies the characteristic of the underlying field.

Examples

```
julia> println(homology(simplicial_torsion_space(6,2)))
[1, 1, 1]

julia> println(homology(simplicial_torsion_space(6,3)))
[1, 1, 1]

julia> println(homology(simplicial_torsion_space(6,5)))
[1, 0, 0]
```

source

15.2 Point Sets and Triangulated Surfaces

`ConleyDynamics.poisson_points_box` – Function.

```
poisson_points_box(bmin::AbstractVector{<:Real}, bmax::AbstractVector{<:Real},
                   pdist::Real; attempts::Int=20,
                   rng::AbstractRNG=Random.default_rng())
                   -> Vector{Vector{Float64}}
```

Create a random sample of points in a box with a minimum distance between the points (Poisson-disk sampling).

The box is given by `[bmin[1],bmax[1]] x ... x [bmin[d],bmax[d]]`, and it can have any dimension `d`. The function selects random points inside this box such that any two points have distance at least `pdist`, while trying to fill the box as densely as possible. The argument `attempts` specifies how many candidate points are tried around each accepted point before giving up on it. The standard value is 20, and larger values tend to fill holes better, but at the expense of runtime. The function uses Bridson's algorithm, and its runtime is proportional to the number of generated points. Every candidate point is compared with the points in `(2*ceil(sqrt(d))+1)^d` neighboring grid cells, so the function is only practical in low dimensions, up to about `d = 5`.

The random number generator can be specified via `rng`. Passing a seeded generator, such as `Random.Xoshiro(1234)`, makes the result reproducible.

The points are returned as a vector of coordinate vectors. They can be used, for example, as nodes for a Delaunay triangulation.

Example

```
using Random
pts = poisson_points_box([0,0,0], [2,1,1], 0.1; rng=Xoshiro(42))
```

source

ConleyDynamics.trisurface_sphere – Function.

```
trisurface_sphere(center::AbstractVector{<:Real}, r::Real, n::Int)
                  -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated sphere with center `center` and radius `r`.

The triangulation is an icosphere: an icosahedron whose triangles are subdivided `n` times, with all vertices projected onto the sphere. The result has `10*4^n + 2` vertices and `20*4^n` triangles, which are all of roughly the same size and shape.

The function returns the vertex coordinates `V` and the triangles `T`, where every triangle is a vector of three indices into `V`. The triangles are consistently oriented, with normals pointing outward.

The returned surface can be used as a constraint or boundary surface for mesh generation, or it can be turned into a two-dimensional simplicial complex in three dimensions via `tetmesh_to_euclidean(V, T)`.

source

ConleyDynamics.trisurface_ellipsoid – Function.

```
trisurface_ellipsoid(center::AbstractVector{<:Real},
                     radii::AbstractVector{<:Real}, n::Int)
                     -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated ellipsoid with center `center` and semi-axes `radii` along the three coordinate directions.

The triangulation is obtained by scaling an icosphere with `n` subdivisions, see `trisurface_sphere`. The function returns the vertex coordinates `V` and the triangles `T`, consistently oriented with outward normals.

source

ConleyDynamics.trisurface_cylinder – Function.

```
trisurface_cylinder(p0::AbstractVector{<:Real}, p1::AbstractVector{<:Real},
                    r::Real, ntheta::Int, nz::Int; capped::Bool=true)
                    -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated cylinder of radius `r` around the axis from `p0` to `p1`.

The lateral surface is subdivided into `ntheta` intervals around the axis and `nz` intervals along the axis. If `capped=true` (default), the two ends are closed by triangle fans around the points `p0` and `p1`, so that the result is a closed surface. Otherwise, the result is an open tube.

The function returns the vertex coordinates `V` and the triangles `T`, consistently oriented with outward normals.

source

`ConleyDynamics.trisurface_torus` – Function.

```
trisurface_torus(center::AbstractVector{<:Real}, R::Real, r::Real,
                 nu::Int, nv::Int; axis::AbstractVector{<:Real}=[0,0,1])
                 -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated torus with center `center`, major radius `R`, and minor radius `r`.

The torus is rotationally symmetric around the line through `center` in the direction `axis`. It is subdivided into `nu` intervals around this axis, and into `nv` intervals around the tube. The function returns the vertex coordinates `V` and the triangles `T`, consistently oriented with outward normals. The result is a closed surface of genus one.

source

`ConleyDynamics.trisurface_plane_patch` – Function.

```
trisurface_plane_patch(origin::AbstractVector{<:Real}, u::AbstractVector{<:Real},
                       v::AbstractVector{<:Real}, nu::Int, nv::Int)
                       -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated planar parallelogram.

The parallelogram consists of the points `origin + s*u + t*v` for `s` and `t` in `[0,1]`, and it is subdivided into `nu` intervals in the direction `u` and `nv` intervals in the direction `v`. The function returns the vertex coordinates `V` and the triangles `T`, consistently oriented with normals in the direction of the cross product of `u` and `v`. The result is an open surface.

source

`ConleyDynamics.trisurface_parametric` – Function.

```
trisurface_parametric(f, urange, vrange, nu::Int, nv::Int;
                      uperiodic::Bool=false, vperiodic::Bool=false,
                      tol::Real=1e-10)
                      -> (V::Vector{Vector{Float64}}, T::Vector{Vector{Int}})
```

Create a triangulated parametric surface.

The surface is the image of the function `f(u,v)`, which returns a coordinate vector for parameters `u` in `urange = (umin, umax)` and `v` in `vrange = (vmin, vmax)`. The parameter rectangle is subdivided into `nu` intervals in `u` and `nv` intervals in `v`, and every subrectangle is split into two triangles. The triangles are oriented so that their normals point in the direction of the cross product of the partial derivatives of `f` with respect to `u` and `v`.

If `uperiodic=true`, then `f(umin,v)` and `f(umax,v)` are assumed to be the same point, and the two seams are glued together. The keyword `vperiodic` does the same in the v direction. In addition, vertices which are closer than `tol` are merged, and triangles which then become degenerate are removed. This takes care of collapsed edges, such as the poles of a sphere in spherical coordinates. The tolerance `tol` has to be positive, and since it is an absolute distance, it has to be scaled with the size of the surface.

Example

```
fsph(u,v) = [cos(u)*sin(v), sin(u)*sin(v), cos(v)]
V, T = trisurface_parametric(fsph, (0,2*pi), (pi,0), 24, 12; uperiodic=true)
```

In this example, the poles are merged automatically, and the range `(pi,0)` for v gives outward normals.

source

15.3 Cubical Complexes

`ConleyDynamics.create_cubical_complex` – Function.

```
create_cubical_complex(cubes::Vector{String}; p::Int=2, euclidean::Bool=false)
```

Initialize a Lefschetz complex from a cubical complex. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The vector cubes contains a list of all the highest-dimensional cubes necessary to define the cubical complex. Every cube is represented as a string as follows:

- d integers, which correspond to the coordinates of a point in d-dimensional Euclidean space
- a point `.`
- d integers 0 or 1, which give the interval length in the respective dimension

The first d integers all have to occupy the same number of characters. In addition, if the occupied space is L characters for each coordinate, the coordinates only can take values from 0 to `10^L - 2`. This is due to the fact that the boundary operator will add one to certain coordinates, and they still need to be representable withing the same L digits.

For example, the string `030600.101` corresponds to the point `(3,6,0)` in three dimensions. The dimensions are 1, 0, and 1, and therefore this string corresponds to the cube `[3,4] x [6] x [0,1]`. The same cube could have also been represented by `360.101` or by `003006000.101`.

When `euclidean=false` (default), the function returns:

- A cubical complex `cc::LefschetzComplex`
- A vector `coords::Vector{Vector{Float64}}` of vertex coordinates

When `euclidean=true`, it returns a `EuclideanComplex` with embedded coordinates derived from the cube labels.

Warning

Note that the labels all have to have the same format!

Example

```
julia> cubes = ["00.11", "01.01", "02.10", "11.10", "11.01", "22.00"];

julia> lc, coords = create_cubical_complex(cubes);

julia> lc.ncells
17

julia> homology(lc)
3-element Vector{Int64}:
 2
 1
 0
```

source

ConleyDynamics.create_cubical_rectangle – Function.

```
create_cubical_rectangle(nx::Int, ny::Int;
                         p::Int=2, randomize::Real=0.0, euclidean::Bool=false)
```

Create a cubical complex covering a rectangle in the plane. The complex is over the rationals if p=0, and over GF(p) if p>0.

The rectangle is given by the subset [0,nx] x [0,ny] of the plane, and each unit square gives a two-dimensional cube in the resulting cubical complex.

When euclidean=false (default), the function returns:

- A cubical complex cc::LefschetzComplex
- A vector coords::Vector{Vector{Float64}} of vertex coordinates

When euclidean=true, it returns a EuclideanComplex with embedded coordinates.

If the optional parameter randomize is assigned a positive real fraction r less that 0.5, then the actual coordinates will be randomized. They are chosen uniformly from discs of radius r centered at each vertex.

source

ConleyDynamics.create_cubical_box – Function.

```
create_cubical_box(nx::Int, ny::Int, nz::Int;
                   p::Int=2, randomize::Real=0.0, euclidean::Bool=false)
```

Create a cubical complex covering a box in space. The complex is over the rationals if p=0, and over GF(p) if p>0.

The box is given by the subset [0,nx] x [0,ny] x [0,nz] of space, and each unit cube gives a three-dimensional cube in the resulting cubical complex.

When euclidean=false (default), the function returns:

- A cubical complex `cc::LefschetzComplex`
- A vector `coords::Vector{Vector{Float64}}` of vertex coordinates

When `euclidean=true`, it returns a `EuclideanComplex` with embedded coordinates.

If the optional parameter `randomize` is assigned a positive real fraction `r` less that 0.5, then the actual coordinates will be randomized. They are chosen uniformly from balls of radius `r` centered at each vertex.

source

`ConleyDynamics.cube_field_size` – Function.

```
cube_field_size(cube::String)
```

Determine the field sizes of a given cube label.

The function returns the dimension of the ambient space in the first output parameter `pointdim`, and the length of the individual coordinate fields in the second return variable `pointlen`.

Example

```
julia> cube_field_size("011654003020.0110")
(4, 3)
```

source

`ConleyDynamics.cube_information` – Function.

```
cube_information(cube::String)
```

Compute a cube's coordinate information.

The function returns an integer vector with the cubes coordinate information. The return vector `intinfo` contains in its components the following data:

- `1:pointdim`: Coordinates of the anchor point
- `1+pointdim:2*pointdim`: Interval length in each dimension
- `1+2*pointdim`: Dimension of the cube

Note that `pointdim` equals the dimension of the points specifying the cube.

Example

```
julia> cube_information("011654003.011")
7-element Vector{Int64}:
  11
 654
   3
   0
   1
   1
   2
```

source

ConleyDynamics.cube_label – Function.

```
cube_label(pointdim::Int, pointlen::Int, pointinfo::Vector{Int})
```

Create a label from a cube's coordinate information.

The dimension of the ambient Euclidean space is `pointdim`, while the field length for each coordinate is `pointlen`. The vector `pointinfo` has to be of length at least two times `pointdim`. The first `pointdim` entries contain the coordinates of the anchor point, while the next `pointdim` entries are either 0 or 1 depending on the size of the interval. For example, if `poindim = 3` and `pointinfo = [1,2,3,1,0,1]`, then we represent the cube in three-dimensional space given by `[1,2] x [2] x [3 4]`.

Example

```
julia> cube_label(3,2,[10,23,5,1,1,0])
"102305.110"
```

source

ConleyDynamics.is_cube_label – Function.

```
is_cube_label(cube::String)
```

Tests whether a given string is a cube label.

source

ConleyDynamics.get_cubical_coords – Function.

```
get_cubical_coords(cc::AbstractComplex)
```

Compute the vertex coordinates for a cubical complex.

The variable `cc` has to contain a cubical complex, and the function returns a vector of coordinates for the vertices of the complex, that can then be used for plotting. “‘

source

15.4 Golden Ratio Rectangles

ConleyDynamics.create_golden_ratio_rectangle – Function.

```
create_golden_ratio_rectangle(bmin, bmax;
                              sigma      = (sqrt(5)-1)/2,
                              sdfunction = (_,_) -> false,
                              sdmin      = 0,
                              sdmax      = 7,
                              p          = 2)
```

Create a planar `EuclideanComplex` over a field of characteristic `p` by recursively subdividing the rectangle `[bmin[1],bmax[1]] × [bmin[2],bmax[2]]` using the golden-ratio subdivision rule.

Subdivision rule

At each recursive step the current box is examined:

- If its recursion depth is > `sdmax`, it becomes a leaf (hard cap).
- If its depth is < `sdmin`, it is always split (guaranteed minimum).
- Otherwise `sdfunction([xmin,ymin],[xmax,ymax])` decides: `true` splits, `false` makes it a leaf.

The default `sdfunction` always returns `false`, so with default arguments exactly `sdmin` levels of uniform subdivision are performed.

When a box is split, the longer edge is divided into two segments of lengths `σ·a` and `(1-σ)·a`; which piece goes to which sub-box is chosen uniformly at random. The default `sigma = (√5-1)/2 ≈ 0.618` is the golden ratio minus 1: starting from a square it produces sub-rectangles whose aspect ratios stay within the three values {1, φ, φ²} under further subdivision (see Fig. 3.1 of Cochran et al., SISC 2013).

Return value

An `EuclideanComplex` whose 2-cells are all rectangles (leaf boxes). The boundary of each rectangle is made up of the minimal set of line segments whose endpoints come from the corners of adjacent leaf boxes, oriented exactly as in a cubical complex: each bottom or right segment has coefficient +1 and each top or left segment has coefficient -1 in the boundary of its enclosing rectangle.

source

15.5 Cell Subset Helper Functions

`ConleyDynamics.convert_cells` – Function.

```
convert_cells(lc::AbstractComplex, cl::Vector{Int})
```

Convert cell list `cl` in the Lefschetz complex `lc` from index form to label form.

source

```
convert_cells(lc::AbstractComplex, cl::Vector{String})
```

Convert cell list `cl` in the Lefschetz complex `lc` from label form to index form.

source

`ConleyDynamics.convert_cellsubsets` – Function.

```
convert_cellsubsets(lc::AbstractComplex, clsub::Vector{Vector{Int}})
```

Convert CellSubsets `clsub` in the Lefschetz complex `lc` from index form to label form.

source

```
convert_cellsubsets(lc::AbstractComplex, clsub::Vector{Vector{String}})
```

Convert CellSubsets `clsub` in the Lefschetz complex `lc` from label form to index form.

source

`ConleyDynamics.cellsubsets_to_cells` – Function.

```
cellsubsets_to_cells(lc::AbstractComplex, css::CellSubsets)
```

Convert the cell subsets `css` into a vector of cells, by simply taking the union of all subsets. The cells are returned in ordered form, based on their index form in `lc`.

source

`ConleyDynamics.cellsubset_bounding_box` – Function.

```
cellsubset_bounding_box(lc, coords, csubset)
```

Compute the bounding box for a cell subset.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, this function computes the smallest enclosing box for the closure of the cell subset given in `csubset`. The function returns the coordinates `bmin` and `bmax` of the minimal and maximal corners of the box, respectively.

source

```
cellsubset_bounding_box(ec, csubset)
```

Compute the bounding box for a cell subset of a `EuclideanComplex`.

For the `EuclideanComplex` `ec`, this function computes the smallest enclosing box for the closure of the cell subset given in `csubset`. Vertex coordinates are taken directly from the embedded `ec.coords` field. The function returns the coordinates `bmin` and `bmax` of the minimal and maximal corners of the box, respectively.

source

`ConleyDynamics.cellsubset_distance` – Function.

```
cellsubset_distance(lc, coords, csubset, dpoint)
```

Compute the distance of a cell subset from a point.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, this function computes the smallest distance of the vertices in the closure of the cell subset given in `csubset` from the point given in `dpoint`.

source

```
cellsubset_distance(ec, csubset, dpoint)
```

Compute the distance of a cell subset from a point for a `EuclideanComplex`.

For the `EuclideanComplex` `ec`, this function computes the smallest distance of the vertices in the closure of the cell subset given in `csubset` from the point given in `dpoint`. Vertex coordinates are taken directly from the embedded `ec.coords` field.

source

`ConleyDynamics.cellsubset_planar_area` – Function.

```
cellsubset_planar_area(lc, coords, csubset)
```

Compute the area of a planar cell subset.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, this function computes the area of the cell subset given in `csubset`. The function assumes that the complex is two-dimensional and that the maximal 2-cells in the cell subset are all polygonal with straight boundary edges. If these conditions are not met an error is raised.

source

```
cellsubset_planar_area(ec, csubset)
```

Compute the area of a planar cell subset of a `EuclideanComplex`.

For the `EuclideanComplex` `ec`, this function computes the area of the cell subset given in `csubset`. The function assumes that the complex is two-dimensional and that the maximal 2-cells in the cell subset are all polygonal with straight boundary edges. Vertex coordinates for every cell are taken directly from the embedded `ec.coords` field, so the function works correctly even when vertex cells are no longer present in the complex.

source

`ConleyDynamics.locate_planar_cellsubsets` – Function.

```
locate_planar_cellsubsets(lc, coords, csubsets, rmin, rmax)
```

Locate cell subsets relative to a planar rectangle.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, and the cell subsets in `csubsets` this function extracts the indices of all cell subset closures which lie in the interior, or intersect the boundary of the rectangle specified by the minimal and maximal corners `rmin` and `rmax`, respectively. The function returns

- `indexI::Vector{Int}`: indices of cell subset closures inside the rectangle,
- `indexB::Vector{Int}`: indices of cell subset closures which intersect the rectangle boundary.

source

```
locate_planar_cellsubsets(lc, coords, csubsets, c, r)
```

Locate cell subsets relative to a planar circle.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, and the cell subsets in `csubsets` this function extracts the indices of all cell subset closures which lie in the interior, or intersect the boundary of the circle specified by the center point `c` and radius `r`. The function returns

- `indexI::Vector{Int}`: indices of cell subset closures inside the circle,
- `indexB::Vector{Int}`: indices of cell subset closures which intersect the circle.

source

```
locate_planar_cellsubsets(ec, csubsets, rmin, rmax)
```

Locate cell subsets relative to a planar rectangle for a `EuclideanComplex`.

For the `EuclideanComplex` `ec` and the cell subsets in `csubsets` this function extracts the indices of all cell subset closures which lie in the interior, or intersect the boundary of the rectangle specified by the minimal and maximal corners `rmin` and `rmax`, respectively. Vertex coordinates are taken directly from the embedded `ec.coords` field. The function returns

- `indexI::Vector{Int}`: indices of cell subset closures inside the rectangle,
- `indexB::Vector{Int}`: indices of cell subset closures which intersect the rectangle boundary.

source

```
locate_planar_cellsubsets(ec, csubsets, c, r)
```

Locate cell subsets relative to a planar circle for a `EuclideanComplex`.

For the `EuclideanComplex` `ec` and the cell subsets in `csubsets` this function extracts the indices of all cell subset closures which lie in the interior, or intersect the boundary of the circle specified by the center point `c` and radius `r`. Vertex coordinates are taken directly from the embedded `ec.coords` field. The function returns

- `indexI::Vector{Int}`: indices of cell subset closures inside the circle,
- `indexB::Vector{Int}`: indices of cell subset closures which intersect the circle.

source

`ConleyDynamics.cellsubset_location_rectangle` – Function.

```
cellsubset_location_rectangle(lc, coords, csubset, rmin, rmax)
```

Determine the location of a cell subset relative to a rectangle.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, this function determines the location of the closure of the cellsubset given in `csubset` relative to the rectangle specified by the minimal and maximal corners `rmin` and `rmax`, respectively. The function returns

- 1 if the set lies in the interior of the rectangle,
- 2 of the set intersects the rectangle boundary, and
- 3 if the set lies in the exterior of the rectangle.

source

```
cellsubset_location_rectangle(ec, csubset, rmin, rmax)
```

Determine the location of a cell subset relative to a rectangle for a `EuclideanComplex`.

For the `EuclideanComplex` `ec`, this function determines the location of the closure of the cellsubset given in `csubset` relative to the rectangle specified by the minimal and maximal corners `rmin` and `rmax`, respectively. Vertex coordinates are taken directly from the embedded `ec.coords` field. The function returns

- 1 if the set lies in the interior of the rectangle,
- 2 if the set intersects the rectangle boundary, and
- 3 if the set lies in the exterior of the rectangle.

source

`ConleyDynamics.cellsubset_location_circle` – Function.

```
cellsubset_location_circle(lc, coords, csubset, c, r)
```

Determine the location of a cell subset relative to a circle.

For the Lefschetz complex `lc`, with vertex coordinates `coords`, this function determines the location of the closure of the cellsubset given in `csubset` relative to the circle specified by the center point `c` and the radius `r`. The function returns

- 1 if the set lies in the interior of the circle,
- 2 of the set intersects the circle, and
- 3 if the set lies in the exterior of the circle.

source

```
cellsubset_location_circle(ec, csubset, c, r)
```

Determine the location of a cell subset relative to a circle for a `EuclideanComplex`.

For the `EuclideanComplex` `ec`, this function determines the location of the closure of the cellsubset given in `csubset` relative to the circle specified by the center point `c` and the radius `r`. Vertex coordinates are taken directly from the embedded `ec.coords` field. The function returns

- 1 if the set lies in the interior of the circle,
- 2 if the set intersects the circle, and
- 3 if the set lies in the exterior of the circle.

source

15.6 Geometry Helper Functions

`ConleyDynamics.convert_planar_coordinates` – Function.

```
convert_planar_coordinates(coords::Vector{<:Vector{<:Real}},
                           p0::Vector{<:Real},
                           p1::Vector{<:Real})
```

Convert a given collection of planar coordinates.

The vector `coords` contains pairs of coordinates, which are then transformed to fit into the box with vertices `p0 = (p0x,p0y)` and `p1 = (p1x,p1y)`. It is assumed that `p0` denotes the lower left box corner, while `p1` is the upper right corner. The function shifts and scales the coordinates in such a way that every side of the box contains at least one point. Upon completion, it returns a new coordinate vector `coordsNew`.

More precisely, if the x-coordinates are spanning the interval `[xmin,xmax]` and the y-coordinates span `[ymin,ymax]`, then the point `(x,y)` is transformed to `(xn,yn)` with:

- `xn = p0x + (p1x-p0x) * (x-cxmin) / (cxmax-cxmin)`
- `yn = p0y + (p1y-p0y) * (y-cymin) / (cymax-cymin)`

source

`ConleyDynamics.convert_spatial_coordinates` – Function.

```
convert_spatial_coordinates(coords::Vector{<:Vector{<:Real}},
                            p0::Vector{<:Real},
                            p1::Vector{<:Real})
```

Convert a given collection of spatial coordinates.

The vector `coords` contains triples of coordinates, which are then transformed to fit into the box with vertices `p0 = (p0x,p0y,p0z)` and `p1 = (p1x,p1y,p1z)`. It is assumed that each coordinate of `p0` is strictly smaller than the corresponding coordinate of `p1`. The function shifts and scales the coordinates in such a way that every side of the box contains at least one point. Upon completion, it returns a new coordinate vector `coordsNew`.

More precisely, if the coordinates span the box `[xmin,xmax] x [ymin,ymax] x [zmin,zmax]`, then the point `(x,y,z)` is transformed to `(xn,yn,zn)` with:

- `xn = p0x + (p1x-p0x) * (x-cxmin) / (cxmax-cxmin)`
- `yn = p0y + (p1y-p0y) * (y-cymin) / (cymax-cymin)`
- `zn = p0z + (p1z-p0z) * (z-czmin) / (czmax-czmin)`

source

`ConleyDynamics.signed_distance_rectangle` – Function.

```
signed_distance_rectangle(p, rmin, rmax)
```

Compute the signed distance from a point to a rectangle.

The point in question is given as `p`. The rectangle has to be parallel to the coordinate axes and is given via its lower left corner `rmin` and its upper right corner `rmax`. The function returns the signed distance, which is negative inside the rectangle, and positive outside.

source

`ConleyDynamics.signed_distance_circle` – Function.

```
signed_distance_circle(p, c, r)
```

Compute the signed distance from a point to a circle.

The point is specified as `p`, while the circle is given by its center point `c` and radius `r`. The function returns the signed distance, which is negative inside the circle, and positive outside.

source

`ConleyDynamics.segment_intersects_rectangle` – Function.

```
segment_intersects_rectangle(p1, p2, rmin, rmax)
```

Determine whether a line segment intersects a rectangle boundary.

The endpoints of the line segment are specified in the form `p1`, `p2`. The rectangle is parallel to the coordinate axes and is given via its lower left corner `rmin` and its upper right corner `rmax`. The function returns `true` if the line segment intersects the rectangle boundary, otherwise `false`.

source

`ConleyDynamics.segment_intersects_circle` – Function.

```
segment_intersects_circle(p1, p2, c, r)
```

Determine whether a line segment intersects a circle.

The endpoints of the line segment are specified in the form `p1`, `p2`, while the circle is given by its center point `c` and radius `r`. The function returns `true` if the line segment intersects the circle, otherwise it returns `false`.

source

`ConleyDynamics.split_segments` – Function.

```
split_segments(segments; eps_snap=1e-6, eps_abs=1e-10)
    -> (Vector{Vector{Float64}}, Vector{Tuple{Int,Int}})
```

Compute a planar straight-line arrangement from a collection of line segments: the output covers the same point-set as the input but no two output segments share an interior point (no crossings, no overlaps).

Arguments

- `segments`: a `Vector` of segments, where each segment is a length-2 vector of 2D endpoints, e.g. `[[x1, y1], [x2, y2]]`.

Keyword arguments

- `eps_snap = 1e-6`: distance threshold for snap-rounding. Any two candidate points (original endpoints or computed intersection points) within this Euclidean distance are identified as the same vertex.
- `eps_abs = 1e-10`: arithmetic guard for degenerate configurations (parallel or zero-length segments). Automatically raised to `eps_snap * 1e-4` if smaller.

Returns

A tuple `(points, edges)` where:

- `points :: Vector{Vector{Float64}}` is the deduplicated vertex list (centroids of snap-rounding clusters), and
- `edges :: Vector{Tuple{Int,Int}}` lists each output sub-segment as a pair `(i, j)` with `i < j`, using 1-based indices into `points`.

Algorithm

1. **Candidate collection** (parallelised): all 2n endpoints and every pairwise intersection point are collected into a flat candidate array. Each thread accumulates into its own private buffer to avoid locking.
2. **Snap-rounding**: all candidate points are clustered with Union-Find so that any two within distance `eps_snap` are identified. The representative of each cluster is its centroid.
3. **Segment splitting** (parallelised): each input segment is split at every snapped candidate point that lies on it (within `eps_snap`), yielding a list of consecutive cluster-index pairs.
4. **Output assembly**: duplicate cluster-index pairs are removed and returned in canonical `(i, j)` form with `i < j`.

Threading

The two O(n²) loops are parallelised with `Threads.@threads`. Start Julia with `--threads auto` or set `JULIA_NUM_THREADS` to use all available cores; the algorithm is correct with any number of threads, including 1.

Example

```
using ConleyDynamics

# Two crossing diagonals of the unit square
segs = [[[0.0, 0.0], [1.0, 1.0]],
        [[0.0, 1.0], [1.0, 0.0]]]

points, edges = split_segments(segs)
# points has 5 entries: the 4 corners plus the crossing at [0.5, 0.5]
# edges  has 4 entries: one sub-segment on each side of the crossing
```

source

Chapter 16

Homology Functions

16.1 Regular Homology

`ConleyDynamics.homology` – Function.

```
homology(lc::AbstractComplex; [algorithm::String])
```

Compute the homology of a Lefschetz complex.

The homology is returned as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`. The computations are performed over the field associated with the Lefschetz complex boundary matrix. It is possible to invoke the computation with one of three different algorithms, by passing the optional argument `algorithm::String`:

- `algorithm = "matrix"` selects the standard matrix algorithm, as for example described in the book by Edelsbrunner, Harer,
- `algorithm = "morse"` selects an algorithm based on Morse reductions,
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions.

By default, the function uses the Morse reduction algorithm `morse`.

source

```
homology(lc::AbstractComplex, subc::Cells; [algorithm::String])
```

Compute the homology of a Lefschetz complex, given as a subcomplex.

This method of the `homology` function computes the homology of the Lefschetz complex given by the locally closed subset `subc` of the ambient Lefschetz complex `lc`. An error is raised if `subc` is not locally closed. The homology is returned as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`. The computations are performed over the field associated with the Lefschetz complex boundary matrix. It is possible to invoke the computation with one of three different algorithms, by passing the optional argument `algorithm::String`:

- `algorithm = "matrix"` selects the standard matrix algorithm, as for example described in the book by Edelsbrunner, Harer,

- `algorithm = "morse"` selects an algorithm based on Morse reductions,
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions.

By default, the function uses the Morse reduction algorithm `morse`.

source

`ConleyDynamics.relative_homology` – Function.

```
relative_homology(lc::AbstractComplex,subc::Cells)
```

Compute the relative homology of a Lefschetz complex with respect to a subcomplex. The computation is performed over the field associated with the Lefschetz boundary matrix.

The subcomplex is the closure of the cells in `subc`, which can be given either as indices or labels. The homology is returned as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`.

source

```
relative_homology(lc::AbstractComplex,subc::Cells,subc0::Cells)
```

Compute the relative homology of a Lefschetz complex with respect to a subcomplex. The computation is performed over the field associated with the Lefschetz boundary matrix.

In this implementation, relative homology of the pair `cl(subc), cl(subc0))` is computed. An error is raised if `cl(subc0)` is not a subset of `cl(subc)`. The homology is returned as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`.

source

16.2 Persistent Homology

`ConleyDynamics.persistent_homology` – Function.

```
persistent_homology(lc::AbstractComplex, filtration::Vector{Int};
                    [algorithm::String])
```

Compute the persistent homology of a Lefschetz complex filtration over the field associated with the Lefschetz complex boundary matrix.

The function returns the two values

- `phsingles::Vector{Vector{Int}}`
- `phpairs::Vector{Vector{Tuple{Int,Int}}}`

It assumes that the order given by the filtration values is admissible, i.e., the permuted boundary matrix is strictly upper triangular. The function returns the starting filtration values for infinite length persistence intervals in `phsingles`, and the birth- and death-filtration values for finite length persistence intervals in `phpairs`. It is possible to invoke the persistent homology computation with one of three different algorithms, by passing the optional argument `algorithm::String`:

- `algorithm = "matrix"` selects the standard matrix algorithm, as for example described in the book by Edelsbrunner, Harer,
- `algorithm = "morse"` selects an algorithm based on Morse reductions,
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions.

By default, the function uses the Morse reduction algorithm `morse`.

source

16.3 Reduction Algorithms

`ConleyDynamics.ph_morse_reduce` – Function.

```
ph_morse_reduce(lc::AbstractComplex, filtration::Vector{Int};
                [parallel::Bool])
```

Compute the persistent homology of a Lefschetz complex filtration over the field associated with the Lefschetz complex boundary matrix.

The function returns the two values

- `phsingles::Vector{Vector{Int}}`
- `phpairs::Vector{Vector{Tuple{Int,Int}}}`

It assumes that the order given by the filtration values is admissible, i.e., the permuted boundary matrix is strictly upper triangular. The function returns the starting filtration values for infinite length persistence intervals in `phsingles`, and the birth- and death-filtration values for finite length persistence intervals in `phpairs`. This function uses an algorithm based on Morse reductions. If the optional argument `parallel=true` is added, then a parallelized version is used.

source

`ConleyDynamics.ph_matrix_reduce` – Function.

```
ph_matrix_reduce(lc::AbstractComplex, filtration::Vector{Int})
```

Compute the persistent homology of a Lefschetz complex filtration over the field associated with the Lefschetz complex boundary matrix.

The function returns the two values

- `phsingles::Vector{Vector{Int}}`
- `phpairs::Vector{Vector{Tuple{Int,Int}}}`

It assumes that the order given by the filtration values is admissible, i.e., the permuted boundary matrix is strictly upper triangular. The function returns the starting filtration values for infinite length persistence intervals in `phsingles`, and the birth- and death-filtration values for finite length persistence intervals in `phpairs`.

source

Chapter 17

Conley Theory Functions

17.1 Multivector Fields

`ConleyDynamics.create_mvf_hull` – Function.

```
create_mvf_hull(lc::AbstractComplex, mvfbase::Vector{Vector{Int}})
```

Create the smallest multivector field containing the given sets.

The resulting multivector field has the property that every set of the form `mvfbase[k]` is contained in a minimal multivector. Notice that these sets do not have to be disjoint, and that not even their locally closed hulls have to be disjoint. In the latter case, this leads to two such sets having to be contained in the same multivector. If the sets in `mvfbase` are poorly chosen, one might end up with extremely large multivectors due to the above potential merging of locally closed hulls. The merging only recomputes the locally closed hulls of multivectors which have grown, so its cost is governed by the sizes of the merged multivectors, and not by the size of the complex.

source

```
create_mvf_hull(lc::AbstractComplex, mvfbase::Vector{Vector{String}})
```

Create the smallest multivector field containing the given sets.

The resulting multivector field has the property that every set of the form `mvfbase[k]` is contained in a minimal multivector. Notice that these sets do not have to be disjoint, and that not even their locally closed hulls have to be disjoint. In the latter case, this leads to two such sets having to be contained in the same multivector. If the sets in `mvfbase` are poorly chosen, one might end up with extremely large multivectors due to the above potential merging of locally closed hulls.

source

`ConleyDynamics.mvf_information` – Function.

```
mvf_information(lc::AbstractComplex, mvf::CellSubsets)
```

Extract basic information about a multivector field.

The input argument `lc` contains the Lefschetz complex, and `mvf` describes the multivector field. The function returns the information in the form of a `Dict{String,Any}`. You can use the command keys to see the keyset of the return dictionary:

- `"N mv"`: Number of multivectors
- `"N critical"`: Number of critcal multivectors
- `"N regular"`: Number of regular multivectors
- `"Lengths critical"`: Length distribution of critical multivectors
- `"Lengths regular"`: Length distribution of regular multivectors

In the last two cases, the dictionary entries are vectors of pairs `(length,frequency)`, where each pair indicates that there are `frequency` multivectors of length `length`.

source

`ConleyDynamics.mvf_length` – Function.

```
mvf_length(lc::AbstractComplex, mvf::CellSubsets)
```

Number of multivectors in a multivector field.

The function returns the number of all multivectors in a given multivector field. This not only includes the multivectors explicitly listed in the vector `mvf`, but also the number of all singletons, which are trivially critical multivectors.

Example

```
julia> lc, mvf= example_forman1d();

julia> mvf
5-element Vector{Vector{String}}:
 ["A", "AD"]
 ["D", "CD"]
 ["C", "AC"]
 ["B", "BE"]
 ["E", "EF"]

julia> mvf_length(lc, mvf)
8
```

source

`ConleyDynamics.mvf_critical` – Function.

```
mvf_critical(lc::AbstractComplex, mvf::CellSubsets)
```

Determines all critical multivectors.

The function returns all critical multivectors of the specified multivector field `mvf`. This includes both the implied singletons, as well as the multivectors listed in `mvf` which have nontrivial homology.

Example

```
julia> lc, mvf= example_forman1d();

julia> mvf
5-element Vector{Vector{String}}:
 ["A", "AD"]
 ["D", "CD"]
 ["C", "AC"]
 ["B", "BE"]
 ["E", "EF"]

julia> mvf_critical(lc, mvf)
3-element Vector{Vector{String}}:
 ["F"]
 ["BF"]
 ["DE"]
```

source

ConleyDynamics.mvf_regular – Function.

```
mvf_regular(lc::AbstractComplex, mvf::CellSubsets)
```

Determines all regular multivectors.

The function returns all regular multivectors of the specified multivector field `mvf`, i.e., all multivectors whose homology groups are all trivial.

Example

```
julia> lc, mvf= example_forman1d();

julia> mvf
5-element Vector{Vector{String}}:
 ["A", "AD"]
 ["D", "CD"]
 ["C", "AC"]
 ["B", "BE"]
 ["E", "EF"]

julia> mvf_regular(lc, mvf)
5-element Vector{Vector{String}}:
 ["A", "AD"]
 ["B", "BE"]
 ["C", "AC"]
 ["D", "CD"]
 ["E", "EF"]
```

source

ConleyDynamics.mvf_is_acyclic – Function.

```
mvf_is_acyclic(lc::AbstractComplex, mvf::CellSubsets)
```

Determine whether a multivector field is acyclic.

The function returns a boolean which indicates whether a multivector field forms an acyclic partition of the Lefschetz complex `lc` or not.

source

`ConleyDynamics.mvf_forward_orbit` – Function.

```
mvf_forward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                  sources::Cells)
```

Determine the forward orbit of a collection of cells.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cells in `sources` in forward time. Critical cells in the forward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_forward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                  source::Cell)
```

Determine the forward orbit of a single cell.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cell `source` in forward time. Critical cells in the forward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_forward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                  sources::Cells, nsteps::Int)
```

Determine a time-restricted forward orbit of a collection of cells.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cells in `sources` in at most `nsteps` transitions in forward time. Critical cells in the forward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_forward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                  source::Cell, nsteps::Int)
```

Determine a time-restricted forward orbit of a single cell.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cell `source` in at most `nsteps` transitions in forward time. Critical cells in the forward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

ConleyDynamics.mvf_backward_orbit – Function.

```
mvf_backward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                   sources::Cells)
```

Determine the backward orbit of a collection of cells.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cells in `sources` in backward time. In other words, it contains all multivectors that flow to `sources` in forward time. Critical cells in the backward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_backward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                   source::Cell)
```

Determine the backward orbit of a single cell.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cell `source` in backward time. In other words, it contains all multivectors that flow to `source` in forward time. Critical cells in the backward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_backward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                   sources::Cells, nsteps::Int)
```

Determine a time-restricted backward orbit of a collection of cells.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cells in `sources` in at most `nsteps` transitions in backward time. In other words, it contains all multivectors that flow to `sources` in at most `nsteps` transitions in forward time. Critical cells in the backward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_backward_orbit(lc::AbstractComplex, mvf::CellSubsets,
                   source::Cell, nsteps::Int)
```

Determine a time-restricted backward orbit of a single cell.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cell `source` in at most `nsteps` transitions in backward time. In other words, it contains all multivectors that flow to `sources` in at most `nsteps` transitions in forward time. Critical cells in the backward orbit will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

ConleyDynamics.mvf_neighborhood – Function.

```
mvf_neighborhood(lc::AbstractComplex, mvf::CellSubsets,
                 sources::Cells, n::Int)
```

Determine a multivector neighborhood of a set of cells.

The function returns all multivectors of the multivector field `mvf` that form a neighborhood of the given cells with a collar width of `n`. More precisely, for `n = 0` one obtains the multivectors intersecting the cells in `sources`, while for `n >= 1` the function returns all multivectors from level `n-1`, together with all multivectors whose closure intersects the closure of the level `n-1` neighborhood. Critical cells in this neighborhood will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

```
mvf_neighborhood(lc::AbstractComplex, mvf::CellSubsets,
                 source::Cell, n::Int)
```

Determine a multivector neighborhood of a cell.

The function returns all multivectors of the multivector field `mvf` that can be reached from the cell `source` in at most `n` steps forward or backward time. In other words, it computes a neighborhood of the multivector containing the cell `source` with a thickness of `n` layers of multivectors. Critical cells in this neighborhood will be returned as singletons. The return type of the multivectors is the same as the type of `mvf`.

source

`ConleyDynamics.extract_multivectors` – Function.

```
extract_multivectors(lc::AbstractComplex, mvf::Vector{Vector{Int}},
                     scells::Vector{Int})
```

Extract all multivectors containing a provided selection of cells.

The function returns all multivectors which contain at least one of the cells in the input vector `scells`. The return argument has type `Vector{Vector{Int}}`.

source

```
extract_multivectors(lc::AbstractComplex, mvf::Vector{Vector{String}},
                     scells::Vector{String})
```

Extract all multivectors containing a provided selection of cells.

The function returns all multivectors which contain at least one of the cells in the input vector `scells`. The return argument has type `Vector{Vector{String}}`.

source

17.2 Multivector Fields via Flows

`ConleyDynamics.create_planar_mvf` – Function.

```
create_planar_mvf(lc::AbstractComplex, coords::Vector{<:Vector{<:Real}}, vf)
```

Create a planar multivector field from a regular vector field.

The function expects a planar Lefschetz complex `lc` and a coordinate vector `coords` of coordinates for all the 0-dimensional cells in the complex. Moreover, the underlying vector field is specified by the function `vf(z::Vector{Float64})::Vector{Float64}`, where both the input and output vectors have length two. The function `create_planar_mvf` returns a multivector field `mvf` on `lc`, which can then be further analyzed using for example the function `connection_matrix`.

The input data `lc` and `coords` can be generated using one of the following methods:

- `create_cubical_rectangle`
- `create_simplicial_rectangle`
- `create_simplicial_delaunay`

In each case, the provided coordinate vector can be transformed to the correct bounding box values using the conversion function `convert_planar_coordinates`.

Example 1

Suppose we define a sample vector field using the commands

```
function samplevf(x::Vector{Float64})
    #
    # Sample vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    y1 = x1 * (1.0 - x1*x1 - 3.0*x2*x2)
    y2 = x2 * (1.0 - 3.0*x1*x1 - x2*x2)
    return [y1, y2]
end
```

One first creates a triangulation of the enclosing box, which in this case is given by `[-2,2] x [-2,2]` using the commands

```
n = 21
lc, coords = create_simplicial_rectangle(n,n);
coordsN = convert_planar_coordinates(coords,[-2.0,-2.0],[2.0,2.0]);
```

The multivector field is then generated using

```
mvf = create_planar_mvf(lc,coordsN,samplevf);
```

and the commands

```
cm = connection_matrix(lc, mvf);
cm.conley
full_from_sparse(cm.matrix)
```

finally show that this vector field gives rise to a Morse decomposition with nine Morse sets, and twelve connecting orbits. Using the commands

```
fname = "morse_test.pdf"
plot_planar_simplicial_morse(lc, coordsN, fname, cm.morse, pv=true)
```

these Morse sets can be visualized. The image will be saved in `fname`.

Example 2

An example with periodic orbits can be generated using the vector field

```
function samplevf2(x::Vector{Float64})
    #
    # Sample vector field with nontrivial Morse decomposition
    #
    x1, x2 = x
    c0 = x1*x1 + x2*x2
    c1 = (c0 - 4.0) * (c0 - 1.0)
    y1 = -x2 + x1 * c1
    y2 =  x1 + x2 * c1
    return [-y1, -y2]
end
```

The Morse decomposition can now be computed via

```
n2 = 51
lc2, coords2 = create_cubical_rectangle(n2,n2);
coords2N = convert_planar_coordinates(coords2,[-4.0,-4.0],[4.0,4.0]);
mvf2 = create_planar_mvf(lc2,coords2N,samplevf2);
cm2 = connection_matrix(lc2, mvf2);
cm2.conley
cm2.poset
full_from_sparse(cm2.matrix)

fname2 = "morse_test2.pdf"
plot_planar_cubical_morse(lc2, fname2, cm2.morse, pv=true)
```

In this case, one obtains three Morse sets: One is a stable equilibrium, one is an unstable periodic orbit, and the last is a stable periodic orbit.

source

```
create_planar_mvf(ec::EuclideanComplex, vf)
```

Create a planar multivector field from a `EuclideanComplex` and a regular vector field `vf`.

This method extracts the vertex coordinates from the embedded coordinates of `ec` and delegates to the standard `create_planar_mvf` implementation. The complex has to be embedded in two dimensions, otherwise an `ArgumentError` is raised. For complexes in three dimensions, use `create_spatial_mvf`.

source

`ConleyDynamics.create_spatial_mvf` – Function.

```
create_spatial_mvf(lc::AbstractComplex, coords::Vector{<:Vector{<:Real}}, vf)
```

Create a spatial multivector field from a regular vector field.

The function expects a three-dimensional Lefschetz complex `lc` and a coordinate vector `coords` of coordinates for all the 0-dimensional cells in the complex. Moreover, the underlying vector field is specified by the function `vf(z::Vector{Float64})::Vector{Float64}`, where both the input and output vectors have length three. The function `create_spatial_mvf` returns a multivector field `mvf` on `lc`, which can then be further analyzed using for example the function `connection_matrix`.

The input data `lc` and `coords` has to be of one of the following two types:

- `lc` is a tetrahedral mesh of a region in three dimensions. In other words, the underlying Lefschetz complex is in fact a simplicial complex, and the vector `coords` contains the vertex coordinates.
- `lc` is a three-dimensional cubical complex, i.e., it is the closure of a collection of three-dimensional cubes in space. The vertex coordinates can be slightly perturbed from the original position in the cubical lattice, as long as the overall structure of the complex stays intact. In that case, the faces are interpreted as Bezier surfaces with straight edges.

Example 1

Suppose we define a sample vector field using the commands

```
function samplevf(x::Vector{Float64})
    #
    # Sample vector field with nontrivial Morse decomposition
    #
    x1, x2, x3 = x
    y1 = x1 * (1.0 - x1*x1)
    y2 = -x2
    y3 = -x3
    return [y1, y2, y3]
end
```

One first creates a cubical complex covering the interesting dynamics, say the trapping region `[-1.5,1.5] x [-1,1] x [-1,1]`, using the commands

```
lc, coords = create_cubical_box(3,3,3);
coordsN = convert_spatial_coordinates(coords,[-1.5,-1.0,-1.0],[1.5,1.0,1.0]);
```

The multivector field is then generated using

```
mvf = create_spatial_mvf(lc,coordsN,samplevf);
```

and the commands

```
cm = connection_matrix(lc, mvf);
cm.conley
full_from_sparse(cm.matrix)
```

finally show that this vector field gives rise to a Morse decomposition with three Morse sets, and two connecting orbits.

source

```
create_spatial_mvf(ec::EuclideanComplex, vf)
```

Create a spatial multivector field from a `EuclideanComplex` and a regular vector field `vf`.

This method extracts the vertex coordinates from the embedded coordinates of `ec` and delegates to the standard `create_spatial_mvf` implementation. The complex has to be three-dimensional and embedded in three dimensions, such as a tetrahedral mesh created by `tetgen_to_simplicial` or `gmsh_to_simplicial`, or a cubical complex created by `create_cubical_box`. Otherwise, an `ArgumentError` is raised. In particular, triangulated surfaces in three dimensions are not supported.

source

`ConleyDynamics.planar_nontransverse_edges` – Function.

```
planar_nontransverse_edges(lc::AbstractComplex, coords::Vector{<:Vector{<:Real}}, vf;
                           npts::Int=100)
```

Find all edges of a planar Lefschetz complex which are not flow transverse.

The Lefschetz complex is given in `lc`, the coordinates of all vertices of the complex in `coords`, and the vector field is specified in `vf`. The optional parameter `npts` determines how many points along an edge are evaluated for the transversality check. The function returns a list of nontransverse edges as `Vector{Int}`, which contains the edge indices.

source

```
planar_nontransverse_edges(ec::EuclideanComplex, vf; npts::Int=100)
```

Find all edges of a planar `EuclideanComplex` which are not flow transverse.

The `EuclideanComplex` is given in `ec` and the vector field is specified in `vf`. The optional parameter `npts` determines how many points along an edge are evaluated for the transversality check. Edge endpoint coordinates are taken directly from the embedded `ec.coords` field. The function returns a list of nontransverse edges as `Vector{Int}`, which contains the edge indices.

source

17.3 Conley Index Computations

`ConleyDynamics.isoinvset_information` – Function.

```
isoinvset_information(lc::AbstractComplex, mvf::CellSubsets, iis::Cells)
```

Compute basic information about an isolated invariant set.

The input argument `lc` contains the Lefschetz complex, and `mvf` describes the multivector field. The isolated invariant set is specified in the argument `iis`. The function returns the information in the form of a `Dict{String,Any}`. The command `keys` can be used to see the keyset of the return dictionary. These describe the following information:

- `"Conley index"` contains the Conley index of the isolated invariant set.
- `"N multivectors"` contains the number of multivectors in the isolated invariant set.

source

`ConleyDynamics.conley_index` – Function.

```
conley_index(lc::AbstractComplex, subcomp::Vector{Int};
             [algorithm::String="auto", validate::Bool=true])
```

Determine the Conley index of a Lefschetz complex subset.

The Conley index of the locally closed set `subcomp` is returned as a vector `betti` of Betti numbers, where `betti[k]` is the Betti number in dimension `k-1`. The computations are performed over the field associated with the Lefschetz complex boundary matrix. Since the Conley index of a locally closed set is the Lefschetz homology of the set itself, only the cells in `subcomp` are used, and the computational cost does not depend on the size of `lc`. The following optional arguments are supported:

- `algorithm::String` selects the homology algorithm:
  - `algorithm = "auto"` chooses between `rank` and `morse` based on the size of `subcomp`, see below. This is the default.
  - `algorithm = "morse"` selects an algorithm based on Morse reductions,
  - `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions,
  - `algorithm = "matrix"` selects the standard matrix algorithm, as for example described in the book by Edelsbrunner, Harer,
  - `algorithm = "rank"` determines the Betti numbers from the ranks of the boundary maps restricted to `subcomp`, which are computed via row reduction. This avoids the construction of a subcomplex, and it is the fastest choice for small sets such as individual multivectors, while `morse` is much faster for large sets.
- `validate::Bool` determines whether the function checks that `subcomp` is locally closed, and raises an error otherwise. The default is `true`. Passing `validate=false` skips this check, which should only be done if it is known that the set is locally closed, since otherwise the result is meaningless.

All algorithms return the same Betti numbers, they only differ in their speed. The choice made by `algorithm = "auto"` is based on the following observations. The `rank` algorithm row reduces the boundary matrix restricted to `subcomp`, without any further setup. For small sets this is very fast, but due to fill-in during the row reduction its cost grows roughly quadratically with the size of the set. The `morse` algorithm first extracts the subcomplex and constructs a Morse matching, which adds a fixed overhead, but afterwards its cost grows almost linearly. In benchmarks on cubical and simplicial complexes, `rank` is three to

seven times faster than `morse` for sets with up to about ten cells, which is the typical size of a multivector. The two algorithms take about the same time for sets with 100 to 150 cells, and for larger sets `morse` is faster by a quickly growing factor, for example by a factor of 5 to 10 for 1000 cells, and by a factor of more than 100 for 50000 cells. Therefore `auto` uses `rank` for sets with at most 100 cells, and `morse` for larger sets. Near this threshold the two algorithms differ by less than a factor of 1.5, so its precise value is not critical. The decision only uses the number of cells in `subcomp`, and therefore has no cost.

source

```
conley_index(lc::AbstractComplex, subcomp::Vector{String};
             [algorithm::String="auto", validate::Bool=true])
```

Determine the Conley index of a Lefschetz complex subset.

This method expects the set `subcomp` in label form, and accepts the same optional arguments `algorithm` and `validate` as the method for subsets in index form.

source

`ConleyDynamics.morse_sets` – Function.

```
morse_sets(lc::AbstractComplex, mvf::CellSubsets; poset::Bool=false)
```

Find the nontrivial Morse sets of a multivector field on a Lefschetz complex.

The input argument `lc` contains the Lefschetz complex, and `mvf` describes the multivector field. The function returns the nontrivial Morse sets as a `Vector{Vector{Int}}`. If the optional argument `poset=true` is added, then the function returns both the Morse sets and the adjacency matrix of the Hasse diagram of the underlying poset.

source

`ConleyDynamics.morse_interval` – Function.

```
morse_interval(lc::AbstractComplex, mvf::CellSubsets,
               ms::CellSubsets)
```

Find the isolated invariant set for a Morse set interval.

The input argument `lc` contains the Lefschetz complex, and `mvf` describes the multivector field. The collection of Morse sets are contained in`ms`. All of these sets should be Morse sets in the sense of being strongly connected components of the flow graph. (Nevertheless, this will be enforced in the function!) In other words, the sets in `ms` should be determined using the function `morse_sets`!

The function returns the smallest isolated invariant set which contains the Morse sets and their connections as a `Vector{Int}`.

source

`ConleyDynamics.restrict_dynamics` – Function.

```
restrict_dynamics(lc::AbstractComplex, mvf::CellSubsets, lcsub::Cells;
                  [validate::Bool=true])
```

Restrict a multivector field to a Lefschetz subcomplex.

For a given multivector field `mvf` on a Lefschetz complex `lc`, and a subcomplex which is given by the locally closed set represented by `lcsub`, create the associated Lefschetz subcomplex `lcreduced` and the induced multivector field `mvfreduced` on the subcomplex. The multivectors of the new multivector field are the intersections of the original multivectors and the subcomplex.

By default, the function raises an error if `lcsub` is not locally closed. This check can be skipped by passing the optional argument `validate=false`, which should only be done if it is already known that the set is locally closed, for example if it was computed using `morse_interval`.

source

`ConleyDynamics.remove_exit_set` – Function.

```
remove_exit_set(lc::AbstractComplex, mvf::CellSubsets)
```

Exit set removal for a multivector field on a Lefschetz subcomplex.

It is assumed that the Lefschetz complex `lc` is a topological manifold and that `mvf` contains a multivector field that is created via either `create_planar_mvf` or `create_spatial_mvf`. The function identifies cells on the boundary at which the flows exits the region covered by the Lefschetz complex. If this exit set is closed, we have found an isolated invariant set and the function returns a Lefschetz complex `lcr` restricted to it, as well as the restricted multivector field `mvfr`. If the exit set is not closed, a warning is displayed and the function returns the restricted Lefschetz complex and multivector field obtained by removing the closure of the exit set. In the latter case, unexpected results might be obtained.

source

17.4 Connection Matrix Computation

`ConleyDynamics.connection_matrix` – Function.

```
connection_matrix(lc::AbstractComplex, mvf::CellSubsets;
                  [algorithm::String],
                  [returnbasis::Bool])
```

Compute a connection matrix for the multivector field `mvf` on the Lefschetz complex `lc` over the field associated with the Lefschetz complex boundary matrix.

The function returns an object of type `ConleyMorseCM`. If the optional argument `returnbasis::Bool=true` is given, then the function also returns a dictionary which gives the basis for the connection matrix columns in terms of the original labels. Finally, it is possible to invoke the connection matrix computation with one of four different algorithms, by passing the optional argument `algorithm::String`:

- `algorithm = "DLMS"` selects the algorithm due to Dey, Lipinski, Mrozek, Slechta (SIAM Journal on Applied Dynamical Systems, 2024),

- `algorithm = "DHL"` selects the algorithm due to Dey, Haas, Lipinski (SIAM Journal on Applied Dynamical Systems, 2026),
- `algorithm = "HMS"` selects the algorithm due to Harker, Mischaikow, Spendlove (Journal of Applied and Computational Topology, 2021),
- `algorithm = "pmorse"` selects a parallelized algorithm based on Morse reductions.

By default, the function uses the parallelized Morse reduction algorithm `pmorse`.

Passing `returnbasis=true` automatically switches it to the slower matrix-based `algorithm = "DLMS"` instead.

source

`ConleyDynamics.cm_reduce_dlms24!` – Function.

```
cm_reduce_dlms24!(matrix::SparseMatrix, psetvec::Vector{Int};
                  [returnbasis::Bool],[returntm::Bool])
```

Compute the connection matrix.

This function uses the algorithm from the paper Dey, Lipinski, Mrozek, and Slechta (SIAM Journal on Applied Dynamical Systems, 2024). It assumes that `matrix` is upper triangular and filtered according to `psetvec`. Modifies the argument `matrix`.

Return values:

- `cmatrix`: Connection matrix
- `cmatrix_cols`: Columns of the connection matrix in the boundary
- `basisvecs` (optional): If the argument `returnbasis=true` is given, this returns information about the computed basis. The k-th entry of `basisvecs` is a vector containing the columns making up the k-th basis vector, which corresponds to column `cmatrix_cols[k]`.
- `tmatrix` (optional): If the argument `returntm=true` is given in addition to `returnbasis=true`, then instead of `basisvecs` the function returns the complete transformation matrix. In this case, `basicvecs` is not returned.

source

`ConleyDynamics.cm_reduce_dhl26!` – Function.

```
cm_reduce_dhl26!(matrix::SparseMatrix, psetvec::Vector{Int})
```

Compute the connection matrix.

This function uses the algorithm from the paper Dey, Haas, and Lipinski (SIAM Journal on Applied Dynamical Systems, 2026). Assumes that `matrix` is upper triangular and filtered according to `psetvec`. Modifies the argument `matrix`.

Return values:

- `cmatrix`: Connection matrix
- `cmatrix_cols`: Columns of the connection matrix in the boundary

source

ConleyDynamics.cm_reduce_hms21 – Function.

```
cm_reduce_hms21(matrix::SparseMatrix, psetvec::Vector{Int})
```

Compute the connection matrix.

This function uses the algorithm from the paper Harker, Mischaikow, Spendlove (Journal of Applied and Computational Topology, 2021). Assumes that `matrix` is upper triangular and filtered according to `psetvec`.

Return values:

- `cmatrix`: Connection matrix
- `cmatrix_cols`: Columns of the connection matrix in the boundary

source

ConleyDynamics.cm_reduce_pmorse26 – Function.

```
cm_reduce_pmorse26(matrix::SparseMatrix, psetvec::Vector{Int})
```

Compute the connection matrix.

This function uses a parallelized Morse matching algorithm. Assumes that `matrix` is upper triangular and filtered according to `psetvec`.

Return values:

- `cmatrix`: Connection matrix
- `cmatrix_cols`: Columns of the connection matrix in the boundary

source

17.5 Forman Vector Fields

ConleyDynamics.forman_critical_cells – Function.

```
forman_critical_cells(lc::AbstractComplex, fvf::CellSubsets)
```

Find all critical cells of a Forman vector field.

This function returns all critical cells of the Forman vector field `fvf` on the Lefschetz complex `lc`.

Example

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> lc = create_simplicial_complex(labels, simplices, p=5);

julia> fvf = [["ab","b"], ["bc","c"]];

julia> c = forman_critical_cells(lc, fvf);

julia> println(c)
["a", "d", "cd"]
```

source

ConleyDynamics.forman_all_cell_types – Function.

```
forman_all_cell_types(lc::AbstractComplex, fvf::CellSubsets)
```

Find all cell types of a Forman vector field.

This function returns all critical cells, all arrow sources, and all arrow targets of the Forman vector field `fvf` on the Lefschetz complex `lc`.

Example

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> lc = create_simplicial_complex(labels, simplices, p=5);

julia> fvf = [["ab","b"], ["bc","c"]];

julia> c, s, t = forman_all_cell_types(lc, fvf);

julia> println(c)
["a", "d", "cd"]

julia> println(s)
["b", "c"]

julia> println(t)
["ab", "bc"]
```

source

ConleyDynamics.forman_conley_maps – Function.

```
forman_conley_maps(lc::AbstractComplex, fvf::CellSubsets)
```

Compute the maps associated with the Conley complex of a Forman gradient field

This function returns the chain maps and chain homotopy associated with the Conley complex of a Forman gradient vector field. These maps are computed via the stabilized combinatorial flow. The function returns the following variables:

- `pp:SparseMatrix`: The chain equivalence which maps the Lefschetz complex to the Conley complex
- `jj:SparseMatrix`: The chain equivalence which maps the Conley complex to the Lefschetz complex
- `hh:SparseMatrix`: The chain homotopy between `jj*pp` and the identity
- `cc:Cells`: The list of critical cells which make up the Conley complex

Example

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> lc = create_simplicial_complex(labels, simplices, p=5);

julia> fvf = [["ab","b"], ["bc","c"]];

julia> pp, jj, hh, cc = forman_conley_maps(lc, fvf);

julia> sparse_show(pp, cc, lc.labels)
   ┆  a  b  c  d ab bc cd
---┆--------------------
 a┆  1  1  1  .  .  .  .
 d┆  .  .  .  1  .  .  .
cd┆  .  .  .  .  .  .  1

julia> sparse_show(jj, lc.labels, cc)
   ┆  a  d cd
---┆---------
 a┆  1  .  .
 b┆  .  .  .
 c┆  .  .  .
 d┆  .  1  .
ab┆  .  .  1
bc┆  .  .  1
cd┆  .  .  1

julia> sparse_show(hh, lc.labels, lc.labels)
   ┆  a  b  c  d ab bc cd
---┆--------------------
 a┆  .  .  .  .  .  .  .
 b┆  .  .  .  .  .  .  .
 c┆  .  .  .  .  .  .  .
 d┆  .  .  .  .  .  .  .
ab┆  .  4  4  .  .  .  .
bc┆  .  .  4  .  .  .  .
cd┆  .  .  .  .  .  .  .

julia> sparse_show(jj*pp - lc.boundary*hh - hh*lc.boundary)
 1 . . . . . .
 . 1 . . . . .
```

```
 . . 1 . . . .
 . . . 1 . . .
 . . . . 1 . .
 . . . . . 1 .
 . . . . . . 1
```

source

ConleyDynamics.forman_comb_flow – Function.

```
forman_comb_flow(lc::AbstractComplex, fvf::Vector{Vector{String}})
```

Compute Forman's combinatorial flow and the associated chain homotopy.

This function returns matrix representations of Forman's combinatorial flow `phi`, and of the associated chain homotopy `gamma`.

Example

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> lc = create_simplicial_complex(labels, simplices, p=5);

julia> fvf = [["ab","b"], ["bc","c"]];

julia> phi, gamma = forman_comb_flow(lc, fvf);

julia> full_from_sparse(gamma)
7×7 Matrix{Int64}:
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  4  0  0  0  0  0
 0  0  4  0  0  0  0
 0  0  0  0  0  0  0

julia> full_from_sparse(phi)
7×7 Matrix{Int64}:
 1  1  0  0  0  0  0
 0  0  1  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  1  0  0  0
 0  0  0  0  0  1  0
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1

julia> full_from_sparse(phi*phi)
7×7 Matrix{Int64}:
 1  1  1  0  0  0  0
 0  0  0  0  0  0  0
```

```
 0  0  0  0  0  0  0
 0  0  0  1  0  0  0
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1
```

source

```
forman_comb_flow(lc::AbstractComplex, fvf::Vector{Vector{String}})
```

Compute Forman's combinatorial flow and the associated chain homotopy.

This function returns matrix representations of Forman's combinatorial flow `phi`, and of the associated chain homotopy `gamma`.

source

`ConleyDynamics.forman_stab_flow` – Function.

```
forman_stab_flow(lc::AbstractComplex, fvf::CellSubsets; maxit::Int=25)
```

Compute Forman's stabilized combinatorial flow and the associated chain homotopy which relates it to the identity.

This function returns matrix representations of Forman's stabilized combinatorial flow `phiI`, and of the associated chain homotopy `gammaI`. The third return argument is a boolean flag which indicates whether or not the combinatorial flow stabilized. If it did not, then either the underlying Forman vector field is not gradient (this is not checked!), or the maximal number of iterations has been reached. In the latter case, one has to pass the optional paramter `maxit` with a larger number of allowed iterations.

Example

```
julia> labels = ["a","b","c","d"];

julia> simplices = [["a","b"], ["b","c"], ["c","d"]];

julia> lc = create_simplicial_complex(labels, simplices, p=5);

julia> fvf = [["ab","b"], ["bc","c"]];

julia> phiI, gammaI, stabilized = forman_stab_flow(lc, fvf);

julia> stabilized
true

julia> full_from_sparse(gammaI)
7×7 Matrix{Int64}:
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  4  4  0  0  0  0
```

```
 0  0  4  0  0  0  0
 0  0  0  0  0  0  0

julia> full_from_sparse(phiI)
7×7 Matrix{Int64}:
 1  1  1  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  0  0  0  0
 0  0  0  1  0  0  0
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1
 0  0  0  0  0  0  1
```

source

`ConleyDynamics.forman_gpaths` – Function.

```
forman_gpaths(lc::AbstractComplex, fvf::CellSubsets, x::Cell)
```

Find all Forman gradient paths starting at a source cell.

For the Forman gradient vector field `fvf` on the Lefschetz complex `lc` this function finds all maximal gradient paths starting at `x`. These are solution paths which consist exclusively of Forman vectors, i.e., they contain an even number of cells whose dimensions alternate between `dim(x)` and `dim(x) + 1`. Every cell of dimension `dim(x)` is an arrow source which is followed by its arrow target, while every cell of dimension `dim(x) + 1` is an arrow target which is succeeded by a cell in its boundary which is the source of a different arrow, as long as such a cell exists.

source

```
forman_gpaths(lc::AbstractComplex, fvf::CellSubsets,
                  x::Cell, y::Cell)
```

Find all Forman gradient paths between two cells.

For the Forman gradient vector field `fvf` on the Lefschetz complex `lc` this function finds all gradient paths between the cells `x` and `y`. The dimensions of these cells have to satisfy one of the following three conditions:

- `dim(x) = dim(y) - 1`: In this case the function returns all solution paths between `x` and `y` which consist entirely of Forman arrows. All the sources have the dimension of `x`, and the targets the dimension of `y`.
- `dim(x) = dim(y)`: In this case the function returns all solution paths `p` between `x` and `y` for which `p[1:end-1]` is a Forman gradient path in the above sense, and `p[end]` lies in the boundary of `p[end-1]` and is different from the cell `p[end-2]`.
- `dim(x) = dim(y) + 1`: In this case the function returns all solution paths `p` between `x` and `y` for which `p[2:end-1]` is a Forman gradient path in the sense of the first item, where `p[2]` lies in the boundary of `p[1]`, and `p[end]` is contained in the boundary of `p[end-1]`.

In all other cases an empty collection is returned.

source

`ConleyDynamics.forman_path_weight` – Function.

```
forman_path_weight(lc::AbstractComplex, path::Cells)
```

Compute the weight of a Forman gradient path.

For the Lefschetz complex `lc` this function computes the weight of the Forman gradient path given in `path`. It is expected that the dimensions of the first and the last cell in the path differ by at most 1. In case they have equal dimension, and the path has length larger than 1, the first cell has to be an arrow source.

source

```
forman_path_weight(lc::AbstractComplex, paths::CellSubsets)
```

Accumulated weight of a collection of Forman gradient paths.

For the Lefschetz complex `lc` this function computes the sum of the weights of the collection of Forman gradient paths given in `paths`.

source

`ConleyDynamics.chain_vector` – Function.

```
chain_vector(lc::AbstractComplex, chcells::Vector{Int})
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has a 1 in every cell location indicated by the entries in the input argument `chcells`.

source

```
chain_vector(lc::AbstractComplex, chcells::Vector{String})
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has a 1 for every cell indicated by the labels listed in the input argument `chcells`.

source

```
chain_vector(lc::AbstractComplex, chcell::Int)
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has a 1 at the cell index specified in the second argument.

source

```
chain_vector(lc::AbstractComplex, chcell::String)
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has a 1 at the cell specified by the second argument.

source

```
chain_vector(lc::AbstractComplex, chcells::Vector{Int}, chcoeff)
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has the entry `chcoeff[k]` in the `chcells[k]`-th row. In other words, it constructs the vector representation of the chain consisting of the cells specified in `chcells`, with coefficients as specified in `chcoeff`.

source

```
chain_vector(lc::AbstractComplex, chcells::Vector{String}, chcoeff)
```

Create a sparse vector representing a chain.

This function returns a sparse matrix in the form of a column vector, which has the entry `chcoeff[k]` in the row corresponding to the cell with label `chcells[k]`. In other words, it constructs the vector representation of the chain consisting of the cells specified in `chcells`, with coefficients as specified in `chcoeff`.

source

```
chain_vector(lc::AbstractComplex, chcell::Int, chcoeff)
```

Create a sparse vector representing a chain.

This short-cut method specifies a chain consiting of one cell and its coefficient. The cell is given as its index.

source

```
chain_vector(lc::AbstractComplex, chcell::String, chcoeff)
```

Create a sparse vector representing a chain.

This short-cut method specifies a chain consiting of one cell and its coefficient. The cell is given via its label.

source

`ConleyDynamics.chain_support` – Function.

```
chain_support(lc::AbstractComplex, cvec::SparseMatrix; coeff::Bool=false)
```

Determine the support of a chain given as a sparse vector.

This function returns the support of a chain, given in the form of a sparse vector. In the default usage, the function returns a `Vector{String}` which contains the labels of all the cells which have nonzero coefficients in the chain. If one passes the optional parameter `coeff=true`, then the function returns two arguments: In addition to the vector of labels as above, it also returns the vector of associated coefficients.

source

Chapter 18

Acyclic Partition Functions

18.1 Construction

`ConleyDynamics.construct_ap_space` – Function.

```
construct_ap_space(lc::AbstractComplex; connected::Bool=true)
```

Construct the space of acyclic partitions of `lc`.

Starting from the partition of `lc` into singletons (Morse vector `f(X)`, the finest possible acyclic partition), repeatedly merges pairs of blocks of an already-found acyclic partition into a single block, keeping only merges that preserve acyclicity (`mvf_is_acyclic`), until every pair of blocks at every level has been tried. This reaches every element of the poset `AP^d(X)` (see `stratum_partition` and `stratum_adjacency` for its Morse-vector stratification), ordered by atomic refinement (`is_atomic_refinement`).

If `connected=true` (the default), a merge is only attempted when the two blocks are topologically adjacent (`lefschetz_neighbors`), so only partitions built from connected multivectors are reached – this is `AP(X)`, the sub-poset of practical interest for e.g. Forman-vector-field style examples, and is dramatically cheaper to enumerate. If `connected=false`, every pair of blocks is tried regardless of adjacency, reaching the full, unrestricted `AP^d(X)`, including partitions with disconnected multivectors; this is needed whenever a question genuinely requires disconnected pieces (a weight-1 Morse-vector jump, for instance, is only guaranteed uniform when drawn from the unrestricted `AP^d(X)` – see `stratum_adjacency`). The unrestricted enumeration can be substantially larger than the connected one, and may not be tractable for complexes with more than a handful of cells; `construct_ap_stratum` targets a single Morse-vector stratum directly and can remain tractable well past that point.

Returns a `Vector{Vector{Vector{Int}}}`: each entry is one element of `AP^d(X)` (or `AP(X)`) as a multivector field in integer form (singletons omitted, matching the usual `CellSubsets` convention).

source

`ConleyDynamics.construct_ap_stratum` – Function.

```
construct_ap_stratum(lc::AbstractComplex, target::Vector{Int}; connected::Bool=true)
```

Construct only the elements of `AP^d(X)` (or `AP(X)` if `connected=true`, the default) whose Morse vector equals `target`, without enumerating the rest of the poset.

Uses the same merge-based search as `construct_ap_space`, but prunes a branch the moment its current Morse vector fails to dominate `target` in every coordinate: refinement never decreases the Morse vector (`delta = beta(A)+beta(B)-beta(C) >= 0` for any merge of two blocks `A`, `B` into `C`; this monotonicity holds for all refinements, not only atomic ones), so once some coordinate has dropped below `target[k]`, no further merge can ever bring it back and the branch can safely be dropped. Nodes whose Morse vector matches `target` exactly are still expanded further, since additional Morse-vector- preserving (`delta=0`) merges can still produce further distinct partitions belonging to the same stratum.

Each accepted merge updates the running Morse vector incrementally from the two merged blocks' already-known Conley indices (`beta(A)+beta(B) - beta(C)`, memoized in a shared cache keyed by block) rather than recomputing the whole partition's Morse vector from scratch at every node. This incremental update is only ever applied to blocks belonging to an already-`mvf_is_acyclic`-accepted partition – `conley_index` errors on a subset that is not locally closed, and a rejected merge candidate's merged block carries no such guarantee, so the acyclicity check always runs first.

Whether this is actually faster than calling `construct_ap_space` and filtering by Morse vector afterward is target-dependent, not automatic: `target = beta(X)` is a global lower bound on the Morse vector of every element of `AP^d(X)` (the "Extremes" property), so pruning against it never triggers and this degenerates to the full enumeration with extra per-node overhead. For a genuinely intermediate target, however, most branches do get cut, and the saving can be large – e.g. for a 13-cell example where the connected `AP(X)` has 40232 elements, fixing an intermediate stratum of 537 elements takes on the order of a tenth of a second here versus several seconds for full enumeration plus filtering. The main intended use is targeting a single stratum of the unrestricted `AP^d(X)` on complexes where full unrestricted enumeration (`construct_ap_space(lc; connected=false)`) is no longer tractable at all.

Returns a `Vector{Vector{Vector{Int}}}`, in the same format as `construct_ap_space`.

source

18.2 Conversion

`ConleyDynamics.convert_partition_mvf` – Function.

```
convert_partition_mvf(sp::Set{Set{Int}})
```

Convert a set partition into a multivector field.

`sp` is a full partition of a complex's cells into blocks, including singleton blocks (the representation used internally by `construct_ap_space` and `construct_ap_stratum`). Returns the corresponding `CellSubsets`-style multivector field: singleton blocks are dropped (they are implicit critical cells) and every remaining block is returned as a sorted `Vector{Int}`.

source

`ConleyDynamics.convert_mvf_partition` – Function.

```
convert_mvf_partition(lc::AbstractComplex, mvf::CellSubsets)
```

Convert a multivector field into a set partition.

The inverse of `convert_partition_mvf`: adds the implicit singleton blocks (every cell of `lc` not already covered by `mvf`) and returns the result as a `Set{Set{Int}}`, the full-partition representation used internally by `construct_ap_space` and `construct_ap_stratum`.

source

18.3 Atomic Refinement

ConleyDynamics.is_atomic_refinement – Function.

```
is_atomic_refinement(lc::AbstractComplex, mvf1::CellSubsets, mvf2::CellSubsets)
```

Determine whether mvf1 is an atomic refinement of mvf2, i.e. whether mvf1 and mvf2 are elements of AP^d(X) with mvf1 obtained from mvf2 by splitting exactly one block of mvf2 into two. Equivalently, mvf1 and mvf2 cover each other in the atomic-refinement order used throughout the construct_ap_space, atomic_distances, and stratum_adjacency functions.

source

ConleyDynamics.atomic_distances – Function.

```
atomic_distances(lc::AbstractComplex, ap::Vector{Vector{Vector{Int}}}; p::Int=99991)
```

Compute the atomic-refinement adjacency matrix of ap (typically the output of construct_ap_space).

Returns a sparse matrix A (in the ConleyDynamics sparse format, over the prime field p) with A[i,j] == 1 iff ap[i] is an atomic refinement of ap[j] (is_atomic_refinement(lc, ap[i], ap[j])). This is the expensive step of the pipeline (checking every pair of elements at adjacent lengths), and is what stratum_adjacency uses to determine coverage between Morse-vector strata.

source

18.4 Connectivity

ConleyDynamics.is_connected_block – Function.

```
is_connected_block(lc::AbstractComplex, block::Cells)
```

Whether block is connected as a subspace of lc.

This is a genuinely different question from block_rho's rho_0 and beta_0: those come from the restricted boundary operator on the block and measure homological cancellation, which is 0 for any regular (non-critical) multivector regardless of whether it is topologically connected – e.g. a Forman pair {v,e} with v a face of e is connected as a subspace but has beta(V) = (0,0,...) because it is non-critical. Connectivity here instead means: is the comparability graph of block (edges wherever one cell is a direct face of another, both cells inside block) a single connected component. Reachability via direct face relations already captures full order-comparability, since any longer chain x < y < z decomposes into direct-face steps.

source

ConleyDynamics.is_connected_partition – Function.

```
is_connected_partition(lc::AbstractComplex, mvf::CellSubsets)
```

Whether every explicit multivector of `mvf` is connected (`is_connected_block`); implicit singletons are always connected. This is exactly the membership test for `AP(X)`, the sub-poset that `construct_ap_space(lc; connected=true)` enumerates directly.

source

`ConleyDynamics.is_closed_in_block` – Function.

```
is_closed_in_block(lc::AbstractComplex, A::Vector{Int}, blockset::Set{Int})
```

Whether `A` is closed in the block `blockset`, i.e., whether `cl(A) cap blockset = A`.

source

18.5 Morse Vector

`ConleyDynamics.morse_vector` – Function.

```
morse_vector(lc::AbstractComplex, mvf::CellSubsets)
```

Morse vector `M(W) = sum_{V in W} beta(V)` of a multivector field/partition `mvf`. Every non-critical block contributes 0, so this is just the sum of the Conley indices of the critical multivectors.

source

`ConleyDynamics.block_rho` – Function.

```
block_rho(lc::AbstractComplex, block::Cells)
```

The rank vector `(rho_0(V),...,rho_n(V))` of a single locally closed multivector `block`, recovered from `beta(V)` and `f(V)`: `f(V) - beta(V) = sum_k rho_k(V) (e_k + e_{k-1})`.

Uses `beta(V) = conley_index(lc, block)`, which coincides with the absolute homology of the restricted boundary operator for any locally closed `V`.

source

18.6 Block Split Analysis

`ConleyDynamics.block_split_deltas` – Function.

```
block_split_deltas(lc::AbstractComplex, block::Cells)
```

Enumerate every atomic split `C = A ⊔ B` of `block` with `A` closed in `C`, and return the achieved `delta = beta(A) + beta(B) - beta(C)` together with `A` and `B` for each (the split spectrum `D({C})`). Brute force over all subsets, so only intended for small blocks (`length(block) <= 20`).

source

ConleyDynamics.split_spectrum – Function.

```
split_spectrum(lc::AbstractComplex, block::Cells)
```

The set D({C}) of distinct delta vectors achievable by a single atomic split of block.

source

ConleyDynamics.connected_split_deltas – Function.

```
connected_split_deltas(lc::AbstractComplex, block::Cells)
```

Like block_split_deltas, but restricted to splits C = A ⊔ B with both A and B topologically connected (is_connected_block) – the deltas actually reachable by a single atomic refinement step while staying inside AP(X), as opposed to the unrestricted AP^d(X).

source

18.7 Stratification

ConleyDynamics.stratum_partition – Function.

```
stratum_partition(lc::AbstractComplex, ap::Vector{Vector{Vector{Int}}})
```

Group the enumerated acyclic partitions ap = construct_ap_space(lc) by their Morse vector. Returns Dict{Vector{Int},Vector{Int}} mapping M to the indices into ap with morse_vector(lc, ap[i]) == M.

source

ConleyDynamics.stratum_jump_weight – Function.

```
stratum_jump_weight(delta::Vector{Int})
```

The weight |c| of the canonical decomposition delta = sum_k c_k (e_k+e_{k-1}).

source

ConleyDynamics.stratum_adjacency – Function.

```
stratum_adjacency(lc::AbstractComplex, ap::Vector{Vector{Vector{Int}}}; A=nothing)
```

For every pair of distinct strata M1 > M2 for which at least one element of AP_{M2} admits an atomic refinement into AP_{M1}, report the jump delta = M1-M2, its weight |c|, how many of AP_{M2}'s elements are covered from M1, and whether coverage is uniform.

A, if supplied, must be atomic_distances(lc, ap); otherwise it is computed internally (this is the expensive step, so pass it in if already available).

source

ConleyDynamics.stratum_reachability – Function.

```
stratum_reachability(lc::AbstractComplex, ap::Vector{Vector{Vector{Int}}}; A=nothing)
```

For every element of ap, compute the full set of Morse vectors reachable by any chain of atomic refinements of any length – not just the single step stratum_adjacency checks. atomic_distances already contains every such one-step edge, including the ones between two elements of the same stratum (stratum_adjacency deliberately skips M1 == M2 pairs when reporting, but the edges are there); this walks the full poset via those edges to answer the "can you always eventually get there" question, regardless of how many intermediate steps – possibly staying within the source stratum for a while – it takes.

Since every atomic refinement increases the total block count (mvf_length) by exactly 1, AP(X) (or AP^d(X)) is graded by block count and hence acyclic; reachability is computed with a single dynamic- programming pass in decreasing block-count order (finest elements, which have no successors, first), rather than a full pairwise transitive closure: reach[v] = {M(v)} \cup \bigcup_{v \to w} reach[w].

A, if supplied, must be atomic_distances(lc, ap); otherwise computed internally.

Returns (reach, edges):

- reach::Vector{Set{Vector{Int}}}: reach[i] is the set of Morse vectors reachable from ap[i], including morse_vector(lc, ap[i]) itself.
- edges::Vector{NamedTuple}: one entry per pair of distinct strata M2, M1 (M1 >= M2 componentwise) for which at least one element of AP_{M2} eventually reaches AP_{M1}, in the same field layout as stratum_adjacency's output (M1, M2, delta, weight, ncovered, ntotal, uniform, uncovered), except ncovered/uniform now count eventual rather than one-step reachability.

source

18.8 Top Layer

ConleyDynamics.is_top_layer – Function.

```
is_top_layer(lc::AbstractComplex, mvf::CellSubsets; connected::Bool=true)
```

Whether the acyclic partition mvf lies in the top layer of its own Morse stratum, i.e. whether it admits no atomic refinement that preserves its Morse vector.

By the local characterization of the top layer (every block is either a hyperbolic singleton, a Forman doubleton, or a genuinely rigid multivector), this comes down to a per-block test: mvf is in the top layer iff none of its explicit blocks admits an atomic split with delta = 0. Implicit singletons need no check, since they cannot be split further.

If connected=true (the default, matching construct_ap_space), only splits into two topologically connected pieces are considered valid atomic refinements (connected_split_deltas, the restricted AP(X) sense). If connected=false, any conservative split counts (block_split_deltas, the unrestricted AP^d(X) sense). This must match how mvf itself was constructed/enumerated, or the test answers a different question than intended.

Since block_split_deltas brute-forces all subsets of a block, this inherits its length(block) <= 20 limitation.

Example

```
julia> lc, mvf= example_forman1d();

julia> is_top_layer(lc, mvf)
true
```

source

ConleyDynamics.stratum_top_layer – Function.

```
stratum_top_layer(lc::AbstractComplex, ap::Vector{Vector{Vector{Int}}}; connected::Bool=true)
```

For every Morse-vector stratum represented in ap, find the indices of ap belonging to its top layer – the acyclic partitions of that Morse vector which cannot be further refined without increasing the Morse vector.

ap is typically the output of construct_ap_space or construct_ap_stratum. connected must match how ap was constructed, since it selects whether a block's conservative-splittability is tested via connected_split_deltas (restricted AP(X), the default) or block_split_deltas (unrestricted AP^d(X)); see is_top_layer.

Returns Dict{Vector{Int},Vector{Int}}, in the same shape as stratum_partition, but every value is filtered down to just the indices in the top layer of that stratum (a subset of stratum_partition(lc, ap)[M]).

A block's rigidity is memoized across the whole call (many partitions, possibly across different strata, reuse identical blocks), so each distinct block occurring anywhere in ap has its split spectrum computed only once.

source

ConleyDynamics.construct_ap_stratum_top – Function.

```
construct_ap_stratum_top(lc::AbstractComplex, target::Vector{Int}; connected::Bool=true)
```

Directly compute the top layer of the Morse stratum with Morse vector target: the elements of AP_target(X) (or AP^d_target(X) if connected=false) that admit no atomic refinement preserving target.

Equivalent to

```
ap = construct_ap_stratum(lc, target; connected=connected)
filter(w -> is_top_layer(lc, w; connected=connected), ap)
```

provided as a single call for convenience. Since the top layer is only meaningful relative to the stratum it sits inside, this still has to construct the whole stratum first (via construct_ap_stratum) before filtering it down.

Returns a Vector{Vector{Vector{Int}}}, in the same format as construct_ap_space and construct_ap_stratum.

source

18.9 Internal Helpers

`ConleyDynamics._rank_decomposition` – Function.

```
_rank_decomposition(d::Vector{Int})
```

Solve `d_k = rho_k + rho_{k+1}` for `rho`, given `rho_0 = 0` and `d` indexed `1:n+1` for dimensions `0:n`. The same recursion also extracts the `c_k` in the canonical decomposition of a Morse-vector jump `delta = sum c_k(e_k+e_{k-1})`.

Internal helper, not exported.

source

`ConleyDynamics._stratum_transitive_reduction` – Function.

```
_stratum_transitive_reduction(edges::Vector{<:NamedTuple})
```

Transitive reduction of the stratum graph described by `edges` (as returned by `stratum_reachability`, or any `NamedTuple` vector with `.M1` and `.M2` fields defining a directed arc `M2 -> M1`): drop an arc `M2 -> M1` whenever some other direct successor `w` of `M2` (`w != M1`) already reaches `M1` via the remaining arcs, since that makes `M2 -> M1` redundant for reachability – it is implied by the two-hop-or-longer path `M2 -> w -> ... -> M1`. The underlying graph is a DAG (`stratum_reachability` only ever reports `M1 >= M2` componentwise), so this reduction is unique.

Used by `plot_morse_reachability` to draw only the covering relation of the reachability order (a proper Hasse diagram) instead of every comparable pair, which `stratum_reachability`'s raw `edges` – being closed under transitivity – reports for all of them.

Returns the filtered subset of `edges` (same `NamedTuple`s, just fewer).

source

Chapter 19

Example Functions

19.1 Examples from Batko et al.

`ConleyDynamics.example_forman1d` – Function.

```
lc, mvf = example_forman1d()
```

Create the simplicial complex and multivector field for the example from Figure 1 in the FoCM 2020 paper by Batko, Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` and the multivector field `mvf`. The Lefschetz complex is defined over the finite field GF(2). If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case a `EuclidenComplex` with embedded coordinates is returned for `lc`.

Examples

```
julia> lc, mvf = example_forman1d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
  ┆  A CD  F BF DE
--┆---------------
 A┆  .  .  .  .  1
CD┆  .  .  .  .  .
 F┆  .  .  .  .  1
BF┆  .  .  .  .  .
DE┆  .  .  .  .  .

julia> sparse_show(cm.matrix)
 . . . . 1
 . . . . .
 . . . . 1
 . . . . .
 . . . . .

julia> println(cm.labels)
["A", "CD", "F", "BF", "DE"]
```

```
julia> lc2, mvf2 = example_forman1d(euclidean=true);

julia> typeof(lc2)
EuclideanComplex

julia> lc2.coords
13-element Vector{Vector{Vector{Float64}}}:
 [[50.0, 100.0]]
 [[250.0, 100.0]]
 [[0.0, 0.0]]
 [[100.0, 0.0]]
 [[200.0, 0.0]]
 [[300.0, 0.0]]
 [[50.0, 100.0], [0.0, 0.0]]
 [[50.0, 100.0], [100.0, 0.0]]
 [[250.0, 100.0], [200.0, 0.0]]
 [[250.0, 100.0], [300.0, 0.0]]
 [[0.0, 0.0], [100.0, 0.0]]
 [[100.0, 0.0], [200.0, 0.0]]
 [[200.0, 0.0], [300.0, 0.0]]
```

source

ConleyDynamics.example_forman2d – Function.

```
lc, mvf = example_forman2d()
```

Create the simplicial complex and multivector field for the example from Figure 3 in the FoCM 2020 paper by Batko, Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` over the finite field GF(2) and the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case a `EuclidenComplex` with embedded coordinates is returned for `lc`.

Examples

```
julia> lc, mvf = example_forman2d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  D  E  F  IJ  BF  EF  HI ADE FGJ
───┼──────────────────────────────────
  D┆  .  .  .  .   1  .   1  .   .
  E┆  .  .  .  .   .  1   .  .   .
  F┆  .  .  .  .   1  1   1  .   .
 IJ┆  .  .  .  .   .  .   .  .   1
 BF┆  .  .  .  .   .  .   .  1   .
 EF┆  .  .  .  .   .  .   .  .   .
 HI┆  .  .  .  .   .  .   .  1   .
ADE┆  .  .  .  .   .  .   .  .   .
FGJ┆  .  .  .  .   .  .   .  .   .
```

```
julia> sparse_show(cm.matrix)
 .  .  .  .  1  .  1  .  .
 .  .  .  .  .  1  .  .  .
 .  .  .  .  1  1  1  .  .
 .  .  .  .  .  .  .  .  1
 .  .  .  .  .  .  .  1  .
 .  .  .  .  .  .  .  .  .
 .  .  .  .  .  .  1  .
 .  .  .  .  .  .  .  .  .
 .  .  .  .  .  .  .  .  .

julia> print(cm.labels)
["D", "E", "F", "IJ", "BF", "EF", "HI", "ADE", "FGJ"]

julia> lc2, mvf2 = example_forman2d(euclidean=true);

julia> typeof(lc2)
EuclideanComplex
```

source

19.2 Examples from Mrozek & Wanner

ConleyDynamics.example_julia_logo – Function.

```
lc, mvf = example_julia_logo()
```

Create the simplicial complex and multivector field for the example from Figure 2.1 in the connection matrix book by Mrozek & Wanner.

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_julia_logo();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  D  BC ABC
---┆------------
  D┆  .   .   .
 BC┆  .   .   1
ABC┆  .   .   .

julia> print(cm.labels)
["D", "BC", "ABC"]
```

source

ConleyDynamics.example_three_cm – Function.

```
lc, mvf = example_three_cm(mvftype)
```

Create the simplicial complex and multivector field for the example from Figure 2.2 in the connection matrix book by Mrozek & Wanner.

Depending on the value of `mvftype`, return the periodic orbit (0=default) or one of the three gradient (1,2,3) examples.

The function returns the Lefschetz complex `lc` over the rational field and the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`, which creates `lc` as a `EuclideanComplex` with embedded coordinates.

Examples

```
julia> lc, mvf = example_three_cm(0);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> print(cm.labels)
["A", "C", "DE", "AC", "BD", "DF", "ABC", "EFG"]

julia> full_from_sparse(cm.matrix)
8×8 Matrix{Rational{Int64}}:
 0  0  0  -1  -1  0   0   0
 0  0  0   1   1  0   0   0
 0  0  0   0   0  0   0  -1
 0  0  0   0   0  0  -1   0
 0  0  0   0   0  0   1   0
 0  0  0   0   0  0   0   1
 0  0  0   0   0  0   0   0
 0  0  0   0   0  0   0   0
```

source

`ConleyDynamics.example_multiflow` – Function.

```
lc, mvf = example_multiflow()
```

Create the Lefschetz complex and multivector field for the example from Figure 2.3 in the connection matrix book by Mrozek & Wanner.

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_multiflow();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
  ┆ BD DF AC CE
--┆------------
```

```
BD┆  .  .  .  .
DF┆  .  .  .  .
AC┆  .  .  .  .
CE┆  .  .  .  .

julia> print(cm.labels)
["BD", "DF", "AC", "CE"]
```

source

ConleyDynamics.example_small_periodicity – Function.

```
lc1, lc2, mvf = example_small_periodicity()
```

Create two representations of the Lefschetz complex and the multivector field for the example from Figure 2.4 in the connection matrix book by Mrozek & Wanner.

The complexes `lc1` and `lc2` are just two representations of the same complex, but they lead to different connection matrices. Both Lefschetz complexes are defined over the finite field GF(2).

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector field `mvf`.

Examples

```
julia> lc1, lc2, mvf = example_small_periodicity();

julia> cm1 = connection_matrix(lc1, mvf, algorithm="DLMS");

julia> cm2 = connection_matrix(lc2, mvf, algorithm="DLMS");

julia> full_from_sparse(cm1.matrix)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  1
 0  0  0  1
 0  0  0  0

julia> print(cm1.labels)
["A", "a", "b", "alpha"]

julia> full_from_sparse(cm2.matrix)
4×4 Matrix{Int64}:
 0  0  0  0
 0  0  0  0
 0  0  0  1
 0  0  0  0

julia> print(cm2.labels)
["A", "c", "b", "alpha"]
```

source

ConleyDynamics.example_subdivision – Function.

```
lc, mvf = example_subdivision(mvftype)
```

Create the Lefschetz complex and multivector field for the example from Figure 7.1 in the connection matrix book by Mrozek & Wanner.

Depending on the value of `mvftype`, return the multivector (0=default) or one of the two combinatorial vector field (1,2) examples.

The function returns the Lefschetz complex `lc` over the rationals and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_subdivision(1);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> full_from_sparse(cm.matrix)
5×5 Matrix{Rational{Int64}}:
 0  0  -1  -1  -1
 0  0   1   0   0
 0  0   0   0   0
 0  0   0   0   0
 0  0   0   0   0
```

source

19.3 General Examples

`ConleyDynamics.example_critical_simplex` – Function.

```
lc, mvf = example_critical_simplex(dim)
```

Create a simplicial complex of dimension `dim` as well as a multivector field on it in which every cell is critical.

The function returns the Lefschetz complex `lc` over GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_critical_simplex(2);

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  A   B   C  AB  AC  BC ABC
───┆────────────────────────────
  A┆  .   .   .   1   1   .   .
  B┆  .   .   .   1   .   1   .
  C┆  .   .   .   .   1   1   .
 AB┆  .   .   .   .   .   .   1
 AC┆  .   .   .   .   .   .   1
```

```
 BC┆   .   .   .   .   .   .   1
ABC┆   .   .   .   .   .   .   .

julia> print(cm.labels)
["A", "B", "C", "AB", "AC", "BC", "ABC"]
```

source

ConleyDynamics.example_moebius – Function.

```
lc1, mvf1, lc2, mvf2 = example_moebius(p)
```

Create two simplicial complexes for a cylinder and Moebius strip, respectively, together with associated multivector fields on them.

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector fields `mvf1` and `mvf2`. Both complexes are over a field with characteristic `p`. Positive prime characteristic uses the finite field GF(p), while zero characteristic gives the rationals.

The multivector field is the same, and it has one critical cell each in dimension 1 and 2 in the interior of the strip. The boundary consists of two periodic orbits for `lc1` and `mvf1`, and of one periodic orbit in the Moebius case `lc2` and `mvf2`. The latter case leads to different connection matrices for the fields GF(2) and GF(7), for example.

Examples

```
julia> lc1, mvf1, lc2, mvf2 = example_moebius(0);

julia> lc2p2 = lefschetz_gfp_conversion(lc2,2);

julia> lc2p7 = lefschetz_gfp_conversion(lc2,7);

julia> cmp2 = connection_matrix(lc2p2, mvf2, algorithm="DHL");

julia> cmp7 = connection_matrix(lc2p7, mvf2, algorithm="DHL");

julia> sparse_show(cmp2.matrix)
 . . . .
 . . . 1
 . . . .
 . . . .

julia> sparse_show(cmp7.matrix)
 . . . .
 . . . 1
 . . . 2
 . . . .
```

source

ConleyDynamics.example_nonunique – Function.

```
lc1, lc2, mvf = example_nonunique()
```

Create two representations of a simplicial complex and one multivector field which illustrates nonunique connection matrices.

The two complexes `lc1` and `lc2` represent the same simplicial complex over GF(2), but differ in the ordering of the labels.

The function returns the Lefschetz complexes `lc1` and `lc2`, as well as the multivector field `mvf`. If desired for plotting, the function can be invoked with the optional argument `euclidean=true`. In that case, `lc1` and `lc2` are created as `EuclideanComplex` with embedded coordinates.

Examples

```
julia> lc1, lc2, mvf = example_nonunique();

julia> cm1 = connection_matrix(lc1, mvf, algorithm="DLMS");

julia> cm2 = connection_matrix(lc2, mvf, algorithm="DLMS");

julia> sparse_show(cm1.matrix)
 . . . 1 . 1 . . .
 . . . 1 . 1 . . .
 . . . . . . . 1 1
 . . . . . . 1 1 .
 . . . . . . . 1 .
 . . . . . . 1 1 .
 . . . . . . . . .
 . . . . . . . . .
 . . . . . . . . .

julia> print(cm1.labels)
["2", "7", "79", "29", "45", "67", "168", "349", "789"]

julia> sparse_show(cm2.matrix)
 . . . 1 . 1 . . .
 . . . 1 . 1 . . .
 . . . . . . 1 . 1
 . . . . . . 1 1 .
 . . . . . . . 1 .
 . . . . . . 1 1 .
 . . . . . . . . .
 . . . . . . . . .
 . . . . . . . . .

julia> print(cm2.labels)
["2", "8", "78", "29", "45", "67", "168", "349", "789"]
```

source

ConleyDynamics.example_clorenz – Function.

```
lc, mvf = example_clorenz()
```

Create the simplicial complex and multivector field for the example from Figure 3 in the JCD 2016 paper by Kaczynski, Mrozek, and Wanner.

The function returns the Lefschetz complex `lc` over the finite field GF(2) and the multivector field `mvf`.

Examples

```
julia> lc, mvf = example_clorenz();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
  ┆  i jp  g hm bc
--┆---------------
 i┆  .  .  .  .  1
jp┆  .  .  .  .  .
 g┆  .  .  .  .  1
hm┆  .  .  .  .  .
bc┆  .  .  .  .  .

julia> print(cm.labels)
["i", "jp", "g", "hm", "bc"]

julia> ms, ps = morse_sets(lc, mvf, poset=true);

julia> [conley_index(lc, mset) for mset in ms]
4-element Vector{Vector{Int64}}:
 [1, 1, 0]
 [1, 1, 0]
 [0, 1, 0]
 [0, 0, 0]

julia> ps
4×4 Matrix{Bool}:
 0  0  1  0
 0  0  1  0
 0  0  0  1
 0  0  0  0
```

source

`ConleyDynamics.example_dunce_chaos` – Function.

```
sc, vfG, vfC = example_dunce_chaos()
```

Create a minimal simplicial complex representation of the Dunce hat, as well as two Forman vector fields.

The function returns the simplicial representation of the Dunce hat in `sc` over the finite field `GF(2)`. The Forman vector field `vfG` is a gradient vector field with unique connection matrix. The field `vfC` is a modification of this field which merges the critical cells of dimensions 1 and 2 into a Forman arrow. The resulting Forman vector field is no longer gradient, and in fact exhibits Lorez-like chaos.

Examples

```
julia> sc, vfG, vfC = example_dunce_chaos();

julia> homology(sc)
3-element Vector{Int64}:
 1
 0
 0

julia> cmG = connection_matrix(sc, vfG, algorithm="DHL");

julia> sparse_show(cmG)
   ┆   1  12 128
---┆------------
  1┆   .   .   .
 12┆   .   .   1
128┆   .   .   .

julia> print(cmG.labels)
["1", "12", "128"]

julia> cmC = connection_matrix(sc, vfC, algorithm="DHL");

julia> sparse_show(cmC)
 ┆ 1
-┆--
1┆ .

julia> print(cmC.labels)
["1"]

julia> msC, psC = morse_sets(sc, vfC, poset=true);

julia> [conley_index(sc, mset) for mset in msC]
2-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 0, 0]

julia> psC
2×2 Matrix{Bool}:
 0  1
 0  0

julia> msC
2-element Vector{Vector{String}}:
 ["1"]
 ["12", "14", "15", "25", "28", "56", "68", "78", "124", "125", "128", "145", "256", "278",
↪  "568", "678"]
```

source

ConleyDynamics.example_torsion_chaos – Function.

```
sc, vfG, vfC = example_torsion_chaos(n::Int, p::Int)
```

Create a triangulation of a space with 1-dimensional torsion, as well as two Forman vectr fields on this complex.

The function returns a simplicial complex `sc` which has the following integer homology groups:

- In dimension 0 it is the group of integers.
- In dimension 1 it is the integers modulo n.
- In dimension 2 it is the trivial group.

In other words, the simplicial complex has nontrivial torsion in dimension 1. It is a triangulation of an n-gon, in which all boundary edges are oriented counterclockwise, and all of these edges are identified. The parameter p specifies the characteristic of the underlying field.

In addition, two Forman vector fields `vfG` and `vfC` are returned. The first one is a gradient vector field whose connection matrix has a large connection matrix entry. In fact, if p is any prime larger than n then there will be an entry n in the matrix. The scond Forman vector field contains a chaotic Morse set. This Morse set will have trivial Morse index for most p. On the other hand, for prime `p = n` the set has the Morse index of an unstable periodic orbit.

Examples

```
julia> sc, vfG, vfC = example_torsion_chaos(3,7);

julia> homology(sc)
3-element Vector{Int64}:
 1
 0
 0

julia> cmG = connection_matrix(sc, vfG, algorithm="DHL");

julia> sparse_show(cmG)
      ┆     0w   0w0x 0w0x1s
──────┆──────────────────────
    0w┆      .      .      .
  0w0x┆      .      .      3
0w0x1s┆      .      .      .

julia> print(cmG.labels)
["0w", "0w0x", "0w0x1s"]

julia> cmC = connection_matrix(sc, vfC, algorithm="DHL");

julia> sparse_show(cmC)
  ┆ 0w
──┆───
0w┆  .

julia> print(cmC.labels)
["0w"]
```

```
julia> msC, psC = morse_sets(sc, vfC, poset=true);

julia> [conley_index(sc, mset) for mset in msC]
2-element Vector{Vector{Int64}}:
 [1, 0, 0]
 [0, 0, 0]

julia> psC
2×2 Matrix{Bool}:
 0  1
 0  0

julia> length.(msC)
2-element Vector{Int64}:
  1
 26
```

source

Chapter 20

Extension Functions

20.1 DelaunayTriangulation.jl

The following functions are available when `DelaunayTriangulation.jl` is loaded alongside `ConleyDynamics.jl`.

`ConleyDynamics.delaunay_points_bnd_rectangle` – Function.

```
delaunay_points_bnd_rectangle(bmin, bmax)
    -> (Vector{Tuple{Float64,Float64}}, Vector{Vector{Int}})
```

Create the initial point list and closed outer boundary curve for a rectangular domain, for use as input to `triangulate` from `DelaunayTriangulation.jl`.

The arguments `bmin = [xmin, ymin]` and `bmax = [xmax, ymax]` give the lower-left and upper-right corners of the bounding rectangle. The four corners are stored as `Float64` tuples in counter-clockwise order. The returned `bndcurve` encodes the closed loop in the format expected by the `boundary_nodes` keyword of `triangulate`.

Returns a tuple `(points, bndcurve)` where:

- `points` is a `Vector{Tuple{Float64,Float64}}` containing the four corner vertices,
- `bndcurve` is a `Vector{Vector{Int}}` that closes the rectangular boundary.

After this call, constraint edges and interior points can be appended with `delaunay_points_add_segment`. The result is then passed to `triangulate` from `DelaunayTriangulation.jl`, and the triangulation converted with `delaunay_to_simplicial` to obtain a `EuclideanComplex`.

This function is only available when `DelaunayTriangulation.jl` is loaded.

Example

```
using DelaunayTriangulation
using ConleyDynamics

points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])
tri = triangulate(points; boundary_nodes = bndcurve)
sc  = delaunay_to_simplicial(tri)
```

source

`ConleyDynamics.delaunay_points_add_nodes` – Function.

```
delaunay_points_add_nodes(points, newpoints)
    -> Vector{Tuple{Float64,Float64}}
```

Append a list of interior nodes to the given points for a Delaunay triangulation.

`newpoints` is a `Vector{Vector{<:Real}}` where each element `[x, y]` is a 2D point which describes an interior node. All points in `newpoints` are appended to `points` (converted to `Float64`), and the function returns the updated `points`.

This function is only available when `DelaunayTriangulation.jl` is loaded.

Example

```
using DelaunayTriangulation
using ConleyDynamics

# Rectangular domain with a circular hole and an open cut
points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])

newpoints = [-4 .+ 8 .* rand(2) for k in 1:12]   # 12 random points
points    = delaunay_points_add_nodes(points, newpoints)

tri = triangulate(points; boundary_nodes = bndcurve)
sc  = delaunay_to_simplicial(tri)
```

source

`ConleyDynamics.delaunay_points_add_segment` – Function.

```
delaunay_points_add_segment(points, segments, newseg)
    -> (Vector{Tuple{Float64,Float64}}, Set{Tuple{Int,Int}})
delaunay_points_add_segment(points, newseg)
    -> (Vector{Tuple{Float64,Float64}}, Set{Tuple{Int,Int}})
```

Append a polygonal curve to the point list and constraint-edge set for a constrained Delaunay triangulation.

`newseg` is a `Vector{Vector{<:Real}}` where each element `[x, y]` is a 2D point on the curve. If the first and last entries of `newseg` coincide (Euclidean distance less than `1e-9`) the curve is treated as closed: only one copy of the shared endpoint is stored and the closing edge connects the last new point back to the first. Otherwise the curve is treated as open and both endpoints are stored separately.

All points in `newseg` are appended to `points` (converted to `Float64`) and all successive pairs are added as directed constraint edges to `segments`.

The three-argument form takes an existing `segments::Set{Tuple{Int,Int}}` and extends it. The two-argument form is a convenience wrapper that initialises `segments` as an empty set; use it when adding the first constraint curve after `delaunay_points_bnd_rectangle`.

Returns the updated `(points, segments)`.

This function is only available when `DelaunayTriangulation.jl` is loaded.

Example

```julia
using DelaunayTriangulation
using ConleyDynamics

# Rectangular domain with a circular hole and an open cut
points, bndcurve = delaunay_points_bnd_rectangle([-5, -5], [5, 5])

n = 12
theta  = range(0, 2*pi, length = n+1)
circle = [[4.0*cos(t), 4.0*sin(t)] for t in theta]  # closed curve
cut    = [[-2.0, -2.0], [0.0, 1.0], [2.0, -1.0]]    # open curve

points, segments = delaunay_points_add_segment(points, circle)
points, segments = delaunay_points_add_segment(points, segments, cut)

tri = triangulate(points; boundary_nodes = bndcurve, segments = segments)
sc  = delaunay_to_simplicial(tri)
```

source

ConleyDynamics.delaunay_points_add_split_segs – Function.

```julia
delaunay_points_add_split_segs(points, newsegments; verbose=false)
    -> (Vector{Tuple{Float64,Float64}}, Set{Tuple{Int,Int}})
delaunay_points_add_split_segs(points, segments, newsegments; verbose=false)
    -> (Vector{Tuple{Float64,Float64}}, Set{Tuple{Int,Int}})
```

Compute a planar straight-line arrangement from `newsegments`, add all resulting vertices and sub-segments to `points` and `segments`, and return the updated pair.

Use this instead of `delaunay_points_add_segment` when the input curves may intersect each other or have collinear overlaps.

`newsegments` is a `Vector` of segments, each given as a length-2 vector of `[x, y]` endpoints, e.g. `[[x1, y1], [x2, y2]]`. The function calls `split_segments` internally to resolve all crossings and overlaps into a non-crossing arrangement, then appends the resulting vertices and edges to the existing `points` and `segments`.

The two-argument form is a convenience wrapper that initialises `segments` as an empty set. The three-argument form takes an existing `segments::Set{Tuple{Int,Int}}` and extends it.

If the keyword argument `verbose` is `true`, the function prints a brief summary: number of input segments, number of intersection points created, and number of output vertices and edges added to `points`.

Returns the updated `(points, segments)`.

Three-argument form: overlaps with existing segments are not checked

When calling the three-argument form, the new segments in `newsegments` are split among themselves by `split_segments`, but **no check is performed for intersections or overlaps with the segments already present in `segments`**. It is the caller's responsibility to ensure that the newly added sub-segments are compatible with the existing constraint-edge set. Incorrect use can produce an invalid segment set that silently causes `DelaunayTriangulation.jl` to drop constraints.

This function is only available when `DelaunayTriangulation.jl` is loaded.

Example

```
using DelaunayTriangulation
using ConleyDynamics

points, bndcurve = delaunay_points_bnd_rectangle([-2, -2], [2, 2])

# Two crossing diagonals — split_segments resolves the crossing automatically
segs = [[[-1.0, -1.0], [1.0, 1.0]],
        [[-1.0,  1.0], [1.0, -1.0]]]

points, segments = delaunay_points_add_split_segs(points, segs; verbose=true)

tri = triangulate(points; boundary_nodes = bndcurve, segments = segments)
sc  = delaunay_to_simplicial(tri)
```

source

`ConleyDynamics.delaunay_to_simplicial` – Function.

```
delaunay_to_simplicial(tt; p::Int=2) -> EuclideanComplex
```

Convert a Delaunay triangulation from `DelaunayTriangulation.jl` into an `EuclideanComplex`.

The argument `tt` is a triangulation object as returned by `triangulate` from `DelaunayTriangulation.jl`. Ghost triangles and ghost vertices (the package's boundary sentinel elements) are automatically excluded via the `each_solid_*` iterators, so the result contains only the geometric simplices.

The keyword argument p specifies the field characteristic of the resulting complex: `p=0` gives the rationals, `p>0` gives GF(p). The default is `p=2`.

Vertex labels are zero-padded decimal strings (`"001"`, `"002"`, ...) whose width is determined by the total vertex count. The resulting `EuclideanComplex` carries the embedded 2D coordinates of each simplex.

This function requires `DelaunayTriangulation.jl` to be loaded before `ConleyDynamics.jl`.

Example

```
using DelaunayTriangulation
using ConleyDynamics

pts = [(0.0, 0.0), (1.0, 0.0), (0.0, 1.0)]
tri = triangulate(pts)
sc  = delaunay_to_simplicial(tri)
sc3 = delaunay_to_simplicial(tri; p=3)
```

source

20.2 TetGen.jl

The following functions create tetrahedral meshes when `TetGen.jl` is loaded alongside `ConleyDynamics.jl`.

`ConleyDynamics.tetgen_mesh` – Function.

```
tetgen_mesh(dom::TetDomain; maxvolume::Union{Real,Nothing}=nothing,
            minratio::Union{Real,Nothing}=1.414,
            mindihedral::Union{Real,Nothing}=nothing,
            steiner_on_boundary::Bool=true, quiet::Bool=true,
            switches::Union{String,Nothing}=nothing) -> TetGenResult
```

Create a constrained tetrahedral mesh of a domain with `TetGen.jl`.

The domain `dom` is described by a `TetDomain`, see for example `tetgen_domain_box` and `tetgen_domain_add_surface`. The keyword arguments control the mesh quality and size:

- `maxvolume`: Maximum volume of the tetrahedra. If `nothing`, there is no global volume constraint. Regional volume constraints of the domain are always applied. Note that TetGen treats volume constraints as approximate: typically about one percent of the tetrahedra exceed them, by up to about fifty percent.
- `minratio`: Maximum ratio of circumradius to shortest edge length of the tetrahedra. Smaller values give better shaped tetrahedra, but more of them. The default `1.414` is the standard value of TetGen. Values below 1 are rejected, since TetGen does not terminate for them. If both `minratio` and `mindihedral` are `nothing`, no quality refinement is performed.
- `mindihedral`: Minimum dihedral angle of the tetrahedra, in degrees, strictly between `0` and `90`. If `nothing`, only `minratio` is used. If `minratio` is `nothing`, but `mindihedral` is given, the ratio bound `2.0` is used. Note that TetGen does not guarantee this bound, and that large values can make TetGen fail.
- `steiner_on_boundary`: If `false`, no additional points are added on the boundary and constraint facets. Note that this also severely limits the refinement close to these facets.
- `quiet`: If `true`, the output of TetGen is suppressed.
- `switches`: A complete TetGen switch string, which overrides all other keyword arguments. It should contain `p` if the facets of the domain are to be respected, and it should not contain `f`, since otherwise interior triangles are reported as boundary triangles with marker `0`.

Quality refinement is switched on by default, since flat tetrahedra can make the construction of multivector fields fragile. The values of `maxvolume`, `minratio`, and `mindihedral` have to be finite, otherwise an `ArgumentError` is raised.

If the facets of the domain intersect, or if the quality bounds are too strict, TetGen fails, and an `ArgumentError` is raised. In that case, check the constraint surfaces and curves, relax the quality bounds, or use `Gmsh.jl` for intersecting geometry. The debugging files `tetgen-tmpfile*` which TetGen writes into the current directory in such cases are removed. An `ArgumentError` is also raised if the mesh does not contain any tetrahedra, which happens if the outer boundary is not closed, or if a hole point does not lie inside a closed constraint surface.

The resulting `TetGenResult` can be converted into an `EuclideanComplex` with `tetgen_to_simplicial`.

This function requires `TetGen.jl` to be loaded.

Example

```
using TetGen, ConleyDynamics

dom = tetgen_domain_box([-2,-2,-2], [2,2,2])
V, T = trisurface_sphere([0,0,0], 1.0, 2)
```

```
dom = tetgen_domain_add_surface(dom, V, T)
res = tetgen_mesh(dom; maxvolume = 0.01)
ec  = tetgen_to_simplicial(res)
sphere_cells = tetgen_marker_cells(res, ec, 1)
```

source

ConleyDynamics.create_simplicial_delaunay3d – Function.

```
create_simplicial_delaunay3d(bmin::AbstractVector{<:Real},
                             bmax::AbstractVector{<:Real},
                             pdist::Real, attempts::Int;
                             p::Int=2, box_corners::Bool=false,
                             rng::AbstractRNG=Random.default_rng())
                             -> EuclideanComplex
```

Create a three-dimensional Delaunay triangulation of a random point sample inside a box. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The function selects a random sample of points inside the box `[bmin[1],bmax[1]] x [bmin[2],bmax[2]] x [bmin[3],bmax[3]]`, with minimum distance `pdist` between the points, see `poisson_points_box` for the meaning of `attempts`. It then computes the Delaunay tetrahedralization of these points with `TetGen.jl`, and returns it as an `EuclideanComplex`. The random number generator can be specified via `rng` for reproducible results.

As in the planar case `create_simplicial_delaunay`, the complex covers only the convex hull of the sample, so it does not fill the box completely near the boundary, and the tetrahedra on the boundary of the convex hull can be very flat. If `box_corners=true`, the eight corners of the box are added to the sample, so that the convex hull is the whole box. For quality meshes of a box, use `tetgen_mesh` with `tetgen_domain_box` instead.

This function requires `TetGen.jl` to be loaded.

source

The domain descriptions and the mesh results are plain Julia data. The following types and functions are part of `ConleyDynamics.jl` itself, and they can be used without loading `TetGen.jl`.

ConleyDynamics.TetDomain – Type.

```
TetDomain
```

Description of a three-dimensional domain for tetrahedral mesh generation with `TetGen.jl`, as a piecewise linear complex.

The struct has the following fields:

- `points::Vector{NTuple{3,Float64}}`: The vertices of the domain description.
- `facets::Vector{Vector{Int}}`: The facets, each given as a vector of indices into `points`. A facet with at least three vertices is a planar polygon, a facet with two vertices is a segment, and a facet with one vertex is a single point. All of them are contained in the final mesh.

- `markers::Vector{Int}`: One marker for every facet. By convention, the outer boundary has marker `0`, and the k-th added constraint surface or curve has marker k.
- `holes::Vector{NTuple{3,Float64}}`: For every hole, one point inside it. The region around this point which is enclosed by facets is not meshed.
- `regions::Vector{Tuple{NTuple{3,Float64},Int,Float64}}`: For every region, a point inside it, an integer attribute, and the maximum tetrahedron volume in that region. A nonpositive maximum volume means that there is no regional volume constraint.

A `TetDomain` is usually created with `tetgen_domain_box` or `tetgen_domain_from_surface`, and then extended with the functions `tetgen_domain_add_nodes`, `tetgen_domain_add_surface`, `tetgen_domain_add_curve`, `tetgen_domain_add_hole`, and `tetgen_domain_add_region`. None of these functions require `TetGen.jl`. The domain is then meshed with `tetgen_mesh`.

All facets have to be non-intersecting, except along shared vertices and edges. In particular, constraint surfaces cannot cross each other or the outer boundary.

source

`ConleyDynamics.TetGenResult` – Type.

```
TetGenResult
```

Tetrahedral mesh created by `tetgen_mesh`.

The struct has the following fields:

- `points::Vector{Vector{Float64}}`: The vertex coordinates of the mesh, including all points added by the mesh generator.
- `tetrahedra::Vector{Vector{Int}}`: The tetrahedra, as vectors of four indices into `points`.
- `trifaces::Vector{Vector{Int}}`: The boundary and constraint triangles, as vectors of three indices into `points`.
- `trifacemarkers::Vector{Int}`: The marker of the facet which contains each triangle in `trifaces`.
- `tetattributes::Vector{Int}`: The region attribute of every tetrahedron, or an empty vector if the domain has no regions.
- `curvesegments::Vector{Tuple{NTuple{3,Float64},NTuple{3,Float64},Int}}`: The segments of all constraint curves of the domain, given by their endpoints and their marker.

The mesh can be converted into an `EuclideanComplex` with `tetgen_to_simplicial`, and the cells on constraint surfaces and curves, or in regions, can be found with `tetgen_marker_cells` and `tetgen_region_cells`.

source

`ConleyDynamics.tetgen_domain_box` – Function.

```
tetgen_domain_box(bmin::AbstractVector{<:Real}, bmax::AbstractVector{<:Real};
                  markers::AbstractVector{<:Integer}=zeros(Int,6)) -> TetDomain
```

Create a TetDomain for the box [bmin[1],bmax[1]] x [bmin[2],bmax[2]] x [bmin[3],bmax[3]].

The domain consists of the eight corners of the box and its six faces, which are quadrilateral facets with outward orientation. By default, all faces have marker 0. The optional argument markers assigns individual markers to the faces, in the order x = bmin[1], x = bmax[1], y = bmin[2], y = bmax[2], z = bmin[3], and z = bmax[3]. This makes it possible to find the cells on some of the faces with tetgen_marker_cells, for example to describe an exit set. This is the three-dimensional analogue of delaunay_points_bnd_rectangle.

source

ConleyDynamics.tetgen_domain_from_surface – Function.

```
tetgen_domain_from_surface(V::AbstractVector, T::AbstractVector) -> TetDomain
```

Create a TetDomain whose outer boundary is the closed triangulated surface given by the vertices V and the triangles T.

The surface is usually created by one of the trisurface_* functions, and it has to be closed, i.e., every edge has to belong to exactly two triangles. Otherwise an ArgumentError is raised. Vertices with the same coordinates are identified before this check, so the surface can also be given as a list of separate triangles, each with its own copies of the vertices, as in STL files. Vertices which do not belong to any triangle are ignored. For example, the output of trisurface_sphere gives a ball, and the output of trisurface_torus gives a solid torus. All facets get marker 0.

source

ConleyDynamics.tetgen_domain_add_nodes – Function.

```
tetgen_domain_add_nodes(dom::TetDomain, newpoints::AbstractVector) -> TetDomain
```

Add nodes to a TetDomain.

The vector newpoints contains the coordinates of the new nodes, as vectors or tuples of length three. Each of these nodes inside the domain becomes a vertex of the final mesh. Nodes outside of the domain or in holes are ignored by the mesh generator, and nodes which coincide with points of the domain are merged with them. Nodes must not lie on a facet or a constraint curve without being one of its vertices, since TetGen reports such nodes as self-intersections and fails. The function returns a new TetDomain, and the original domain is not changed.

source

ConleyDynamics.tetgen_domain_add_surface – Function.

```
tetgen_domain_add_surface(dom::TetDomain, V::AbstractVector, T::AbstractVector;
                          marker::Union{Int,Nothing}=nothing,
                          hole::Bool=false,
                          region::Union{Tuple{Int,Real},Nothing}=nothing)
                          -> TetDomain
```

Add a triangulated constraint surface to a TetDomain.

The surface is given by its vertices V and its triangles T, for example as created by one of the `trisurface_*` functions. Its triangles become facets of the domain, so the final mesh will contain the surface as a union of triangles. The facets get the marker `marker`, and if no marker is given, the next free marker `maximum(dom.markers) + 1` is used, or 1 if this maximum is zero or less. The cells of the final complex on the surface can be found with `tetgen_marker_cells`.

For closed surfaces, two additional options are available, which cannot be combined:

- `hole=true` removes the inside of the surface from the domain.
- `region=(attr, maxvol)` marks the inside of the surface as a region with integer attribute `attr`, and with maximum tetrahedron volume `maxvol` (use a nonpositive value for no volume constraint). The tetrahedra in this region can be found with `tetgen_region_cells`.

In both cases, the surface has to be closed and consistently oriented, and a point inside the surface is determined automatically. This point avoids the surfaces which are already part of the domain, so that a region bounded by a sphere around a previously added hole is the shell between the two surfaces. If the surface consists of several connected components, such as two disjoint spheres, one point is determined for every component, so that all of them become holes or regions. Note that for nested components, such as two concentric spheres, both the inner ball and the shell between the spheres are removed or marked. If this is not intended, add the surfaces separately, starting with the inner one, or use `tetgen_domain_add_hole` or `tetgen_domain_add_region` with explicit points.

Vertices of the surface which coincide exactly with each other or with existing points of the domain are identified, before the surface is checked. Triangles which then have repeated vertices are rejected, and vertices which do not belong to any triangle are ignored. Apart from such shared vertices and edges, the surface must not intersect other surfaces or the boundary of the domain. For intersecting surfaces, use `Gmsh.jl` instead, see the manual. The function returns a new `TetDomain`, and the original domain is not changed.

source

`ConleyDynamics.tetgen_domain_add_curve` – Function.

```
tetgen_domain_add_curve(dom::TetDomain, curve::AbstractVector;
                        marker::Union{Int,Nothing}=nothing) -> TetDomain
```

Add a polygonal constraint curve to a `TetDomain`.

The vector `curve` contains the consecutive points of the curve, as vectors or tuples of length three. If the first and the last point agree up to `1e-9` times the size of the curve, the curve is closed. Every segment of the curve becomes a facet of the domain, so the final mesh will contain the curve as a union of edges. Repeated consecutive points and segments which occur more than once are ignored. The facets get the marker `marker`, and if no marker is given, the next free marker is used as in `tetgen_domain_add_surface`.

Vertices of the curve which coincide exactly with existing points of the domain are identified with them. Apart from such shared vertices, the curve must not intersect any facets of the domain, and in particular it cannot lie inside a face of the outer boundary. Curves outside of the domain or inside holes are ignored by TetGen, and `tetgen_mesh` prints a warning in this case. The function returns a new `TetDomain`, and the original domain is not changed.

source

`ConleyDynamics.tetgen_domain_add_hole` – Function.

```
tetgen_domain_add_hole(dom::TetDomain, point::AbstractVector{<:Real}) -> TetDomain
```

Add a hole to a `TetDomain`.

The point `point` has to lie inside a region of the domain which is completely enclosed by facets. This region is then removed from the mesh. A hole point which is not enclosed by facets removes the whole domain. The function returns a new `TetDomain`.

source

`ConleyDynamics.tetgen_domain_add_region` – Function.

```
tetgen_domain_add_region(dom::TetDomain, point::AbstractVector{<:Real},
                         attr::Int, maxvol::Real=-1.0) -> TetDomain
```

Add a region to a `TetDomain`.

The region is the part of the domain which contains `point` and which is bounded by facets. All tetrahedra in this region get the integer attribute `attr`, and they can be found with `tetgen_region_cells`. If `maxvol` is positive, the tetrahedra in the region will have volume at most `maxvol`, up to the tolerance of TetGen described in `tetgen_mesh`. The function returns a new `TetDomain`.

source

`ConleyDynamics.tetgen_to_simplicial` – Function.

```
tetgen_to_simplicial(res::TetGenResult; p::Int=2) -> EuclideanComplex
```

Convert a tetrahedral mesh created by `tetgen_mesh` into an `EuclideanComplex`. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

Vertices of the mesh which do not belong to any tetrahedron are removed. Vertex labels are zero-padded decimal strings as in `tetmesh_to_euclidean`.

source

`ConleyDynamics.tetgen_marker_cells` – Function.

```
tetgen_marker_cells(res::TetGenResult, ec::EuclideanComplex, marker::Int;
                    closure::Bool=true) -> Vector{Int}
```

Find the cells of a complex which lie on the facets with a given marker.

The complex `ec` has to be created from `res` via `tetgen_to_simplicial`, otherwise an `ArgumentError` is raised. The function returns the indices of all triangles of `ec` which lie on facets of the domain with marker `marker`, as well as all edges on constraint curves with this marker. If `closure=true` (default), all faces of these cells are included as well, so the result is a closed subcomplex. For a constraint surface, this gives the surface as a two-dimensional subcomplex, and for the marker `0` it gives the outer boundary of the domain.

The returned cells can be used, for example, to define exit sets or to restrict a multivector field to a constraint surface.

source

`ConleyDynamics.tetgen_region_cells` – Function.

```
tetgen_region_cells(res::TetGenResult, ec::EuclideanComplex, attr::Int;
                    closure::Bool=true) -> Vector{Int}
```

Find the tetrahedra of a complex which lie in the regions with a given attribute.

The complex `ec` has to be created from `res` via `tetgen_to_simplicial`, otherwise an `ArgumentError` is raised, and the domain has to contain regions, see `tetgen_domain_add_region` and `tetgen_domain_add_surface`. The function returns the indices of all tetrahedra with region attribute `attr`. If `closure=true` (default), all faces of these tetrahedra are included as well, so the result is a closed subcomplex.

Note that TetGen assigns new attributes, larger than all given ones, to the parts of the domain which do not contain a region point. For example, if a single region with attribute 1 is defined inside a closed constraint surface, then the tetrahedra outside of this surface get attribute 2. The attributes of all tetrahedra are stored in `res.tetattributes`.

source

20.3 Gmsh.jl

The following functions are available when `Gmsh.jl` is loaded alongside `ConleyDynamics.jl`.

`ConleyDynamics.gmsh_to_simplicial` – Function.

```
gmsh_to_simplicial(; dim::Int=3, p::Int=2,
                   physical::Union{Tuple{Integer,Integer},Nothing}=nothing)
                   -> EuclideanComplex
```

Convert the mesh of the current `Gmsh.jl` model into an `EuclideanComplex`. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The function reads the mesh elements of dimension `dim` from the current model, which has to be meshed already, for example by calling `gmsh.model.mesh.generate(3)`. For `dim=3`, these are the tetrahedra, and the result is a tetrahedral mesh. For `dim=2`, the triangles are used, which gives a surface complex embedded in three dimensions, and for `dim=1`, the edges are used, which gives a graph. All faces of these elements are added automatically. Since `Gmsh` only stores the triangles on the surfaces and the edges on the curves of the model, and not the interior faces of the tetrahedra, `dim=2` gives only the triangles on the surfaces of the model, and `dim=1` only the edges on its curves. If `physical=(pdim, ptag)` is given, only the elements of the physical group with dimension `pdim` and tag `ptag` are used, and `pdim` has to equal `dim`.

Only linear simplices are supported, i.e., edges, triangles, and tetrahedra whose nodes are their vertices. If the elements of dimension `dim` include other types, an `ArgumentError` is raised. This happens for quadrangles, hexahedra, prisms, and pyramids, which `Gmsh` creates with its recombination options. It also happens for second order elements, which `gmsh.model.mesh.setOrder(2)` creates by adding nodes on the edges

to follow curved geometry. Only the elements of dimension `dim` are checked, so that, for example, a hexahedral mesh can still be converted with `dim=1`. Nodes which do not belong to any of the used elements are dropped. Vertex labels are zero-padded decimal strings as in `tetmesh_to_euclidean`.

If two distinct nodes of the used elements have the same coordinates, a warning is printed. This usually means that touching or intersecting entities were meshed separately, without `gmsh_common_refinement` or a `gmsh.model.occ.fragment` operation, so that the complex is not connected across them. A warning is also printed if some of the used entities of dimension `dim`, i.e., surfaces for `dim=2` and volumes for `dim=3`, contain no mesh elements. This happens if the meshing failed for them, so that the complex has holes there.

The function only reads the current model. It neither initializes nor finalizes `Gmsh`, so the model can be used afterwards, for example to find the cells of physical groups with `gmsh_marker_cells`.

This function requires `Gmsh.jl` to be loaded.

Example

```
using Gmsh: gmsh
using ConleyDynamics

gmsh.initialize()
gmsh.model.occ.addBox(0, 0, 0, 1, 1, 1)
gmsh.model.occ.synchronize()
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.2)
gmsh.model.mesh.generate(3)
ec = gmsh_to_simplicial()
gmsh.finalize()
```

source

`ConleyDynamics.gmsh_file_to_simplicial` – Function.

```
gmsh_file_to_simplicial(filename::AbstractString; dim::Int=3, p::Int=2)
                        -> EuclideanComplex
```

Read a mesh or geometry file with `Gmsh.jl` and convert it into an `EuclideanComplex`. The complex is over the rationals if `p=0`, and over `GF(p)` if `p>0`.

The file can be in any format which `Gmsh` can open, such as a mesh file (`.msh`) or a geometry file (`.geo`). If the file does not contain mesh elements of dimension `dim`, a mesh of this dimension is generated with the mesh options specified in the file. The mesh is then converted with `gmsh_to_simplicial`.

If `Gmsh` is not initialized, the function initializes it and finalizes it at the end. Otherwise, the file is opened in the running `Gmsh` session. This creates a new model, which becomes the current model, while the previous models remain available.

This function requires `Gmsh.jl` to be loaded.

source

`ConleyDynamics.gmsh_marker_cells` – Function.

```
gmsh_marker_cells(ec::EuclideanComplex, dim::Integer, tag::Integer;
                  physical::Bool=true, closure::Bool=true) -> Vector{Int}
```

Find the cells of a complex which belong to a physical group or an entity of the current `Gmsh.jl` model.

The complex `ec` has to be created from the current model, for example with `gmsh_to_simplicial`. If `physical=true` (default), the function returns the indices of the cells of `ec` formed by the mesh elements of the physical group with dimension `dim` and tag `tag`. If `physical=false`, the elements of the model entity with dimension `dim` and tag `tag` are used instead. Depending on `dim`, these are vertices, edges, triangles, or tetrahedra. Elements which are not cells of `ec` are ignored. If `closure=true` (default), all faces of these cells are included as well, so the result is a closed subcomplex.

Physical groups are the most convenient way to keep track of geometry after boolean operations such as `gmsh.model.occ.fragment`, since these operations renumber the entities. The cells of `ec` are identified by the coordinates of their vertices, so the numbering of the nodes does not matter.

This function requires `Gmsh.jl` to be loaded.

source

`ConleyDynamics.gmsh_common_refinement` – Function.

```
gmsh_common_refinement(objects::AbstractVector;
                       names::Union{AbstractVector{<:AbstractString},Nothing}=nothing)
                       -> Vector{Int}
```

Split the given geometric objects of the current `Gmsh.jl` model along all their mutual intersections, and create one physical group for every object.

Every element of `objects` describes one geometric object by a single entity `(dim, tag)` or by a vector of such entities, all of the same dimension. For example, a box is given by `(3, box)`, a rectangle by `(2, rect)`, and the surface of a sphere by the vector of its boundary surfaces, see the manual. The objects have to be created with the OpenCASCADE kernel `gmsh.model.occ`.

The function calls `gmsh.model.occ.fragment` for all entities of all objects, which replaces them by pieces that only meet along common faces, edges, and points. Afterwards, the mesh of every object consists of the mesh elements of its pieces, and any two objects share exactly the mesh elements of their intersection. In other words, the mesh is a common refinement of all objects. For every object, the function then creates a physical group of the same dimension, which contains all pieces of the object, and it returns the tags of these groups in the order of `objects`. If `names` is given, it contains a name for every physical group.

After meshing, the cells of the objects are found with `gmsh_marker_cells`, and the cells on the intersection of two objects are the intersection of their cell lists. Pieces of an object which lie outside of all volumes, for example the part of a plane outside of a box, are not part of the tetrahedral mesh, and their elements are ignored by `gmsh_marker_cells`.

This function requires `Gmsh.jl` to be loaded.

Example

```
using Gmsh: gmsh
using ConleyDynamics

gmsh.initialize()
```

```
box    = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
sphere = gmsh.model.occ.addSphere(0, 0, 0, 1.0)
plane  = gmsh.model.occ.addRectangle(-2, -2, 0.3, 4, 4)
gmsh.model.occ.synchronize()
sphsurf = gmsh.model.getBoundary([(3, sphere)], false, false)
gmsh.model.occ.remove([(3, sphere)])

pgbox, pgplane, pgsphere = gmsh_common_refinement([(3, box), (2, plane), sphsurf])
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.25)
gmsh.model.mesh.generate(3)

ec     = gmsh_to_simplicial()
circle = intersect(gmsh_marker_cells(ec, 2, pgplane), gmsh_marker_cells(ec, 2, pgsphere))
gmsh.finalize()
```

source

ConleyDynamics.gmsh_add_trisurface – Function.

```
gmsh_add_trisurface(V::AbstractVector, T::AbstractVector)
                    -> Vector{Tuple{Int,Int}}
```

Add a triangulated surface to the current `Gmsh.jl` model, so that it can be used as an object of `gmsh_common_refinement`.

The surface is given by the vertices `V` and the triangles `T`, for example as created by one of the `trisurface_*` functions, or as read from an STL file. Every triangle becomes a flat surface of the OpenCASCADE kernel `gmsh.model.occ`, and adjacent triangles share their points and edges, so the surface is the piecewise linear interpolation of its vertices. The function returns the entities `(2, tag)` of these surfaces, which form one object of `gmsh_common_refinement`. The surface does not have to be closed or consistently oriented, and an edge may belong to more than two triangles.

Vertices with the same coordinates are identified, so the surface can also be given as a list of separate triangles, each with its own copies of the vertices. Vertices which do not belong to any triangle are ignored. An `ArgumentError` is raised for triangles with repeated vertices, for degenerate triangles whose vertices lie on a line, for triangles which occur more than once, and for triangles with an edge of length at most `2e-7`, since OpenCASCADE identifies points closer than `1e-7`. If OpenCASCADE fails nevertheless, the entities created so far are removed from the model before the error is passed on.

Since every edge of the triangulation becomes a curve of the model, the mesh near the surface is at least as fine as the triangulation, even if `Mesh.MeshSizeMax` is larger. The triangulation should therefore not be finer than needed. The function calls `gmsh.model.occ.synchronize()`.

This function requires `Gmsh.jl` to be loaded.

Example

```
using Gmsh: gmsh
using ConleyDynamics

gmsh.initialize()
box  = gmsh.model.occ.addBox(-2, -2, -2, 4, 4, 4)
V, T = trisurface_parametric((x, y) -> [x, y, (x^2 + y^2) / 2 - 1],
```

```
                            (-1.5, 1.5), (-1.5, 1.5), 12, 12)
surf = gmsh_add_trisurface(V, T)

pgbox, pgsurf = gmsh_common_refinement([(3, box), surf])
gmsh.option.setNumber("Mesh.MeshSizeMax", 0.25)
gmsh.model.mesh.generate(3)

ec    = gmsh_to_simplicial()
cells = gmsh_marker_cells(ec, 2, pgsurf)
gmsh.finalize()
```

source

Chapter 21

Plotting Functions

21.1 Visualizing Simplicial Complexes (Luxor)

`ConleyDynamics.plot_planar_simplicial` – Function.

```
plot_planar_simplicial(sc::AbstractComplex,
                       coords::Vector{<:Vector{<:Real}},
                       fname::String;
                       [mvf::CellSubsets=Vector{Vector{Int}}([]),]
                       [labeldir::Vector{<:Real}=Vector{Int}([]),]
                       [labeldis::Real=8,]
                       [hfac::Real=1.2,]
                       [vfac::Real=1.2,]
                       [sfac::Real=0,]
                       [pdim::Vector{Bool}=[true,true,true],]
                       [pv::Bool=false])
```

Create an image of a planar simplicial complex, and if specified, a Forman vector field on it.

The vector `coords` contains coordinates for every one of the vertices of the simplicial complex `sc`. The image will be saved in the file with name `fname`, and the ending determines the image type. Accepted are `.pdf`, `.svg`, `.png`, and `.eps`.

If the optional `mvf` is specified and is a Forman vector field, then this Forman vector field is drawn as well. The optional vector `labeldir` contains directions for the vertex labels, and `labeldis` the distance from the vertex. The directions have to be reals between 0 and 4, with 0,1,2,3 corresponding to E,N,W,S. The optional constants `hfac` and `vfac` contain the horizontal and vertical scale vectors, while `sfac` describes a uniform scale. If `sfac=0` the latter is automatically determined. The vector `pdim` specifies in which dimensions cells are drawn; the default shows vertices, edges, and triangles. Finally if one passes the argument `pv=true`, then in addition to saving the file a preview is displayed.

Examples

Suppose we have created a simplicial complex using the commands

```
sc, coords = create_simplicial_delaunay(300, 300, 30, 20)
fname = "sc_plot_test.pdf"
```

Then the following code creates an image of the simplicial complex without labels, but with a preview:

```
plot_planar_simplicial(sc, coords, fname, pv=true)
```

If we want to see the labels, we can use

```
ldir = fill(0.5, sc.ncells);
plot_planar_simplicial(sc, coords, fname, labeldir=ldir, labeldis=10, pv=true)
```

This command puts all labels in the North-East direction at a distance of 10.

source

```
plot_planar_simplicial(ec::EuclideanComplex,
                       fname::String;
                       kwargs...)
```

Create an image of a planar simplicial complex from an `EuclideanComplex`.

The vertex positions for every cell are taken directly from `ec.coords[k]`, which stores the complete vertex coordinates for each cell `k`. This means the function works correctly even when some vertices are no longer present as cells in the complex (e.g. after subcomplex creation), as long as the higher-dimensional cells still carry their coordinates.

source

`ConleyDynamics.plot_planar_simplicial_morse` – Function.

```
plot_planar_simplicial_morse(sc::AbstractComplex,
                             coords::Vector{<:Vector{<:Real}},
                             fname::String,
                             morsesets::CellSubsets;
                             [hfac::Real=1.2,]
                             [vfac::Real=1.2,]
                             [sfac::Real=0,]
                             [pdim::Vector{Bool}=[false,true,true],]
                             [pv::Bool=false,]
                             [ci::Bool=false])
```

Create an image of a planar simplicial complex, together with Morse sets, or also selected multivectors.

The vector `coords` contains coordinates for every one of the vertices of the simplicial complex `sc`. The image will be saved in the file with name `fname`, and the ending determines the image type. Accepted are `.pdf`, `.svg`, `.png`, and `.eps`.

The vector `morsesets` contains a list of Morse sets, or more general, subsets of the simplicial complex. For every `k`, the set described by `morsesets[k]` will be shown in a distinct color.

The optional constants `hfac` and `vfac` contain the horizontal and vertical scale vectors for the margins, while `sfac` describes a uniform scale. If `sfac=0` the latter is automatically determined. The vector `pdim` specifies in which dimensions cells are drawn; the default only shows edges and triangles. If one passes the argument `pv=true`, then in addition to saving the file a preview is displayed. Lastly, passing the argument `ci=true` will color code the Morse sets according to their respective Conley indices.

source

```
plot_planar_simplicial_morse(ec::EuclideanComplex,
                             fname::String,
                             morsesets::CellSubsets;
                             kwargs...)
```

Create an image of a planar simplicial complex with Morse sets from an `EuclideanComplex`.

The vertex positions for every cell are taken directly from `ec.coords[k]`, which stores the complete vertex coordinates for each cell k. This means the function works correctly even when some vertices are no longer present as cells in the complex (e.g. after subcomplex creation), as long as the higher-dimensional cells still carry their coordinates.

source

21.2 Visualizing Cubical Complexes (Luxor)

`ConleyDynamics.plot_planar_cubical` – Function.

```
plot_planar_cubical(cc::AbstractComplex,
                    coords::Vector{<:Vector{<:Real}},
                    fname::String;
                    [hfac::Real=1.2,]
                    [vfac::Real=1.2,]
                    [cubefac::Real=0,]
                    [pdim::Vector{Bool}=[true,true,true],]
                    [pv::Bool=false])
```

Create an image of a planar cubical complex.

The vector `coords` contains coordinates for every one of the vertices of the cubical complex `cc`. The image will be saved in the file with name `fname`, and the ending determines the image type. Accepted are `.pdf`, `.svg`, `.png`, and `.eps`. The optional constants `hfac` and `vfac` contain the horizontal and vertical scale vectors. The optional argument `cubefac` specifies the side length of an elementary cube for plotting, and it will be automatically determined otherwise. The vector `pdim` specifies which cell dimensions should be plotted, with `pdim[k]` representing dimension `k-1`. Finally if one passes the argument `pv=true`, then in addition to saving the file a preview is displayed.

Examples

Suppose we have created a cubical complex using the commands

```
cubes = ["00.11", "01.01", "02.10", "11.10", "11.01", "22.00"]
coords = [[0,0],[0,1],[0,2],[1,0],[1,1],[1,2],[2,1],[2,2]]
cc = create_cubical_complex(cubes)
fname = "cc_plot_test.pdf"
```

Then the following code creates an image of the simplicial complex without labels, but with a preview:

```
plot_planar_cubical(cc, coords, fname, pv=true)
```

If one only wants to plot the edges in the complex, but not the vertices or rectangles, then one can use:

```
plot_planar_cubical(cc, coords, fname, pv=true, pdim=[false,true,false])
```

source

```
plot_planar_cubical(cc::AbstractComplex,
                    fname::String;
                    [hfac::Real=1.2,]
                    [vfac::Real=1.2,]
                    [cubefac::Real=0,]
                    [pdim::Vector{Bool}=[true,true,true],]
                    [pv::Bool=false])
```

Create an image of a planar cubical complex.

This is an alternative method which does not require the specification of the vertex coordinates. They will be taken from the cube vertex labels.

source

```
plot_planar_cubical(ec::EuclideanComplex,
                    fname::String;
                    kwargs...)
```

Create an image of a planar cubical complex from an `EuclideanComplex`.

The vertex positions for every cell are taken directly from `ec.coords[k]`, which stores the complete vertex coordinates for each cell k. This means the function works correctly even when some vertices are no longer present as cells in the complex (e.g. after subcomplex creation), as long as the higher-dimensional cells still carry their coordinates.

source

`ConleyDynamics.plot_planar_cubical_morse` – Function.

```
plot_planar_cubical_morse(cc::AbstractComplex,
                          coords::Vector{<:Vector{<:Real}},
                          fname::String,
                          morsesets::CellSubsets;
                          [hfac::Real=1.2,]
                          [vfac::Real=1.2,]
                          [cubefac::Real=0,]
                          [pdim::Vector{Bool}=[false,true,true],]
                          [pv::Bool=false])
```

Create an image of a planar cubical complex, together with Morse sets, or also selected multivectors.

The vector `coords` contains coordinates for every one of the vertices of the cubical complex `cc`. The image will be saved in the file with name `fname`, and the ending determines the image type. Accepted are `.pdf`, `.svg`, `.png`, and `.eps`.

The vector `morsesets` contains a list of Morse sets, or more general, subsets of the cubical complex. For every k, the set described by `morsesets[k]` will be shown in a distinct color.

The optional constants `hfac` and `vfac` contain the horizontal and vertical scale vectors for the margins, while `cubefac` describes a uniform scale. If `cubefac=0` the latter is automatically determined. The vector `pdim` specifies in which dimensions cells are drawn; the default only shows edges and squares. Finally if one passes the argument `pv=true`, then in addition to saving the file a preview is displayed.

source

```
plot_planar_cubical_morse(cc::AbstractComplex,
                          fname::String,
                          morsesets::CellSubsets;
                          [hfac::Real=1.2,]
                          [vfac::Real=1.2,]
                          [cubefac::Real=0,]
                          [pdim::Vector{Bool}=[false,true,true],]
                          [pv::Bool=false])
```

Create an image of a planar cubical complex, together with Morse sets, or also selected multivectors.

This is an alternative method which does not require the specification of the vertex coordinates. They will be taken from the cube vertex labels.

source

```
plot_planar_cubical_morse(ec::EuclideanComplex,
                          fname::String,
                          morsesets::CellSubsets;
                          kwargs...)
```

Create an image of a planar cubical complex with Morse sets from an `EuclideanComplex`.

The vertex positions for every cell are taken directly from `ec.coords[k]`, which stores the complete vertex coordinates for each cell k. This means the function works correctly even when some vertices are no longer present as cells in the complex (e.g. after subcomplex creation), as long as the higher-dimensional cells still carry their coordinates.

source

21.3 Visualizing Acyclic Partition Strata (Luxor)

`ConleyDynamics.plot_morse_strata` – Function.

```
plot_morse_strata(lc, ap, fname; A=nothing, title="", figw=nothing, figh=nothing, pv=false)
```

Hasse-style diagram of the Morse-vector strata of AP^d(X). Nodes are distinct Morse vectors, drawn as circles labeled `(M)` and `n=...` (the number of AP^d(X) partitions with that Morse vector), placed by level sum(M) (monotone under refinement) – note that "adjacent" levels need not differ by 1 in sum(M): every weight-1 jump already raises it by 2, since ek + e{k-1} sums to 2.

Edges are drawn between every pair of strata found adjacent by `stratum_adjacency`, labeled `|c|=<weight>, <covered>/<total>` (how many of the coarser stratum's partitions admit an atomic refinement into the finer one, out of how many total), and styled by the classification spelled out in the figure's own legend:

- solid green: weight |c| = 1 (always uniform)
- solid gray: weight |c| >= 2, but every partition happens to be covered for this particular complex anyway
- dashed red: weight |c| >= 2 and not every partition is covered (a realized counterexample to the naive question)

An edge that skips an intermediate level is bent aside so it doesn't draw on top of the shorter edges passing through that level.

`figw` and `figh` default to `nothing`, which auto-sizes the canvas: the full layout (nodes and every bend point) is computed first, then the canvas is sized and the whole picture shifted to fit it exactly (with a floor wide enough for the legend text even when there's very little else to draw), so nothing is ever cut off regardless of how far a bend ends up needing to reach. Pass explicit values to override – explicit `figw` and `figh` are used as-is, not auto-corrected. `title`, if given, is drawn at the top of the figure.

See also `plot_morse_reachability` for the analogous diagram of eventual (multi-step) reachability between strata, rather than single-step adjacency.

Examples

Consider the triangle ABC with a pendant edge CD attached at C:

```
labels    = ["A", "B", "C", "D"]
simplices = [[1, 2, 3], [3, 4]]
lc = create_simplicial_complex(labels, simplices)

ap = construct_ap_space(lc)
plot_morse_strata(lc, ap, "strata.pdf", title="Triangle ABC with pendant edge CD")
```

If both `plot_morse_strata` and `plot_morse_reachability` are wanted for the same complex, precompute `atomic_distances` once and pass it to both, since it is the expensive step and is otherwise recomputed by each call:

```
A = atomic_distances(lc, ap)
plot_morse_strata(lc, ap, "adjacency.pdf"; A=A)
plot_morse_reachability(lc, ap, "reachability.pdf"; A=A)
```

source

`ConleyDynamics.plot_morse_reachability` – Function.

```
plot_morse_reachability(lc, ap, fname; A=nothing, title="", figw=nothing, figh=nothing,
↪  pv=false)
```

Hasse-style diagram of eventual reachability between the Morse-vector strata of AP^d(X), i.e. the multi-step generalization of `plot_morse_strata`: an edge `M2 -> M1` here means some element of `AP_{M2}` reaches `AP_{M1}` via a chain of atomic refinements of any length (`stratum_reachability`), not just a single one (`stratum_adjacency`). Nodes are laid out exactly as in `plot_morse_strata` – distinct Morse vectors, drawn as circles labeled `(M)` and `n=...`, placed by level sum(M).

Because eventual reachability is transitive, `stratum_reachability` reports an edge for every comparable pair of strata reachable by some chain – far denser than the single-step covering relation

`plot_morse_strata` draws. To keep the picture legible, only the transitive reduction of that relation is drawn, computed by `_stratum_transitive_reduction`. An edge `M2 -> M1` is omitted whenever it is already implied by some other drawn path `M2 -> ... -> M1`. This means the absence of a drawn edge between two strata does not mean one is unreachable from the other – only that a shown path already establishes it. The full (unreduced) edge set and underlying element-level reachability are both available in the return value for anything needing the raw data.

Edges are labeled `|c|=<weight>, <covered>/<total>` (how many of the coarser stratum's partitions eventually reach the finer one, out of how many total), and colored:

- solid green: uniform – every partition of the coarser stratum eventually reaches the finer one
- dashed red: not uniform – at least one partition of the coarser stratum never reaches the finer one

Unlike `plot_morse_strata`, there is no weight-based three-way split: the weight-1 uniformity guarantee is specifically about single atomic refinements and does not extend to multi-step chains, so no analogous "guaranteed" tier exists here to draw a distinction around.

`figw`, `figh`, `title`, and `pv` behave exactly as in `plot_morse_strata`.

Examples

Consider the triangle `ABC` with a pendant edge `CD` attached at `C`:

```
labels    = ["A", "B", "C", "D"]
simplices = [[1, 2, 3], [3, 4]]
lc = create_simplicial_complex(labels, simplices)

ap = construct_ap_space(lc)
plot_morse_reachability(lc, ap, "reachability.pdf",
                        title="Triangle ABC with pendant edge CD")
```

If both `plot_morse_reachability` and `plot_morse_strata` are wanted for the same complex, precompute `atomic_distances` once and pass it to both, since it is the expensive step and is otherwise recomputed by each call:

```
A = atomic_distances(lc, ap)
plot_morse_reachability(lc, ap, "reachability.pdf"; A=A)
plot_morse_strata(lc, ap, "adjacency.pdf"; A=A)
```

source

21.4 Plot Data Types (Plots.jl)

`ConleyDynamics.SimplicialComplexPlot` – Type.

```
SimplicialComplexPlot
```

Wrapper type for plotting a planar simplicial complex via Plots.jl.

Constructed automatically by `plot_simplicial`, but instances can also be passed directly to `Plots.plot`.

source

`ConleyDynamics.CubicalComplexPlot` – Type.

```
CubicalComplexPlot
```

Wrapper type for plotting a planar cubical complex via Plots.jl.

source

`ConleyDynamics.SimplicialMorsePlot` – Type.

```
SimplicialMorsePlot
```

Wrapper type for plotting a planar simplicial complex with Morse sets via Plots.jl.

source

`ConleyDynamics.CubicalMorsePlot` – Type.

```
CubicalMorsePlot
```

Wrapper type for plotting a planar cubical complex with Morse sets via Plots.jl.

source

`ConleyDynamics.MVFPlot` – Type.

```
MVFPlot
```

Wrapper type for plotting a multivector field on a planar complex via Plots.jl. Each multivector is rendered as a stadium (pill) shaped region.

source

`ConleyDynamics.MVRegionPlot` – Type.

```
MVRegionPlot
```

Wrapper type for plotting a single multivector on a planar complex via Plots.jl. The multivector is rendered as an inflated convex hull per maximal cell.

source

21.5 Visualizing Simplicial Complexes (Plots.jl)

`ConleyDynamics.plot_simplicial` – Function.

```
plot_simplicial(ec::EuclideanComplex; kwargs...) -> Plots.Plot
```

Plot a planar simplicial complex. Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `mvf::CellSubsets=[]`: Forman vector field to overlay (arrows + critical cells)
- `labeldir::Vector{<:Real}=[]`: label directions per cell (0=E,1=N,2=W,3=S)
- `labeldis::Real=0.05`: label offset in data-space units
- `pdim::Vector{Bool}=[true,true,true]`: which dimensions (0,1,2) to draw

source

`ConleyDynamics.plot_simplicial_morse` – Function.

```
plot_simplicial_morse(ec::EuclideanComplex, morsesets::CellSubsets;
                      kwargs...) -> Plots.Plot
```

Plot a planar simplicial complex with Morse sets highlighted.

Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `pdim::Vector{Bool}=[false,true,true]`: which dimensions (0,1,2) to draw
- `ci::Bool=false`: color Morse sets by their Conley index
- `addcritical::Bool=false`: when true, cells absent from morsesets are shown as implicit singletons

source

`ConleyDynamics.plot_simplicial_mvf` – Function.

```
plot_simplicial_mvf(ec::EuclideanComplex, mvf::CellSubsets;
                    kwargs...) -> Plots.Plot
```

Plot a planar simplicial complex with multivector field regions shaded. Each multivector is rendered as a single colored region, inset from cell boundaries so adjacent multivectors are visually distinct. Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `pdim::Vector{Bool}=[true,true,true]`: which dimensions to show in background
- `tubefac::Real=0.05`: tube half-width as fraction of average edge length
- `mvfcolor::Union{String,Vector{String}}="darkorange"`: color(s) for multivector regions; a single string applies to all, a vector cycles through multivectors in order
- `mvfalpha::Real=0.3`: fill opacity (0.0 = transparent, 1.0 = opaque)
- `addcritical::Bool=true`: when true, cells absent from the MVF are shown as implicit singletons

- `ci::Bool=false`: color each multivector by its Conley index instead of using `mvfcolor`

source

`ConleyDynamics.plot_simplicial_mv` – Function.

```
plot_simplicial_mv(ec::EuclideanComplex, mv; kwargs...) -> Plots.Plot
```

Plot a single multivector on a planar simplicial complex background. The multivector is rendered as an inflated convex hull of barycenters, one hull per maximal cell of the multivector. Requires `using Plots` before `using ConleyDynamics`.

`mv` may be a `Vector{Int}` (cell indices) or `Vector{String}` (cell labels).

Optional keyword arguments:

- `pdim::Vector{Bool}=[true,true,true]`: which background dimensions to draw
- `tubefac::Real=0.05`: inflation radius as fraction of average edge length
- `mvfcolor::String="darkorange"`: X11 color name for the region
- `mvfalpha::Real=0.3`: fill opacity (0.0 = transparent, 1.0 = opaque)

source

21.6 Visualizing Cubical Complexes (Plots.jl)

`ConleyDynamics.plot_cubical` – Function.

```
plot_cubical(ec::EuclideanComplex; kwargs...) -> Plots.Plot
```

Plot a planar cubical complex. Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `mvf::CellSubsets=[]`: multivector field to overlay (arrows + critical-cell dots)
- `pdim::Vector{Bool}=[true,true,true]`: which dimensions (0,1,2) to draw

source

`ConleyDynamics.plot_cubical_morse` – Function.

```
plot_cubical_morse(ec::EuclideanComplex, morsesets::CellSubsets;
                   kwargs...) -> Plots.Plot
```

Plot a planar cubical complex with Morse sets highlighted.

Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `pdim::Vector{Bool}=[false,true,true]`: which dimensions (0,1,2) to draw
- `ci::Bool=false`: color Morse sets by their Conley index
- `addcritical::Bool=false`: when true, cells absent from morsesets are shown as implicit singletons

source

`ConleyDynamics.plot_cubical_mvf` – Function.

```
plot_cubical_mvf(ec::EuclideanComplex, mvf::CellSubsets;
                 kwargs...) -> Plots.Plot
```

Plot a planar cubical complex with multivector field regions shaded. Requires `using Plots` before `using ConleyDynamics`.

Optional keyword arguments:

- `pdim::Vector{Bool}=[true,true,true]`: which dimensions to show in background
- `tubefac::Real=0.05`: tube half-width as fraction of average edge length
- `mvfcolor::Union{String,Vector{String}}="darkorange"`: color(s) for multivector regions
- `mvfalpha::Real=0.3`: fill opacity (0.0 = transparent, 1.0 = opaque)
- `addcritical::Bool=true`: when true, cells absent from the MVF are shown as implicit singletons
- `ci::Bool=false`: color each multivector by its Conley index instead of using `mvfcolor`

source

`ConleyDynamics.plot_cubical_mv` – Function.

```
plot_cubical_mv(ec::EuclideanComplex, mv; kwargs...) -> Plots.Plot
```

Plot a single multivector on a planar cubical complex background. See `plot_simplicial_mv` for details.

source

21.7 Visualizing Complexes in Three Dimensions (Makie)

`ConleyDynamics.plot_simplicial3d` – Function.

```
plot_simplicial3d(ec::EuclideanComplex; kwargs...) -> Makie.Figure
plot_simplicial3d!(ax, ec::EuclideanComplex; kwargs...)
```

Plot a simplicial complex embedded in three dimensions with Makie.

The first form creates a new figure and returns it, the second form draws into an existing Makie axis `ax`, such as an `LScene` or an `Axis3`. The complex can be a tetrahedral mesh, or a two-dimensional complex embedded in three dimensions, such as a triangulated surface. By default, only the boundary surface of the tetrahedra is shown, semi-transparent, together with its edges.

Optional keyword arguments:

- `pdim::Vector{Bool}=[false,true,true,true]`: which cell dimensions (0, 1, 2, 3) to draw. Three-dimensional cells are drawn as surfaces.
- `hull_only::Bool=true`: draw only the boundary surface of the tetrahedra and its edges, instead of all faces and edges.
- `clip=nothing`: a pair `(n, offset)` with a normal vector `n` and a number `offset`. Cells which are not faces of other cells, such as the tetrahedra of a mesh, are removed if their centroid `c` satisfies `n⋅c > offset`, together with all faces which do not belong to a remaining cell. This shows the interior of the complex.
- `alpha::Real=0.3`: opacity of the faces.
- `facecolor`, `edgecolor`, `vertexcolor`: colors of the faces, edges, and vertices. The defaults are `colorant"steelblue1"`, `colorant"royalblue3"`, and `colorant"royalblue4"`.
- `linewidth::Real=0.75`: width of the edges.
- `linealpha::Real=1.0`: opacity of the edges.
- `markersize=nothing`: radius of the vertex spheres, in data units. The default is a fraction of the typical edge length.

The first form also accepts the following keyword arguments:

- `axis::Symbol=:scene`: `:scene` creates an interactive LScene without axis decorations, and `:axis3` creates an `Axis3` with ticks and equal aspect ratios.
- `size=(800,600)`: the figure size.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`. For interactive rotation and the best handling of transparency, use `GLMakie`. For figures in files, use `CairoMakie`.

source

`ConleyDynamics.plot_simplicial3d!` – Function.

```
plot_simplicial3d!(ax, args...; kwargs...)
```

Draw into an existing Makie axis `ax`. See `plot_simplicial3d`.

source

`ConleyDynamics.plot_simplicial3d_morse` – Function.

```
plot_simplicial3d_morse(ec::EuclideanComplex, morsesets::CellSubsets;
                        kwargs...) -> Makie.Figure
plot_simplicial3d_morse!(ax, ec::EuclideanComplex, morsesets::CellSubsets;
                         kwargs...)
```

Plot a simplicial complex embedded in three dimensions with Morse sets highlighted, using Makie.

The complex is drawn faintly in the background, and every Morse set is drawn in its own color, according to its dimension: tetrahedra by the boundary surface of their union, triangles which are not faces of tetrahedra in the Morse set as triangles, edges which are not faces of higher-dimensional cells as thick lines, and isolated vertices as spheres.

Optional keyword arguments:

- `ci::Bool=false`: color the Morse sets by their Conley index.
- `addcritical::Bool=false`: cells which do not belong to any Morse set are shown as additional singleton Morse sets.
- `colors=nothing`: a vector of colors for the Morse sets. By default, the same distinguishable colors as in the planar plots are used.
- `alpha::Real=0.9`: opacity of the Morse sets.
- `bgalpha::Real=0.08`: opacity of the background faces.
- `pdim::Vector{Bool}=[false,true,true,true]`: which cell dimensions to draw in the background.
- `hull_only`, `clip`, `linewidth`, `markersize`: as for `plot_simplicial3d`. The `clip` keyword applies to the background and the Morse sets, and `linewidth=4` applies to the edges of Morse sets.

The first form also accepts the keywords `axis` and `size`, see `plot_simplicial3d`.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`.

source

`ConleyDynamics.plot_simplicial3d_morse!` – Function.

```
plot_simplicial3d_morse!(ax, args...; kwargs...)
```

Draw into an existing Makie axis `ax`. See `plot_simplicial3d_morse`.

source

`ConleyDynamics.plot_simplicial3d_mv` – Function.

```
plot_simplicial3d_mv(ec::EuclideanComplex, mv; kwargs...) -> Makie.Figure
plot_simplicial3d_mv!(ax, ec::EuclideanComplex, mv; kwargs...)
```

Plot a single multivector of a simplicial complex embedded in three dimensions, using Makie.

The multivector `mv` is a vector of cell indices or labels. It is drawn in the same way as a Morse set in `plot_simplicial3d_morse`, on top of the faint background complex. This is useful for checking the multivectors created by `create_spatial_mvf`.

Optional keyword arguments:

- `color=colorant"darkorange"`: the color of the multivector.
- `alpha`, `bgalpha`, `pdim`, `hull_only`, `clip`, `linewidth`, `markersize`: as for `plot_simplicial3d_morse`.

The first form also accepts the keywords `axis` and `size`, see `plot_simplicial3d`.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`.

source

`ConleyDynamics.plot_simplicial3d_mv!` – Function.

```
plot_simplicial3d_mv!(ax, args...; kwargs...)
```

Draw into an existing Makie axis ax. See plot_simplicial3d_mv.

source

ConleyDynamics.plot_simplicial3d_mvf – Function.

```
plot_simplicial3d_mvf(ec::EuclideanComplex, mvf::CellSubsets;
                      kwargs...) -> Makie.Figure
plot_simplicial3d_mvf!(ax, ec::EuclideanComplex, mvf::CellSubsets;
                       kwargs...)
```

Plot a multivector field on a simplicial complex embedded in three dimensions, using Makie.

Every multivector with at least two cells is drawn in the same way as a Morse set in plot_simplicial3d_morse. The colors are chosen from a palette by a greedy coloring, so that adjacent multivectors get different colors whenever the palette is large enough. Since multivectors fill the complex, such plots are only readable for small complexes, or together with the clip keyword.

Optional keyword arguments:

- colors=nothing: the color palette. By default, twelve light distinguishable colors are used.
- ci::Bool=false: color each multivector by its Conley index instead.
- addcritical::Bool=false: also draw the singleton multivectors of mvf and the cells which do not belong to any multivector of mvf, which are critical cells.
- alpha::Real=0.8: opacity of the multivectors.
- bgalpha, pdim, hull_only, clip, linewidth, markersize: as for plot_simplicial3d_morse.

The first form also accepts the keywords axis and size, see plot_simplicial3d.

Requires Makie.jl, which is loaded by GLMakie, CairoMakie, or WGLMakie.

source

ConleyDynamics.plot_simplicial3d_mvf! – Function.

```
plot_simplicial3d_mvf!(ax, args...; kwargs...)
```

Draw into an existing Makie axis ax. See plot_simplicial3d_mvf.

source

ConleyDynamics.plot_cubical3d – Function.

```
plot_cubical3d(ec::EuclideanComplex; kwargs...) -> Makie.Figure
plot_cubical3d!(ax, ec::EuclideanComplex; kwargs...)
```

Plot a cubical complex embedded in three dimensions with Makie.

Squares are drawn as two triangles each, and cubes by their boundary squares. The keyword arguments are the same as for `plot_simplicial3d`.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`.

source

`ConleyDynamics.plot_cubical3d!` – Function.

```
plot_cubical3d!(ax, args...; kwargs...)
```

Draw into an existing Makie axis `ax`. See `plot_cubical3d`.

source

`ConleyDynamics.plot_cubical3d_morse` – Function.

```
plot_cubical3d_morse(ec::EuclideanComplex, morsesets::CellSubsets;
                     kwargs...) -> Makie.Figure
plot_cubical3d_morse!(ax, ec::EuclideanComplex, morsesets::CellSubsets;
                      kwargs...)
```

Plot a cubical complex embedded in three dimensions with Morse sets highlighted, using Makie. The keyword arguments are the same as for `plot_simplicial3d_morse`.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`.

source

`ConleyDynamics.plot_cubical3d_morse!` – Function.

```
plot_cubical3d_morse!(ax, args...; kwargs...)
```

Draw into an existing Makie axis `ax`. See `plot_cubical3d_morse`.

source

`ConleyDynamics.plot_cubical3d_mv` – Function.

```
plot_cubical3d_mv(ec::EuclideanComplex, mv; kwargs...) -> Makie.Figure
plot_cubical3d_mv!(ax, ec::EuclideanComplex, mv; kwargs...)
```

Plot a single multivector of a cubical complex embedded in three dimensions, using Makie. The keyword arguments are the same as for `plot_simplicial3d_mv`.

Requires `Makie.jl`, which is loaded by `GLMakie`, `CairoMakie`, or `WGLMakie`.

source

`ConleyDynamics.plot_cubical3d_mv!` – Function.

```
plot_cubical3d_mv!(ax, args...; kwargs...)
```

Draw into an existing Makie axis ax. See plot_cubical3d_mv.

source

ConleyDynamics.plot_cubical3d_mvf – Function.

```
plot_cubical3d_mvf(ec::EuclideanComplex, mvf::CellSubsets;
                   kwargs...) -> Makie.Figure
plot_cubical3d_mvf!(ax, ec::EuclideanComplex, mvf::CellSubsets;
                    kwargs...)
```

Plot a multivector field on a cubical complex embedded in three dimensions, using Makie. The keyword arguments are the same as for plot_simplicial3d_mvf.

Requires Makie.jl, which is loaded by GLMakie, CairoMakie, or WGLMakie.

source

ConleyDynamics.plot_cubical3d_mvf! – Function.

```
plot_cubical3d_mvf!(ax, args...; kwargs...)
```

Draw into an existing Makie axis ax. See plot_cubical3d_mvf.

source

ConleyDynamics.plot_cells3d! – Function.

```
plot_cells3d!(ax, ec::EuclideanComplex, cells; kwargs...)
```

Draw a subset of cells of a complex embedded in three dimensions into an existing Makie axis ax.

The vector cells contains cell indices or labels. The cells are drawn according to their dimension, in the same way as a Morse set in plot_simplicial3d_morse. This is the basic building block of all three-dimensional plotting functions, and it can be used to add further cell subsets to a plot, such as exit sets or constraint surfaces.

Optional keyword arguments:

- color=colorant"darkorange": the color of the cells.
- alpha::Real=1.0: opacity of the faces.
- pdim::Vector{Bool}=[true,true,true,true]: which cell dimensions to draw.
- hull_only, clip, markersize: as for plot_simplicial3d.
- linewidth::Real=4: width of the edges.

Requires Makie.jl, which is loaded by GLMakie, CairoMakie, or WGLMakie.

source

21.8 Internal Helpers

`ConleyDynamics._plot_strata_diagram` – Function.

```
_plot_strata_diagram(strata, edges, fname; title, figw, figh, pv, legendh,
                     edge_style, legend_header, legend_swatches)
```

Shared Luxor rendering core behind `plot_morse_strata` and `plot_morse_reachability`: lays out one node per key of `strata` (a `Dict{Vector{Int},Vector{Int}}` as returned by `stratum_partition`) by level sum(M), and draws an edge for every element of `edges` (`NamedTuples` shaped like the output of `stratum_adjacency` and `stratum_reachability`, using `.M1`, `.M2`, `.weight`, `.ncovered`, `.ntotal`).

The legend is built from two pieces:

- `legend_header`: plain text lines shown at the top of the legend.
- `legend_swatches`: one colored line-and-caption block per entry, given as a vector of named tuples with fields `color`, `dash`, `width`, and `caption` (a `Vector{String}`).

`edge_style(e)` maps an edge to its `(color, dash, width)` – this is the one piece of classification logic that differs between callers. `legendh` is the fixed height reserved for the legend block; callers size it to their own header/swatch content.

Returns `(pos, figwI, foghI)` for the caller to fold into its own return value alongside whatever edge/strata data it wants to expose.

source

Chapter 22

Sparse Matrix Functions

22.1 Internal Sparse Matrix Representation

ConleyDynamics.SparseMatrix – Type.

```
SparseMatrix{T}
```

Composite data type for a sparse matrix with entries of type T.

The struct has the following fields:

- `const nrow::Int`: Number of rows
- `const ncol::Int`: Number of columns
- `const char::Int`: Characteristic of type T
- `const zero::T`: Number 0 of type T
- `const one::T`: Number 1 of type T
- `entries::Vector{Vector{T}}`: Matrix entries corresponding to `columns`
- `columns::Vector{Vector{Int}}`: `columns[k]` points to nonzero entries in column k
- `rows::Vector{Vector{Int}}`: `rows[k]` points to nonzero entries in the k-th row

source

22.2 Access Functions

ConleyDynamics.sparse_get_entry – Function.

```
sparse_get_entry(matrix::SparseMatrix, ri::Int, ci::Int)
```

Get the sparse matrix entry at location `(ri,ci)`.

source

Base.getindex – Method.

```
Base.getindex(matrix::SparseMatrix, ri::Int, ci::Int)
```

Get the sparse matrix entry at location `(ri,ci)`.

source

`ConleyDynamics.sparse_set_entry!` – Function.

```
sparse_set_entry!(matrix::SparseMatrix, ri::Int, ci::Int, val)
```

Set the sparse matrix entry at location `(ri,ci)` to `val`.

source

```
sparse_set_entry!(matrix::SparseMatrix{Int}, ri::Int, ci::Int, val)
```

Set the sparse matrix entry at location `(ri,ci)` to `val`.

This method assumes that the sparse matrix is over the integers, which means that it is interpreted over a finite field. Thus, the new entry is automatically reduced via modular arithmetic.

source

`Base.setindex!` – Method.

```
Base.setindex!(matrix::SparseMatrix, val, ri::Int, ci::Int)
```

Set the sparse matrix entry at location `(ri,ci)` to `val`.

source

`ConleyDynamics.sparse_get_column` – Function.

```
sparse_get_column(matrix::SparseMatrix, ci::Int)
```

Get the ci-th column of the sparse matrix.

source

`ConleyDynamics.sparse_get_nz_column` – Function.

```
sparse_get_nz_column(matrix::SparseMatrix, ci::Int)
```

Get the row indices for the nonzero entries in the ci-th column of the sparse matrix.

source

`ConleyDynamics.sparse_get_nz_row` – Function.

```
sparse_get_nz_row(matrix::SparseMatrix, ri::Int)
```

Get the column indices for the nonzero entries in the ri-th row of the sparse matrix.

source

ConleyDynamics.sparse_minor – Function.

```
smp = sparse_minor(sm::SparseMatrix, rvec::Vector{Int}, cvec::Vector{Int})
```

Create sparse submatrix by specifying the desired row and column indices.

source

22.3 Basic Functions

ConleyDynamics.sparse_size – Function.

```
sparse_size(matrix::SparseMatrix, dim::Int)
```

Number of rows (dim=1) or columns (dim=2) of a sparse matrix.

source

```
sparse_size(matrix::SparseMatrix)
```

Return the size of a sparse matrix.

The function returns a tuple which contains the number of rows and the number of columns of the given matrix.

source

ConleyDynamics.sparse_low – Function.

```
sparse_low(matrix::SparseMatrix, col::Int)
```

Row index of the lowest nonzero matrix entry in column col.

source

ConleyDynamics.sparse_is_zero – Function.

```
sparse_is_zero(sm::SparseMatrix)
```

Test whether the sparse matrix sm is the zero matrix.

source

ConleyDynamics.sparse_is_identity – Function.

```
sparse_is_identity(A::SparseMatrix)
```

Test whether the sparse matrix A is the identity matrix.

source

ConleyDynamics.sparse_is_equal – Function.

```
sparse_is_equal(A::SparseMatrix, B::SparseMatrix)
```

Test whether the sparse matrices A and B are equal.

For equality, the two sparse matrices not only have to have the same size and the same entries, they also have to be of the same type and be defined over the same field.

source

Base.:(==) – Method.

```
Base.:(==)(A::SparseMatrix,B::SparseMatrix)
```

Test whether the sparse matrices A and B are equal.

For equality, the two sparse matrices not only have to have the same size and the same entries, they also have to be of the same type and be defined over the same field.

source

ConleyDynamics.sparse_is_rref – Function.

```
sparse_is_rref(A::SparseMatrix)
```

Check whether a matrix is in reduced row Echelon format.

This function determines whether a given matrix is in reduced row Echelon format. It returns the appropriate boolean value.

source

ConleyDynamics.sparse_is_sut – Function.

```
bool = sparse_is_sut(sm::SparseMatrix)
```

Check whether the sparse matrix is strictly upper triangular.

source

ConleyDynamics.sparse_fullness – Function.

```
sparse_fullness(sm::SparseMatrix)
```

Return the fullness of the sparse matrix `sm`, which equals the percentage of nonzero elements.

source

`ConleyDynamics.sparse_sparsity` – Function.

```
sparse_sparsity(sm::SparseMatrix)
```

Return the sparsity of the sparse matrix `sm`, which equals the percentage of zero entries.

source

`ConleyDynamics.sparse_nz_count` – Function.

```
sparse_nz_count(sm::SparseMatrix)
```

Return the number of nonzero entries of the sparse matrix `sm`.

source

`ConleyDynamics.sparse_show` – Function.

```
sparse_show(sm::SparseMatrix)
```

Display a sparse matrix in readable format.

This function will print the complete matrix, so be careful with sparse matrices of large size!

source

```
sparse_show(sm::SparseMatrix, rlabels::Vector{String}, clabels::Vector{String})
```

Display a sparse matrix in readable format, with labels.

The input parameter `clabels` contains the labels for the columns, while `rlabels` give the row labels.

Examples

```
julia> lc, mvf = example_forman1d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm.matrix, cm.labels, cm.labels)
   ┆  A CD  F BF DE
---┆---------------
 A ┆  .  .  .  .  1
CD ┆  .  .  .  .  .
 F ┆  .  .  .  .  1
BF ┆  .  .  .  .  .
DE ┆  .  .  .  .  .
```

source

```
sparse_show(sm::SparseMatrix, labels::Vector{String})
```

Display a sparse matrix in readable format, with labels.

This function assumes that the matrix is square, and that the labels are the same for rows and columns. They are provided in `labels`.

source

```
sparse_show(cm::ConleyMorseCM)
```

Display the connection matrix with labels.

This function displays the (sparse) connection matrix including its labels. It uses `sparse_show(cm.matrix, cm.labels, cm.labels)`.

Examples

```
julia> lc, mvf = example_forman1d();

julia> cm = connection_matrix(lc, mvf, algorithm="DHL");

julia> sparse_show(cm)
   ┆  A CD  F BF DE
---┆---------------
 A ┆  .  .  .  .  1
CD ┆  .  .  .  .  .
 F ┆  .  .  .  .  1
BF ┆  .  .  .  .  .
DE ┆  .  .  .  .  .
```

source

`Base.show` – Method.

```
Base.show(io::IO, ::MIME"text/plain", sm::SparseMatrix)
```

Display the sparse matrix `sm` when hitting return in REPL.

source

22.4 Conversion Functions

`ConleyDynamics.sparse_from_full` – Function.

```
sparse_from_full(matrix::Matrix{Int}; [p::Int=2])
```

Create sparse matrix from full integer matrix. If the optional argument p is specified and positive, then the returned matrix is an integer matrix which is interpreted over GF(p). On the other hand, if p is equal to zero, then the return matrix has rational type.

source

```
sparse_from_full(vec::Vector{Int}; [p::Int=2])
```

Create sparse column matrix from a full integer vector. If the optional argument p is specified and positive, then the returned matrix is an integer matrix which is interpreted over GF(p). On the other hand, if p is equal to zero, then the return matrix has rational type.

source

ConleyDynamics.full_from_sparse – Function.

```
full_from_sparse(sm::SparseMatrix)
```

Create full matrix from sparse matrix.

source

ConleyDynamics.sparse_from_lists – Function.

```
sparse_from_lists(nr, nc, tchar, tzero, tone, r, c, v)
```

Create sparse matrix from lists describing the entries.

The vectors r, c, and v have to have the same length and the matrix has entry v[k] at (r[k],c[k]). Zero entries will be ignored, and multiple entries for the same matrix position raise an error.

The input arguments have the following meaning:

- nr::Int: Number of rows
- nc::Int: Number of columns
- tchar: Field characteristic if T==Int
- tzero::T: Number 0 of type T
- tone::T: Number 1 of type T
- r::Vector{Int}: Vector of row indices
- c::Vector{Int}: Vector of column indices
- v::Vector{T}: Vector of matrix entries

If tchar>0, then the entries in v are all replaced by their values mod tchar.

source

ConleyDynamics.lists_from_sparse – Function.

```
nr, nc, tchar, tzero, tone, r, c, v = lists_from_sparse(sm::SparseMatrix)
```

Create list representation from sparse matrix.

The output variables are exactly what is needed to create a sparse matrix object using `sparse_from_lists`.

source

22.5 Matrix Creation

`ConleyDynamics.sparse_identity` – Function.

```
sparse_identity(n::Int; p::Int=2)
```

Create a sparse identity matrix with n rows and columns.

The optional argument p specifies the field characteristic. If `p=0` then the sparse matrix is over the rationals, while if `p>0` is a prime, then the matrix is an integer matrix whose entries are interpreted in `GF(p)`.

source

`ConleyDynamics.sparse_zero` – Function.

```
sparse_zero(nr::Int, nc::Int; p::Int=2)
```

Create a sparse zero matrix with `nr` rows and `nc` columns.

The optional argument p specifies the field characteristic. If `p=0` then the sparse matrix is over the rationals, while if `p>0` is a prime, then the matrix is an integer matrix whose entries are interpreted in `GF(p)`.

source

`ConleyDynamics.sparse_diagonal` – Function.

```
sparse_diagonal(nr::Int, nc::Int, d::Vector{<:Real};
                p::Int=2, offset::Int=0)
```

Create sparse diagonal matrix.

The function creates a sparse matrix with `nr` rows and `nc` columns, over a field with characteristic `p`. Without additional arguments, the matrix has the entries in the vector `d` along the main diagonal. If the optional argument `offset` is given and positive, then the diagonal is placed `offset` positions above the main diagonal, if it is negative, then it indicates how many positions below the diagonal it is placed. The function places as many entries from `d` as possible, i.e., if the length of `d` is too short, the remaining diagonal entries are zero, if it is too long, then the later entries are ignored. The function tries to convert the entries in `d` to the correct format based on `p`. Errors are raised if this is not possible.

source

`ConleyDynamics.sparse_hcat` – Function.

```
sparse_hcat(A::SparseMatrix, B::SparseMatrix)
```

Concatenate two sparse matrices horizontally.

Exceptions are raised if the matrices are not defined over the same field, or if they have different numbers of rows.

source

```
sparse_hcat(S1, S2, S3, ...)
```

Concatenate several sparse matrices horizontally.

This method of `sparse_hcat` allows for any number of arguments. If you want to concatenate a vector `vec` of sparse matrices, use the call `sparse_hcat(vec...)`.

Exceptions are raised if the matrices are not defined over the same field, or if they have different numbers of rows.

source

`ConleyDynamics.sparse_vcat` – Function.

```
sparse_vcat(A::SparseMatrix, B::SparseMatrix)
```

Concatenate two sparse matrices vertically.

Exceptions are raised if the matrices are not defined over the same field, or if they have different numbers of columns.

source

```
sparse_vcat(S1, S2, S3, ...)
```

Concatenate several sparse matrices vertically.

This method of `sparse_vcat` allows for any number of arguments. If you want to concatenate a vector `vec` of sparse matrices, use the call `sparse_vcat(vec...)`.

Exceptions are raised if the matrices are not defined over the same field, or if they have different numbers of columns.

source

`ConleyDynamics.sparse_hvcat` – Function.

```
sparse_hvcat(rblocks::Tuple{Vararg{Int}}, A::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

The fist argument has to be a tuple which specifies how many blocks have to be combined in each row. This means that the total number of sparse matrices has to be equal to the sum of the entries in `rblocks`.

Exceptions are raised if the dimensions of the matrices are not compatible, or if the matrices are not defined over the same field.

source

```
sparse_hvcat(rblocks::Int, A::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

The fist argument specifies how many blocks are contained in each row. Thus, the number of sparse matrices in Avec has to be divisible by `rblocks`. Exceptions are raised if the dimensions of the matrices are not compatible, or if the matrices are not defined over the same field.

source

`ConleyDynamics.sparse_hvncat` – Function.

```
sparse_hvncat(dims::Tuple{Int,Int}, rowfirst::Bool, A::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

The function expects dimensions `(d1,d2)`, as well as `d1*d2` sparse matrices. The boolean argument `rowfirst` indicates whether the sparse matrices are arranged row-first or column-first. The matrices are then arranged in an `d1 x d2`-times rectangular array to create a new sparse matrix. Exceptions are raised if the dimensions of the matrices are not compatible, or if the matrices are not defined over the same field.

source

`ConleyDynamics.sparse_cat` – Function.

```
sparse_cat(A::SparseMatrix...; dims=1)
```

Concatenate sparse matrices along a dimension.

The integer `dims` specifies along which dimension the sparse matrices are combined. In other words, `dims=1` reduces to `vcat`, while `dims=2` reduces to `hcat`. In addition, this function accepts the argument `dims=(1,2)`. In this case, the sparse matrices are placed as blocks along the diagonal of a large sparse block matrix, with all remaining blocks being zero.

source

`Base.hcat` – Method.

```
Base.hcat(::SparseMatrix...)
```

Concatenate several sparse matrices horizontally.

This method allows one to use the shortform `[S1 S2 S3...]` for the horizontal concatenation, just like in the matrix case.

source

`Base.vcat` – Method.

```
Base.vcat(::SparseMatrix...)
```

Concatenate several sparse matrices vertically.

This method allows one to use the shortform `[S1; S2; S3; ...]` for the vertical concatenation, just like in the matrix case.

source

`Base.hvcat` – Method.

```
Base.hvcat(::Tuple{Vararg{Int}}, ::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

This method allows one to use the shortform `[S1 S2; S3 S4 S5]` etc for the concatenation, just like in the matrix case. It is based on the function `sparse_hvcat`. For more information, refer to its documentation. The number of blocks in each row can vary, as long as the dimensions are overall compatible.

source

`Base.hvcat` – Method.

```
Base.hvcat(::Int, ::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

This method allows one to use the shortform `[S1 S2; S3 S4]` etc for the concatenation, just like in the matrix case. It is based on the function `sparse_hvcat`. For more information, refer to its documentation.

source

`Base.hvncat` – Method.

```
Base.hvncat(::Tuple{Int,Int}, ::Bool, ::SparseMatrix...)
```

Arrange sparse matrices in rectangular form.

This method allows one to use the shortform `[S1; S2;; S3; S4]` etc for the concatenation, just like in the matrix case. It is based on the function `sparse_hvncat`. For more information, refer to its documentation. This form does require that the same number of blocks are used in each row and each column. For varying numbers please use the functions `hvcat` or `sparse_hvcat`.

source

22.6 Elementary Matrix Operations

`ConleyDynamics.sparse_add_column!` – Function.

```
sparse_add_column!(matrix::SparseMatrix, ci1::Int, ci2::Int, cn, cd)
```

Replace column[ci1] by column[ci1] + (cn/cd) * column[ci2].

source

```
sparse_add_column!(matrix::SparseMatrix{Int}, ci1::Int, ci2::Int,
                   cn::Int, cd::Int)
```

Replace column[ci1] by column[ci1] + (cn/cd) * column[ci2].

The computation is performed mod p, where the characteristic is taken from matrix.char. An error is thrown if matrix.char==0.

source

ConleyDynamics.sparse_add_row! – Function.

```
sparse_add_row!(matrix::SparseMatrix, ri1::Int, ri2::Int, cn, cd)
```

Replace row[ri1] by row[ri1] + (cn/cd) * row[ri2].

source

```
sparse_add_row!(matrix::SparseMatrix{Int}, ri1::Int, ri2::Int,
                cn::Int, cd::Int)
```

Replace row[ri1] by row[ri1] + (cn/cd) * row[ri2].

The computation is performed mod p, where the characteristic is taken from matrix.char. An error is thrown if matrix.char==0.

source

ConleyDynamics.sparse_rref! – Function.

```
sparse_rref!(A::SparseMatrix)
```

Compute the reduced row Echelon form of the given matrix.

This function computes the reduced row Echelon form of a sparse matrix in place. The input matrix will be altered!

Example

```
julia> Afull = [1 0 1 2 4 3 2; 3 4 6 2 3 4 2; 2 0 2 4 1 1 1];

julia> A = sparse_from_full(Afull, p=7);

julia> sparse_rref!(A)
```

```
julia> sparse_show(A)
 1 . 1 2 4 . 3
 . 1 6 6 3 . 5
 . . . . . 1 2
```

source

ConleyDynamics.sparse_rref – Function.

```
sparse_rref(A::SparseMatrix)
```

Compute the reduced row Echelon form of the given matrix.

This function computes and returns the reduced row Echelon form of a sparse matrix. The argument A is not altered.

Example

```
julia> Afull = [1 0 1 2 4 3 2; 3 4 6 2 3 4 2; 2 0 2 4 1 1 1];

julia> A = sparse_from_full(Afull, p=2);

julia> sparse_show(A)
 1 . 1 . . 1 .
 1 . . . 1 . .
 . . . . 1 1 1

julia> sparse_show(sparse_rref(A))
 1 . . . . 1 1
 . . 1 . . . 1
 . . . . 1 1 1
```

source

ConleyDynamics.sparse_solve – Function.

```
sparse_solve(A::SparseMatrix, b::SparseMatrix)
```

Find a solution of a sparse linear system.

This function computes a solution of the sparse linear system Ax = b. It is expected that A and b are sparse matrices over the same field, and that both have the same number of rows. Usually, this function is used with a column vector b. However, if b is a matrix, then the respective matrix equation is solved. The function returns a particular solution of the system, if one exists. Otherwise it returns `nothing`.

In order to obtain all solutions of the system, one has to add an arbitrary linear combination of the basis vectors determined via `sparse_basis_kernel(A)`.

Example

```
julia> Afull = [1 0 1 2 4 3 2; 3 4 6 2 3 4 2; 2 0 2 4 1 1 1];

julia> A = sparse_from_full(Afull, p=7);

julia> bfull = [1 2 5 3; 3 6 6 4; 2 4 2 1];

julia> b = sparse_from_full(bfull, p=7);

julia> sparse_show(A)
 1 . 1 2 4 3 2
 3 4 6 2 3 4 2
 2 . 2 4 1 1 1

julia> sparse_show(b)
 1 2 5 3
 3 6 6 4
 2 4 2 1

julia> x = sparse_solve(A, b);

julia> sparse_show(x)
 1 2 3 .
 . . 5 .
 . . . .
 . . . .
 . . . .
 . . 3 1
 . . . .

julia> A*x == b
true
```

source

ConleyDynamics.sparse_basis_kernel – Function.

```
sparse_basis_kernel(A::SparseMatrix)
```

Compute a kernel basis for a sparse matrix.

This function determines a basis for the kernel of the given matrix A. The result is returned as a vector of sparse matrices. The latter are column vectors which span the kernel.

Example

```
julia> Afull = [1 0 1 2 4 3 2; 3 4 6 2 3 4 2; 2 0 2 4 1 1 1];

julia> A = sparse_from_full(Afull, p=7);

julia> sparse_rref!(A)

julia> sparse_show(A)
 1 . 1 2 4 . 3
```

```
 . 1 6 6 3 . 5
 . . . . . 1 2

julia> kbasis = sparse_basis_kernel(A);

julia> length(kbasis)
4

julia> sparse_show(sparse_transpose(kbasis[1]))
 6 1 1 . . . .

julia> sparse_show(sparse_transpose(kbasis[2]))
 5 1 . 1 . . .

julia> sparse_show(sparse_transpose(kbasis[3]))
 3 4 . . 1 . .

julia> sparse_show(sparse_transpose(kbasis[4]))
 4 2 . . . 5 1
```

source

ConleyDynamics.sparse_basis_range – Function.

```
sparse_basis_range(A::SparseMatrix)
```

Compute a basis for the range of a sparse matrix.

This function determines a basis for the range of the given matrix A. The result is returned as a vector of sparse matrices. The latter are column vectors which span the range, also known as column space.

Example

```
julia> Afull = [1 0 1 2 4 3 2; 3 4 6 2 3 4 2; 2 0 2 4 1 1 1];

julia> A = sparse_from_full(Afull, p=7);

julia> sparse_show(A)
 1 . 1 2 4 3 2
 3 4 6 2 3 4 2
 2 . 2 4 1 1 1

julia> rbasis = sparse_basis_range(A);

julia> length(rbasis)
3

julia> sparse_show(sparse_transpose(rbasis[1]))
 1 3 2

julia> sparse_show(sparse_transpose(rbasis[2]))
 . 4 .
```

```
julia> sparse_show(sparse_transpose(rbasis[3]))
 3 4 1

julia> sparse_show(sparse_rref(A))
 1 . 1 2 4 . 3
 . 1 6 6 3 . 5
 . . . . . 1 2
```

source

`ConleyDynamics.sparse_permute` – Function.

```
sparse_permute(sm::SparseMatrix, pr::Vector{Int}, pc::Vector{Int})
```

Create sparse matrix by permuting the row and column indices.

The vector `pr` describes the row permutation, and `pc` the column permutation.

source

`ConleyDynamics.sparse_transpose` – Function.

```
sparse_transpose(A::SparseMatrix)
```

Create the transpose of a sparse matrix.

This does seem pretty self-explanatory, doesn't it?

source

`Base.adjoint` – Method.

```
Base.adjoint(A::SparseMatrix)
```

Create the transpose of a sparse matrix.

This method allows one to use `A'` for the computation of the transpose matrix.

source

`ConleyDynamics.sparse_inverse` – Function.

```
sparse_inverse(matrix::SparseMatrix)
```

Compute the inverse of a sparse matrix.

The function automatically performs the computations over the underlying field of the sparse matrix.

source

`ConleyDynamics.sparse_remove!` – Function.

```
sparse_remove!(matrix::SparseMatrix, ri::Int, ci::Int)
```

Remove the sparse matrix entry at location `(ri,ci)`.

source

`ConleyDynamics.sparse_add` – Function.

```
sparse_add(A::SparseMatrix, B::SparseMatrix)
```

Add two sparse matrices.

Exceptions are raised if the matrix sum is not defined or the entry types do not match.

source

`ConleyDynamics.sparse_subtract` – Function.

```
sparse_subtract(A::SparseMatrix, B::SparseMatrix)
```

Subtract two sparse matrices.

The function returns `A - B`. Exceptions are raised if the matrix difference is not defined or the entry types do not match.

source

`ConleyDynamics.sparse_multiply` – Function.

```
sparse_multiply(A::SparseMatrix, B::SparseMatrix)
```

Multiply two sparse matrices.

Exceptions are raised if the matrix product is not defined or the entry types do not match.

source

```
sparse_multiply(alpha::Int, A::SparseMatrix)
```

Multiply a scalar with a sparse matrix.

This method is for finite fields. An exception is raised if the sparse matrix is not defined over a finite field.

source

```
sparse_multiply(alpha::Rational{Int}, A::SparseMatrix)
```

Multiply a scalar with a sparse matrix.

This method is for the rational numbers. An exception is raised if the sparse matrix is not defined over field of rationals.

source

`ConleyDynamics.sparse_scale` – Function.

```
sparse_scale(sfac, A::SparseMatrix)
```

Scale a sparse matrix by a scalar.

An exception is raised if the entry types do not match.

source

`Base.:+` – Method.

```
Base.:+(A::SparseMatrix, B::SparseMatrix)
```

Add two sparse matrices.

Exceptions are raised if the matrix sum is not defined or the entry types do not match.

source

`Base.:-` – Method.

```
Base.:-(A::SparseMatrix, B::SparseMatrix)
```

Subtract two sparse matrices.

Exceptions are raised if the matrix difference is not defined or the entry types do not match.

source

`Base.:*` – Method.

```
Base.:*(A::SparseMatrix, B::SparseMatrix)
```

Multiply two sparse matrices.

Exceptions are raised if the matrix product is not defined or the entry types do not match.

source

`Base.:*` – Method.

```
Base.:*(alpha::Int, A::SparseMatrix)
```

Multiply a scalar with a sparse matrix.

This method is for finite fields. An exception is raised if the sparse matrix is not defined over a finite field.

source

`Base.:*` – Method.

```
Base.:*(alpha::Rational{Int}, A::SparseMatrix)
```

Multiply a scalar with a sparse matrix.

This method is for the rational numbers. An exception is raised if the sparse matrix is not defined over field of rationals.

source

`Base.:*` – Method.

```
Base.:*(sfac, A::SparseMatrix)
```

Compute the scalar product of `sfac` and the sparse matrix `A`.

An exception is raised if the scalar `sfac` does not have the same type as the matrix entries.

source

22.7 Sparse Helper Functions

`ConleyDynamics.scalar_inverse` – Function.

```
scalar_inverse(s, p::Int)
```

Compute the inverse of a scalar.

source

```
scalar_inverse(s::Int, p::Int)
```

Compute the inverse of a scalar.

This function computes the inverse in modular arithmetic with base `p`.

source

`ConleyDynamics.scalar_multiply` – Function.

```
scalar_multiply(s1, s2, p::Int)
```

Compute the product of two scalars.

source

```
scalar_multiply(s1::Int, s2::Int, p::Int)
```

Compute the product of two scalars.

This function computes the product in modular arithmetic with base `p`.

source

`ConleyDynamics.scalar_add` – Function.

```
scalar_add(s1, s2, p::Int)
```

Compute the sum of two scalars.

source

```
scalar_add(s1::Int, s2::Int, p::Int)
```

Compute the sum of two scalars.

This function computes the sum in modular arithmetic with base `p`.

source

Chapter 23

Complete API Index

23.1 Composite Data Structures

- ConleyDynamics
- ConleyDynamics.AbstractComplex
- ConleyDynamics.Cell
- ConleyDynamics.CellSubsets
- ConleyDynamics.Cells
- ConleyDynamics.ConleyMorseCM
- ConleyDynamics.EuclideanComplex
- ConleyDynamics.LefschetzComplex
- Base.show
- Base.show
- Base.show

23.2 Lefschetz Complex Functions

- ConleyDynamics.compose_reductions
- ConleyDynamics.create_lefschetz_gf2
- ConleyDynamics.create_random_filter
- ConleyDynamics.euclidean_to_lefschetz
- ConleyDynamics.filter_induced_mvf
- ConleyDynamics.filter_shallow_pairs
- ConleyDynamics.lefschetz_boundary
- ConleyDynamics.lefschetz_cell_count
- ConleyDynamics.lefschetz_clomo_pair

- ConleyDynamics.lefschetz_closed_subcomplex
- ConleyDynamics.lefschetz_closure
- ConleyDynamics.lefschetz_coboundary
- ConleyDynamics.lefschetz_field
- ConleyDynamics.lefschetz_filtration
- ConleyDynamics.lefschetz_filtration_mvf
- ConleyDynamics.lefschetz_gfp_conversion
- ConleyDynamics.lefschetz_information
- ConleyDynamics.lefschetz_interior
- ConleyDynamics.lefschetz_is_closed
- ConleyDynamics.lefschetz_is_cubical
- ConleyDynamics.lefschetz_is_locally_closed
- ConleyDynamics.lefschetz_is_simplicial
- ConleyDynamics.lefschetz_is_subcubical
- ConleyDynamics.lefschetz_lchull
- ConleyDynamics.lefschetz_neighbors
- ConleyDynamics.lefschetz_newbasis
- ConleyDynamics.lefschetz_newbasis_maps
- ConleyDynamics.lefschetz_openhull
- ConleyDynamics.lefschetz_reduction
- ConleyDynamics.lefschetz_reduction_maps
- ConleyDynamics.lefschetz_skeleton
- ConleyDynamics.lefschetz_subcomplex
- ConleyDynamics.lefschetz_to_euclidean
- ConleyDynamics.lefschetz_topboundary
- ConleyDynamics.manifold_boundary
- ConleyDynamics.permute_lefschetz_complex
- ConleyDynamics.rescale_coords
- ConleyDynamics.validate_lefschetz_complex

23.3 Special Complex Functions

- ConleyDynamics.cellsubset_bounding_box
- ConleyDynamics.cellsubset_distance
- ConleyDynamics.cellsubset_location_circle
- ConleyDynamics.cellsubset_location_rectangle
- ConleyDynamics.cellsubset_planar_area
- ConleyDynamics.cellsubsets_to_cells
- ConleyDynamics.convert_cells
- ConleyDynamics.convert_cellsubsets
- ConleyDynamics.convert_planar_coordinates
- ConleyDynamics.convert_spatial_coordinates
- ConleyDynamics.create_cubical_box
- ConleyDynamics.create_cubical_complex
- ConleyDynamics.create_cubical_rectangle
- ConleyDynamics.create_golden_ratio_rectangle
- ConleyDynamics.create_simplicial_complex
- ConleyDynamics.create_simplicial_delaunay
- ConleyDynamics.create_simplicial_rectangle
- ConleyDynamics.cube_field_size
- ConleyDynamics.cube_information
- ConleyDynamics.cube_label
- ConleyDynamics.get_cubical_coords
- ConleyDynamics.is_cube_label
- ConleyDynamics.locate_planar_cellsubsets
- ConleyDynamics.poisson_points_box
- ConleyDynamics.segment_intersects_circle
- ConleyDynamics.segment_intersects_rectangle
- ConleyDynamics.signed_distance_circle
- ConleyDynamics.signed_distance_rectangle
- ConleyDynamics.simplicial_klein_bottle
- ConleyDynamics.simplicial_projective_plane
- ConleyDynamics.simplicial_torsion_space

- ConleyDynamics.simplicial_torus
- ConleyDynamics.split_segments
- ConleyDynamics.tetmesh_to_euclidean
- ConleyDynamics.trisurface_cylinder
- ConleyDynamics.trisurface_ellipsoid
- ConleyDynamics.trisurface_parametric
- ConleyDynamics.trisurface_plane_patch
- ConleyDynamics.trisurface_sphere
- ConleyDynamics.trisurface_torus

23.4 Homology Functions

- ConleyDynamics.homology
- ConleyDynamics.persistent_homology
- ConleyDynamics.ph_matrix_reduce
- ConleyDynamics.ph_morse_reduce
- ConleyDynamics.relative_homology

23.5 Conley Theory Functions

- ConleyDynamics.chain_support
- ConleyDynamics.chain_vector
- ConleyDynamics.cm_reduce_dhl26!
- ConleyDynamics.cm_reduce_dlms24!
- ConleyDynamics.cm_reduce_hms21
- ConleyDynamics.cm_reduce_pmorse26
- ConleyDynamics.conley_index
- ConleyDynamics.connection_matrix
- ConleyDynamics.create_mvf_hull
- ConleyDynamics.create_planar_mvf
- ConleyDynamics.create_spatial_mvf
- ConleyDynamics.extract_multivectors
- ConleyDynamics.forman_all_cell_types
- ConleyDynamics.forman_comb_flow

- ConleyDynamics.forman_conley_maps
- ConleyDynamics.forman_critical_cells
- ConleyDynamics.forman_gpaths
- ConleyDynamics.forman_path_weight
- ConleyDynamics.forman_stab_flow
- ConleyDynamics.isoinvset_information
- ConleyDynamics.morse_interval
- ConleyDynamics.morse_sets
- ConleyDynamics.mvf_backward_orbit
- ConleyDynamics.mvf_critical
- ConleyDynamics.mvf_forward_orbit
- ConleyDynamics.mvf_information
- ConleyDynamics.mvf_is_acyclic
- ConleyDynamics.mvf_length
- ConleyDynamics.mvf_neighborhood
- ConleyDynamics.mvf_regular
- ConleyDynamics.planar_nontransverse_edges
- ConleyDynamics.remove_exit_set
- ConleyDynamics.restrict_dynamics

23.6 Acyclic Partition Functions

- ConleyDynamics._rank_decomposition
- ConleyDynamics._stratum_transitive_reduction
- ConleyDynamics.atomic_distances
- ConleyDynamics.block_rho
- ConleyDynamics.block_split_deltas
- ConleyDynamics.connected_split_deltas
- ConleyDynamics.construct_ap_space
- ConleyDynamics.construct_ap_stratum
- ConleyDynamics.construct_ap_stratum_top
- ConleyDynamics.convert_mvf_partition
- ConleyDynamics.convert_partition_mvf

- ConleyDynamics.is_atomic_refinement
- ConleyDynamics.is_closed_in_block
- ConleyDynamics.is_connected_block
- ConleyDynamics.is_connected_partition
- ConleyDynamics.is_top_layer
- ConleyDynamics.morse_vector
- ConleyDynamics.split_spectrum
- ConleyDynamics.stratum_adjacency
- ConleyDynamics.stratum_jump_weight
- ConleyDynamics.stratum_partition
- ConleyDynamics.stratum_reachability
- ConleyDynamics.stratum_top_layer

23.7 Example Functions

- ConleyDynamics.example_clorenz
- ConleyDynamics.example_critical_simplex
- ConleyDynamics.example_dunce_chaos
- ConleyDynamics.example_forman1d
- ConleyDynamics.example_forman2d
- ConleyDynamics.example_julia_logo
- ConleyDynamics.example_moebius
- ConleyDynamics.example_multiflow
- ConleyDynamics.example_nonunique
- ConleyDynamics.example_small_periodicity
- ConleyDynamics.example_subdivision
- ConleyDynamics.example_three_cm
- ConleyDynamics.example_torsion_chaos

23.8 Extension Functions

- ConleyDynamics.TetDomain
- ConleyDynamics.TetGenResult
- ConleyDynamics.create_simplicial_delaunay3d
- ConleyDynamics.delaunay_points_add_nodes
- ConleyDynamics.delaunay_points_add_segment
- ConleyDynamics.delaunay_points_add_split_segs
- ConleyDynamics.delaunay_points_bnd_rectangle
- ConleyDynamics.delaunay_to_simplicial
- ConleyDynamics.gmsh_add_trisurface
- ConleyDynamics.gmsh_common_refinement
- ConleyDynamics.gmsh_file_to_simplicial
- ConleyDynamics.gmsh_marker_cells
- ConleyDynamics.gmsh_to_simplicial
- ConleyDynamics.tetgen_domain_add_curve
- ConleyDynamics.tetgen_domain_add_hole
- ConleyDynamics.tetgen_domain_add_nodes
- ConleyDynamics.tetgen_domain_add_region
- ConleyDynamics.tetgen_domain_add_surface
- ConleyDynamics.tetgen_domain_box
- ConleyDynamics.tetgen_domain_from_surface
- ConleyDynamics.tetgen_marker_cells
- ConleyDynamics.tetgen_mesh
- ConleyDynamics.tetgen_region_cells
- ConleyDynamics.tetgen_to_simplicial

23.9 Plotting Functions

- ConleyDynamics.CubicalComplexPlot
- ConleyDynamics.CubicalMorsePlot
- ConleyDynamics.MVFPlot
- ConleyDynamics.MVRegionPlot
- ConleyDynamics.SimplicialComplexPlot

- ConleyDynamics.SimplicialMorsePlot
- ConleyDynamics._plot_strata_diagram
- ConleyDynamics.plot_cells3d!
- ConleyDynamics.plot_cubical
- ConleyDynamics.plot_cubical3d
- ConleyDynamics.plot_cubical3d!
- ConleyDynamics.plot_cubical3d_morse
- ConleyDynamics.plot_cubical3d_morse!
- ConleyDynamics.plot_cubical3d_mv
- ConleyDynamics.plot_cubical3d_mv!
- ConleyDynamics.plot_cubical3d_mvf
- ConleyDynamics.plot_cubical3d_mvf!
- ConleyDynamics.plot_cubical_morse
- ConleyDynamics.plot_cubical_mv
- ConleyDynamics.plot_cubical_mvf
- ConleyDynamics.plot_morse_reachability
- ConleyDynamics.plot_morse_strata
- ConleyDynamics.plot_planar_cubical
- ConleyDynamics.plot_planar_cubical_morse
- ConleyDynamics.plot_planar_simplicial
- ConleyDynamics.plot_planar_simplicial_morse
- ConleyDynamics.plot_simplicial
- ConleyDynamics.plot_simplicial3d
- ConleyDynamics.plot_simplicial3d!
- ConleyDynamics.plot_simplicial3d_morse
- ConleyDynamics.plot_simplicial3d_morse!
- ConleyDynamics.plot_simplicial3d_mv
- ConleyDynamics.plot_simplicial3d_mv!
- ConleyDynamics.plot_simplicial3d_mvf
- ConleyDynamics.plot_simplicial3d_mvf!
- ConleyDynamics.plot_simplicial_morse
- ConleyDynamics.plot_simplicial_mv
- ConleyDynamics.plot_simplicial_mvf

23.10 Sparse Matrix Functions

- ConleyDynamics.SparseMatrix
- Base.:(==)
- Base.:*
- Base.:*
- Base.:*
- Base.:*
- Base.:+
- Base.:-
- Base.adjoint
- Base.getindex
- Base.hcat
- Base.hvcat
- Base.hvcat
- Base.hvncat
- Base.setindex!
- Base.show
- Base.vcat
- ConleyDynamics.full_from_sparse
- ConleyDynamics.lists_from_sparse
- ConleyDynamics.scalar_add
- ConleyDynamics.scalar_inverse
- ConleyDynamics.scalar_multiply
- ConleyDynamics.sparse_add
- ConleyDynamics.sparse_add_column!
- ConleyDynamics.sparse_add_row!
- ConleyDynamics.sparse_basis_kernel
- ConleyDynamics.sparse_basis_range
- ConleyDynamics.sparse_cat
- ConleyDynamics.sparse_diagonal
- ConleyDynamics.sparse_from_full
- ConleyDynamics.sparse_from_lists

- ConleyDynamics.sparse_fullness
- ConleyDynamics.sparse_get_column
- ConleyDynamics.sparse_get_entry
- ConleyDynamics.sparse_get_nz_column
- ConleyDynamics.sparse_get_nz_row
- ConleyDynamics.sparse_hcat
- ConleyDynamics.sparse_hvcat
- ConleyDynamics.sparse_hvncat
- ConleyDynamics.sparse_identity
- ConleyDynamics.sparse_inverse
- ConleyDynamics.sparse_is_equal
- ConleyDynamics.sparse_is_identity
- ConleyDynamics.sparse_is_rref
- ConleyDynamics.sparse_is_sut
- ConleyDynamics.sparse_is_zero
- ConleyDynamics.sparse_low
- ConleyDynamics.sparse_minor
- ConleyDynamics.sparse_multiply
- ConleyDynamics.sparse_nz_count
- ConleyDynamics.sparse_permute
- ConleyDynamics.sparse_remove!
- ConleyDynamics.sparse_rref
- ConleyDynamics.sparse_rref!
- ConleyDynamics.sparse_scale
- ConleyDynamics.sparse_set_entry!
- ConleyDynamics.sparse_show
- ConleyDynamics.sparse_size
- ConleyDynamics.sparse_solve
- ConleyDynamics.sparse_sparsity
- ConleyDynamics.sparse_subtract
- ConleyDynamics.sparse_transpose
- ConleyDynamics.sparse_vcat
- ConleyDynamics.sparse_zero